1/1 Силлабус

текущий урок

Дисциплина: Внедрение моделей машинного обучения

Общая информация

Длительность

6 недели

Семестр

2 семестр

Аттестация

Экзамен

Авторы и преподаватели

Кирилл Власов

- «Цифра», руководитель разработки и внедрения решений промышленного искусственного интеллекта.
- Ведение проектов в области искусственного интеллекта в области цифровизации бизнеса и государственного сектора.
- Преподаёт дисциплины по Machine Learning и DevOps в МФТИ.

Константин Некрасов

- АО «Газпромбанк», директор по разработке ML-моделей.
- Ведение проектов в области машинного обучения для решения прикладных задач финтеха.
- Эксперт в глубоком обучении, в частности в направлениях обработки естественного языка (NLP).
- Разрабатывает и внедряет ML-модели в продуктовую среду.
- Преподаёт дисциплины по Data Science, Data Engineering, Machine Learning, DevOps и Web Development в ведущих вузах страны: МИФИ, МФТИ, ВШЭ.

О дисциплине

Машинное обучение лежит в основе многих инновационных технологий искусственного интеллекта. На этой дисциплине студенты узнают, как программы, разработанные с помощью алгоритмов машинного обучения, умеют предсказывать поломки оборудования, предугадывать поведение клиентов и принимать логические и аналитические решения. Кроме того, в рамках курса студенты узнают, как компании используют машинное обучение, и рассмотрят примеры применения на реальных кейсах.

Дисциплина охватывает ключевые аспекты подготовки моделей машинного обучения к использованию в промышленной среде. Студенты научатся разрабатывать, деплоить и управлять ML-моделями с применением современных практик MLOps, обеспечивать воспроизводимость, надёжность и масштабируемость решений, а также проводить A/B-тестирование для оценки эффективности моделей.

Цель дисциплины

Подготовить студентов к работе на позициях Junior ML-инженера и Junior MLOps-инженера, обучив их ключевым практическим и теоретическим навыкам, необходимым для успешного внедрения и сопровождения моделей машинного обучения в продакшене.

В рамках дисциплины вы:

- научитесь готовить и деплоить модели машинного обучения в промышленную среду;
- освоите инструменты контейнеризации и оркестрации для управления ML-приложениями;
- изучите методы обеспечения воспроизводимости экспериментов;
- освоите проведение A/B-тестирования для оценки эффективности моделей;
- приобретёте навыки работы с сервисной архитектурой и интеграцией моделей в бизнес-процессы.

Пререквизиты и постреквизиты

Что нужно знать для прохождения дисциплины: классическое машинное обучение.

Что можно будет изучать после прохождения дисциплины: введение в глубокое обучение, продвинутые методы глубокого обучения.

Содержание дисциплины

№	Раздел	Образовательные результаты модуля
1	Модуль 1. Подготовка модели к продакшену и деплой	• Понимание процессов подготовки моделей к продакшену. • Навыки настройки среды для деплоя ML-моделей. • Знание основных инструментов и платформ для деплоя (Flask, FastAPI, AWS, Azure)
2	Модуль 2. Воспроизводимость и контейнеризация приложений	• Владение инструментами для обеспечения воспроизводимости (DVC, MLflow). • Навыки контейнеризации с использованием Docker. • Опыт работы с системами управления зависимостями и средами
3	Модуль 3. Сервисная архитектура и оркестрация приложений	• Знание принципов микросервисной архитектуры. • Навыки оркестрации с использованием Kubernetes. • Опыт настройки и мониторинга сервисов в продакшене
4	Модуль 4. A/B-тестирование	• Освоение методов A/B-тестирования для оценки моделей. • Навыки анализа результатов и принятия решений на основе данных. • Понимание статистических основ проведения тестов

Практика

Практическая подготовка по дисциплине осуществляется в формате синхронных онлайн-занятий и асинхронной самостоятельной работы на образовательной платформе.

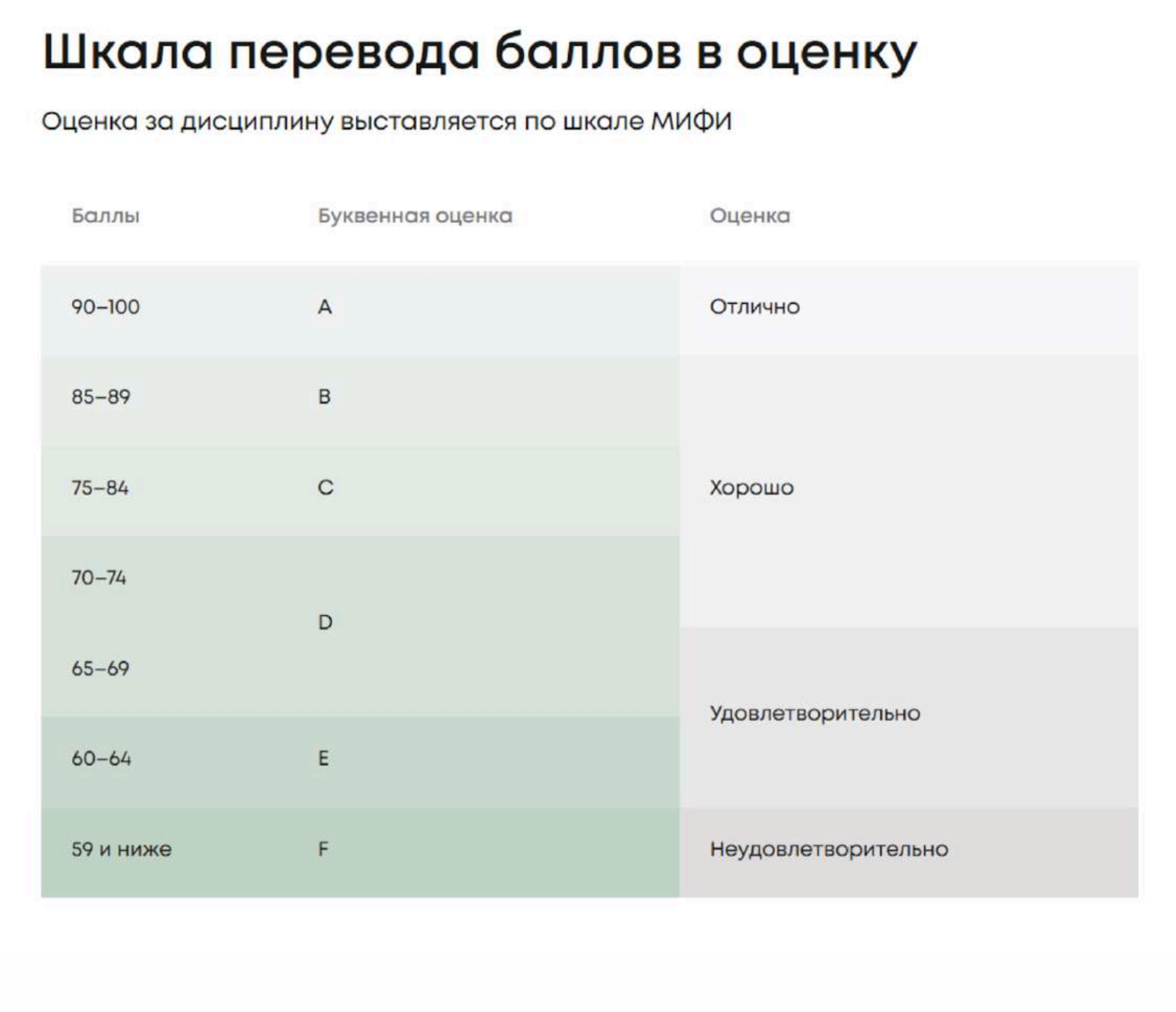
Асинхронная практика включает интерактивные задания: тесты (выбор одного или нескольких ответов), задания с вводом ответа и написание кода с автоматической проверкой. Упражнения направлены на освоение ключевых этапов внедрения ML-моделей: обучение, оценку качества, сохранение моделей и основы развёртывания.

Синхронные занятия проводятся в формате практико-ориентированных онлайн-семинаров с разбором прикладных кейсов и архитектурных решений.

- По завершении дисциплины выполняется семестровое задание — **итоговое тестирование.** Срок выполнения — до трёх недель после окончания курса. Во время сессии задание сдать уже нельзя.
- Итогом дисциплины является **проект (сессионное задание)** — кейс «Разработка и внедрение сервиса прогнозирования дефолта по кредитным картам». Проект сдаётся в течение двух недель после завершения дисциплины либо в период сессии.

Оценивание

Оценка за дисциплину суммируется из оценок за выполнение итогового тестирования (максимальное количество баллов — 50) и сессионного задания, за которое вы также сможете получить максимально 50 баллов.



Шкала перевода баллов в оценку

Оценка за дисциплину выставляется по шкале МИФИ

Баллы	Буквенная оценка	Оценка
90–100	A	Отлично
85–89	B	Хорошо
75–84	C	
70–74	D	
65–69		Удовлетворительно
60–64	E	
59 и ниже	F	Неудовлетворительно

Предыдущий урок Следующий урок

1/11 Введение

текущий урок

Добро пожаловать на дисциплину «Внедрение моделей машинного обучения».

Модель машинного обучения сама по себе не создаёт ценности — она становится полезной только тогда, когда встроена в работающую систему. Только через корректное внедрение, сопровождение и масштабирование ML-решения начинают работать стабильно, приносить измеримую пользу и использоваться в промышленной эксплуатации.

В течение следующих 6 недель вы изучите, как превратить обученную модель в полноценный производственный сервис.

В ЭТОМ РАЗДЕЛЕ ВЫ:

Узнаете:

- о различных способах применения *ML*-моделей;
- таких понятиях, как изолированность, оркестрация и контейнеризация;
- больше о жизненном цикле *ML*-моделей и о том, как происходит их разработка в *DS*-командах на стадии выведения модели в *production*;
- как учесть требование воспроизводимости среды в *DS*-проекте;
- особенности монолитной и микросервисной архитектур, организации взаимодействия между сервисами;

Научитесь:

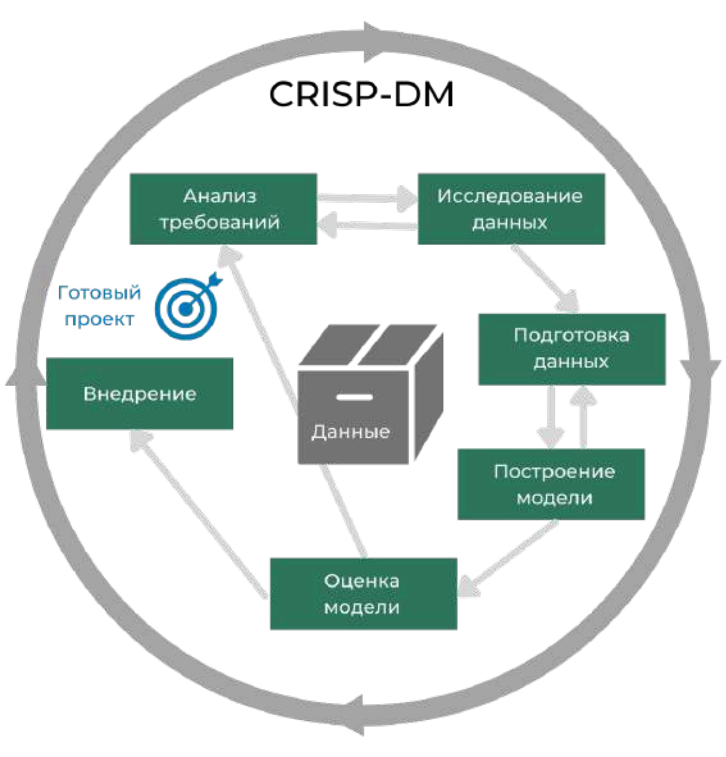
- формулировать требования продакшн-среды к *ML*-моделям;
- сохранять *ML*-модели в зависимости от этих требований;
- разрабатывать простейшие веб-сервисы для деплоя моделей на *Flask*;
- создавать контейнеры для своих моделей и работать с *Docker* и *Docker-compose*;
- оценивать качество моделей после внедрения в *production*.

Важно задать себе следующие вопросы:

- Какими будут основные потребители результатов? Это будут другие сервисы? Или модель будет встроена в мобильное приложение?
- Как и в каком виде потребители ожидают получать результаты? По времени или по запросу? Если по запросу, то по каким правилам они будут его осуществлять?
- Какая для этого существует (или планируется) инфраструктура?

Модель CRISP-DM

Существует множество методологий для управления Data Science-проектами. Одной из классических методологий является Cross-Industry Standard Process for Data Mining, или CRISP-DM. Давайте взглянем на её этапы:



На **этапе внедрения** мы должны понять, будет ли происходить деградация модели во времени в связи с изменением распределений входных данных и возможно ли автоматизировать оценку качества, обновление моделей и их деплой.

> Как учесть все эти сложности в процессе разработки моделей? Вот с этим мы и будем разбираться.

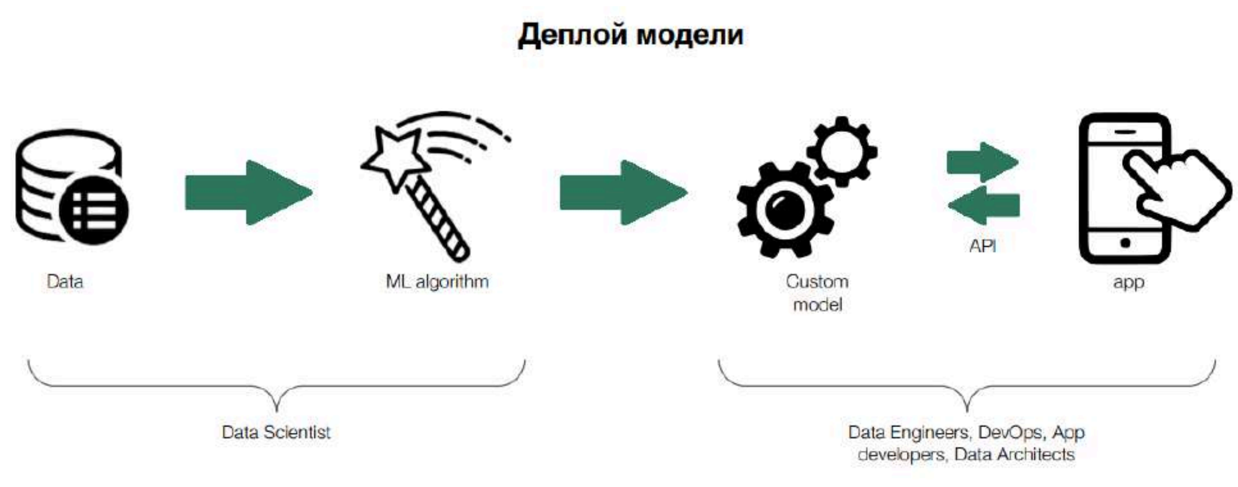
ДЕПЛОЙ МОДЕЛИ

> Задача развёртывания приложения на сервере называется **деплой** (от англ. *deployment* — внедрение, развёртывание). По сути, это процесс трансформации исходного кода вашего приложения в рабочее состояние на конкретном сервере.

Для того чтобы внедрить модель в продакшн (в минимальном варианте), необходимо:

1. Сохранить обученную модель в файл.
2. Поднять сервер.
3. Доставить и запустить на нём свою модель.

Звучит довольно просто, не так ли?



К сожалению, при моделировании зачастую не учитываются вопросы, связанные с деплоем модели и её внедрением в уже существующие системы. Для чего мы обучили модель? Может быть, мы будем использовать её для чат-бота? Может, она должна быть встроена в приложение для *iPhone*? Как часто потребуется переобучать модель?

> Опыт подсказывает, что если модель создаётся «в вакууме», то в итоге её просто нельзя будет вывести в продакшн.

Дополнительной проблемой является то, что большинство библиотек для машинного обучения сфокусированы именно на **обучении**, а не на **предсказании**.

Кроме того, в крупных компаниях дата-сайентисты, дата-инженеры, *ML*-инженеры, занимающиеся внедрением моделей в продакшн, могут быть не просто разными людьми, но и разными командами. На практике может оказаться так, что готовая обученная модель будет внедрена другой командой на другом языке программирования. При этом обычным разработчикам модели машинного обучения могут представляться непрозрачными загадочными чёрными ящиками.

В этом модуле мы будем учиться подготавливать модели к внедрению в продакшн и разворачивать собственный веб-сервис. Как вы увидите дальше, код, который мы написали в *Jupyter Notebook*, практически никогда не попадает в продакшн без изменений.

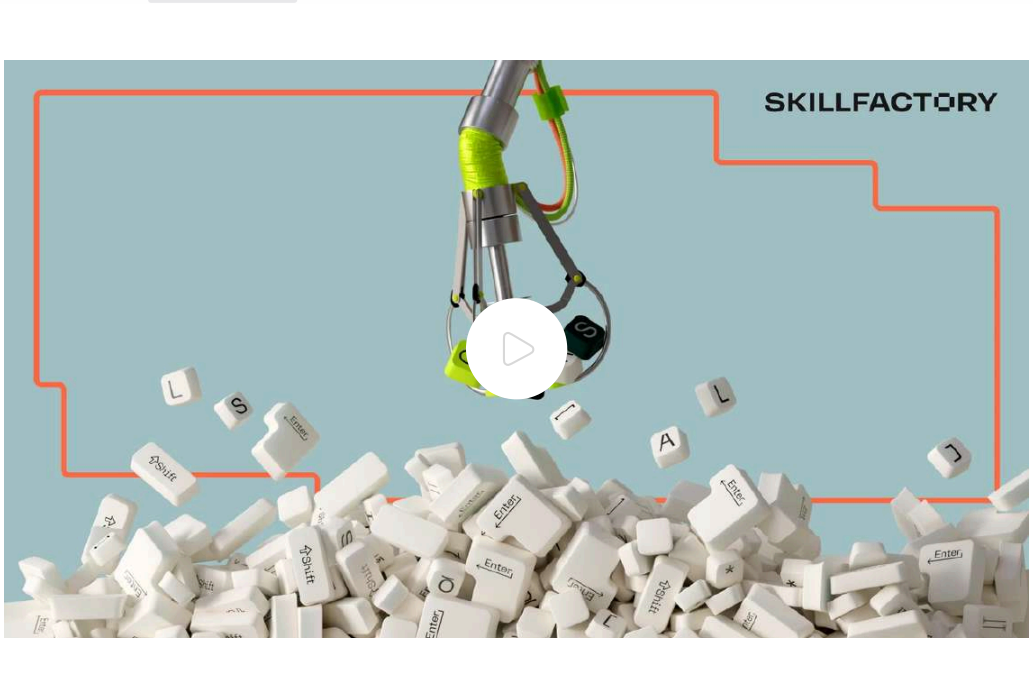
Цели этого модуля:

- Понять, что такое инференс модели и как организуется сохранение и загрузка модели.
- Узнать, что такое сериализация и десериализация и научиться их различать.
- Понять, что делать, если итоговый проект реализован на другом языке программирования, а ваша модель обучена на *Python.*
- Рассмотреть вопросы сетевого взаимодействия и узнать о настройке взаимодействия между серверами по сети.
- Закрепить умение писать запросы к серверу.
- Разобрать основные отличия и области применения фреймворков для разработки веб-сервисов (*Django*, *FastAPI* и, конечно, *Flask*).
- Научиться использовать фреймворк *Flask* для реализации простейших сервисов моделей.

‹ Предыдущий урок   Следующий урок ›

2/11 Сохранение и загрузка моделей: pickle и joblib

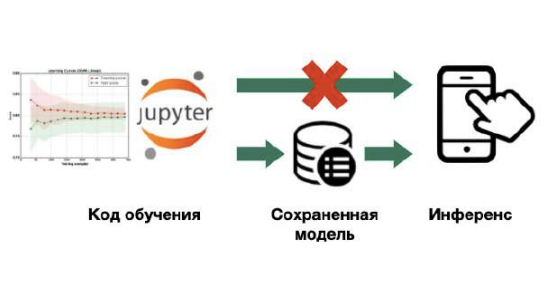
текущий урок



Представьте ситуацию: вы садитесь за руль автомобиля, и… вам нужно заново учиться водить! Сложная ситуация, не правда ли? В реальности мы просто заводим машину, включаем передачу и начинаем движение.

Примерно того же продакшн-среда требует от моделей. И это понятно, ведь намного дешевле с точки зрения расходования ресурсов хранить данные в виде готовой модели, чем каждый раз переобучать модель при каждом запросе/запуске сервиса.

Именно поэтому код, который был написан для обучения модели и оценки её качества, крайне редко используется для **инференса** (от англ inference — вывод). Так называется непрерывная работа алгоритма машинного обучения в конечном приложении. По этой причине при внедрении моделей в продакшн их принято сохранять в готовом виде, то есть уже обученными и готовыми решать реальные задачи.



Изучив материалы в этом юните, вы сможете:

1. Подробно разобраться с тем, как с помощью библиотеки pickle сохранять обученные модели машинного обучения, загружать их обратно и заново обращаться к ним для предсказаний.
2. Узнать о способах передачи моделей для внедрения на языках программирования, отличных от Python.
3. Разобраться с темой **сериализации**.

Совет. Обязательно попробуйте воспроизвести примеры, которые мы будем разбирать далее.

СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ

Как и почти всё в языке программирования Python, обученная модель является **объектом**. Этот объект не простой, поскольку модель содержит сложную иерархию классов — в каждом классе есть набор полей, ссылающихся на объекты других классов, и так далее.

Например, объект класса RandomForestClassifier из библиотеки sklearn содержит множество полей, часть из которых устанавливается во время инициализации модели (максимальная глубина, количество деревьев в ансамбле, критерий информативности и т. д.), а часть определяется во время обучения модели (последовательность предикатов внутри каждого дерева в ансамбле, значимость признаков). Наша задача — «законсервировать» этот объект (модель), сохранив значение всех полей, которые мы задали при инициализации объекта и получили по итогам обучения. То есть мы должны сохранить модель, включая её внешние и внутренние параметры.

Чтобы гарантировать сохранение всей структуры данных и получить её при загрузке обратно, используется сериализация.

> **Сериализация** — это процесс трансформации любой структуры данных, поддерживаемой в языке, в последовательность битов (или байтов). Обратной операцией является **десериализация**.

Зачем это нужно?

То, в каком виде данные записаны на диск, зачастую может сильно отличаться от того, в каком виде они существуют в памяти программы. И там, и там это будут байты информации, но в файле будет представлена просто их последовательность, а в памяти программы это может быть какой-то объект или даже несколько объектов со структурой.

Приведём простой пример с форматом CSV. Пусть у нас есть файл с одной строкой, где несколько слов записаны через запятую. На диске это будет записано просто как последовательность байтов.

Однако когда мы считываем эту последовательность внутри программы, мы хотим работать с ней как со **списком строк**, то есть нам нужно не только считать эти данные, но и применить к ним некоторое преобразование:

```
line = 'word1, word2, word3'

line.split(",")

## ['word1', 'word2', 'word3']
```

Можно заметить, что после этого преобразования появилось много объектов (каждое слово списка) и некоторая структура (то, как слова расположены в этом списке). Процесс, который мы только что выполнили, называется **десериализацией**.

Также возможен обратный процесс, когда мы хотим сохранить в виде последовательности байтов какие-то данные из памяти программы. Например, нам нужно снова получить начальную строку:

```
",".join(['word1', 'word2', 'word3'])

## word1,word2,word3
```

Этот процесс как раз и называется **сериализацией**.

Мы разобрали простейший пример, но на практике всё гораздо сложнее. Сериализовывать можно не только в текст (как с CSV, JSON и подобными форматами), но и в **бинарный** формат, который человек не сможет прочитать.

Бывают форматы, которые могут описывать более сложные структуры (тот же JSON). Также можно добавить сжатие итогового набора битов.

Заметим, что программа должна потратить некоторый ресурс CPU, чтобы преобразовать объект в набор байтов и наоборот.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Для чего нужна сериализация?

○ Позволяет упаковать структуры данных и функций в единый компонент.

○ Обеспечивает способность функции обрабатывать структуры данных разных типов.

○ Выделяет главные характеристики структуры данных и отбрасывает второстепенные.

● Позволяет трансформировать структуры данных в последовательность байтов.

✓

Show answer

Отправить

✓ Верных

Задание 2

0 points possible (ungraded)

Гарантируется ли, что после процесса сериализации и десериализации вы получите те же самые структуры данных?

● Да, гарантируется.

○ Нет, при десериализации всегда получаются структуры данных, отличные от исходных.

✓

Show answer

Отправить

✓ Верных

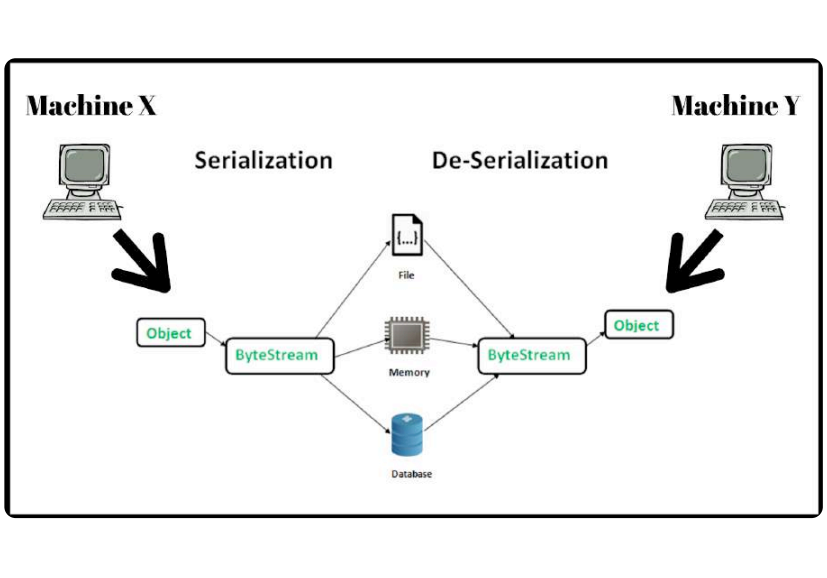
ИНСТРУМЕНТЫ СЕРИАЛИЗАЦИИ: PICKLE

Посмотрим, с помощью каких средств происходит сериализация объектов в Python.

В стандартную библиотеку Python входит модуль pickle, который служит для сериализации почти всех объектов произвольного типа.

Мы помним, что объекты находятся в оперативной памяти и направляются в байтовые потоки ввода-вывода. В байтовые потоки может быть направлен любой файлоподобный объект.

В ходе десериализации исходный объект воссоздаётся в оперативной памяти с теми же самыми значениями, но с новой идентичностью — новым адресом в памяти.



Обратите внимание на предупреждение в официальной документации:

«Warning. The pickle module is not secure. Only unpickle data you trust.»

Так как законсервирован может быть абсолютно любой объект, в нём могут быть «спрятаны» различные вредоносные программы или данные. Поэтому будьте внимательны и не проводите десериализацию бинарных файлов, в происхождении которых вы не уверены.

Не переживайте: файлы, предлагаемые для десериализации в рамках нашей дисциплины, безопасны.

Для иллюстрации работы модуля pickle последовательно пройдёмся по всем шагам и разберёмся с тем, как сериализовать уже обученные алгоритмы машинного обучения.

ШАГ №1

Обучим модель линейной регрессии на встроенном датасете о диабете — Diabetes dataset.

В данном датасете представлены десять исходных признаков: возраст, пол, индекс массы тела, среднее артериальное давление и шесть измерений сыворотки крови были получены для каждого из 442 пациентов с сахарным диабетом. Интерес представляет количественный показатель прогресса заболевания, замеренный через год после исходного измерения. Тип задачи — регрессия.

Примечание. Для простоты и наглядности мы опустим процесс предобработки данных, разведывательный анализ, разделение выборки на обучающую и тестовую, валидацию модели и подбор гиперпараметров, так как для наших целей это неважно. Мы уверены, что вы уже способны произвести эти этапы самостоятельно.

В качестве модели, прогнозирующей целевую переменную, возьмём простейшую линейную регрессию, и обучим её на исходных данных:

```
from sklearn.linear_model import LinearRegression

from sklearn.datasets import load_diabetes

# Загружаем датасет о диабете
X, y = load_diabetes(return_X_y=True)
# Инициализируем модель линейной регрессии
regressor = LinearRegression()
# Обучаем модель
regressor.fit(X,y)

## LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

В результате выполнения кода получился объект класса `LinearRegression`, на который ссылается переменная `regressor`. При этом атрибуты объекта (веса модели линейной регрессии) были сформированы во время обучения. То есть объект `regressor` теперь является обученной моделью.

ШАГ №2

Далее, когда мы получили обученную модель, нам необходимо сериализовать её, превратив объект Python в поток байтов. Для этого импортируем модуль pickle и воспользуемся функцией `dumps()`, в которую нужно передать объект Python.

```
import pickle

# Производим сериализацию обученной модели
model = pickle.dumps(regressor)

print(type(model))
print(type(regressor))
## bytes
## sklearn.linear_model._base.LinearRegression
```

Как видим, мы создали объект `model` типа `bytes`.

ШАГ №3

Давайте попробуем восстановить (десериализовать) объект Python. Для этого в модуле pickle есть функция `loads()`, в которую нужно передать сериализованный объект (поток байтов).

```
# Производим десериализацию
regressor_from_bytes = pickle.loads(model)
regressor_from_bytes
## LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

В результате десериализации мы смогли восстановить исходный объект (модель).

ШАГ №4

Сохраним сериализованный объект прямо в файл. Для этого в pickle есть функция `dump()` (без `s` на конце). В неё необходимо передать имя файла или ссылку на открытый файл. Файл назовём myfile, его расширение — .pkl (формат данных pickle):

```
# Производим сериализацию и записываем результат в файл формата pkl
with open('myfile.pkl', 'wb') as output:
    pickle.dump(regressor, output)
```

Теперь у нас есть бинарный файл с готовой моделью, и мы можем передать его, например, ML-инженерам, которые будут заниматься деплоем модели на сервер.

ШАГ №5

Посмотрим на код, который восстанавливает (десериализует) обученную модель из файла myfile.pkl. Для этого в pickle есть функция `load()` (без `s` на конце). В неё необходимо передать имя файла или ссылку на открытый файл.

```
# Производим десериализацию и извлекаем модель из файла формата pkl
with open('myfile.pkl', 'rb') as pkl_file:
    regressor_from_file = pickle.load(pkl_file)

regressor_from_file
## LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

ШАГ №6

Убедимся, что методы и результаты предсказаний обученной модели и модели, загруженной из файла, совпадают:

```
# Проверяем, что все элементы массивов предсказаний совпадают между собой
all(regressor.predict(X) == regressor_from_bytes.predict(X))
## True
all(regressor.predict(X) == regressor_from_file.predict(X))
## True
```

Как видите, исходная и восстановленная из байтов и файла модели дают одинаковые предсказания. Это значит, что теперь мы можем импортировать наши обученные модели в любое Python-приложение и пользоваться ими, минуя этап обучения и все этапы, предшествующие ему.

ОГРАНИЧЕНИЯ

Как мы упоминали, у pickle есть ограничения. Например, мы не можем сериализовать лямбда-функции. Давайте посмотрим, что нам вернёт следующий код:

```
my_lambda = lambda x: x*2
with open('my_lambda.pkl', 'wb') as output:
    pickle.dump(my_lambda, output)

##"PicklingError: Can't pickle <function <lambda>"
```

Совет. В таких случаях лучше пользоваться пакетом dill.

Задание 3

0 points possible (ungraded)

Что делает функция dumps() из библиотеки *pickle*?

○ Позволяет сохранить сериализованный объект в файл.

● Возвращает сериализованный объект.

○ Загружает объект из файла и десериализует его.

○ Загружает объект из потока байтов.

✓

Show answer

Отправить

✓ Верных

Задание 4

0 points possible (ungraded)

Что делает функция load() из библиотеки *pickle*?

○ Позволяет сохранить сериализованный объект в файл.

○ Возвращает сериализованный объект.

● Загружает объект из файла и десериализует его.

○ Загружает объект из потока байтов.

✓

Show answer

Отправить

✓ Верных

СОХРАНЕНИЕ ПАЙПЛАЙНА

Ранее мы посмотрели простейший пример сериализации готовой модели.

У вас мог возникнуть вопрос: что делать, если перед подачей данных в модель их необходимо предобработать, например произвести стандартизацию, исключить неинформативные признаки? Неужели придётся прописывать все эти шаги в коде инференса модели? А что если вопросами инференса занимаются совершенно другие специалисты, которые вообще ничего не знают о машинном обучении и не умеют производить предобработку данных?

Конечно, мы должны передать результаты в таком виде, чтобы ими можно было воспользоваться без лишних манипуляций.

Мы уже упоминали, что pickle работает с любыми объектами Python. Поэтому для сохранения может быть доступна не просто обученная модель, но и целый **пайплайн**, включающий предобработку данных.

Например, мы хотим сериализовать пайплайн, который включает в себя min-max-нормализацию и отбор пяти наиболее важных факторов на основе корреляции Пирсона. Полученные в результате данные отправляются на вход модели линейной регрессии.

```
import pickle

from sklearn.linear_model import LinearRegression

from sklearn.datasets import load_diabetes

from sklearn.feature_selection import SelectKBest, f_regression

from sklearn.preprocessing import MinMaxScaler

from sklearn.pipeline import Pipeline

# Загружаем датасет о диабете
X, y = load_diabetes(return_X_y=True)

# Создаём пайплайн, который включает нормализацию, отбор признаков и обучение модели
pipe = Pipeline([
  ('Scaling', MinMaxScaler()),
  ('FeatureSelection', SelectKBest(f_regression, k=5)),
  ('Linear', LinearRegression())
  ])

# Обучаем пайплайн
pipe.fit(X, y)
```

Пайплайн обучен. Давайте сохраним его в файл с помощью pickle:

```
# Сериализуем pipeline и записываем результат в файл
with open('my_pipeline.pkl', 'wb') as output:
    pickle.dump(pipe, output)
```

Если сериализация завершилась успешно, то при инференсе модели мы сможем восстановить её из файла:

```
# Десериализуем pipeline из файла
with open('my_pipeline.pkl', 'rb') as pkl_file:
    loaded_pipe = pickle.load(pkl_file)
```

Проверим, что результаты исходного и десериализованного пайплайнов и идентичны:

```
# Сравниваем предсказания исходного и восстановленного пайплайнов
print(all(pipe.predict(X) == loaded_pipe.predict(X)))

## True
```

Примечание. Если мы хотим сохранять сериализованные пайплайны в виде потока байтов, нужно использовать функции `dumps()` и `loads()` , а не `dump()` и `load()`.

Однако в процессе предобработки могут возникнуть шаги, которые нельзя реализовать стандартными методами sklearn. Например, для решения многих задач в нашем курсе мы часто использовали feature engineering, чтобы повысить качество работы моделей. Как встроить этот шаг в исходный пайплайн?

Для этого в sklearn можно организовать так называемые **кастомные трансформеры**. Такой трансформер должен наследоваться от двух классов: `TransformerMixin` и `BaseEstimator`.

Посмотрим на шаблон кастомного трансформера:

```
from sklearn.base import TransformerMixin, BaseEstimator
class MyTransformer(TransformerMixin, BaseEstimator):
    '''Шаблон кастомного трансформера'''

    def __init__(self):
        '''
        Здесь прописывается инициализация параметров, не зависящих от данных.
        '''
        pass

    def fit(self, X, y=None):
        '''
        Здесь прописывается «обучение» трансформера.
        Вычисляются необходимые для работы трансформера параметры (если они нужны).
        '''

        return self

    def transform(self, X):
        '''
        Здесь прописываются действия с данными.
        '''
        return X
```

У трансформера должно быть **три обязательных метода:**

- `__init__()` — метод, который вызывается при создании объекта данного класса. Он предназначен для инициализации исходных параметров.Например, у трансформера для создания полиномиальных признаков PolynomialFeatures из sklearn в методе `__init__()` параметр `degree` задаёт степень полинома.
- `fit()` — метод, который вызывается для «обучения» трансформера. Он должен возвращать ссылку на сам объект (`self`).Например, в трансформере StandardScaler в методе `fit()` прописано вычисление среднего значения и стандартного отклонения в каждом столбце таблицы, переданной в качестве параметра метода `fit()`.
- `transform()` — метод, который трансформирует приходящие на вход данные. Он должен возвращать преобразованный массив данных.Например, при вызове метода `transform()` у StandardScaler из sklearn внутри происходит преобразование — вычитание из каждого столбца среднего и деление результата на стандартное отклонение. Причём среднее и стандартное отклонение вычисляются заранее в методе `fit()`.

Примечание. Как мы знаем, у некоторых трансформеров из sklearn, например у того же MinMaxScaler, есть ещё и метод `fit_transform()`, который является комбинацией методов `fit()` и `transform()`.

Наш трансформер пока что ничего не делает. Предположим, мы хотим генерировать в данных новый признак, который является простым произведением первых трёх столбцов таблицы. Давайте пропишем в методе `transform()` эти действия.

Для работы такого трансформера нужны только исходные данные без дополнительных параметров, поэтому методы `__init__()` и `fit()` остаются без изменений.

```
class MyTransformer(TransformerMixin, BaseEstimator):
    '''Шаблон кастомного трансформера'''


    def __init__(self):
        '''Здесь прописывается инициализация параметров, не зависящих от данных.'''
        pass


    def fit(self, X, y=None):
        '''
        Здесь прописывается «обучение» трансформера.
        Вычисляются необходимые для работы трансформера параметры (если они нужны).
        '''
        return self


    def transform(self, X):
        '''Здесь прописываются действия с данными.'''
        # Создаём новый столбец как произведение первых трёх
        new_column = X[:, 0] * X[:, 1] * X[:, 2]
        # Для добавления столбца в массив нужно изменить его размер на (n_rows, 1)
        new_column = new_column.reshape(X.shape[0], 1)
        # Добавляем столбец в матрицу измерений
        X = np.append(X, new_column, axis=1)
        return X
```

Посмотрим, как работает наш кастомный трансформер. Создадим объект трансформера, вызовем метод `transform` и посмотрим на результирующий размер таблицы.

```
# Инициализируем объект класса MyTransformer (вызывается метод __init__)
custom_transformer = MyTransformer()
# Чисто формально вызываем метод fit, но у нас он ничего не делает
custom_transformer.fit(X)
# Трансформируем исходные данные (вызывается метод transform)
X_transformed = custom_transformer.transform(X)
print('Shape before transform: {}'.format(X.shape))
print('Shape after transform: {}'.format(X_transformed.shape))

## Shape before transform: (442, 10)
## Shape after transform: (442, 11)
```

Видно, что в результате трансформации в исходную матрицу наблюдений добавился новый столбец.

Теперь давайте встроим этот трансформер в сам пайплайн — для этого достаточно добавить новый шаг в пайплайн.

```
# Создаём пайплайн, который включает Feature Engineering, нормализацию, отбор признаков и обучение модели
pipe = Pipeline([
  ('FeatureEngineering', MyTransformer()),
  ('Scaling', MinMaxScaler()),
  ('FeatureSelection', SelectKBest(f_regression, k=5)),
  ('Linear', LinearRegression())
  ])

# Обучаем пайплайн
pipe.fit(X, y)
```

Наконец можно сериализовать полученный pipeline:

```
# Сериализуем pipeline и записываем результат в файл
with open('my_new_pipeline.pkl', 'wb') as output:
    pickle.dump(pipe, output)
```

Теперь мы можем передать пайплайн и воспользоваться им для инференса, предварительно произведя десериализацию.

Задание 5

0 points possible (ungraded)

Десериализуйте полученный *pipeline* с добавленным в него кастомным трансформером из файла. Затем предскажите значение целевой переменной для наблюдения, которое описывается следующим вектором:

```
features = np.array([[ 0.00538306, -0.04464164,  0.05954058,
-0.05616605,  0.02457414, 0.05286081, -0.04340085,  0.05091436,
-0.00421986, -0.03007245]])
```

В поле для ответа введите предсказанное значение целевой переменной, округлённое **до целого числа**.

Отправить

В дополнение предлагаем вам ознакомиться с примерами использования пайплайнов (на англ. языке):

- Sklearn Pipeline Tutorial;
- Pipeline Guide (GitHub);
- A Simple Guide to Scikit-learn Pipelines.

БИБЛИОТЕКА JOBLIB

Как мы видим, pickle прекрасно справляется со своей задачей: мы можем сериализовать и восстанавливать любые Python-объекты, включая модели и даже пайплайны. Однако иногда массивы данных, на которых обучаются модели, бывают настолько большими, что после загрузки из pickle невозможно восстановить объект полностью.

В таких случаях вместо pickle лучше использовать библиотеку joblib. Этот модуль более эффективен и надёжен для работы с объектами, которые содержат большие массивы данных. Пожалуй, единственный минус этого модуля в том, что он может «консервировать» только в файл, поэтому вы не сможете получить объект в виде бинарной строки и работать с ним. В модуле попросту отсутствуют методы для работы с бинарной строкой. Формат файлов для сохранения — .joblib.

В остальном работа с joblib полностью идентична работе с pickle: после обучения модели производим сериализацию с помощью функции `dump()`, а в коде самого приложения, где нужно использовать модель, выполняем десериализацию с помощью функции `load()`. В каждую из этих функций необходимо передать путь до файла для записи и чтения соответственно.

Для иллюстрации работы сохраним полученную линейную регрессию:

```
import joblib

# Загружаем датасет о диабете
X, y = load_diabetes(return_X_y=True)
# Обучаем модель линейной регрессии
regressor = LinearRegression()
regressor.fit(X, y)
# Производим сериализацию и сохраняем результат в файл формата .joblib
joblib.dump(regressor, 'regr.joblib')

## ['regr.joblib']
```

Загрузим файл заново (загрузка может быть произведена в другом файле с кодом):

```
# Десериализуем модель из файла
clf_from_jobliv = joblib.load('regr.joblib')
# Сравниваем предсказания
```

< Предыдущий урок   Следующий урок >

3/11 **Практика: pickle**

текущий урок

Источник изображения

В первом практическом уроке этого модуля мы будем учиться работать с модулем pickle.

Ваш коллега Василий обучил модель и теперь просит вас проверить её на ваших данных. Он присылает вам pickle-файл. Загрузите модель, используя модуль pickle.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

При загрузке вывелся секретный код. Введите его в поле ниже.

Отправить

Задание 2

0 points possible (ungraded)

Проверьте, объект какого типа получился. Какую модель вам прислал коллега?

○ *LinearRegression*
○ *LogisticRegression*
○ *DecisionTreeClassifier*
○ *DecisionTreeRegressor*
○ *RandomForestClassifier*

Отправить

Задание 3

0 points possible (ungraded)

Теперь необходимо применить модель. Сделайте предсказание для следующего набора фичей: [1, 1, 1, 0.661212487096872]. Введите результат, предварительно округлив его до **трёх знаков после точки-разделителя**.

Отправить

У присланной вам модели есть два поля (атрибута) с именами `a` и `b`. Создайте из них словарь с такими же именами ключей и значениями, а затем сохраните его в файл с помощью модуля pickle.

Чтобы вы могли проверить правильность решения задания, мы создали специальный проверочный скрипт. Скачайте его здесь.

Сохраните его рядом с вашим pickle-файлом (в той же папке) и запустите, передав первым аргументом имя файла. Если вы всё сделали правильно, на экран выведется ответ для следующего задания.

Как запустить скрипт?

В ячейке Jupyter Notebook:

```
!python hw1_check_ol.py имя_pickle_файла.pkl
```

В терминале:

```
cd имя_папки_со_скриптом

python hw1_check_ol.py имя_pickle_файла.pkl
```

Задание 4

0 points possible (ungraded)

Если вы всё сделали правильно, в результате выполнения проверочного скрипта будет выведен секретный код.

Впишите полученный код в окно ниже:

После решения задания проверочный скрипт можно удалить — больше он вам не понадобится.

Отправить

‹ Предыдущий урок   Следующий урок ›

Сохранение и перенос моделей: PMML и ONNX

Среда или требования к **инференсу** модели для вашего проекта могут быть устроены так, что потребуют реализации на языке программирования, отличном от Python. Например, если компания разрабатывает десктопное приложение, то для внедрения модели её потребуется «перевести» на Java или C++. Как это сделать?

PREDICTIVE MODEL MARKUP LANGUAGE

В таких случаях используется генерация файла формата PMML (Predictive Model Markup Language).

> **PMML** — это XML-диалект, который применяется для описания статистических и DS-моделей. PMML-совместимые приложения позволяют легко обмениваться моделями данных между собой. Разработка и внедрение PMML осуществляется IT-консорциумом Data Mining Group.

Подробнее с PMML можно ознакомиться на официальном сайте.

К сожалению, далеко не все библиотеки для машинного обучения (в том числе sklearn) поддерживают возможность сохранения обученной модели в указанном формате. Одним из инструментов является библиотека Nyoka (поддержка и совместимость зависят от версии sklearn).

Давайте сохраним модель из предыдущего блока в формат PMML.

Для установки можно использовать систему управления пакетами pip:

```
!pip3 install nyoka
```

или

```
pip install nyoka
```

Рассмотрим пример работы с библиотекой:

```
from nyoka import skl_to_pmml
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.datasets import load_diabetes

X, y = load_diabetes(return_X_y=True)
cols = load_diabetes()['feature_names']

scaler = MinMaxScaler()
pipe = Pipeline([
            ('Scaling', MinMaxScaler()),
            ('Linear', LinearRegression())
        ])
# Обучение пайплайна, включающего линейную модель и нормализацию признаков
pipe.fit(X, y)
# Сохраним пайплайн в формате pmml в файл pipeline.pmml
skl_to_pmml(pipeline=pipe, col_names=cols, pmml_f_name="pipeline.pmml")
```

Итак, мы построили пайплайн обработки данных и обучили модель линейной регрессии. После этого мы с помощью функции `skl_to_pmml` сохранили модель в файл `pipe.pmml`.

Откройте файл `pipe.pmml` с помощью любого текстового редактора.

Давайте рассмотрим этот файл подробнее:

- Секция `<DataDictionary>` содержит информацию о признаках, включая наименование и тип данных, используемых для построения модели.
- Секция `<TransformationDictionary>` содержит информацию о необходимых преобразованиях для каждого признака. Обратите внимание, что в этом блоке также содержится информация для трансформации. Так как мы использовали `minMaxScaler()`, то в файле записаны минимальное и максимальное значения.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Какая секция содержит информацию о типе используемой модели?

Отправить

Таким образом, в файле содержится вся информация для того, чтобы пайплайн можно было использовать на любом языке программирования.

OPEN NEURAL NETWORK EXCHANGE

В разработке моделей на основе нейронных сетей сегодня наиболее распространён формат ONNX (Open Neural Network Exchange).

> ONNX (Open Neural Network Exchange) — это открытый стандарт для обеспечения совместимости моделей машинного обучения. Он позволяет разработчикам искусственного интеллекта использовать модели с различными инфраструктурами, инструментами, средами исполнения и компиляторами.

Стандарт совместно поддерживается компаниями Microsoft, Amazon, Facebook и другими партнёрами как проект с открытым исходным кодом.

Часто стандарт ONNX и его библиотеки используют для конвертации из одного фреймворка в другой (например, из PyTorch в TensorFlow для использования в продакшене). Для конвертации различных фреймворков (не только DL) в формат ONNX и обратно существует ряд библиотек:

- ONNX-Tensorflow;
- Tensorflow-ONNX;
- Keras-ONNX;
- Sklearn-ONNX.
- и другие.

Также в рамках стандарта ONNX есть инструмент ONNX-runtime. Он служит для ускорения инференса Python-моделей, а также инференса на других языках, например Java, C++.

Задание 2

0 points possible (ungraded)

Возможно ли осуществить инференс модели, обученной в *Python*, на других языках программирования?

● Да, можно.
○ Нет, нельзя.

✓

Show answer

Отправить

✓ Верных

Задание 3

0 points possible (ungraded)

Вы обучили классификатор на *Python* с помощью *sklearn*, инференс предстоит делать также на *Python*. Какую библиотеку вы можете использовать для сериализации модели?

○ *Pickle*
○ *Joblib*
○ *Sklearn-ONNX*
○ *Nyoka*
● Все перечисленные выше библиотеки.

✓

Show answer

Отправить

✓ Верных

‹ Предыдущий урок   Следующий урок ›

5/11 Деплой модели. Протоколы сетевого взаимодействия

Помните вашего коллегу Василия — того, что прислал модель в pkl-формате и просил проверить её на ваших данных? Вот, кстати, и ссылка на модель.

Завтра Василию предстоит защищать решение перед руководителями проекта, а для этого нужно показать работающий прототип (в нашем случае — **задеплоить модель на тестовый сервер**) и рассказать об эффективности предлагаемого решения. За деплоем Василий обратился к вам.

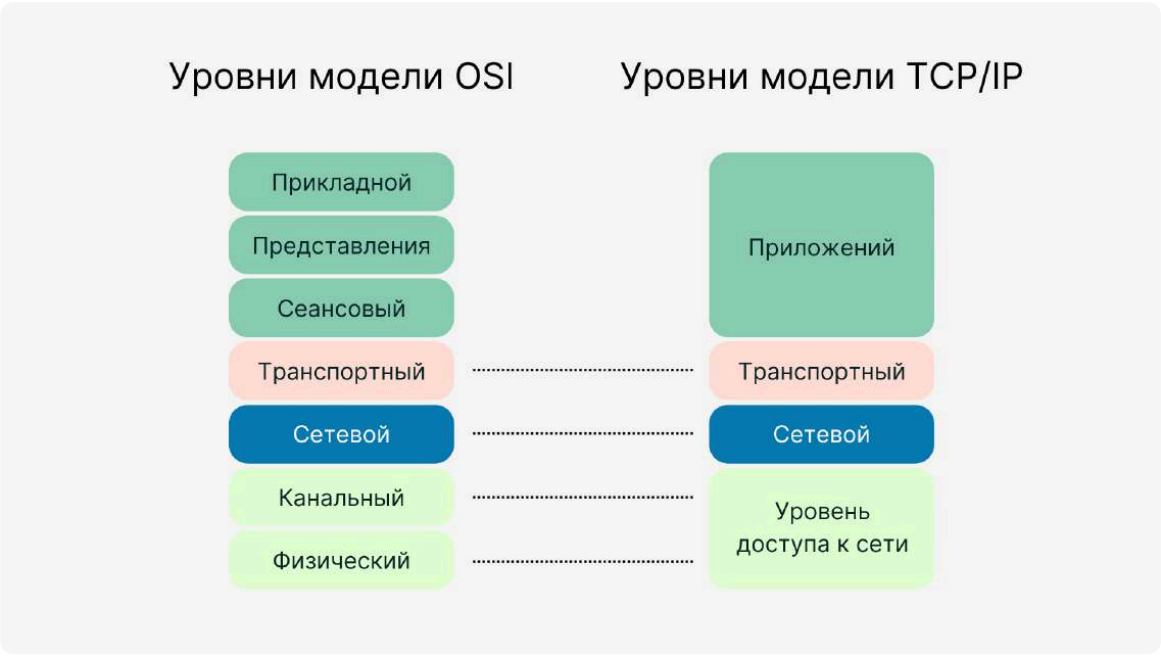
С чего начнём?

- Во-первых, нужно разобраться с тем, как происходит взаимодействие серверов по сети.
- Во-вторых, необходимо узнать, как написать сервер и обернуть в него модель. Какие есть фреймворки? Какой выбрать конкретно в нашем случае? Как его реализовать?

МОДЕЛИ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ

Начнём с первого вопроса и немного поговорим о том, как происходит взаимодействие между серверами по сети, то есть разберём процесс обмена информацией между компьютерами.

Наиболее известные модели сетевого взаимодействия — OSI и TCP/IP.

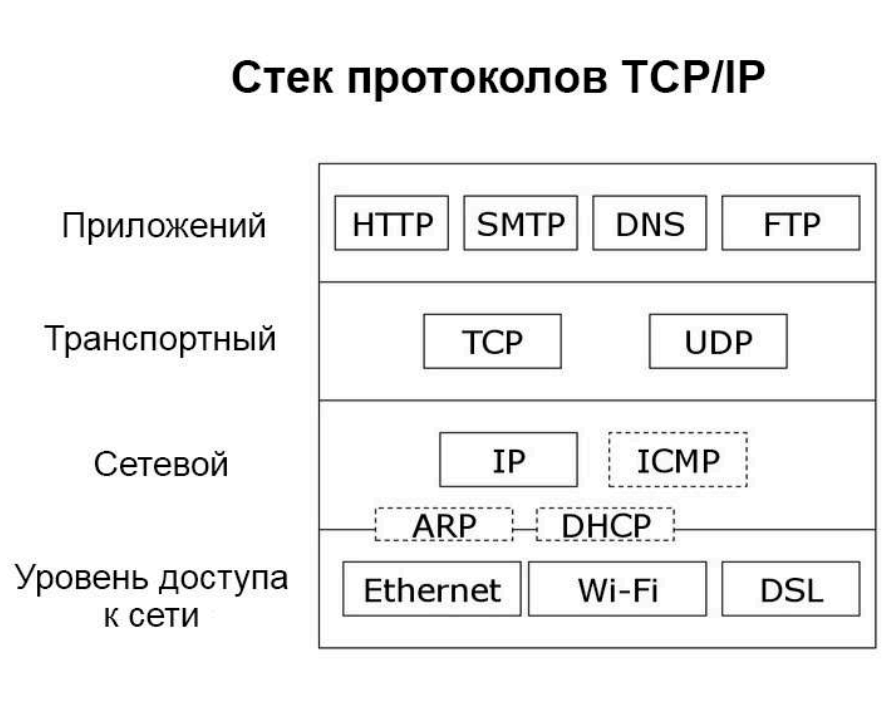


Эти модели распределяют сетевые протоколы по разным уровням взаимодействия. Что такое протокол?

Вообще **протокол** — это некоторый набор правил, определяющий принципы взаимодействия устройств в сети. В нашем случае это правила, по которым программа, получив по сети набор битов, понимает, как его прочитать и что он значит.

Для того чтобы обмен информацией между устройствами проходил успешно, все устройства (участники процесса) должны следовать условиям протокола. В сети поддержка протоколов встраивается или в аппаратную (в «железо»), или в программную часть (в код системы), или в обе этих части.

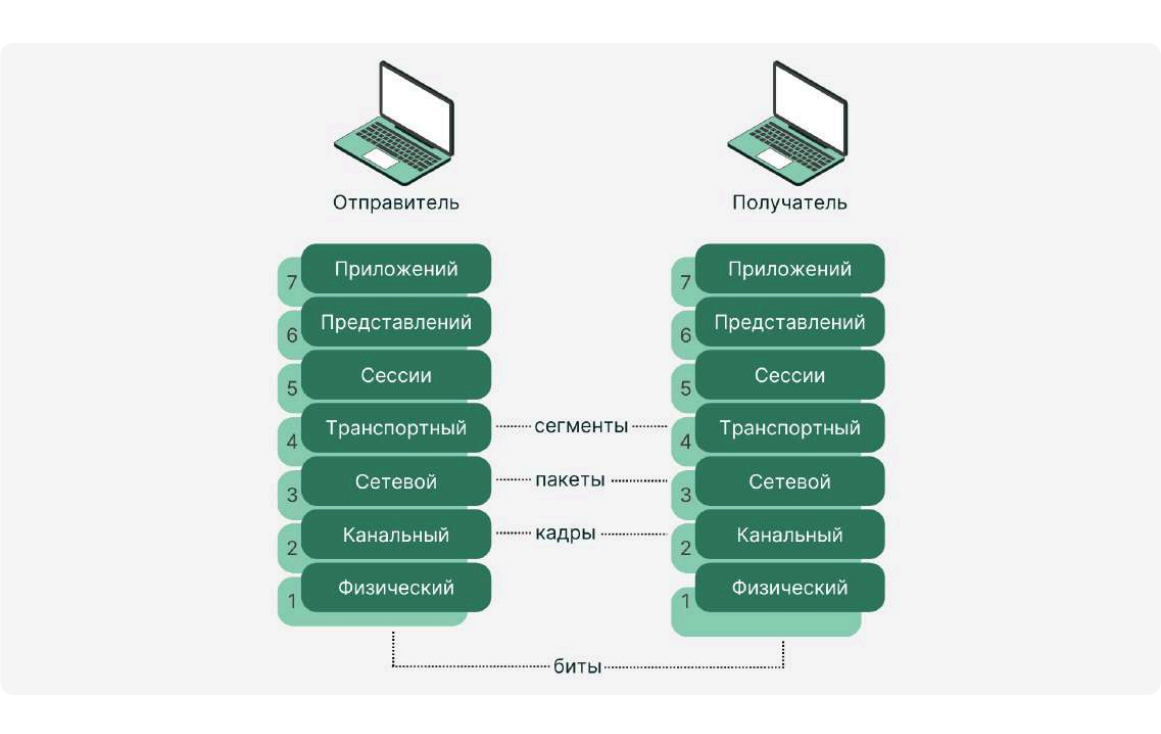
На схеме ниже представлены примеры протоколов, а также уровни их распределения в модели TCP/IP:



С другими протоколами и их назначением вы можете ознакомиться здесь.

В процессе сетевого взаимодействия участвуют как минимум два устройства — устройство-отправитель и устройство-получатель. Говоря простым языком, каждая из моделей сетевых взаимодействий устанавливает правила и регламенты по отправке сообщений между компьютерами.

Отправленное сообщение проходит все уровни, начиная от прикладного уровня приложений и заканчивая физическим уровнем доступа к сети. Когда сообщение доходит до адресата, оно также проходит все уровни в обратном порядке.



Если привести аналогию из жизни, можно сказать, что обе модели описывают правила, по которым мем, который вы отправляете другу в мессенджере, будет преобразован до битов данных, передаваемых по электрическим проводам, а затем будет восстановлен по этим же битам и отображён на экране устройства вашего друга.

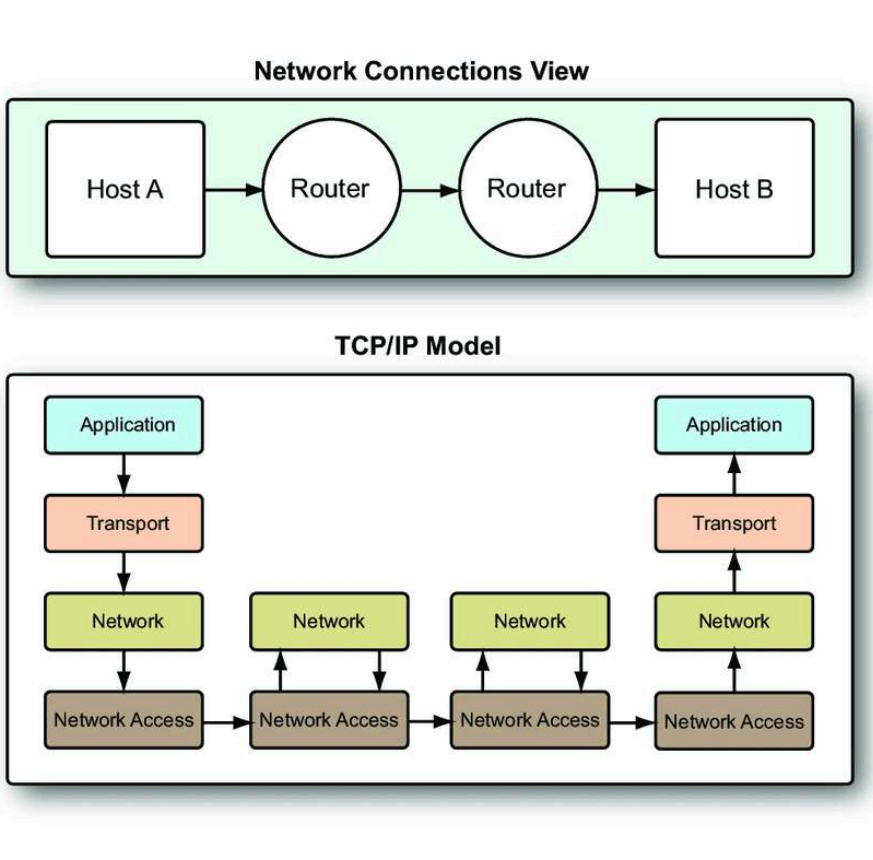
Нам как дата-сайентистам не нужно знать все подробности того, как работают и чем отличаются друг от друга разные уровни взаимодействия. Однако для общего развития и понимания того, как устройства обмениваются информацией, вы можете прочитать об уровнях моделей OSI и TCP/IP здесь и здесь.

Для наших целей (деплой модели в прод) достаточно уметь работать всего с тремя протоколами:

> **IP** — протокол сетевого уровня. Он определяет путь, по которому передаются данные.

> **TCP** — соответствует транспортному уровню, а значит, определяет, как передаются данные.

> **HTTP** — относится к прикладному уровню, описывающему взаимодействие приложений с сетью.



IP

IP (Internet Protocol) — один из главных протоколов сетевого взаимодействия. Он отвечает за маршрутизацию трафика по сети, то есть определяет путь, по которому отправятся данные. Данные передаются пакетами (или **датаграммами**), которые формирует протокол IP.

Важным свойством IP является отсутствие гарантированной доставки пакетов и их цельности: пакеты могут прийти в другой очерёдности (не в той, в которой их отправляли), прийти повреждёнными (тогда они уничтожаются) или вообще не прийти.

Путь, по которому отправятся данные, строится на основе IP-адресов.

IP-адрес — это уникальный адрес, используемый для связи устройств внутри сети.

IP-адрес устроен довольно просто: чаще всего это четыре числа, разделённых точками (такой формат поддерживается в протоколе IPv4). Например, вот один из самых популярных IP-адресов — 192.168.0.1. Вы могли вводить его, чтобы зайти в настройки своего роутера.

Каждое из чисел в адресе — это восьмизначное двоичное число, или, правильнее говорить, октет. Оно может принимать значения от 0000 0000 до 1111 1111 в двоичной системе или от 0 до 255 — в десятичной системе счисления, то есть 256 разных значений.

Получается, что диапазон IP-адресов стартует с 0.0.0.0 и заканчивается 255.255.255.255. Если посчитать количество всех адресов в этом диапазоне, получится чуть больше четырёх миллиардов.

Уникальность IP-адреса может быть глобальной (в рамках всего интернета) или локальной (в рамках локальной подсети). Некоторые IP-адреса не являются общедоступными и зарезервированы для специальных целей, например диапазоны IP-адресов:

127.0.0.1–127.255.255.255	используются для связи внутри локальной машины (localhost)
172.16.0.0–172.31.255.255	используются для частных подсетей (недоступных из интернета)
198.18.0.0–198.19.255.254	используются для тестирования производительности

> **localhost** — зарезервированное доменное имя для IP-адресов из диапазона 127.0.0.1–127.255.255.255 (в сети из одного компьютера — для 127.0.0.1).

В компьютерной сети localhost относится к компьютеру, на котором запущена программа. Компьютер работает как виртуальный сервер. Тем самым создаётся так называемая «внутренняя петля»: обращаясь по IP-адресу localhost, вы, по сути, заставляете компьютер общаться с самим собой (хотя на самом деле внутри всё немного сложнее). Это нужно, например, для разработки и тестирования клиент-серверных приложений на одной машине (то есть и клиент, и сервер находятся на одном компьютере), что позволяет при разработке не использовать сетевое оборудование, дополнительные программные модули и тому подобное.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Какие *IP*-адреса являются общедоступными в сети Интернет?
Отметьте все подходящие варианты ответов.

☐ A 127.0.0.1.
☐ B 172.16.21.7.
☑ C 8.8.8.8.
☐ D 18.301.43.2.
☑ E 83.0.4.1.

✓

Ответ
Верно:
C Верно. Данный адрес является доступным для всего интернета. Кстати, этот *IP*-адрес принадлежит *Google* и часто используется для тестирования сетевых приложений.
E Верно. Данный адрес верно сформирован и является доступным для интернета.

Show answer

Отправить

✓ Верных

TCP

TCP (Transmission Control Protocol) — протокол транспортного уровня. Он отвечает за управление передачей данных и гарантирует:

- доставку пакетов (посылает пакеты повторно, если они не были доставлены);
- последовательность и целостность доставки пакетов (используя нумерацию и контрольные суммы для проверки);
- устраняет дубликаты в случае необходимости.

Важной особенностью TCP является то, что перед отправкой данных он «устанавливает соединение» с получателем — обменивается управляющей информацией. После отправки пакетов источник ждёт подтверждения от получателя, что пакеты были доставлены.

Обычно на одном **узле сети** (сервере, компьютере) работают несколько приложений/процессов одновременно. Для идентификации приложения на источнике и получателе используется **порт**, который задаётся целым неотрицательным числом. Процесс или приложение могут зарезервировать у ОС определённый порт, например, для передачи данных по сети.

Порты разделяют на системные (0–1023), и пользовательские (1024–49151). Некоторые номера портов определены для конкретных приложений, например:

- 22 — протокол SSH для безопасной передачи данных;
- 25 — протокол SMTP для незащищённой передачи e-mail-сообщений;
- 80 — протокол HTTP.

Например, если приложение доступно по адресу 172.16.0.11:8001, это значит следующее:

- 172.16.0.11 — IP-адрес;
- 8001 — TCP-порт, отведённый приложению.

Задание 2

0 points possible (ungraded)

Выберите верные утверждения:

☑ A Модели сетевого взаимодействия *OSI* и *TCP/IP* распределяют одни и те же протоколы по разным уровням.
☐ B*localhost* является адресом вашего компьютера в интернете.
☐ C Протокол *IP* гарантирует доставку пакетов данных от источника к получателю.
☑ D Идентификацию приложения на сервере производят с помощью *TCP*-порта.

✓

Ответ
Верно:
A Верно. Модели *OSI* и *TCP/IP* представляют собой одни и те же протоколы, объединённые в разные уровни.
DВерно. *TCP*-порт является составной частью адреса приложения.

Show answer

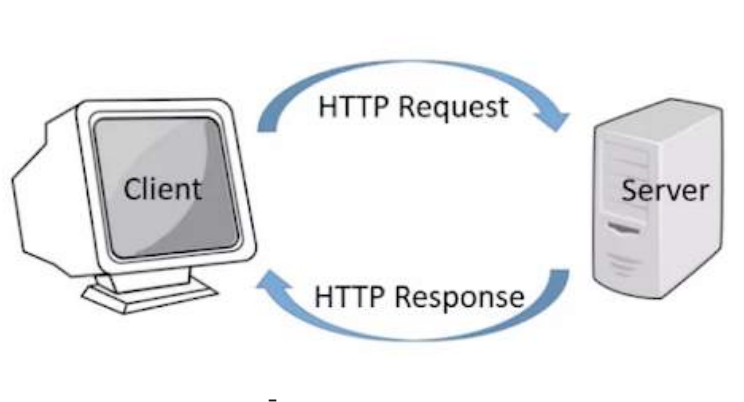
Отправить

✓ Верных

HTTP

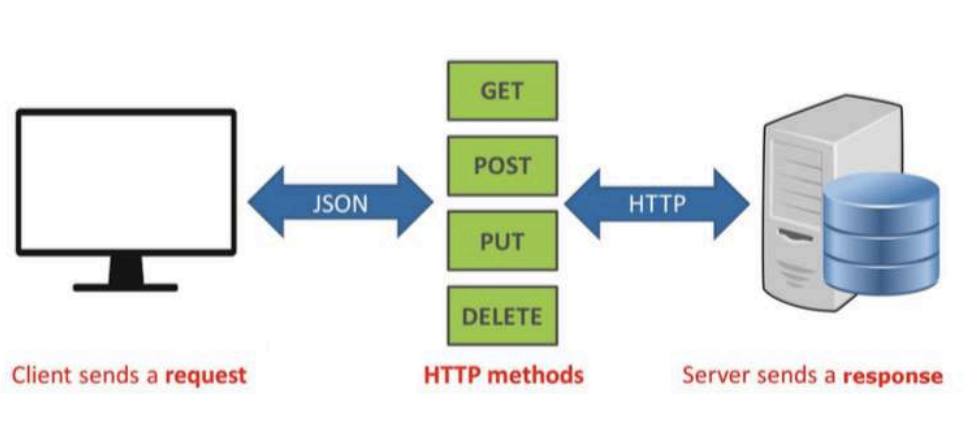
HTTP — это наиболее широко используемый протокол. Все сайты, на которые вы заходите, работают по этому протоколу. Он был разработан именно для передачи содержимого HTML-страниц в интернете, но впоследствии стал использоваться и для других целей. Например, HTTP применяется для налаживания взаимодействия между сервисами в сложных системах. Этим он нам и интересен.

Итак, HTTP — это протокол, который работает по принципу **клиент-сервер**.



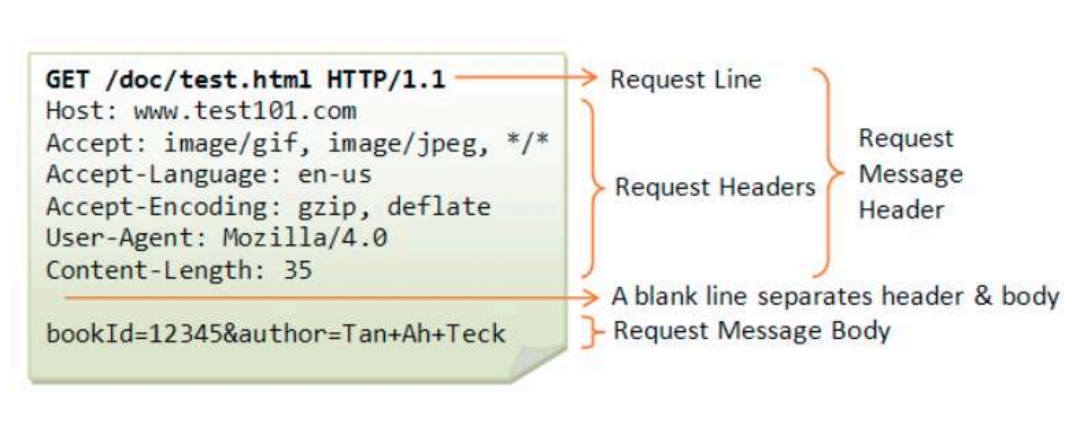
Это означает, что во взаимодействии участвует **две программы**, причём в разных ролях. Одна из них — **клиент**, или «заказчик услуг», формирует запрос и отправляет его к серверу. **Сервер**, или «поставщик услуг», получив запрос, обрабатывает его, формирует ответ и возвращает его клиенту.

СТРУКТУРА HTTP-ЗАПРОСОВ



Запрос и ответ в HTTP являются строками, составленными в соответствии с протоколом.

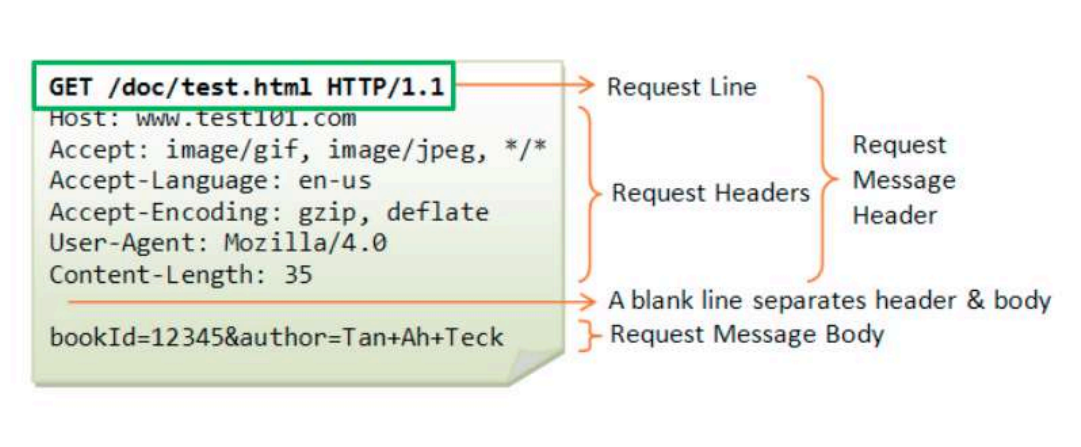
Запрос состоит из трёх частей.



```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Content-Length: 35

bookId=12345&author=Tan+Ah+Teck
```

1. **Стартовая строка**, или Request Line — по ней определяется вид запроса.
2. **Заголовки запроса**, или Request Headers — дополнительные параметры запроса, в которых обычно передаётся служебная информация, например, в каком формате ожидается ответ или информация о клиенте.
3. **Тело запроса**, или Request Message Body — содержит данные для передачи. Эта часть присутствует не всегда.

Давайте чуть подробнее разберём первую часть.



```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Content-Length: 35

bookId=12345&author=Tan+Ah+Teck
```

Первое, что указано в стартовой строке — это **метод**, или тип запроса. Есть набор стандартных методов, но теоретически вы можете создавать и свои.

Основные методы:

- GET — обычно означает получение содержимого ресурса и не содержит тела.

  Например, когда вы заходите в каталог интернет-магазина, вы получаете страницу с товарами — ваш браузер отправляет GET-запрос на сервер интернет-магазина.

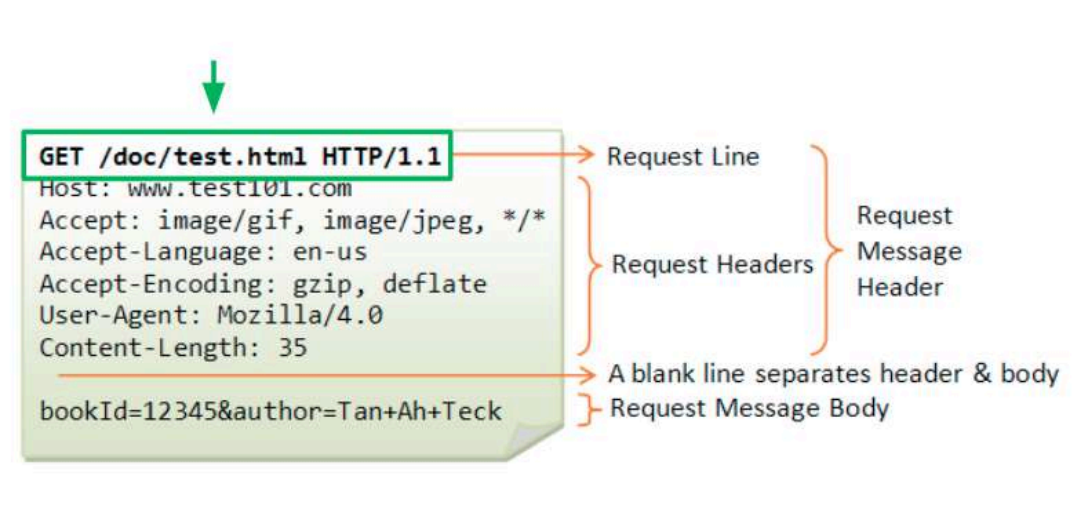
- POST — наоборот, передача данных ресурсу.

  Например, когда вы заполняете форму авторизации на любом сайте и нажимаете кнопку для отправки своих данных, вы совершаете POST-запрос на сервер ресурса.

- PUT — обновление ресурса.
- DELETE — удаление ресурса.

В HTTP мы работаем с ресурсами, которые расположены по некоторому адресу на сервере. Изначально под ресурсами понимались HTML-файлы на сайте (вёрстка сайта), но сейчас это уже некоторое абстрактное понятие.

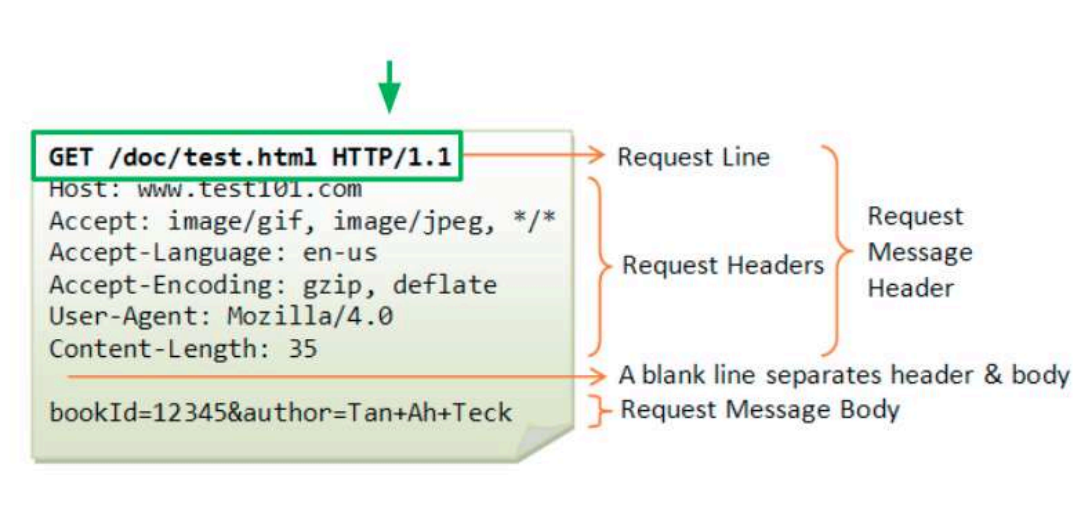
Адрес ресурса, или URI (Uniform Resource Identifier) — это то, что вы видите в адресной строке браузера. Он следует за методом в стартовой строке запроса.



```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Content-Length: 35

bookId=12345&author=Tan+Ah+Teck
```

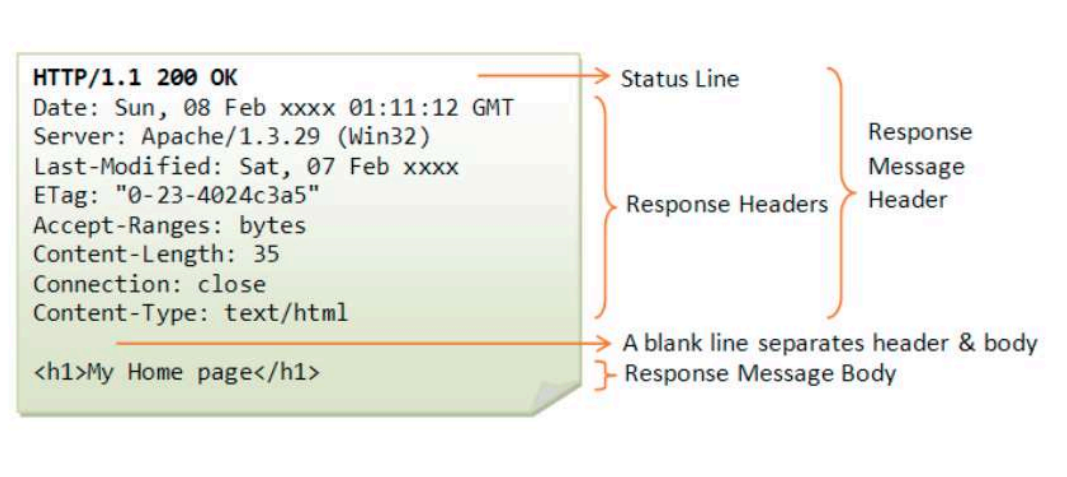
Чем отличаются URI, URL и URN?

HTTP/1.1 долгое время был основным стандартом; сегодня широко используются HTTP/2 и HTTP/3.



```
GET /doc/test.html HTTP/1.1
Host: www.test101.com
Accept: image/gif, image/jpeg, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0
Content-Length: 35

bookId=12345&author=Tan+Ah+Teck
```

Ответ также состоит из стартовой строки, заголовков и тела.



```
HTTP/1.1 200 OK
Date: Sun, 08 Feb xxxx 01:11:12 GMT
Server: Apache/1.3.29 (Win32)
Last-Modified: Sat, 07 Feb xxxx
ETag: "0-23-4024c3a5"
Accept-Ranges: bytes
Content-Length: 35
Connection: close
Content-Type: text/html

<h1>My Home page</h1>
```

Основное отличие — в стартовой строке: там вместо метода и URI указывается код состояния. Это численное значение, которое показывает результат обработки. Коды задаются протоколом, и вы наверняка встречали «ошибку 404» на сайтах.

404 — это как раз код состояния, означающий, что ресурса с заданным URI не существует на сервере.

Группы кодов состояния ответа HTTP-сервера делятся на следующие группы:

- информационные (100–199);
- успешно (200–299);
- перенаправление (300–399);
- ошибка клиента (400–499);
- ошибка сервера (500–599).

Примеры:

Код 200 означает, что запрос обработан успешно.

Код 500 означает, что сервер не смог обработать запрос.

Задание 3

0 points possible (ungraded)

Сопоставьте код состояния с его значением.

Не отвечает.

504 ✓

Слишком много запросов.

429 ✓

Сервис недоступен.

503 ✓

Сервер не смог обработать запрос.

500 ✓

Запрос обработан успешно.

200 ✓

Ресурса с заданным *URI* не существует на сервере.

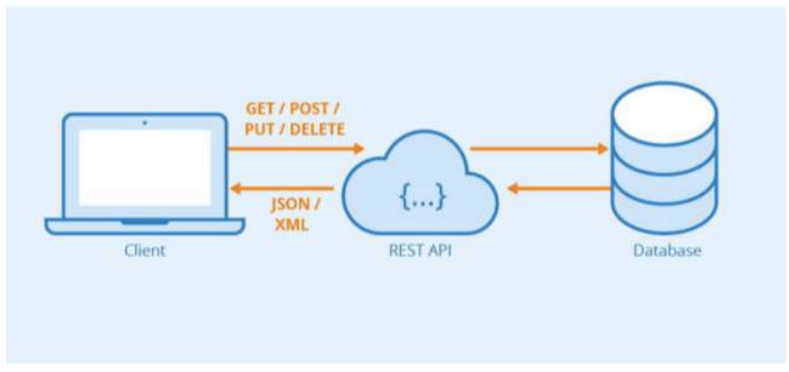
404 ✓

Show answer

Отправить

✓ Верных

REST (Representational State Transfer)



Спецификация HTTP не обязывает сервер понимать все методы, а также не указывает серверу, что он должен делать при получении запроса с тем или иным методом. Поэтому был изобретён архитектурный стиль REST.

Он даёт более верхнеуровневые указания, чем HTTP-протокол, а именно:

- как правильно организовывать адресацию к ресурсам;
- какие методы у этих ресурсов должны быть;
- какой ожидается результат.

Основная концепция философии REST заключается в том, что клиентом RESTful-сервера может быть что/кто угодно: браузер, другое приложение, разработчик. Веб-приложение, спроектированное по правилам REST, предоставляет информацию о себе в форме информации о своих ресурсах.

Ресурс может быть любым объектом, о котором сервер предоставляет информацию. Например, в API Instagram ресурсом может быть пользователь, фотография, хэштег. Каждая единица информации (ресурс) однозначно определяется URL.

- GET-запрос `/rest/users` — получение информации обо всех пользователях.
- GET-запрос `/rest/users/125` — получение информации о пользователе с id=125.
- POST-запрос `/rest/users` — добавление нового пользователя.
- PUT-запрос `/rest/users/125` — изменение информации о пользователе с id=125.
- DELETE-запрос `/rest/users/125` — удаление пользователя с id=125.

Поскольку спецификация REST является общепризнанной и широко распространённой, то следовать ей очень полезно.

Если вы говорите, что ваш сервис RESTful (то есть построен по этой спецификации), то любой разработчик сразу поймёт, как примерно устроен сервис, а значит, сможет быстро начать им пользоваться: отправлять правильные запросы и получать тот результат, который ожидает.

Промежуточные итоги

Для того чтобы наладить взаимодействие по HTTP, на стороне **клиента** вам необходимо сформировать запрос: указать адрес, куда отправится запрос, выбрать метод, а также задать заголовки и тело запроса, если это нужно для выбранного метода.

На стороне **сервера** при получении запроса необходимо знать, какой ресурс запрошен, чтобы понять, каким кодом его обрабатывать. Этот код, в зависимости от того, что он должен делать, может проверить значения заголовков и/или прочитать тело запроса.

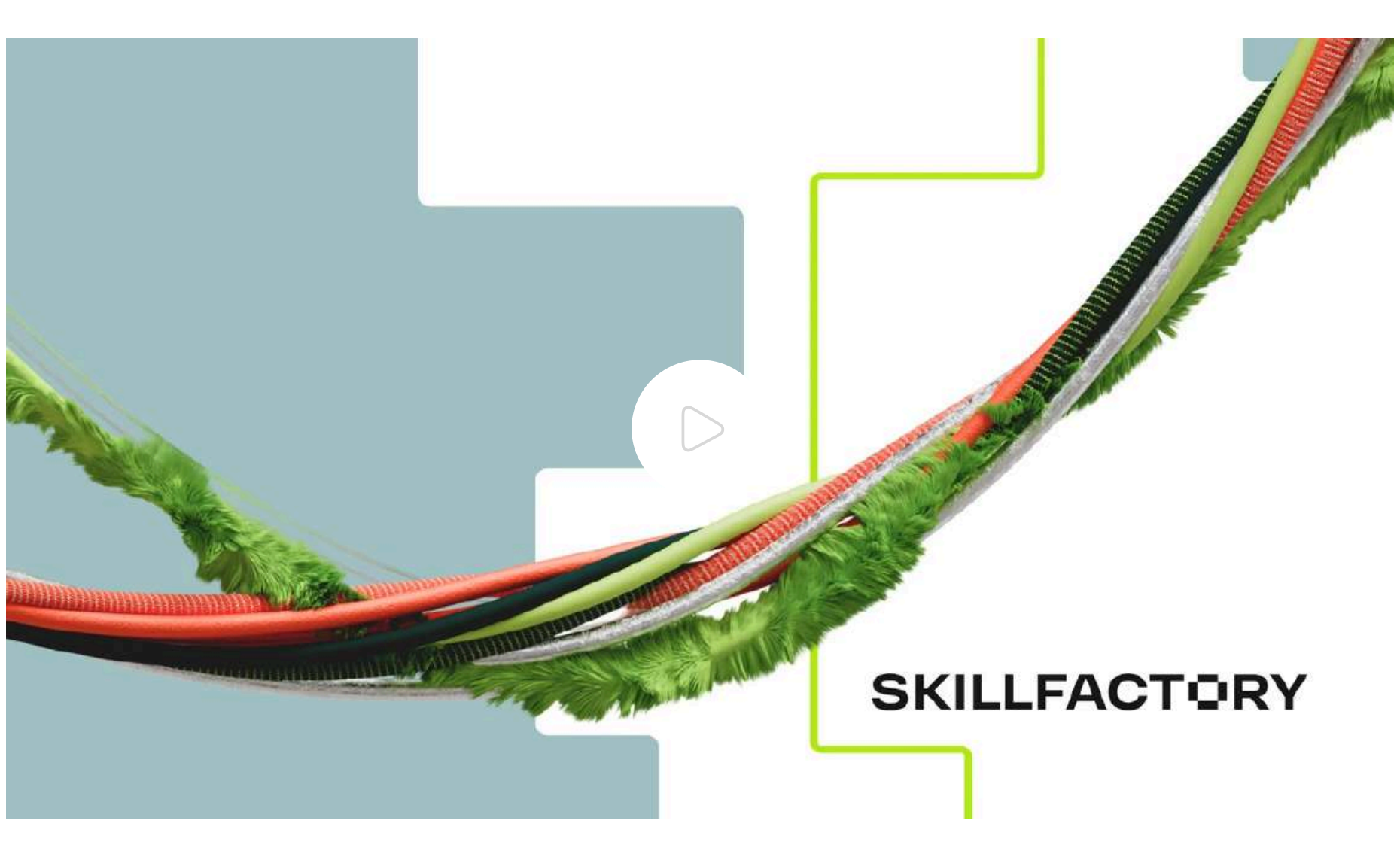
Отлично, со взаимодействием сервисов по сети мы разобрались. Эти теоретические знания обязательно пригодятся вам для построения собственного веб-сервиса. Теперь пора приступать к рассмотрению

‹ Предыдущий урок Следующий урок ›

6/11 Деплой модели. Обзор фреймворков

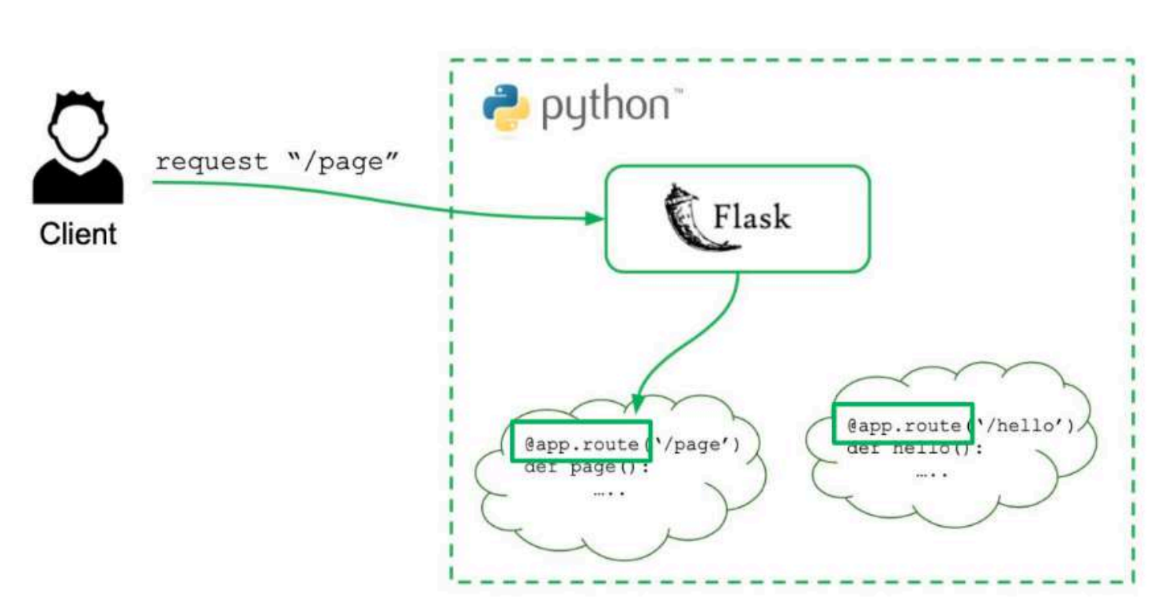
текущий урок

Следующий шаг в подготовке к деплою — рассмотреть, какие существуют веб-фреймворки Python, которые позволяют реализовать серверную часть приложения и обеспечить к нему клиентский доступ.



FLASK

Flask позиционируется как микрофреймворк для написания **легковесных** сервисов, но при этом он достаточно гибкий и кастомизируемый, что позволяет использовать его в проектах любой сложности. Как и все остальные фреймворки, он работает на основе **маршрутизации запросов**, приходящих на сервер, который и должен обработать эти запросы.



В случае с Flask маршрутизация выглядит как обычные функции Python, которые вы помечаете специальным **декоратором** `@app.route`. Он связывает эту функцию с адресом, на который приходит запрос. В этой функции вы должны разобрать **параметры запроса**, возможно, выполнить какую-то логику и вернуть ответ клиенту.

Таким образом, для написания минимально функционального сервера достаточно добавить в скрипт всего несколько строк: необходимые импорты, создание Flask-приложения, декоратор, который свяжет адрес (endpoint, **эндпоинт**) с функцией, и запуск приложения. Последнее как раз и запустит работу Flask: он будет слушать запросы, маршрутизировать их и возвращать ответ.

Что такое эндпоинт?

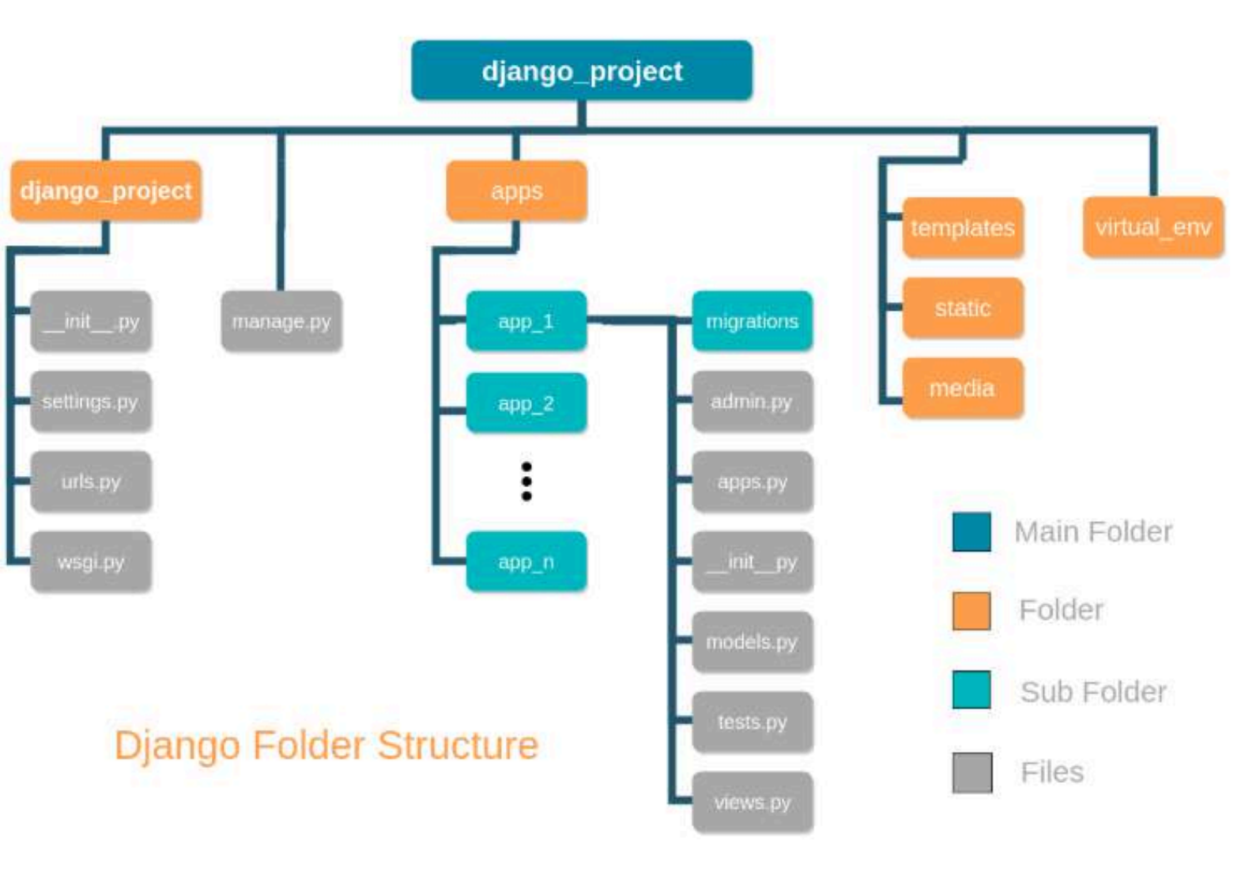
Рассмотрим на примере. Сайт почты Mail.ru имеет адрес https://e.mail.ru. Для того чтобы попасть в папку со входящими письмами, необходимо обратиться к соответствующему интерфейсу приложения. Для этого к адресу сайта добавляется `/inbox/`: https://e.mail.ru/inbox. Интерфейс, который предоставляет доступ к вашим отправленным письмам, находится по адресу https://e.mail.ru/sent/. Получается, что `/inbox/` и `/sent/` — это и есть эндпоинты.

У Flask есть множество продвинутых функций и плагинов, написанных сторонними разработчиками для решения самых разных задач. Однако нам достаточно научиться работать с Flask на минимальном уровне, чтобы вывести свою модель как отдельный маленький сервис (микросервис), решающий одну конкретную задачу — к сервису можно обращаться по сети и получать результаты (предсказания модели).

Реализация глобальной структуры сервиса (веб-приложения) и встраивание этой структуры в микросервис — это уже задача других специалистов, например веб-разработчиков.

DJANGO

В отличие от Flask, Django нельзя назвать микрофреймворком. Напротив, даже минимальный проект для Django обычно генерируется **специальными скриптами** и включает с десяток файлов и папок.



Такая сложная структура нужна для громоздких приложений, чтобы не путаться в коде. Поэтому Django используют, когда нужно что-то **посерьёзней**, например целые сайты с базами данных.

ПРЕИМУЩЕСТВА DJANGO:

- Мощный движок для рендера HTML-страниц, основанный на шаблонах.
- Собственный ORM.
- Огромное количество дополнительных плагинов. Один из них практически так же популярен, как и сам Django — речь о Django Rest Framework. Основываясь на ORM, он позволяет в несколько строк реализовывать эндпоинты протокола RESTful. Сам Django Rest Framework также имеет множество расширений. Благодаря этому написание сложного проекта можно свести в составлению правильной модели данных, установке и настройке подходящих расширений.

FLASK vs DJANGO

Давайте более подробно рассмотрим характеристики Flask и Django и узнаем, чем отличаются и чем похожи эти два фреймворка.

В Django есть модули для очень широкой функциональности. Чтобы получить к ним доступ, надо просто добавить нужную настройку в конфигурацию. Это отлично перекликается с девизом Django: “The web framework for perfectionists with deadlines” («Веб-фреймворк для перфекционистов, которые придерживаются сроков»).

Во Flask всё наоборот: сам фреймворк предоставляет только базовую функциональность. «Из коробки» будет доступно направление по адресам эндпойнтов, обработка запросов и ошибок, дебаггер и некоторые другие функции, явно связанные с сервером. Например, чтобы добавить авторизацию, придётся устанавливать **стороннюю библиотеку**. При этом вы можете выбрать ту, которая лучше подходит текущему проекту, лучше поддерживается или просто более удобна в использовании, а можете вообще не подключать библиотеки.

Flask нужен для оборачивания **инференса модели в API**, когда, например, по поступившему запросу запускается предикт модели. Знание Flask пригодится и на этапе создания прототипа, и на этапе тестирования.

Также некоторые компании строят весь продакшен на Flask, а значит, знать его основы будет полезно.

В связи с тем, что Django выпускается в релиз вместе со всеми сопутствующими модулями, у него достаточно длинные релиз-циклы. С Flask эта проблема менее актуальна: если какой-то функциональности не хватает, часто она есть в какой-то библиотеке. Этот подход жизнеспособен, потому что у Flask большое сообщество разработчиков, которые занимаются поддержкой библиотек для этого фреймворка.

Flask приобрёл популярность именно благодаря небольшим приложениям, для которых Django казался избыточным. Однако сегодня Flask используется не только для таких приложений.

Для справки приведём инструментарий, используемый в Django и Flask, чтобы продемонстировать, как фреймворки отличаются друг от друга по функциональности. Запоминать таблицу не нужно.

Примечание. При разработке на Flask список чаще всего не ограничивается перечисленными в таблице библиотеками — здесь собраны только наиболее популярные и известные. Также есть библиотеки, у которых есть специальные расширения для Flask. Чаще всего они не обязательны для работы с библиотекой, но могут быть полезны.

Функциональность	Django	Flask	Flask extension
Шаблоны	Django templates	Jinja2	
ORM	Django ORM	sqlalchemy	Flask-sqlalchemy
Миграции	Django ORM	Alembic	Flask-alembic
Работа с MongoDB	Django ORM	PyMongo, MongoEngine	Flask-PyMongo, Flask-MongoEngine
Авторизация, аутентификация	Django Contrib Auth	Flask-Login, Flask-Principal	
Админка	Django Admin	Flask-Admin	
Кэширование	Django Cache Framework	Flask-Caching	
REST	Django REST	Flask-RESTful	
Сериализация	Django REST	Marshmallow	Flask-marshmallow

КТО ПОБЕДИЛ?

Как мы видим, для основной функциональности, реализованной в Django, есть аналоги, которые работают с Flask. Нельзя сказать, что какой-то из фреймворков проигрывает другому по большому числу параметров.

Однако стоит понимать, что, выбирая Django, вы оставляете проект со всей экосистемой Django, и что-то изменить впоследствии будет трудно. Выбирая Flask, вы получаете систему, построенную из блоков, каждый из которых проще заменить.

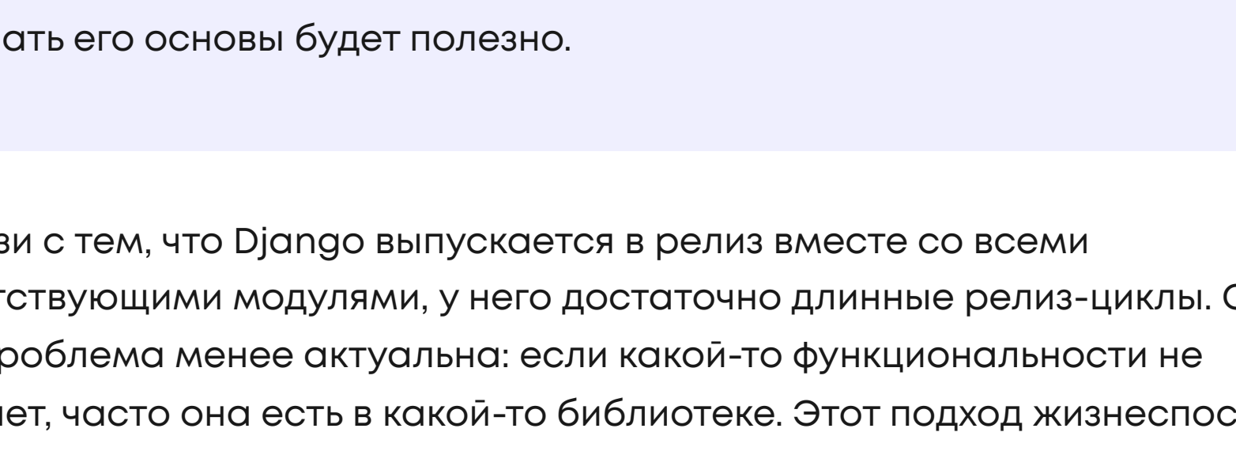
FASTAPI

Это относительно молодой фреймворк, однако он уже успел завоевать некоторую популярность. Возможно, через некоторое время он превратится в стандарт индустрии, заменив Flask.

Примечание. Да-да, именно Flask! FastAPI тоже позиционируется как легковесный фреймворк. Более того, принцип его использования очень похож на Flask. Авторы утверждают, что, вдохновившись Flask, они написали **более быстрый** (за счёт использования более современных протоколов), но при этом не уступающий по функциональности фреймворк.

Основное нововведение в FastAPI — интеграция библиотеки pydantic, которая позволяет `декларативно` описывать структуры запросов.

Также в FastAPI есть **поддержка асинхронных функций** и реализация парадигмы Dependency Injection.



```
from datetime import datetime
from typing import List, Optional
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name = 'John Doe'
    signup_ts: Optional[datetime] = None
    friends: List[int] = []
```

Что такое асинхронные функции?

ДРУГИЕ ФРЕЙМВОРКИ

Фреймворк aiohttp тоже близок к Flask, но основан на асинхронном взаимодействии. Aiohttp — сторонний асинхронный веб-фреймворк, появившийся в эпоху внедрения asyncio (Python 3.4).

Tornado — относительно старый и более общий асинхронный фреймворк для написания сетевых приложений, но также он умеет работать с HTTP. По заверениям разработчиков, он очень хорошо держит нагрузку.

В этом уроке мы рассмотрели основные фреймворки для реализации веб-сервисов на Python. Далее в курсе мы будем использовать фреймворк Flask, однако если вы уже знакомы с другими фреймворками, вы можете использовать их.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Подумайте, какой(-ие) из фреймворков лучше использовать в следующих случаях.
Верными могут быть как один, так и несколько вариантов.

Кейс 1. Микросервис для запросов к модели машинного обучения.

☑ A *Flask*
☐ B *Django*
☑ C *FastAPI*

✓

Ответ
Верно:
A Верно.
C Верно.

Кейс 2. Серверная часть приложения (бэкенд) для интернет-магазина с каталогом товаров и обширной базой клиентов.

☐ A *Flask*
☑ B *Django*
☐ C *FastAPI*

✓

Ответ
Верно:
B Верно.

Show answer

Отправить

✓ Верных

‹ Предыдущий урок   Следующий урок ›

7/11 Пишем сервер на Flask

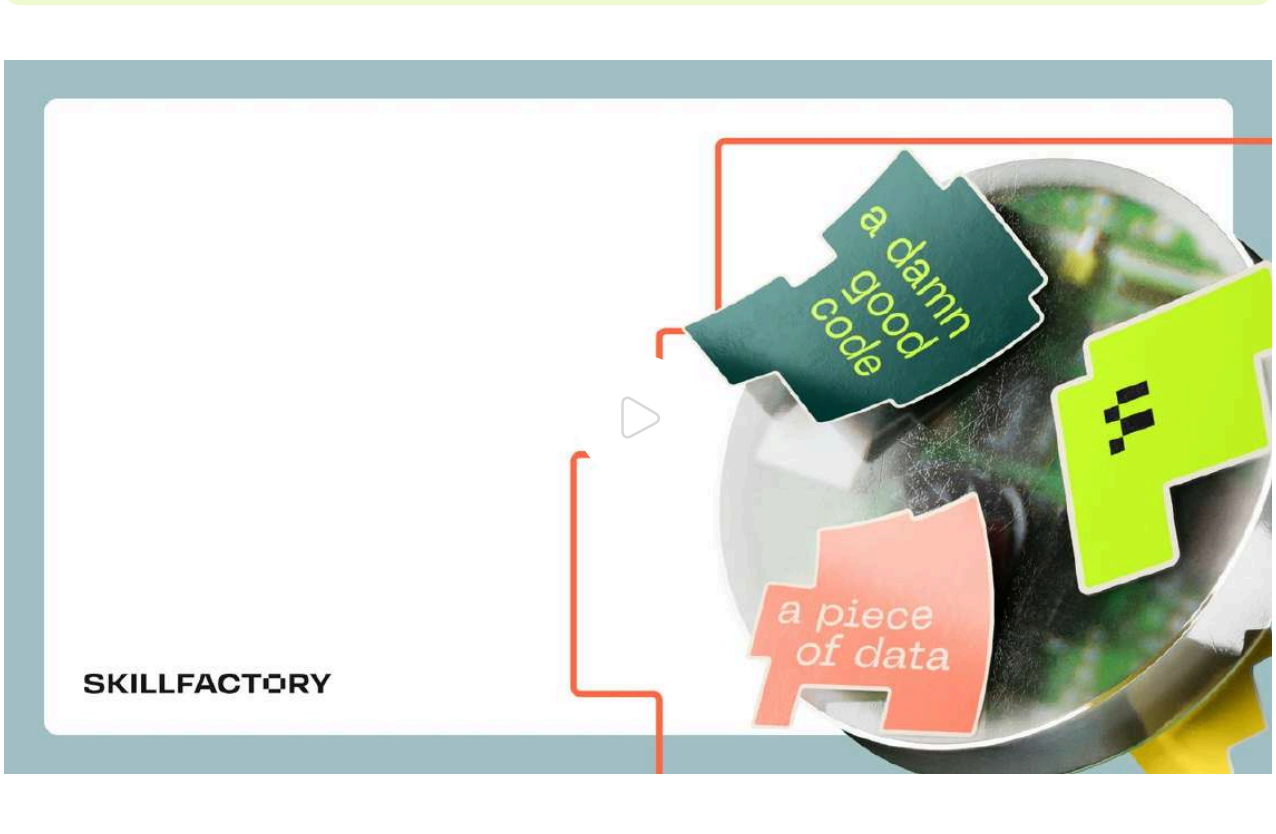
текущий урок

Если бы вы попробовали написать сервер в Jupyter Notebook, то почти сразу поняли бы, что он для этого не предназначен. Как минимум потому, что после запуска вы больше не смогли бы выполнить в ноутбуке ни одной команды — программа сервера переходит в режим прослушки сети и ожидания приходящих запросов. Поэтому разработку нашего веб-сервиса мы будем вести в обычных файлах .py.

Итак, давайте напишем простое приложение на Flask.

ЧАСТЬ I. ИНИЦИАЛИЗАЦИЯ ВЕБ-ПРИЛОЖЕНИЯ

Примечание. В видео эксперт использует IDE PyCharm. Вы можете установить её самостоятельно по инструкции либо использовать ту IDE, к которой вы привыкли, например VS Code.



Сначала необходимо установить Flask. Сделать это можно с помощью pip:

```
pip install flask
```

Теперь импортируем его и создадим объект Flask-приложения.

```
from flask import Flask
app = Flask(__name__)
```

Мы передаём `__name__` при инициализации класса `Flask`, чтобы определить имя, с которым будет использоваться этот модуль. Flask использует расположение файла как точку, к которой он привязывает ресурсы.

Совет. В абсолютном большинстве случаев можно передавать аргумент `__name__` для инициализации класса — приложение будет настроено верно.

Теперь мы можем написать функцию, которая будет обрабатывать запросы, и прикрепить её к какому-то пути (URI). Это делается с помощью специального декоратора `route`.

```
@app.route('/hello')
def hello_func():
    return 'hello!'
```

Наша функция пока не делает никакой обработки и просто отвечает строкой с приветствием. Нам осталось запустить приложение. Для этого выполним метод `run`, не забыв занести его в стандартный `main`.

```
if __name__ == '__main__':

    app.run('localhost', 5000)
```

Примечание. В блоке `if __name__ == '__main__'` прописывается код, который не должен выполняться при импорте модуля. Переменная `__name__` — это специальная переменная, которая будет равна `"__main__"`, только если файл запускается как основная программа, и выставляется равной имени модуля при импорте модуля.

Например, если мы захотим импортировать файл server.py как внешний модуль,

```
from server import *
```

то код, указанный в блоке `if __name__ == '__main__'`, соответствующий запуску сервера, не будет выполнен.

Мы запускаем наш сервис в строке `app.run('localhost', 5000)`, указывая адрес сетевого интерфейса и порт, на котором будет работать сервер. В нашем случае мы работаем на локальной машине по IP-адресу 127.0.0.1, или localhost (то есть доступ к сервису может быть получен только с нашего компьютера), а номер порта, по которому можно отправлять запросы, — 5000.

Если вы работаете в Jupyter Notebook, сохраните весь код в файле server.py (не в ноутбуке!) и запустите его из командной строки, выполнив команду `python server.py`.

В терминале появится сообщение о запуске development-сервера и адрес `http://localhost:5000/`

После запуска программа перейдёт в режим прослушивания сети.

Теперь откройте браузер и зайдите по адресу http://localhost:5000/hello.

Вы увидите текст `hello!`, а в логах — строку с кодом обработки вашего запроса.

Примечание. Чтобы прервать работу программы-сервера в VS Code, нажмите сочетание клавиш CTRL + C, находясь в поле терминала. После этого перезапустите скрипт.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Какой код обработки вернулся в результате выполнения *GET*-запроса на наш сервис?

Отправить

Задание 2

0 points possible (ungraded)

Попробуйте обратиться к веб-сервису по адресу `http://localhost:5000/source`. В поле для ответа введите код обработки, который вернулся в результате выполнения *GET*-запроса по указанному адресу.

Отправить

Задание 3

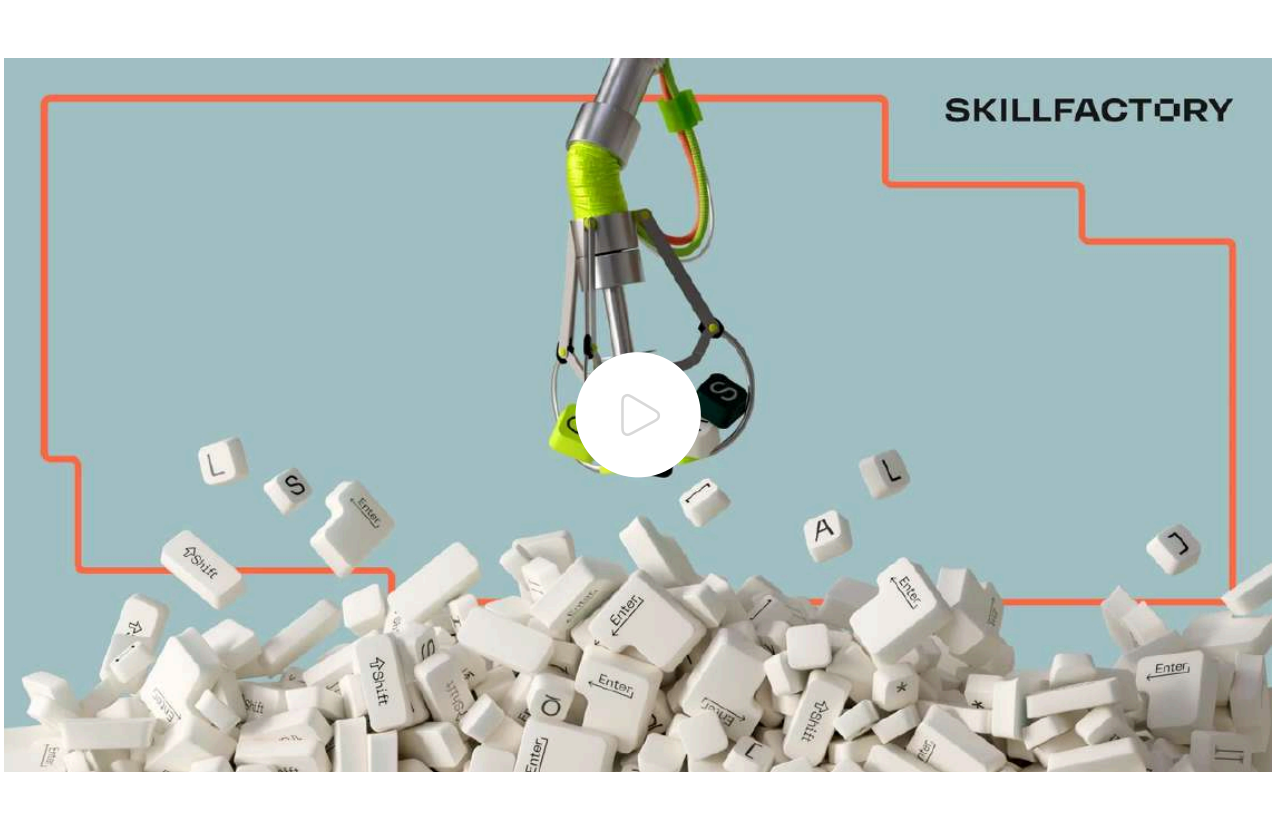
0 points possible (ungraded)

Напишите новую функцию `index()`, которая будет возвращать строку `"Test message. The server is running"`. Оберните эту функцию в декоратор `app.route()`, указав в качестве эндпоинта `'/'`. Данный эндпоинт будет соответствовать обращению к сайту по дефолтному адресу: `http://localhost:5000/`.

Перезапустите веб-сервис и зайдите по этому адресу. В поле для ответа введите код обработки, который вернулся в результате выполнения *GET*-запроса.

Отправить

ЧАСТЬ II. ПЕРЕДАЧА ПАРАМЕТРОВ ЗАПРОСА



Давайте немного усложним логику обработки — создадим функцию, которая принимает какой-то параметр и использует его для вычисления результата.

Параметры можно передавать тремя способами:

- через адресную строку;
- через заголовки;
- через тело.

В заголовках обычно передают сервисные параметры. Так как у метода GET не бывает тела, остаются только **параметры адресной строки**.

Вы наверняка видели параметры на сайтах, когда в конец адреса дописывается что-то вроде `?id=10&page=2`. Если поставить в конце адреса вопросительный знак, то после него можно добавлять параметры. Передавать можно любые параметры — они будут перечисляться через `&` и состоять из имени и значения (`id=10`). Например, если вы будете что-то искать в Google, ваш запрос будет видно в адресной строке в параметре с названием `q` (от англ. query).

Параметры запроса во Flask находятся в специальном объекте `request`, который нужно импортировать. Параметры адресной строки можно найти в поле `args` этого объекта, где `args` — это словарь.

Давайте немного модифицируем наш код — теперь сервер будет здороваться и обращаться по имени. Для этого занесём параметр `name`, полученный из `request`, в переменную и воспользуемся этим значением, поставив его в форматированную строку.

```
from flask import Flask, request

app = Flask(__name__)
@app.route('/hello')

def hello_func():
    name = request.args.get('name')
    return f'hello {name}!'

if __name__ == '__main__':

    app.run('localhost', 5000)
```

Перезапустите сервер и зайдите по адресу http://localhost:5000/hello?name=world.

В результате выполнения запроса вы должны увидеть в браузере текст "hello world".

Поменяйте значение параметра `name` в адресной строке на своё имя и выполните новый запрос к серверу, чтобы проверить, что при изменении параметра адресной строки меняется и возвращаемый результат.

Задание 4

0 points possible (ungraded)

Напишите функцию `current_time`, которая будет возвращать словарь, где ключ — `'time'`, а значение — текущее время.

Примечание. Текущее время можно узнать с помощью модуля `datetime`:

```
import datetime
now = datetime.datetime.now()
```

Оберните эту функцию в декоратор `app.route()` с эндпоинтом `/time`.

Попробуйте обратиться к сервису по заданному эндпоинту.

По какой ссылке вы переходили, чтобы увидеть, правильно ли работает ваш скрипт? В поле ниже вставьте ссылку.

Введите в поле ниже код обработки запроса из логов.

Отправить

ЧАСТЬ III. POST-ЗАПРОСЫ

SKILLFACTORY

Пока что мы написали обработчик GET-запроса.

Если мы хотим обрабатывать и другие методы, например POST-запросы, это необходимо указать в декораторе:

```
@app.route('/add', methods=['POST'])
```

Что должна возвращать функция-обработчик? Она будет возвращать объект `Response`, в котором мы должны были бы задать все компоненты ответа: код обработки, заголовки и тело. К счастью, Flask (а точнее, объект `Response`) умеет и превращать в ответы другие объекты, и самостоятельно формировать ответ.

Например, строка

```
return f'hello {name}!'
```

преобразуется в ответ с кодом `200` и телом, состоящим из этой строки:

```
return f'hello {name}!', 200
```

Также можно возвращать кортеж из двух элементов, строки и числа. Число используется как код обработки.

Примечание. Вспомогательная функция jsonify поможет преобразовать обычный питоновский словарь в ответ в формате JSON, который очень часто используется для передачи структурированных данных.

Вооружившись этими знаниями, напишем обработчик POST-запроса, который будет читать тело запроса в JSON-формате и составлять ответ на основе его содержимого.

```
from flask import Flask, request, jsonify

app = Flask(__name__)
```

Укажем, что функция обрабатывает метод POST:

```
@app.route('/add', methods=['POST'])
```

Напишем саму функцию:

```
def add():

    num = request.json.get('num')
```

Параметры тела доступны в поле `data`. Но если тело — это JSON-строка, то можно использовать поле json.

Напишем проверку и укажем код ошибки:

```
    if num > 10:
        return 'too much', 400

    return jsonify({
        'result': num + 1

    })
```

Тогда функция-обработчик будет иметь следующий вид:

```
@app.route('/add', methods=['POST'])
def add():
    num = request.json.get('num')
    if num > 10:
        return 'too much', 400
    return jsonify({
        'result': num + 1
    })
```

Запускаем:

```
if __name__ == '__main__':

    app.run('localhost', 5000)
```

Обычный переход по ссылке в браузере выполняет GET-запрос. Для отправки POST-запросов удобно использовать Postman или клиентский скрипт на requests.

Если вы любите визуальные редакторы запросов, рекомендуем обратить внимание на Postman.

Напишем простой сервис в **соседнем скрипте**, используя библиотеку requests, которая позволяет отправлять HTTP-запросы. Для этого создадим отдельный файл client.py, в котором опишем работу программы клиента.

Чтобы выполнить POST-запрос, нужно просто вызвать соответствующую функцию и передать ей адрес (URL) и содержимое тела запроса.

```
import requests

if __name__ == '__main__':

    r = requests.post('http://localhost:5000/add', json={'num': 5})
```

Далее выведем статус-код ответа:

```
print(r.status_code)
```

Теперь пропишем проверку:

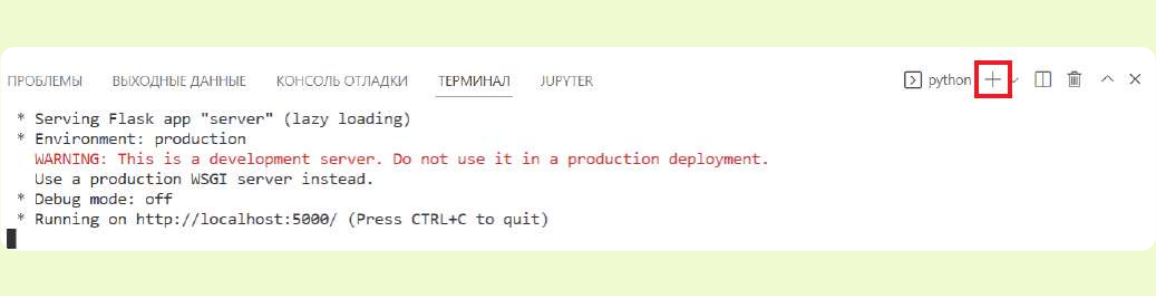
```
if r.status_code == 200:

   print(r.json()['result'])
else:

   print(r.text)
```

Полный код нашего простейшего клиентского приложения будет выглядеть так:

```
import requests

if __name__ == '__main__':
    # выполняем POST-запрос на сервер по эндпоинту add с параметром json
    r = requests.post('http://localhost:5000/add', json={'num': 5})
    # выводим статус запроса
    print(r.status_code)
    # реализуем обработку результата
    if r.status_code == 200:
        # если запрос выполнен успешно (код обработки=200),
        # выводим результат на экран
        print(r.json()['result'])
    else:
        # если запрос завершён с кодом, отличным от 200,
        # выводим содержимое ответа
        print(r.text)
```

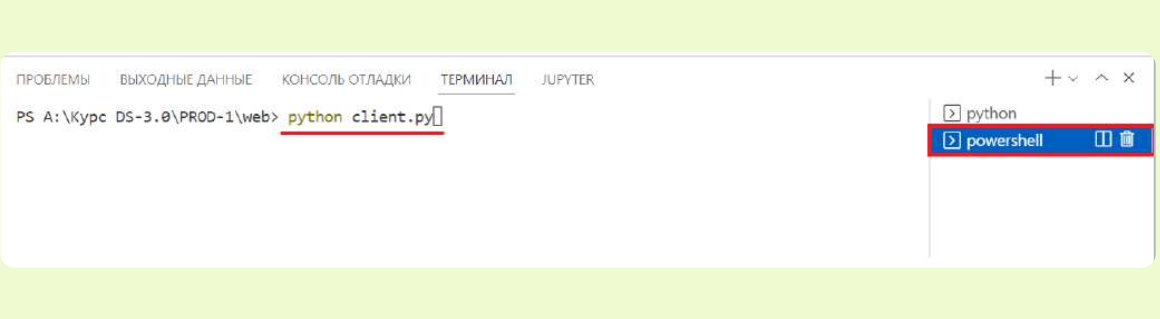
Примечание. Чтобы запустить код клиентского приложения параллельно работающему серверу в VS Code, создайте новый терминал и запустите файл client.py вручную, написав в терминале команду:

```
python client.py
```



```
 * Serving Flask app "server" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://localhost:5000/ (Press CTRL+C to quit)
```

Далее можно переключаться между терминалами и запускать в них необходимые команды.



```
PS A:\Курс DS-3.0\PROD-1\web> python client.py
```

Выполнив данный код, мы произведём POST-запрос на локальный сервер по эндпоинту `add`. Так как переданное значение `num < 10`, то запрос будет выполнен успешно и в терминале, соответствующему программе-клиенту, будет выведено:

```
## 200
## 6
```

Задание 5

0 points possible (ungraded)

В качестве значения параметра num возьмите число 15. Выполните скрипт клиентского приложения. Что будет выведено после выполнения этого скрипта?

Код обработки запроса:

Результат запроса:

Отправить

ДЕПЛОИМ МОДЕЛЬ НА ВЕБ-СЕРВИС

Итак, мы рассмотрели базовые принципы работы с фреймворком Flask и научились писать простейшие веб-сервисы. Этого достаточно, чтобы решить нашу первоначальную задачу — задеплоить модель. Загрузите модель один раз при старте приложения (вне функции-обработчика).

Нам необходимо написать веб-сервис, который позволит клиентам делать запросы к нашей модели. В ответ клиенты должны получать ответы — предсказания модели. Формат предсказаний обсудим ниже.

Файл с сериализованной моделью вы можете найти здесь.

У модели, как всегда, есть метод `predict`, который принимает на вход numpy-массив размерности `(кол-во объектов; кол-во признаков)`. Всего у нас четыре анонимных признака, и их интерпретация сейчас не имеет значения.

Давайте обернём процедуру предсказания модели в обработчик POST-запросов.

Задание 6

0 points possible (ungraded)

Используя только что обретённые знания, напишите *Flask*-приложение, которое по эндпоинту `/predict` будет слушать *POST*-запросы на предсказания.

В теле *POST*-запроса — список из четырёх чисел в формате *JSON* (один объект, четыре признака). Эти числа можно извлечь из запроса следующим образом:

```
features = request.json
```

Так как на вход модели необходимо подать *numpy*-массив определённой размерности, а не список, не забудьте перевести результат в тип `np.array()` и скорректировать его под размер `(1, 4)` с помощью `reshape()`:

```
features = np.array(features).reshape(1, 4)
```

После этого подайте полученный вектор на вход модели и получите результат.

Обратите внимание, что результатом предсказания также станет не число, а *numpy*-массив, пусть и состоящий из одного числа. Нас же интересует именно предсказанное число, поэтому не забудьте извлечь результат из полученного массива (обратившись по индексу `0`).

Ответом на запрос должен быть *JSON*-формат `{"prediction": *число - предсказание модели*}`.

Для проверки запустите свой сервер на порте 5000 локальной машины и выполните проверочный клиентский скрипт.

Если всё в порядке и запросы к серверу выполняются корректно, то после запуска проверочного скрипта вы получите в терминале секретный код. Введите этот код в поле ниже

Отправить

Поздравляем — теперь у вас есть рабочий прототип модели, который работает на веб-сервисе и умеет выдавать предсказания в ответ на POST-запросы клиентов. Однако это ещё не всё: в следующем уроке мы узнаем, как повысить производительность сервиса.

‹ Предыдущий урок   Следующий урок ›

8/11 GIL. uWSGI + NGINX

текущий урок

Этот урок является дополнительным и предназначен для тех, кто хочет немного углубиться в процесс создания настоящих веб-сервисов. Задания в уроке не оцениваются.

В предыдущем уроке мы познакомились с основами веб-разработки и смогли запустить на локальной машине маленький веб-сервер, который может принимать запросы и обрабатывать их. В этой части модуля мы займёмся оптимизацией работы нашего веб-сервиса и поднимем настоящий, полноценный веб-сервер.

Сразу отметим: инструменты, рассматриваемые в этом уроке, предназначены **исключительно для UNIX-операционных систем (Linux и MacOS)**.

В целом, разработка веб-приложений под Windows — не самая популярная идея. Большая часть серверов в продакшн работает на дистрибутивах ОС Linux. Для этого есть много причин, но сейчас мы не будем на них останавливаться. Важно понимать, что некоторые инструменты, такие как uWSGI, который мы будем рассматривать далее, просто не предназначены для ОС Windows, и попытки развернуть такие инструменты на этой ОС бессмысленны. Для Windows существуют свои решения, о которых мы также упомянем.

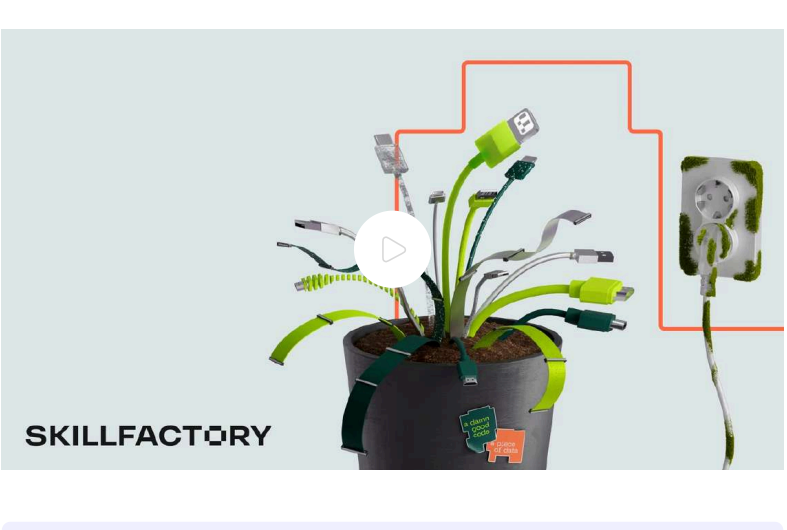
По упомянутой выше причине рекомендуем вам уже сейчас понемногу «перевозить» свою разработку на ОС Linux, если вы не сделали этого раньше. В качестве дистрибутива Linux советуем использовать Ubuntu (Рекомендуем Ubuntu LTS (например, 22.04/24.04 или новее)). Ниже приведены способы установки Ubuntu с подобными инструкциями.

Способы установки Ubuntu:

- Самый безопасный, простой и рекомендуемый нами способ — установить виртуальную машину Ubuntu с помощью программы виртуализации VirtualBox. Виртуальную машину можно представить как операционную систему внутри другой операционной системы. Подробную инструкцию по установке вы можете найти здесь.
- Пользователям Windows также можно использовать Windows Subsystem for Linux (WSL) — подсистему ОС Windows 10, позволяющую разработчикам и тестировщикам запускать нативные приложения Linux, писать скрипты, выполнять команды непосредственно из Windows.
- Самый продвинутый, но в то же время **опасный** способ — установить Linux в качестве второй системы параллельно Windows. Однако при настройке системы неопытным пользователем возможна потеря файлов на исходной ОС. Поэтому, если вы выберете именно этот способ, настоятельно рекомендуем вам быть предельно внимательными и следовать всем приведённым инструкциям. Подробную инструкцию по установке вы можете найти здесь.

Итак, если вы установили ОС Linux и готовы повысить производительность своего веб-сервиса, давайте начнём.

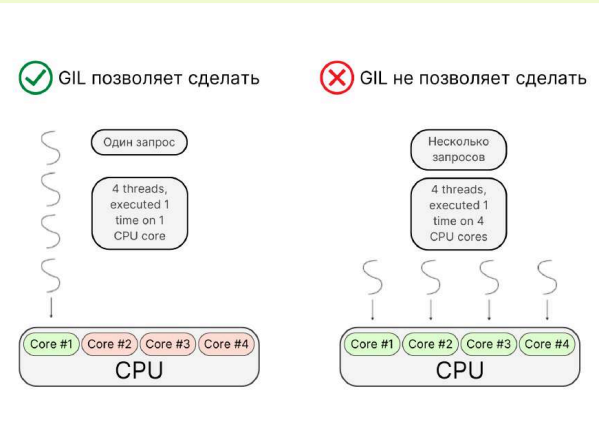
GIL. ОГРАНИЧЕНИЕ НА МНОГОПОТОЧНОСТЬ



Представим ситуацию: мы хотим, чтобы сервер с нашей моделью одновременно обрабатывал **несколько процессов**. Если мы попробуем запустить такой сервер, то либо его производительность будет крайне низкой, либо процессы просто не запустятся. Почему?

Дело в механизме под названием GIL — Global Interpreter Lock. Он заключается в том, что в любом процессе Python одновременно может работать только один **тред** (один конкретный поток).

Примечание. Подробнее об отличиях процесса и потока можно узнать здесь.



Таким образом, процесс Python не может утилизировать многоядерные системы. Для нашей задачи написания веб-сервера это означает, что сервер на Python одновременно может обрабатывать **только один запрос**, и это очень серьёзное ограничение даже для самых простых сервисов.

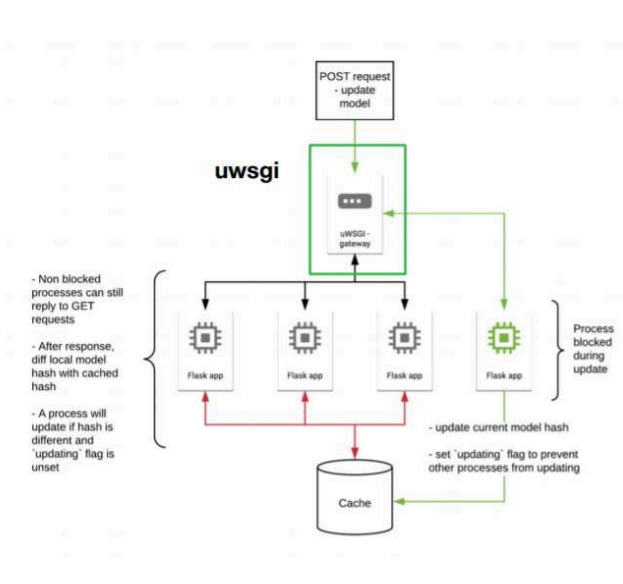
КАК РЕШИТЬ ПРОБЛЕМУ?

Хитрость заключается в том, что, раз один процесс может выполнять только один тред на одном ядре, мы просто запустим несколько процессов, чтобы они работали **параллельно**.

Существует несколько специализированных веб-серверов, решающих эту задачу, и все они имеют разную степень совместимости с веб-фреймворками Python. Однако принцип работы у них один: вы запускаете команду старта веб-сервера и передаёте в неё в качестве параметра указание на скрипт, в котором написано ваше приложение. Эта команда запустит несколько рабочих процессов (копий главного скрипта), а сама будет заниматься **прослушиванием** входящих запросов и передавать их на обработку в один из запущенных процессов.

Если вам интересно узнать, чем специализированные веб-серверы для Python отличаются друг от друга, прочитайте эту статью.

Для Flask используются веб-серверы Gunicorn, связка NGINX + uWSGI и ещё несколько менее популярных.


POST request - update model
uwsgi
uWSGI-gateway
- Non blocked processes can still reply to GET requests
- After response, diff local model hash with cached hash
- A process will update if hash is different and 'updating' flag is unset
Flask app
Flask app
Flask app
Flask app
Process blocked during update
- update current model hash
- set 'updating' flag to prevent other processes from updating
Cache

В этом модуле мы разберём связку NGINX + uWSGI. Для вас не составит труда самостоятельно разобраться с любым другим способом, поскольку принцип их работы одинаков.

Важно! Мы рекомендуем вам всегда выполнять все изложенные шаги при настройке сервера. Игнорируйте это предостережение, только если вы на 100 % уверены, что многопоточность вам не понадобится.

ПРЕДВАРИТЕЛЬНАЯ ПОДГОТОВКА

Перед началом давайте зафиксируем все вводные. Наш проект будет храниться в директории `/home/<имя пользователя>/web`. В этой папке есть папка models, в которой хранится файл с моделью. Файл с Flask-приложением будет также располагаться в этой папке и будет называться server.py. Здесь же будет лежать файл с кодом клиентской части client.py.

```
├web
  ├models
     └model.pkl
  └client.py
  └server.py
```

Веб-приложение будет работать локально на порте с номером 5000. У него будет два эндпоинта (`'/'` и `'/predict'`), каждому из которых соответствует свой интерфейс (функция, обёрнутая в декоратор `app.route`):

- Первый интерфейс будет соответствовать эндпоинту по умолчанию (`'/'`) — `http://localhost:5000`. Он поможет нам в тестировании приложения. В результате выполнения запроса по данному адресу должно быть выведено сообщение о том, что сервер успешно работает.
- Второй интерфейс будет соответствовать эндпоинту `http://localhost:5000/predict`. Он предназначен для обработки поступающих POST-запросов. Это тот самый обработчик, который вы должны были написать в задании 7.6 — вставьте свой код на место пропущенной части.

Файл server.py:

```python
from flask import Flask, request, jsonify
import pickle
import numpy as np

# загружаем модель из файла
with open('models/model.pkl', 'rb') as pkl_file:
    model = pickle.load(pkl_file)

# создаём приложение
app = Flask(__name__)

@app.route('/')
def index():
    msg = "Test message. The server is running"
    return msg

@app.route('/predict', methods=['POST'])
def predict():
    #ваш код здесь
    pass

if __name__ == '__main__':
    app.run('localhost', 5000)
```

Для тестирования выполнения POST-запросов на сервере, которые возвращают предсказания модели, будем использовать следующий клиентский скрипт:

Файл client.py:

```python
import requests

if __name__ == '__main__':
    # выполняем POST-запрос на сервер по эндпоинту add с параметром json
    r = requests.post('http://localhost:5000/predict', json=[1, 0, 1, 24])
    # выводим статус запроса
    print('Status code: {}'.format(r.status_code))
    # реализуем обработку результата
    if r.status_code == 200:
        # если запрос выполнен успешно (код обработки=200),
        # выводим результат на экран
        print('Prediction: {}'.format(r.json()['prediction']))
    else:
        # если запрос завершён с кодом, отличным от 200,
        # выводим содержимое ответа
        print(r.text)
```

Предварительно протестируйте приложение, запустив файл server.py и перейдя в браузере по адресу http://localhost:5000, а также попробовав выполнить клиентский скрипт, формирующий POST-запрос.

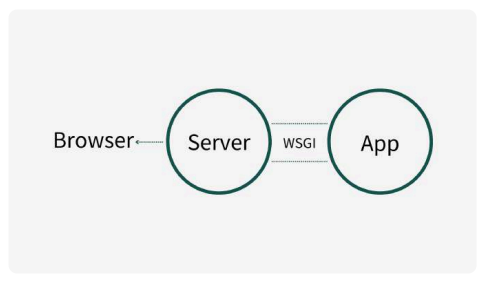
Если всё работает корректно, можно приступать к работе с uWSGI и NGINX.

WSGI И UWSGI

Давайте разберём основную терминологию.

Flask-приложению нужен интерфейс, который будет запускать приложение в нескольких процессах, обрабатывать поступающие запросы и возвращать пользователю ответ.

Для выполнения этих задач был разработан **WSGI (Web-Server Gateway Interface)** — протокол, стандарт взаимодействия между Python-программами, выполняющимися на веб-сервере и самим веб-сервером. Проще говоря, этот стандарт регламентирует, как Python должен обрабатывать запросы от приложений к веб-серверам и обратно.



Во взаимодействии, организованном по WSGI-протоколу, участвуют два компонента:

- Веб-сервер, который принимает поступающие запросы (uWSGI, Nginx, Apache и т. д.). На схеме выше обозначен как Server.
- Веб-приложение, написанное на языке Python. На схеме выше обозначено как App.

> **uWSGI** — это веб-сервер, который запускает Python-приложения через протокол WSGI. У этого веб-сервера есть и специальная версия протокола WSGI, которая называется uwsgi.

> **uwsgi** (не путать с uWSGI!) — это бинарная оптимизированная версия протокола WSGI, которая позволяет достичь ещё большей скорости взаимодействия между сервером и приложением.

Установка uwsgi

Чтобы запустить сервис в настоящем, боевом режиме, установим веб-сервер uWSGI. Это можно сделать с помощью стандартного менеджера пакетов Python pip.

В UNIX-системах (Ubuntu/Mac OS) установка производится довольно легко через выполнение перечисленных ниже команд.

Для Ubuntu:

```
$ sudo apt-get install build-essential python-dev
$ pip install uwsgi
```

Для MacOS:

```
$ brew install build-essential python-dev
$ pip install uwsgi
```

Чтобы проверить, что установка прошла успешно, выполните в терминале команду:

```
$ uwsgi --version
```

Если с установкой возникли проблемы, загляните в инструкцию.

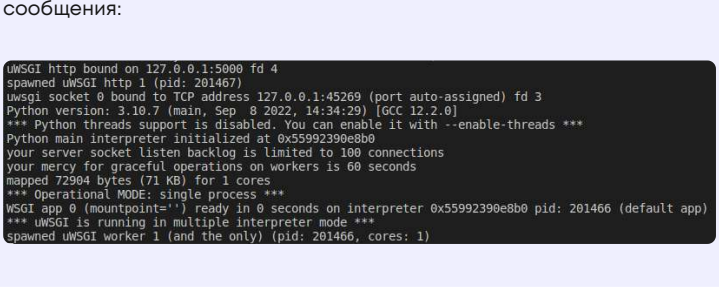
Запустим сервис (файл сервера server.py, который мы писали в прошлых уроках) с помощью веб-сервера uWSGI. Это довольно просто:

```
$ uwsgi --http :5000 --module server:app
```

Разберём аргументы.

- Аргумент `http :5000` указывает, что uWSGI должен обрабатывать HTTP-запросы на порте 5000.
  При необходимости перед двоеточием можно указать **хост** — `localhost`, `127.0.0.1` для локального запуска (он используется по умолчанию, если ничего не указано) или `0.0.0.0`, чтобы сервис был доступен по всем сетевым интерфейсам.
- Аргумент `server:app` указывает, какое приложение будет запускать uWSGI: модуль server.py (до двоеточия) и объект app внутри этого модуля (после двоеточия).

В результате выполнения команды появится большое сообщение с указанием всех параметров организованного соединения (начиная от версии uWSGI и операционной системы, на которой запущено приложение, и заканчивая номером процесса pid, закреплённым за запущенным приложением). Вот финальный фрагмент такого сообщения:



```
uWSGI http bound on 127.0.0.1:5000 fd 4
spawned uWSGI http 1 (pid: 201467)
uwsgi socket 0 bound to TCP address 127.0.0.1:45269 (port auto-assigned) fd 3
Python version: 3.10.7 (main, Sep  8 2022, 14:34:29) [GCC 12.2.0]
*** Python threads support is disabled. You can enable it with --enable-threads ***
Python main interpreter initialized at 0x55992390e8b0
your server socket listen backlog is limited to 100 connections
your mercy for graceful operations on workers is 60 seconds
mapped 72904 bytes (71 KB) for 1 cores
*** Operational MODE: single process ***
WSGI app 0 (mountpoint='') ready in 0 seconds on interpreter 0x55992390e8b0 pid: 201466 (default app)
*** uWSGI is running in multiple interpreter mode ***
spawned uWSGI worker 1 (and the only) (pid: 201466, cores: 1)
```

После этого uWSGI перейдёт в режим ожидания поступающих запросов — прослушивания сети.

Чтобы проверить, что ваш веб-сервер работает, обратитесь по адресу localhost в строке браузера, не забывая указать порт, который мы задали для прослушивания сети — http://localhost:5000. Вы должны увидеть сообщение об успешной работе сервера, которое мы прописали в функции `index()` при реализации приложения.

Давайте проверим количество процессов в системе, ведь нашей целью было с помощью uwsgi создать несколько процессов, каждый из которых занимается прослушиваем сети и обработкой поступающих запросов.

Для этого в Linux есть специальные инструменты, похожие на диспетчер задач в Windows, и самый продвинутый из них — утилита htop. Она позволяет отслеживать процессы в режиме реального времени, что очень полезно для администрирования серверов. Давайте установим её.

Для начала создайте ещё один терминал, не прерывая работу сервера. В новом терминале запустите следующую команду для установки:

```
$ sudo apt-get install htop
```

Теперь запустите утилиту htop, введя в терминал команду:

```
$ htop -t
```

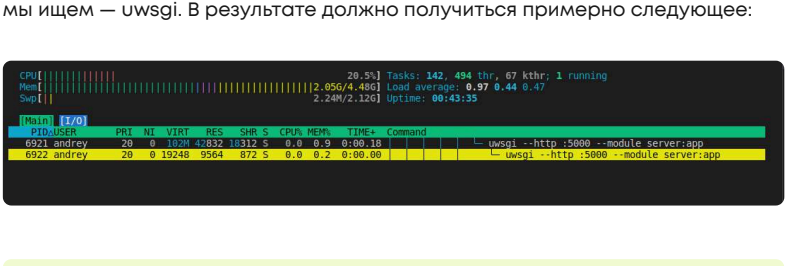
Ключ `-t` означает, что процессы будут выводиться в виде дерева (от англ. tree), то есть в иерархии сверху будет находиться родительский процесс, а ниже будут располагаться те процессы, которые он породил (дочерние).

В результате выполнения данной команды будет выведена огромная таблица со всеми процессами, запущенными в системе. В этой таблице нас будут интересовать следующие поля:

- `PID` — идентификатор процесса;
- `USER` — пользователь, запустивший процесс;
- `TIME+` — время, измеренное в часах (указывает на то, сколько процесс находился в пользовательском и системном времени);
- `Command` — полная командная строка.

Если вы хотите больше узнать о команде htop и её результатах, загляните сюда.

Для того чтобы отфильтровать только нужные нам процессы из полученной таблицы, мы должны нажать клавишу F4, а затем написать слово, которое мы ищем — uwsgi. В результате должно получиться примерно следующее:



```
CPU[||||||||||||                                20.5%] Tasks: 142, 494 thr, 67 kthr; 1 running
Mem[||||||||||||||||||||||||||||||||||||||2.05G/4.48G] Load average: 0.97 0.44 0.47
Swp[|                                   2.24M/2.12G] Uptime: 00:43:35

[Main] [I/O]
  PID USER       PRI  NI  VIRT   RES   SHR S  CPU% MEM%   TIME+  Command
 6921 andrey      20   0  102M 42832 18312 S   0.0  0.9  0:00.18  │  │  │  └─ uwsgi --http :5000 --module server:app
 6922 andrey      20   0 19248  9564   872 S   0.0  0.2  0:00.00  │  │  │     └─ uwsgi --http :5000 --module server:app
```

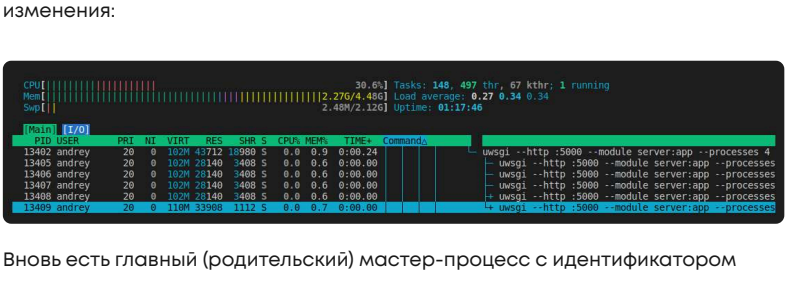
Что мы видим? У нас есть **один основной процесс** (он обозначен идентификатором 6921, но ваш идентификатор может отличаться). Этот процесс порождает один дочерний процесс 6922, который и выполняет прослушивание входящих запросов на нашем сервере.

Эта ситуация стандартная и пока что ничем не отличается от обычного запуска скрипта — работает только один процесс.

Давайте добавим больше процессов, чтобы увеличить пропускную способность сервера. Для этого завершим работу сервера (в терминале с запущенным сервером нажмите сочетание клавиш CTRL+C) и перезапустим его, увеличив количество работающих одновременно процессов до четырёх. Для этого в используемой ранее команде по запуску сервера необходимо дописать параметр `--processes`, указав количество процессов, и параметр `--master` (для чего он нужен — обсудим чуть позже):

```
$ uwsgi --http :5000 --processes 4 --master --module server:app
```

В соседнем терминале, где запущена утилита htop, мы увидим следующие изменения:



```
CPU[||||||||||||||||||                          30.6%] Tasks: 148, 497 thr, 67 kthr; 1 running
Mem[||||||||||||||||||||||||||||||||||||||2.27G/4.48G] Load average: 0.27 0.34 0.34
Swp[|                                   2.48M/2.12G] Uptime: 01:17:46

[Main] [I/O]
  PID USER       PRI  NI  VIRT   RES   SHR S  CPU% MEM%   TIME+  Command
13402 andrey      20   0  102M 43712 18980 S   0.0  0.9  0:00.24       uwsgi --http :5000 --module server:app --processes 4
13405 andrey      20   0  102M 28140  3408 S   0.0  0.6  0:00.00       ├─ uwsgi --http :5000 --module server:app --processes
13406 andrey      20   0  102M 28140  3408 S   0.0  0.6  0:00.00       ├─ uwsgi --http :5000 --module server:app --processes
13407 andrey      20   0  102M 28140  3408 S   0.0  0.6  0:00.00       ├─ uwsgi --http :5000 --module server:app --processes
13408 andrey      20   0  102M 28140  3408 S   0.0  0.6  0:00.00       ├─ uwsgi --http :5000 --module server:app --processes
13409 andrey      20   0  110M 33908  1112 S   0.0  0.7  0:00.00       └─ uwsgi --http :5000 --module server:app --processes
```

Вновь есть главный (родительский) мастер-процесс с идентификатором 13402, который в свою очередь породил пять дочерних процессов. Четыре из пяти дочерних процессов рабочие — они занимаются прослушиваем сети, а о том, занимается пятый процесс, мы поговорим ниже.

Таким образом, мы **увеличили** число рабочих процессов до четырёх. Сделайте несколько запросов к своему веб-сервису и посмотрите на количество Python-процессов в системе.

Итак, мы увеличили пропускную способность нашего сервиса.

Обратите внимание! Это не значит, что сами запросы будут обрабатываться быстрее — в этом смысле код не изменился, и на его выполнение тратится столько же времени. Зато мы получили возможность параллельно обрабатывать до четырёх запросов.

Отметим ещё одну возможность, которая позволит облегчить работу с uWSGI. Ранее мы запускали веб-сервер, указывая параметры непосредственно в команде для его запуска. Однако когда параметров становится слишком много, то каждый раз набирать в терминале команду для запуска сервера становится проблематичным. В таком случае лучше определить конфигурацию в отдельном файле. Давайте назовём этот файл "uwsgi.ini" и разместим его в директории проекта.

Содержимое этого файла похоже на обычный код: слева от "`=`" пишется имя параметра, а справа от него — значение.

В начало файла добавим заголовок `[uwsgi]`, чтобы указать uWSGI на необходимость применения настроек, а далее перечислим нужные нам параметры:

Файл uwsgi_config.ini

```
[uwsgi]
module = server:app

http = 127.0.0.1:5000

processes = 4
master = true
```

Теперь, чтобы запустить uWSGI с указанной в файле конфигурацией, необходимо просто передать путь до файла в параметре `--ini`:

```
$ uwsgi --ini uwsgi_config.ini
```

В результате выполнения данной команды в терминале вы увидите же самое сообщение, что и ранее при запуске uWSGI.

Но и это ещё не всё. Как мы уже отметили, на скриншотах, полученных с помощью htop и после запуска uwsgi, видно, что процессов не четыре, а пять. Откуда взялся ещё один? Этот последний процесс является HTTP-роутером, который слушает HTTP-трафик и отправляет его на воркеры (рабочие процессы), то есть он выступает в роли пункта-распределителя.

Однако uWSGI — это не самая оптимальная реализация HTTP-роутера. Она существует скорее для отладки веб-разработчиками, что не является нашей зоной компетенции, поэтому сами авторы uWSGI советуют использовать полноценный веб-сервер. И тут появляется NGINX.

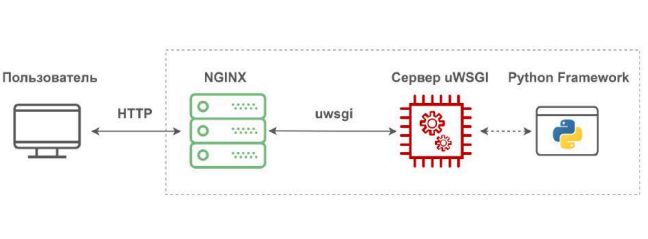
NGINX

NGINX — это веб-сервер, который оптимизирует нагрузку за счёт **асинхронной архитектуры**, управляемой событиями. Говоря простым языком, архитектура использует одно ядро на один процесс. При этом в одном процессе могут быть сотни тысяч входящих запросов от каждого пользователя. Совместная параллельная обработка позволяет не создавать новые потоки для каждого соединения — это быстро и удобно. Такая высокая производительность делает NGINX самым популярным веб-сервером в мире.

На сегодняшний день NGINX поддерживает протокол uwsgi, что позволяет приложениям WSGI, запущенным на uWSGI, лучше взаимодействовать с NGINX. Благодаря этому развёртывание приложений становится очень простым в настройке, чрезвычайно гибким, функциональным, а также может пользоваться преимуществами поставляемых оптимизаций. Всё это делает сочетание uWSGI + NGINX великолепным вариантом для многих сценариев развёртывания.

Что мы хотим увидеть в итоге?

Мы хотим, чтобы входящие HTTP-запросы, которые поступают на наш сервис, принимал NGINX, так как у него очень высокая производительность. Далее необходимо, чтобы поступающие запросы по протоколу uwsgi передавались на uWSGI, в котором запущены процессы-воркеры, выполняющие обработку запросов на Flask.



Для того чтобы подключить сервер NGINX к нашему приложению, сначала нужно его установить.

Установка NGINX

В Linux-системах установка NGINX происходит с помощью одной команды в терминале:

```
$ sudo apt install nginx
```

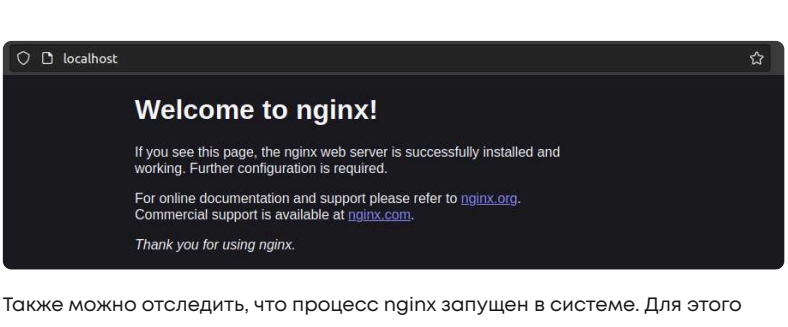
Для MacOS установка NGINX идентична установке в Linux с той лишь разницей, что вместо менеджера пакетов apt используется homebrew*:

* Для перехода по ссылке рекомендуем воспользоваться VPN-сервисом.

```
$ brew install nginx
```

Существует и версия для Windows, но некоторые важные функции не поддерживаются в Windows, что лишает веб-сервер значительной части производительности, и его использование под этой ОС как таковое вызывает сомнения.

После установки сервер NGINX запускается автоматически. Чтобы проверить, что установка была выполнена успешно, достаточно зайти в браузер и в поисковой строке написать http://localhost (без указания порта). В результате вы должны увидеть примерно следующий результат:



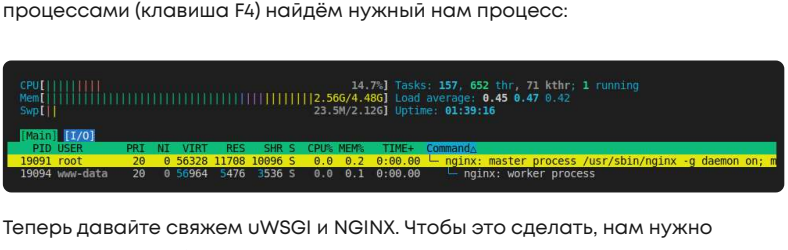
Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

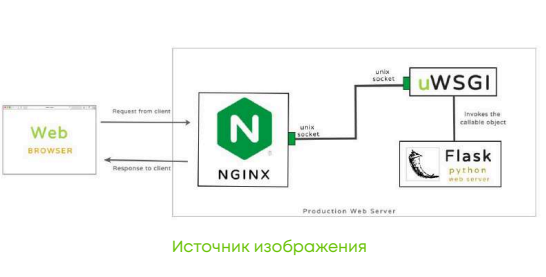
Также можно отследить, что процесс nginx запущен в системе. Для этого опять же воспользуемся утилитой htop и с помощью фильтрации таблицы с процессами (клавиша F4) найдём нужный нам процесс:



```
CPU[|||||||||                                   14.7%] Tasks: 157, 652 thr, 71 kthr; 1 running
Mem[||||||||||||||||||||||||||||||||||||||2.56G/4.48G] Load average: 0.45 0.47 0.42
Swp[|                                   23.5M/2.12G] Uptime: 01:39:16

[Main] [I/O]
  PID USER       PRI  NI  VIRT   RES   SHR S  CPU% MEM%   TIME+  Command
19091 root        20   0 56328 11708 10096 S   0.0  0.2  0:00.00 └─ nginx: master process /usr/sbin/nginx -g daemon on; m
19094 www-data    20   0 56964  5476  3536 S   0.0  0.1  0:00.00    └─ nginx: worker process
```

Теперь давайте свяжем uWSGI и NGINX. Чтобы это сделать, нам нужно запустить uWSGI без роутера — место роутера должен занят NGINX, который будет пересылать поступающий трафик, а uWSGI будет слушать трафик прямиком из **сокета**.

> **Сокет** — это программный интерфейс для взаимодействия между двумя процессами (в нашем случае между NGINX и uWSGI). Можно представить себе сокет как специальный файл в памяти, в который один процесс пишет, а другой из него читает. Тогда наша схема организации взаимодействия между клиентами и веб-приложением будет выглядеть следующим образом:



Источник изображения

Примечание. Если вы всё ещё на Windows, то, к сожалению, сокетов у вас нет и придётся использовать один из TCP-портов по этой инструкции.

Итак, чтобы запустить взаимодействие через сокеты, нам достаточно лишь изменить параметр http на socket и указать в нём путь до файла-сокета, через который мы хотим организовать обмен информацией.Файл с названием socket и расширением .sock будет находиться в специальной директории /tmp, которая в UNIX-системах предназначена для хранения временных объектов (/tmp — от англ. “temporary” — «временный»). Создавать его самим не нужно — за нас это автоматически сделает uWSGI при запуске.

Также мы выполним очистку сокета после остановки процесса uWSGI. Для этого будем использовать параметр `vacuum` — благодаря этой опции файл будет самостоятельно удаляться после отключения сервера. Отредактируем наш файл с конфигурацией uWSGI (uwsgi_confug.ini):

Файл uwsgi_config.ini

```
[uwsgi]
module = server:app

socket = /tmp/socket.sock
vacuum = true

processes = 4
master = true
```

Теперь мы готовы перезапустить uWSGI:

```
$ uwsgi --ini uwsgi_config.ini
```

Убедитесь, что в директории /tmp был создан файл socket.sock. Для этого в соседнем терминале выполните команду:

```
$ ls /tmp/socket.sock
```

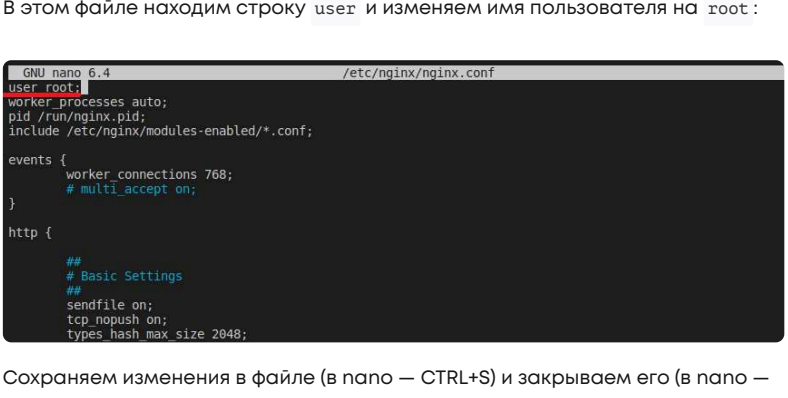
Если команда была выполнена без ошибок, то файл существует.

Последний большой шаг — настроить NGINX на передачу приходящих HTTP-запросов в созданный сокет-файл. Для этого нам нужно сначала немного изменить глобальную конфигурацию NGINX, а именно дать ему права на запуск от имени суперпользователя (root). Так NGINX сможет иметь доступ на чтение, изменение и исполнение всех файлов в системе, в том числе для файлов сокетов.

Открываем во встроенном редакторе nano (или любом другом редакторе) файл с конфигурацией nginx.conf — он находится в директории /etc/nginx:

```
$ sudo nano /etc/nginx/nginx.conf
```

В этом файле находим строку `user` и изменяем имя пользователя на `root`:



```
  GNU nano 6.4                         /etc/nginx/nginx.conf
user root;
worker_processes auto;
pid /run/nginx.pid;
include /etc/nginx/modules-enabled/*.conf;

events {
        worker_connections 768;
        # multi_accept on;
}

http {


        ##
        # Basic Settings
        ##
        sendfile on;
        tcp_nopush on;
        types_hash_max_size 2048;
```

Сохраняем изменения в файле (в nano — CTRL+S) и закрываем его (в nano — CTRL+X).

Формально у NGINX есть только один файл конфигурации — тот, который мы только отредактировали. Теоретически конфигурации для каждого приложения можно было бы прописывать именно в нём. Однако так делать не рекомендуется. Для этого уже существуют папки /etc/nginx/sites-available/ и /etc/nginx/sites-enabled.

- Первая просто содержит файлы конфигурации, в каждом из которых находится отдельный виртуальный хост.
- Вторая папка содержит ссылки на файлы из sites-available и подключена к основному конфигурационному файлу.

Чтобы составить конфигурацию nginx для нового приложения, мы сначала должны создать файл конфигурации в папке sites-available, а затем создать ссылку на него в папке sites-enabled. В результате таких неочевидных, но очень важных для безопасности манипуляций мы добавим в глобальную конфигурацию работы NGINX конфигурацию, нужную для нашего приложения.

Итак, создадим и откроем на редактирование файл server.conf в папке sites-available:

```
$ sudo nano /etc/nginx/sites-available/server.conf
```

Затем введём в него следующее содержимое:

Файл /etc/nginx/sites-available/server.conf

```
server {
    listen 80;
    server_name localhost;
    location / {
        uwsgi_pass unix:/tmp/socket.sock;
        include uwsgi_params;
    }
}
```

Давайте разберёмся, что мы тут прописали. Структура файла очень напоминает JSON-формат. Корневой заголовок `server` указывает на то, что мы создаём конфигурацию нового сервера NGINX.

Внутри этого блока через "`;`" указываются следующие поля:

- `listen` — номер порта, который будет прослушивать сервер NGINX. По умолчанию протокол HTTP работает на порте 80. То есть по умолчанию обращение через браузер по любому IP-адресу (например, `http://198.102.103.8`) — это на самом деле обращение к нему по порту 80 (например, `http://198.102.103.8:80`). В параметре `listen` мы указали, что наш сервер NGINX будет слушать адрес 127.0.0.1 (адрес внутренней петли) на порте 80. То есть для обращения к нему будет достаточно ввести в поисковую строку браузера 127.0.0.1 или `localhost` без указания хоста. Если указать, другой порт, то в адресной строке браузера его также придётся прописывать.
- `server_name` — доменное имя. Так как у нас нет зарегистрированного доменного имени, то в качестве него мы указываем `localhost`.

  Доменное имя (домен) — это легко запоминающееся уникальное название сайта, связанное с определённым *IP*-адресом в интернете. Мы указываем его в адресе сайта после http:// и в адресе электронной почты после символа "@".

  Например, когда мы через браузер заходим на сайт http://skillfactory.ru, за этой ссылкой скрывается некоторый *IP*-адрес сервера, на котором работает сайт *Skillfactory*, а также номер порта (пусть для примера это будет адрес 11.67.78.8:80). Для того чтобы оперировать *IP*-адресами и номерами портов, как раз и были придуманы доменные имена.

  Нельзя просто так присвоить сайту доменное имя — его необходимо приобрести через регистратора. Регистратор — это компания, которая продаёт свободные доменные имена. Часто такие компании также предлагают услуги хостинга.
- В блоке `location` мы указываем эндпоинт (по умолчанию всегда обращаемся к корневому эндпоинту " / "), а также параметры, соответствующие этому блоку:
  - `uwsgi_pass` — команда, указывающая на то, что поступающие запросы будут передаваться на uwsgi. В аргументах этой команды мы отметили, что взаимодействие организовано через UNIX-сокеты, и указали путь до сокета (/tmp/socket.sock), по которому происходит обмен данными между NGINX и uWSGI.
  - `include` — команда для подключения файлов. Мы подключаем файл uwsgi_params, в котором указаны общие параметры работы uWSGI.

После завершения редактирования файл server.conf сохраняем его содержимое (CTRL+S) и выходим из редактора (CTRL+X).

Когда файл с конфигурацией готов, создаём на него символическую ссылку (аналог ярлыков в Windows) в папке sites-enabled. В Linux за это отвечает команда `ln`. Её синтаксис выглядит следующим образом:

```
ln опции файл_источник файл_ссылки
```

Для создания символических ссылок используется ключ `-s`.

Подробнее о команде `ln` вы можете прочитать здесь.

Тогда команда в терминале будет выглядеть следующим образом:

```
$ sudo ln -s /etc/nginx/sites-available/server.conf /etc/nginx/sites-enabled
```

Теперь наша конфигурация активирована.

Проверим, не было ли обнаружено ошибок при сборке конфигурации. Для этого используется команда nginx с ключом `-t` (test).

```
$ sudo nginx -t
```

Если ошибок не будет обнаружено, перезапустим процесс nginx для чтения новой конфигурации и для управления процессами (запуска/остановки и т. д.). Для этого в UNIX-системах есть утилита `systemctl`:

```
$ sudo systemctl restart nginx
```

Чтобы проверить результат, откройте браузер и сделайте запрос http://localhost. Если всё прошло успешно и в браузере вам было выведено отладочное сообщение, поздравляем — вы успешно запустили NGINX и связали его работу с uWSGI.

Обратите внимание, что для доступа к серверу нам теперь не нужно указывать номер порта, ведь мы запустили NGINX на порте 80 (указали это в файле server.conf), а он используется по умолчанию.

Что происходит?

При поступлении запроса на порт 80 NGINX транслирует его в обозначенный сокет по протоколу uwsgi. Процесс uWSGI, подключенный к этому сокету, обработает его на одном из запущенных процессов с нашим кодом, и ответ уйдёт тем же путём в обратной последовательности.

Таким образом, наш сервис стал **производительнее**, ведь он может параллельно обрабатывать несколько запросов.

А что дальше?

Мы проделали большую работу. Однако, как вы могли заметить, наш веб-сервис сейчас работает только локально, то есть доступ к нему можно получить только с вашего компьютера. В рамках нашего курса заводить полноценный веб-сервис, доступный в интернете, нам не нужно — нам было достаточно лишь прикоснуться к веб-программированию и потренироваться разворачивать собственный веб-сервис на локальной машине, поэтому мы смело можем закончить на этом этапе.

В реальности, как вы понимаете, доступ к сервису должен быть у всех компьютеров. Для того чтобы организовать возможность обращения к вашему серверу через интернет, вам нужно получить доменное имя (купить или получить бесплатно), привязанное к IP-адресу вашего сервера. После этого в конфигурации NGINX вашего веб-сервера (server.conf) нужно будет заменить параметр `server_name` с `localhost` на доменное имя или IP-адрес вашего сервера:

```
server_name your_domain www.your_domain;
```

Так ваш сервер сможет обрабатывать запросы, поступающие от внешних пользователей.

Это тренировочный тест. За него не предусмотрены баллы, но он поможет закрепить вам изученный материал.

Задание 1

0 points possible (ungraded)

Выберите верные утверждения:

☐ **A** Процесс *Python* может утилизировать многоядерные системы.
☐ **B** *GIL* обеспечивает одновременную работу только одного потока.
☐ **C** Связка *NGINX* и *uWSGI* обеспечивает запуск нескольких копий скрипта и прослушивает входящие запросы.
☐ **D** У *NGINX* и *uWSGI* нет аналогов — можно использовать только их.

Отправить

Задание 2

0 points possible (ungraded)

Чем занимается *NGINX* в связке *NGINX* + *uWSGI* + *Flask*?

○ Выполняет код *Python* для обработки запросов.
○ Слушает сетевой интерфейс и отправляет входящие запросы на обработку.
○ Запускает процессы-воркеры для параллельной обработки запросов и перенаправляет на них запросы.

Отправить

‹ Предыдущий урок   Следующий урок ›

9/11 **Итоги**

текущий урок

Поздравляем с прохождением первого модуля, посвящённого машинному обучению в продакшене!

В этом модуле вы:

Узнали:

- как сохранять обученные модели в файлы и обмениваться ими;
- как использовать обученные модели в приложениях, реализованных на языках программирования, отличных от Python;
- основные концепции межсетевого взаимодействия;
- несколько самых популярных веб-фреймворков Python и сравнили их между собой;
- как решать проблему GIL для Python с помощью протокола uwsgi;

Научились:

- отличать сериализацию от десериализации;
- реализовывать простейшие веб-сервисы на Flask;
- разворачивать свой сервер на NGINX.

Стоит отметить, что в процессе разработки нашего сервиса мы не учли одну очень важную особенность — воспроизводимость и изолированность. В чём сейчас состоит проблема? Когда мы передадим исходный код своему коллеге Василию, может оказаться так, что он у него попросту не запустится. Этому может быть множество причин: разница в версиях библиотек, в установленных путях до файлов, в конце концов — в операционных системах. Как же тогда быть?

Забегая вперёд, скажем, что для поддержки принципа воспроизводимости кода используются изоляция, виртуализация и контейнеризация. О них мы как раз и поговорим в следующем модуле.

Дополнительные материалы:

- документация по работе с Pickle;
- документация по работе с Joblib;
- список фреймворков для разработки HTTP-серверов на странице;
- обновляемый список ресурсов и плагинов для Flask;
- описание различных уровней взаимодействия сетей по сетевой модели OSI;
- исчерпывающий список кодов состояния протокола HTTP;
- подробное описание REST-архитектуры;
- Postman (для любителей визуальных редакторов запросов).

< Предыдущий урок Следующий урок >

10/11 Подсказки и эталонные ответы к заданиям модуля

текущий урок

2. Сохранение и загрузка моделей: pickle и joblib

Задание 5

```
# Десериализуем pipeline из файла
with open('my_new_pipeline.pkl', 'rb') as pkl_file:
    loaded_pipe = pickle.load(pkl_file)

features =np.array([[ 0.00538306, -0.04464164,  0.05954058, -0.05616605,
0.02457414, 0.05286081, -0.04340085,  0.05091436, -0.00421986, -0.03007245]])

# Формируем предсказание
print(np.round(loaded_pipe.predict(features)))
```

3. Практика: pickle

Задание 1

```
import pickle

with open('model.pkl', 'rb') as pkl_file:
    model = pickle.load(pkl_file)
```

Задание 3

```
print(model.predict([[1, 1, 1, 0.661212487096872]]))
```

Задание 4

```
my_dict = {'a': model.a, 'b': model.b}

with open('my_dict.pkl', 'wb') as pkl_file:
    pickle.dump(my_dict, pkl_file)

!python hw1_check_ol.py my_dict.pkl
```

7. Пишем сервер на Flask

Задание 3

```
@app.route('/')
def index():
    return "Test message. The server is running"
```

Задание 4

```
from flask import Flask
import datetime
app = Flask(__name__)

@app.route('/time')
def current_time():
    return {'time': datetime.datetime.now()}
if __name__ == '__main__':
    app.run('localhost', 5000)
```

Для выполнения запроса используется ссылка http://localhost:5000/time.

Задание 5

```
import requests

if __name__ == '__main__':
    # выполняем POST-запрос на сервер по эндпоинту add с параметром json
    r = requests.post('http://localhost:5000/add', json={'num': 15})
    # выводим статус запроса
    print(r.status_code)
    # реализуем обработку результата
    if r.status_code == 200:
        # если запрос выполнен успешно (код обработки=200),
        # выводим результат на экран
        print(r.json()['result'])
    else:
        # если запрос завершён с кодом, отличным от 200,
        # выводим содержимое ответа
        print(r.text)
```

Задание 6

```
from flask import Flask, request, jsonify
import pickle
import numpy as np

with open('./models/model.pkl', 'rb') as pkl_file:
    model = pickle.load(pkl_file)

app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():
    features = np.array(request.json)
    features = features.reshape(1, 4)
    prediction = model.predict(features)
    return  jsonify({'prediction': prediction[0]})

if __name__ == '__main__':

    app.run('localhost', 5000)
```

‹ Предыдущий урок   Следующий урок ›

1/8 Введение

текущий урок

В предыдущем модуле мы рассмотрели базовые принципы работы с моделями на этапе продакшена.

Мы научились:

- сохранять обученные модели в виде бинарных файлов, а также файлов других форматов, которые поддерживаются языками программирования, отличными от Python;
- локально разворачивать собственный веб-сервис с помощью фреймворка Flask, встраивая в его работу свою модель машинного обучения;
- оптимизировать работу этого веб-сервиса, увеличивая его пропускную способность с помощью таких инструментов, как uWSGI и NGINX.

Однако, как мы уже упомянули в конце прошлого модуля, осталась одна проблема: наш сервис (да и сама модель) работает только на нашем компьютере. Когда мы зальём исходный код на GitHub, а наш коллега Василий склонирует себе репозиторий и попробует запустить сервис на своём «боевом» сервере, нет гарантии, что приложение запустится.

Что, если на сервере Василия нет необходимых зависимостей, таких как Scikit Learn, Flask и так далее? Установить библиотеки несложно, но какие именно их версии нужны Василию? А что, если у него уже стоят некоторые библиотеки, которые вы использовали, но в совершенно другой версии, которая не совместима с вашим приложением?

Кроме того, для обеспечения работы сервиса в связке uWSGI + NGINX Василию придётся проделывать все те манипуляции, которые мы производили в прошлом модуле. Вы уже должны были убедиться, что процесс настройки взаимодействия uWSGI + NGINX + Flask не из лёгких, пусть его и можно обернуть в некоторую инструкцию.

А что, если, ко всему прочему, на сервере Василия стоит другая операционная система, например Windows, которая не поддерживает работу с uWSGI?

> Так много вопросов… И у нас есть на них ответы. Проблема, к которой мы подошли, называется **проблемой воспроизводимости**. О ней, как правило, не задумываются новички, однако она часто проявляется на этапе продакшена. Например, мы хотим перенести проект с локальной машины, где вели разработку, на реальный сервер, но из-за разницы в настройках и конфигурациях этот процесс становится болезненным и требует затрат времени, которого всегда остаётся очень мало на этапе продакшена.

В этом модуле мы разберём, как сделать свой код воспроизводимым и как изолировать свой сервис так, чтобы его можно было запустить на любой машине, а самое главное — как сделать это быстро и просто.

Цели модуля:

- Узнать, что такое воспроизводимость и какими инструментами её можно обеспечить.
- Познакомиться с терминами «виртуализация» и «контейнеризация».
- Научиться создавать виртуальные окружения, изолировать среду разработки, устанавливать и фиксировать зависимости в виртуальных окружениях.
- Познакомиться с инструментом контейнеризации Docker.
- Научиться писать Dockerfile, создавать образы контейнеров, запускать их, а также делиться ими с помощью хостинга docker-образов Docker Hub.
- Создать docker-образ для нашего веб-сервиса и запустить его в контейнере.

С каким проектом будем работать?

В предыдущем модуле мы написали маленький веб-сервис. В нём функционирует модель машинного обучения, которая выполняет предсказания для данных, поступающих через POST-запросы по эндпоинту `'/predict'`.

Перед прохождением модуля давайте зафиксируем, как должна выглядеть директория нашего проекта:

```
├─web
  ├─models
      └─model.pkl
  └─client.py
  └─server.py
```

Проект будет располагаться в директории web, в которой находятся файлы:

1. server.py — содержит интерфейс сервера, реализованный на Flask. В интерфейсе предусмотрено два эндпоинта:
   - `'/'` — корневой, по обращению к которому пользователю возвращается тестовое сообщение;
   - `'/predict'` — предназначенный для обработки POST-запросов. Запросы приходят в виде списка из четырёх чисел в формате `json()`. Результатом выполнения запроса является JSON-словарь с ключом `'prediction'` и значением-предсказанием модели.Вы писали функцию для обработки этого запроса в прошлом модуле.
2. client.py — скрипт для тестирования POST-запросов на сервер.
3. models — папка с моделями, в которой находится файл model.pkl с моделью.

Файл server.py:

```
from flask import Flask, request, jsonify
import pickle
import numpy as np

# загружаем модель из файла
with open('models/model.pkl', 'rb') as pkl_file:
    model = pickle.load(pkl_file)

# создаём приложение
app = Flask(__name__)

@app.route('/')
def index():
    msg = "Тестовое сообщение. Сервер запущен!"
    return msg

@app.route('/predict', methods=['POST'])
def predict():
    #ваш код здесь
    pass

if __name__ == '__main__':
    app.run('localhost', 5000)
```

Файл client.py:

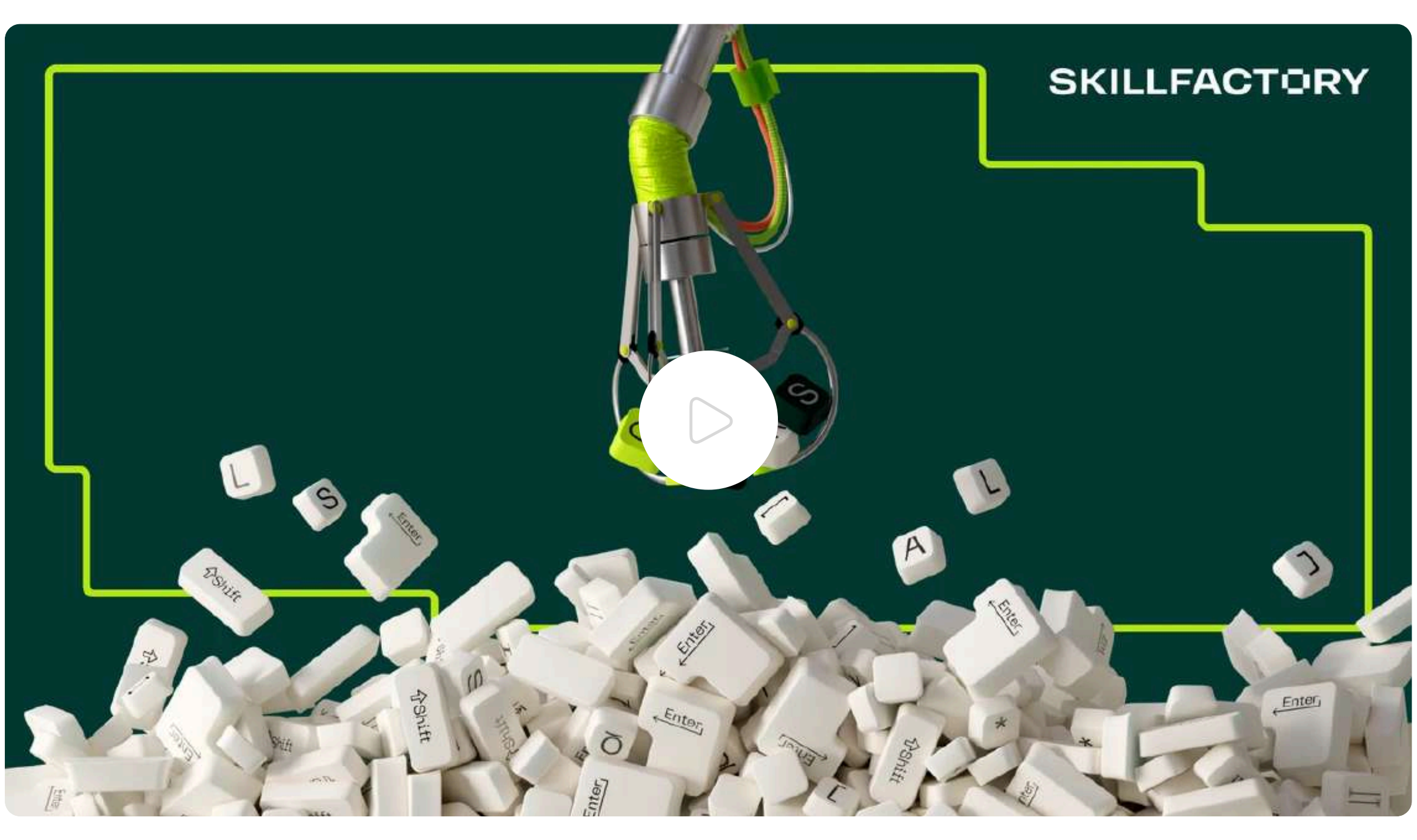
```
import requests

if __name__ == '__main__':
    # выполняем POST-запрос на сервер по эндпоинту add с параметром json
    r = requests.post('http://localhost:5000/predict', json=[1, 0, 1, 24])
    # выводим статус запроса
    print('Status code: {}'.format(r.status_code))
    # реализуем обработку результата
    if r.status_code == 200:
        # если запрос выполнен успешно (код обработки=200),
        # выводим результат на экран
        print('Prediction: {}'.format(r.json()['prediction']))
    else:
        # если запрос завершён с кодом, отличным от 200,
        # выводим содержимое ответа
        print(r.text)
```

В этом модуле мы сделаем наш сервис изолированным и поместим его в контейнер.

‹ Предыдущий урок
Следующий урок ›

2/8 Воспроизводимость

текущий урок



Код, разрабатываемый дата-сайентистом, должен быть воспроизводимым в постоянно меняющихся в условиях:

- меняются данные, поступающие на вход;
- трансформируются пайплайны предобработки данных;
- постоянно изменяются гиперпараметры алгоритмов;
- иногда модифицируются или вовсе удаляются алгоритмы популярных библиотек.

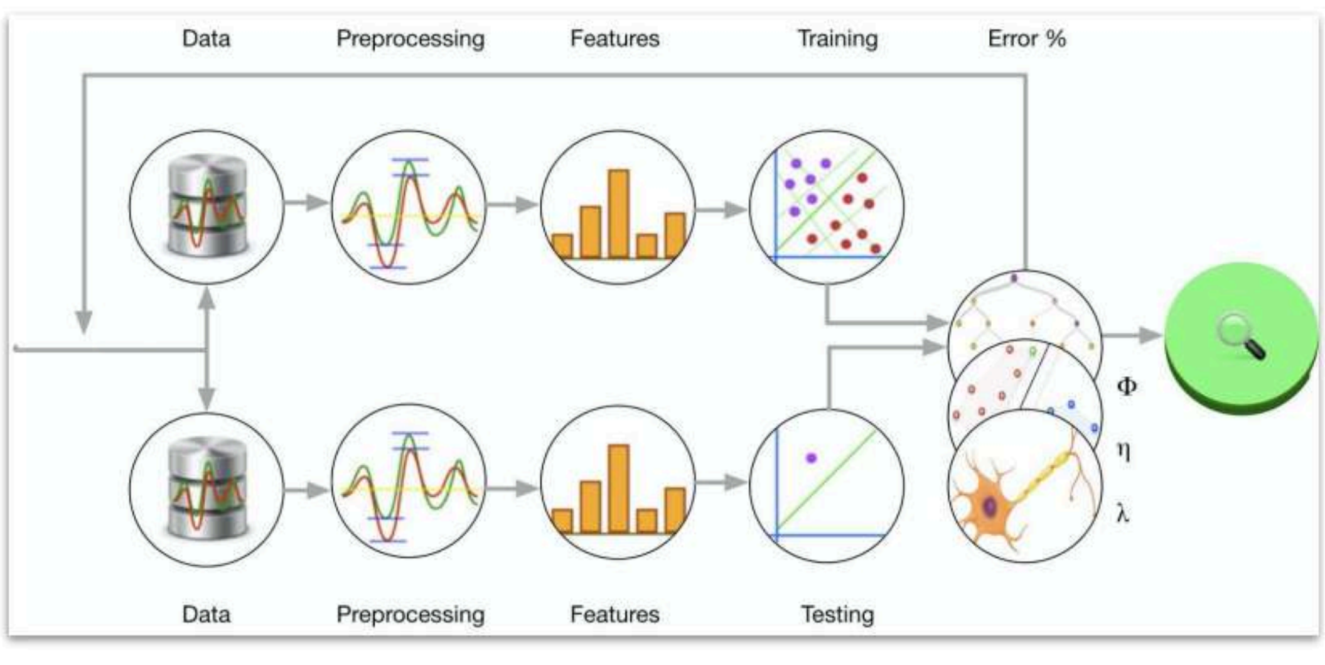
В идеале ожидается, что выполнение кода должно приводить к одинаковым результатам в различных условиях. Именно с этой целью в некоторых заданиях вы встречали требование зафиксировать параметр `random seed` — таким образом, вы уже неоднократно сталкивались с проблемой воспроизводимости на практике.

> **Воспроизводимость результатов** является одним из главных показателей качества модели. Именно поэтому обеспечение воспроизводимости — одна из важнейших проблем в современном ML-инжиниринге.

Мы можем доверять результату, полученному с помощью модели, только когда доподлинно известно, на какой выборке и с каким пайплайном предобработки она была построена. Кроме того, ожидается, что результаты будут консистентными (согласованными с исходными данными), так как необходимо однозначно понимать, какое изменение привело к улучшению или ухудшению результатов.

Наиболее остро вопрос воспроизводимости стоит в случае, когда над одной моделью работает большая DS-команда. Каждый участник команды должен воспроизвести весь пайплайн и получить тот же самый результат. При этом возможные изменения должны доставляться в общий код так же быстро, сохраняя все требования к воспроизводимости.

На схеме приведён классический пайплайн работы над моделью:



Инструменты обеспечения воспроизводимости:

- **Версионирование кода.**
  Обычно весь пайплайн представлен в виде частей кода, для работы с которым используется знакомая нам распределённая система управления версиями Git в совокупности с хостингом GitHub.
- **Версионирование артефактов.**
  В процессе работы над проектом появляются различные артефакты: датасеты, модели, файлы конфигурации и прочее. Для их версионирования обычно используются такие инструменты, как DVC* и Sonatype Nexus.
  * Если какие-то ссылки не открываются, обратитесь к координатору.
- **Виртуализация и контейнеризация.**
  Одним из важнейших аспектов воспроизводимости является настройка окружения.

  > **Виртуализация** — это технология изоляции, сохранения состояния и воспроизведения окружения. Благодаря ей вы можете воссоздать на другом компьютере точную копию вашего окружения и тем самым обеспечить воспроизводимость.

  Однако иногда воссоздания окружения недостаточно и необходимо обеспечить не только воспроизводимость версий библиотек, но и воспроизводимость среды выполнения программы — операционной системы, на которой реализован проект. Для этого прибегают к инструментам **контейнеризации**, таким как Docker.

  Все перечисленные выше инструменты мы подробно обсудим далее.
- **Управление экспериментами.**
  Для каждого запуска обучения необходимо фиксировать настройки гиперпараметров и сохранять их. В этом помогают системы управления экспериментами, например MLflow.

> Над созданием универсального стандарта воспроизводимости работают многие крупные компании. Это означает, что у каждой компании пока что существуют собственные требования к обеспечению воспроизводимости. Когда вы начнёте работать в DS-команде, ваши коллеги будут требовать от вас их соблюдения. Обычно чем крупнее компания, тем более серьёзные требования к воспроизводимости необходимо соблюдать. Поскольку единого стандарта пока не существует, пути и используемые в разных компаниях инструменты могут кардинально отличаться.

> Рекомендуем вам всегда помнить о задаче воспроизводимости результатов модели. Это позволит вам постепенно привить себе тот уровень культуры кода, который будет удовлетворять самые требовательные компании.

> Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 1

0 points possible (ungraded)

Изменение каких факторов может негативно повлиять на воспроизводимость результатов?
Отметьте все подходящие варианты ответов.

☑ A Данные, поступающие на вход модели во время её обучения
☑ B Гиперпараметры алгоритмов
☑ C Новые версии библиотек
☑ D Пайплайн предобработки (препроцессинга) данных

✓

Ответ
Верно:
A Верно.
B Верно.
C Верно.
D Верно.

Show answer

Отправить

✓ Верных

Задание 2

0 points possible (ungraded)

Выберите примеры плохой воспроизводимости:

☑ **A** При перезапуске инференса модели на одинаковых датасетах получаем разные предикты (ответы модели).
☐ **B** При перезапуске инференса модели на разных датасетах получаем одинаковые предикты (ответы модели).
☐ **C** Данные для обучения обновляются один раз в день, модель переобучается один раз в день, инференс модели в разные дни даёт разные предикты (ответы модели).
☑ **D** Ваш коллега обучил модель, получил результат precision=0.73 на тестовом датасете и передал вам код. Вы делаете инференс на том же тестовом датасете и получаете результат precision=0.77.
☐ **E** Вы обучили модель и получили результат инференса на тестовом датасете precision=0.73. Затем вы решили добавить нормализацию одного из признаков перед обучением модели и получили результат precision=0.77.

✓

Ответ
Верно:
A Верно.
D Верно.

Show answer

Отправить

✓ Верных

‹ Предыдущий урок
Следующий урок ›

3/8 Виртуализация и изолированность. Virtualenv

текущий урок

При обсуждении воспроизводимости мы упоминали, что одним из важнейших её требований является **настройка окружения**. Разные версии библиотек в разных проектах могут конфликтовать друг с другом, и становится сложно поддерживать работоспособность всех сервисов. Поэтому необходимо обеспечить не только одинаковые операционные системы, но и установку всех нужных библиотек и их версий.

SKILLFACTORY

Почти в 100 % случаев при работе над реальным проектом вам потребуется воссоздать точное окружение на сервере, чтобы обеспечить единообразие среды выполнения. Это включает в себя не только стандартные установленные зависимости и интерпретатор Python, но и специфические зависимости, которые могут работать по-разному на разных операционных системах или на разных их версиях.

Несколько лет назад для этих задач использовали программные решения для удалённого управления конфигурациями. Они создавали **большой конфигурационный файл**, в котором описывались все настройки, зависимости и библиотеки, и на его основе настраивались все удалённые машины. Одним из таких инструментов является Ansible. Он позволяет через SSH-соединение «проталкивать» файлы конфигурации на множество машин и таким образом обеспечивать единообразие.

Другие системы, например Chef, обычно поступают наоборот: узлы «тянут» (pull) конфигурацию с главной машины. Chef используется и сейчас, когда необходимо привести парк машин к одинаковой конфигурации.

Проблемы

- Когда в системе много сервисов и при этом каждый сервис запускается на разных машинах, становится тяжело следить за полнотой и правильностью конфигурационных файлов.
- На локальном компьютере разработчика может быть не установлено (или наоборот установлено) ПО, напрямую влияющее на стабильность работы приложения.

Намного проще создать для каждого проекта виртуальную среду, изолированную от основной системы. Тогда каждый из проектов будет иметь свои **независимые настройки**, в том числе разные библиотеки и их версии.

Когда мы говорим про **изолированность**, мы имеем в виду, что приложение будет запускаться в своей отдельной среде и таким образом не зависеть от ОС и настроек системы, на которой мы его разворачиваем.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 1

0 points possible (ungraded)

Для чего предназначен инструмент *Ansible*?

○ Создаёт контейнеры и управляет ими.
○ Создаёт виртуальные машины и управляет ими.
○ Управляет виртуальными средами.
● Помогает управлять конфигурациями удалённых серверов.

✓

Show answer

Отправить

✓ Верных

VIRTUALENV

Python
Virtualenv

Источник изображения

Одним из наиболее популярных инструментов для создания изолированных сред в Python является virtualenv. Он обеспечивает работоспособность сервисов вне зависимости от того, какие они имеют зависимости.

Примечание. Сначала мы будем изучать основы работы с виртуальными окружениями на «игрушечных» примерах, а затем рассмотрим, как настроить виртуальное окружение для нашего веб-сервиса.

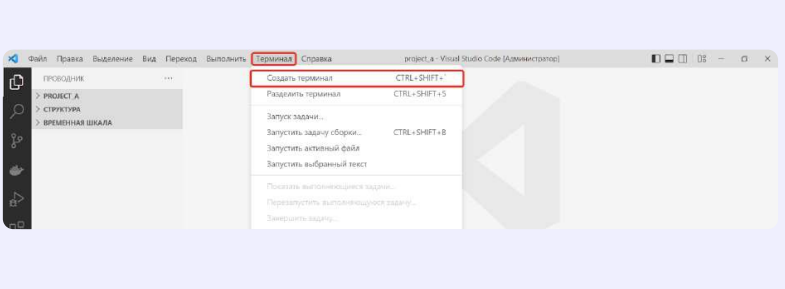
Предварительно создайте в своей операционной системе папку с именем project_a. Эта папка будет имитировать папку проекта, и на её основе мы продемонстрируем работу виртуальных окружений. Перейдите в созданную папку в терминале (команда `cd /путь/до/папки`) или откройте её в вашей IDE.

Чтобы установить virtualenv, необходимо воспользоваться стандартной командой:

```
$ pip install virtualenv
```

Примечание. Здесь и далее символ $ будет означать, что команды выполняются в терминале (для Windows — в командной строке). **Вводить его не нужно** — вводите только текст команды!

Напомним, что в VS Code терминал (в Windows — командную строку) можно открыть с помощью кнопки в верхнем меню IDE:



Так как принципы работы с инструментом незначительно, но отличаются для различных UNIX-систем (Linux и MacOS) и Windows, мы приведём инструкции по работе для каждой из ОС.

Примечание. Чтобы показать, что команды в терминале выполняются в виртуальном окружении, мы будем перед символом `$` писать в скобках имя виртуального окружения, в котором происходит работа. Например:

```
(project_a_venv) $ pip install pandas
```

UNIX Windows

Чтобы создать новую среду, необходимо набрать в терминале команду:

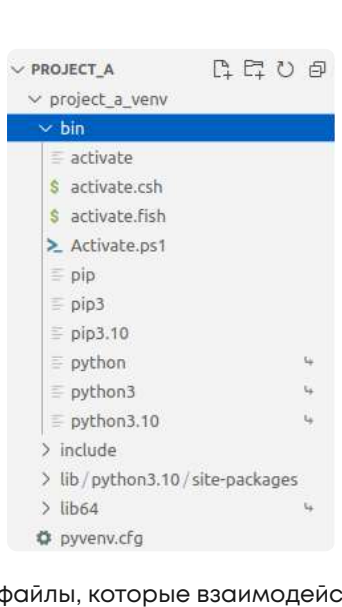
```
$ python3 -m venv <название сервиса>
```

Например, следующая команда создаёт виртуальное окружение с именем project_a_venv:

```
$ python3 -m venv project_a_venv
```

После этого в вашей текущей директории появится папка проекта с именем, которое вы указали в команде `venv`.

В UNIX-системах эта директория будет выглядеть примерно следующим образом (имена файлов могут незначительно отличаться в зависимости от версии Python, которую вы используете):



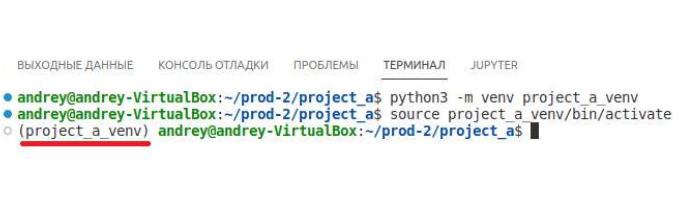
В директории bin лежат файлы, которые взаимодействуют с виртуальной средой, а в lib и lib64 содержится копия версии Python и все зависимости (библиотеки и их версии).

Чтобы активировать виртуальную среду и зайти в неё в UNIX-системах, необходимо запустить в терминале команду:

```
$ source project_a_venv/bin/activate
```

Если команда выполнилась успешно, вы увидите, что перед приглашением в командной строке появилась дополнительная надпись, совпадающая с именем виртуального окружения.

В результате выполнения команды мы увидим следующее:



```
andrey@andrey-VirtualBox:~/prod-2/project_a$ python3 -m venv project_a_venv
andrey@andrey-VirtualBox:~/prod-2/project_a$ source project_a_venv/bin/activate
(project_a_venv) andrey@andrey-VirtualBox:~/prod-2/project_a$
```

Выделенная красным строка говорит нам, что мы находимся в изолированном окружении project_a_venv.

Таким образом вы полностью изолируете окружение своего проекта и можете установить все необходимые для работы проекта версии пакетов — эти версии не будут отражены в глобальном окружении Python и будут зафиксированы только в активированном виртуальном окружении.

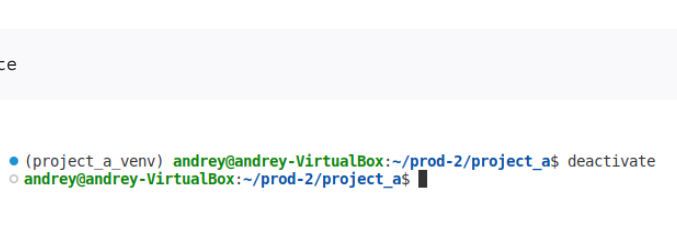
Например, выполним команду для установки библиотеки scikit-learn:

```
(project_a_venv) $ pip install scikit-learn
```

После установки библиотеки вы увидите, что в папке lib/python3.X/site-packages (X — ваша версия Python) появится папка scikit-learn, а также зависимости, необходимые для работы этой библиотеки (например, numpy, scipy и joblib) — они устанавливаются вместе с ней автоматически.

Чтобы выйти из виртуального окружения в область глобального окружения, необходимо ввести в терминале команду:

```
$ deactivate
```



```
(project_a_venv) andrey@andrey-VirtualBox:~/prod-2/project_a$ deactivate
andrey@andrey-VirtualBox:~/prod-2/project_a$
```

Задание 2

0 points possible (ungraded)

Какой командой можно активировать виртуальное окружение *virtualenv*?

○ pip install virtualenv
● source <название_окружения>/bin/activate — для *UNIX* или <название_окружения>/Scripts/Activate.ps1 — для *Windows*
○ python3 -m venv <название окружения> — для *UNIX* или python -m venv <название окружения> — для *Windows*
○ virtualenv -p
○ deactivate

✓

Show answer

Отправить

✓ Верных

Задание 3

0 points possible (ungraded)

Какой командой можно деактивировать виртуальное окружение *virtualenv*?

○ pip install virtualenv
○ source <название_окружения>/bin/activate — для *UNIX* или <название_окружения>/Scripts/Activate.ps1 — для *Windows*
○ python3 -m venv <название_окружения> — для *UNIX* или python -m venv <название_окружения> — для *Windows*
○ virtualenv -p
● deactivate

✓

Show answer

Отправить

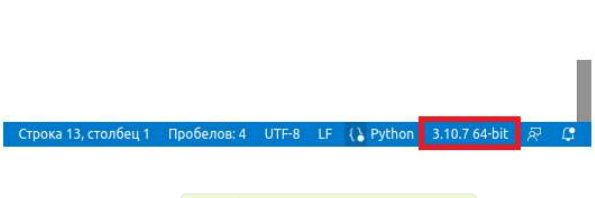
✓ Верных

VIRTUALENV и VS CODE

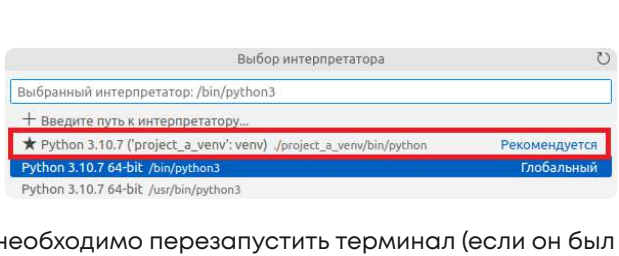
Если вы разрабатываете свои программы в IDE, например в VS Code, то перемещаться между виртуальными окружениями становится совсем просто.

Давайте предварительно создадим в папке нашего проекта пустой py-файл, чтобы VS Code понял, что мы работаем с языком Python. Назовём этот файл app.py.

Чтобы переключиться между окружениями в VS Code, необходимо перейти в раздел выбора интерпретатора Python (правый нижний угол):

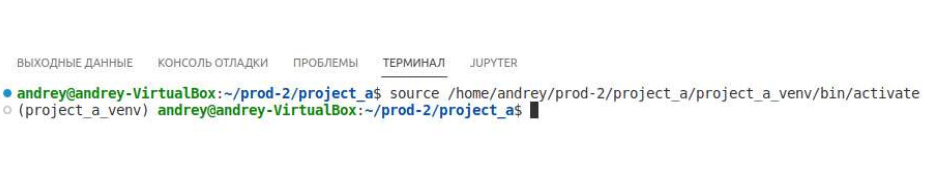


По умолчанию используется `глобальное окружение`. Нам нужно переключиться на только что созданное виртуальное окружение:


Выбор интерпретатора
Выбранный интерпретатор: /bin/python3
+ Введите путь к интерпретатору...
★ Python 3.10.7 ('project_a_venv': venv) ./project_a_venv/bin/python
Рекомендуется
Python 3.10.7 64-bit /bin/python3
Глобальный
Python 3.10.7 64-bit /usr/bin/python3

После этого необходимо перезапустить терминал (если он был открыт).

Результат будет тем же, что и после активации виртуального окружения через командную строку:



```
andrey@andrey-VirtualBox:~/prod-2/project_a$ source /home/andrey/prod-2/project_a/project_a_venv/bin/activate
(project_a_venv) andrey@andrey-VirtualBox:~/prod-2/project_a$
```

Примечание. Если вы устанавливали среду Anaconda, то вашим окружением по умолчанию будет среда base. В пакет Anaconda входит инструмент conda, который одновременно является и менеджером пакетов (как pip), и менеджером среды (как venv). С его помощью также можно создавать виртуальные окружения и управлять ими. Подробнее о conda и её различиях с pip и venv вы можете узнать здесь.

Изоляция зависимостей

Теперь давайте на примере рассмотрим, как работать с виртуальными окружениями.

Рядом с папкой project_a создайте ещё одну папку проекта и назовите её project_b. В этой папке также создайте пустой файл app.py.

Откройте два терминала: в первом перейдите в папку project_a, а во втором — в project_b. Также можно открыть эти папки в двух окнах VS Code.

В папке с проектом A создадим виртуальное окружение с именем project_a_venv, активируем его и установим scikit-learn (если вы не делали этого ранее):

UNIX Windows

```
$ python3 -m venv project_a_venv
$ source project_a_venv/bin/activate
(project_a_venv)$ pip install -q scikit-learn
```

Примечание. Ключ `-q` предназначен для установки без вывода справочной информации — «тихая» установка (от англ. quiet — тихий).

Затем создадим виртуальное окружение в папке project_b с именем project_b_venv, активируем его и установим пакет pandas.

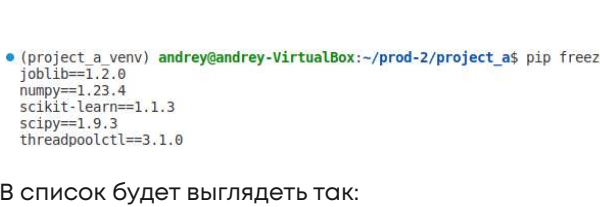
UNIX Windows

```
$ python3 -m venv project_b_venv
$ source project_b_venv/bin/activate
(project_b_venv)$ pip install -q pandas
```

Давайте посмотрим, какие библиотеки доступны внутри каждого из окружений. Для этого воспользуемся командой pip freeze, которая выводит список установленных пакетов с указанием номера их версии. Выполните в каждом из окружений команду:

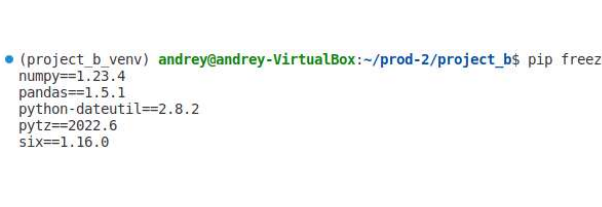
```
(project_{}_venv)$ pip freeze
```

Для проекта A мы увидим примерно следующую картину:



```
(project_a_venv) andrey@andrey-VirtualBox:~/prod-2/project_a$ pip freeze
joblib==1.2.0
numpy==1.23.4
scikit-learn==1.1.3
scipy==1.9.3
threadpoolctl==3.1.0
```

Для проекта B список будет выглядеть так:



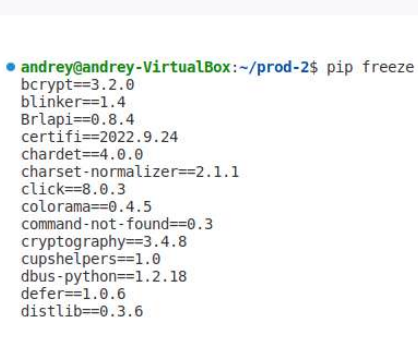
```
(project_b_venv) andrey@andrey-VirtualBox:~/prod-2/project_b$ pip freeze
numpy==1.23.4
pandas==1.5.1
python-dateutil==2.8.2
pytz==2022.6
six==1.16.0
```

Примечание. На скриншотах приведены результаты работы команд только в UNIX-системах, так как они совпадают с результатами в Windows.

Что мы видим? Списки установленных пакетов отличаются: например, библиотеки scikit-learn нет в виртуальном окружении проекта B, а библиотеки pandas нет в виртуальном окружении проекта A. Списки пакетов для проектов A и B пересекаются только в одном — библиотеке numpy. Так происходит потому, что и scikit-learn, и pandas требуют для своей стандартной работы пакет numpy.

Теперь давайте взглянём на список глобально установленных пакетов. Для этого откроем ещё один терминал и, не активируя никаких окружений, напишем в нём команду для вывода списка установленных пакетов:

```
$ pip freeze
```



```
andrey@andrey-VirtualBox:~/prod-2$ pip freeze
bcrypt==3.2.0
blinker==1.4
Brlapi==0.8.4
certifi==2022.9.24
chardet==4.0.0
charset-normalizer==2.1.1
click==8.0.3
colorama==0.4.5
command-not-found==0.3
cryptography==3.4.8
cupshelpers==1.0
dbus-python==1.2.18
defer==1.0.6
distlib==0.3.6
```

В глобальном окружении находится совершенно другой список зависимостей.

Примечание. Пакеты в глобальном окружении на вашем компьютере могут отличаться от приведённых на скриншоте.

Только что мы посмотрели на пример **изоляции**: мы создали отдельные виртуальные окружения для каждого из проектов, зависимости которых изолированы друг от друга. Из приведённого примера становится интуитивно понятно, что виртуальные окружения существуют независимо от глобального и установка пакета в одно из них не означает, что пакет будет установлен в какое-то другое окружение, и наоборот.

Например, если мы установим в глобальное окружение библиотеку numpy определённой версии, например:

```
pip install numpy==1.26.4
```

то версии numpy в виртуальных окружениях проектов A и B не изменятся.

Аналогично, если мы обновим numpy внутри одного из виртуальных окружений, это никак не повлияет на глобальное окружение или другие виртуальные среды.

Такой механизм позволяет работать с проектами A и B независимо друг от друга и даже независимо от глобального окружения, гибко управляя версиями библиотек и предотвращая конфликты зависимостей.

Примечание. Очевидно, что попытка запустить в окружении код, использующий библиотеку, которая в нём не установлена, приведёт к ошибке. Например, если попробовать импортировать библиотеку pandas в файле app.py проекта A,

Файл project_a/app.py:

```
import pandas as pd

df = pd.DataFrame({'col1': [1, 2], 'col2': [3, 4]})
print(df)
```

а после запустить этот файл из-под соответствующего виртуального окружения, мы получим ошибку импорта:

```
(project_a_venv) $ python3 app.py

Traceback (most recent call last):
  File "/home/andrey/prod-2/project_a/app.py", line 1, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
```

Поэтому стоит иметь в виду, что для каждого создаваемого виртуального окружения необходимо отдельно установить все зависимости, которые не входят в стандартную библиотеку Python.

Однако это ещё не всё. Когда разработка проекта завершена и мы готовы загрузить его на свой GitHub и поделиться им с коллегами, мы можем сохранить все те версии библиотек, которые использовали при разработке, в файл. Для этого применяется всё та же команда `pip freeze`, только с указанием имени файла, в который необходимо произвести запись. Традиционно такой файл называют requirements.txt и располагают в корневой директории проекта. Для указания файла используется ключ `-r` или оператор `>`:

```
pip freeze > requirements.txt
```

или

```
(project_a_venv) $ pip freeze > requirements.txt
```

В результате создаётся текстовый файл requirements.txt, который вы можете поместить в свой репозиторий на GitHub.

Когда коллеги будут клонировать ваш проект, им не нужно будет разбираться, какие библиотеки вы использовали в проекте, а тем более — каких версий. Они смогут создать на своём компьютере собственное виртуальное окружение и установить в него все необходимые зависимости, используя лишь одну команду:

```
(имя_виртуального_окружения) pip install -r requirements.txt
```

Удобно, не правда ли?

Теперь давайте заглянем «под капот» и поймём, как именно работает изоляция.

Как работает изоляция

Чтобы понять, как работает изоляция, давайте проверим, где располагаются исполняемые файлы Python в случае глобального и виртуального окружений.

Для начала посмотрим на глобальное окружение. С деактивированной средой запускаем команду:

UNIX Windows

```
$ which python3
/usr/bin/python3
```

Данная команда выводит расположение скрипта, который выполняется при вызове команды `python3`.

Теперь попробуйте запустить ту же самую команду, но уже находясь в одном из виртуальных окружений, например в виртуальном окружении проекта A:

UNIX Windows

```
(project_a_venv) $ which python3
…/project_a/project_a_venv/bin/python3
```

Видно, что, активировав среду, мы получаем другой путь к интерпретатору Python. Но как подключение библиотек зависит от расположения интерпретатора? Напрямую.

Мы не будем подробно останавливаться на том, как происходит внутреннее управление путями в операционных системах, так как это довольно обширная тема и её изложение будет уникально для каждого типа операционных систем. Если говорить вкратце, при активации виртуального окружения вы меняете специальные переменные среды ОС, в результате чего при запуске команды вызова интерпретатора используете не глобальную версию интерпретатора Python, который имеет доступ ко всем установленным пакетам, а его копию, которая лежит в папке с виртуальным окружением. Этот интерпретатор не имеет доступа к папке, где хранятся библиотеки, установленные в глобальное или любое другое виртуальное окружение — у него она своя. Это же правило работает и в обратную сторону.

Примечание. Подробнее о том, как меняются переменные среды, можно узнать в этой статье.

Причины использования виртуальных окружений

У вас мог возникнуть вопрос: зачем это нужно? Зачем так сложно? Раньше мы спокойно работали в глобальном окружении и даже не знали, что оно глобальное, а сейчас для каждого проекта придётся создавать отдельное виртуальное окружение?

Если вы хотите стать профессиональными разработчиками, работать в команде и обеспечивать воспроизводимость библиотек с минимальной затратой времени, то да — создавать виртуальные окружения вам придётся чуть ли не в каждом проекте.

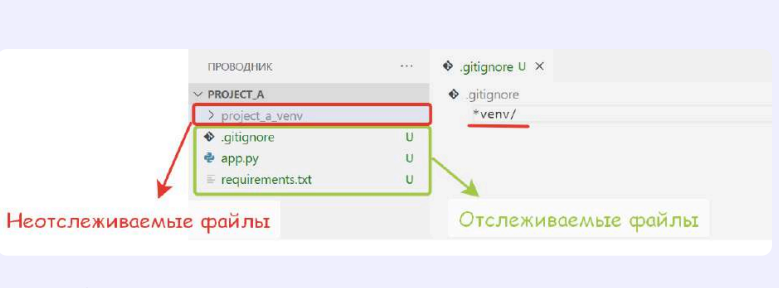
Представим следующую ситуацию: у вас есть два проекта — проект A и проект B. Они оба имеют зависимость от одной и той же библиотеки. Проблема становится явной, когда мы начинаем запрашивать разные версии этой библиотеки. Например, может случиться так, что проект A запрашивает версию 1.0.0, а проект B — версию 2.0.0, причём версия 2.0.0 настолько сильно отличается от 1.0.0, что для адаптации проекта A под новую версию придётся его полностью переписывать. Это большая проблема для Python, ведь работая только в глобальном окружении, мы не можем использовать обе версии библиотеки. Так или иначе, возникнет конфликт, который могут решить виртуальные окружения.

Вторая причина использования виртуальных окружений — удобная коммуникация внутри команды. Разрабатывая проект в виртуальном окружении, мы можем сохранить **только те зависимости и их версии**, которые использовали в проекте, например в файл requirements.txt.

Затем мы можем передать разработку своим коллегам (например, через GitHub), и они смогут установить только те зависимости, которые необходимы для работы нашего кода.

Примечание. Важно отметить, что папку самого виртуального окружения в GitHub помещать не нужно. Всегда добавляйте эту папку в файл .gitignore.

Например, мы можем прописать в файле .gitignore строку `*venv/`, которая будет означать, что всё содержимое папок, названия которых оканчиваются на `venv`, будет игнорироваться при коммитах:



Подробнее о том, как сочетать использование виртуальных окружений с .gitignore, вы можете почитать здесь*.

* Если какие-то ссылки не открываются, обратитесь к координатору.

Виртуальное окружение для FLASK-приложения

Пришло время создать виртуальное окружение для нашего веб-сервиса.

Предварительно перейдите в каталог, где расположен код вашего проекта из предыдущего модуля.

В корневой директории проекта (у нас она называется web) создадим виртуальное окружение с именем project_venv.

UNIX Windows

```
$ python3 -m venv project_venv
```

Локальные копии Python и pip будут установлены в каталог project_venv в каталоге вашего проекта.

Активируем виртуальное окружение:

UNIX Windows

```
$ source project_venv/bin/activate
```

Теперь мы можем переходить к установке пакетов в наше окружение. Сначала установим wheel с локальным экземпляром pip, чтобы убедиться, что наши пакеты будут устанавливаться даже при отсутствии архивов wheel (актуально для старых версий pip):

```
(project_venv) $ pip install wheel
```

Затем установим Flask, requests и scikit-learn. Чтобы установить несколько пакетов сразу, можно просто перечислить их через пробел после команды `install`.

```
(project_venv) $ pip install flask requests scikit-learn
```

Затем запустим наш сервер:

UNIX Windows

```
(project_venv) $ python3 ./server.py
```

Давайте проверим, что мы установили все необходимые для работы веб-сервиса и его тестирования зависимости. Через браузерную строку зайдите по адресу http://localhost:5000 или http://127.0.0.1:5000. Там должно быть выведено сообщение, что ваш сервер запущен.

Также попробуйте отправить POST-запрос на ваш сервер, выполнив клиентский скрипт в соседнем терминале.

UNIX Windows

```
(project_venv) $ python3 ./client.py
```

В результате работы скрипта должно быть выведено сообщение о статусе обработки запроса (он должен быть равен 200) и предсказание модели для отправленных данных.

Если всё работает корректно, GET- и POST-запросы отработали без ошибок, то мы можем зафиксировать версии наших зависимостей и поместить их в файл requirements.txt в корневой директории проекта:

```
(project_venv) $ pip freeze > requirements.txt
```

В нашей директории появится файл requirements.txt — он ещё пригодится в следующих уроках.

Теперь, если мы загрузим наш код на GitHub (предварительно добавив папку project_venv в файл .gitignore), нашему коллеге Василию необходимо будет склонировать себе репозиторий, а после этого создать виртуальное окружение и активировать его. Чтобы в точности воссоздать все версии зависимостей, которые мы использовали, Василию будет достаточно набрать в терминале следующую команду:

```
pip install -r requirements.txt
```

После этого зависимости внутри виртуального окружения, созданного Василием, будут совпадать с нашими.

Virtualenv — полезный инструмент для изоляции Python-зависимостей. Однако он обеспечивает лишь изоляцию на уровне интерпретатора и пакетов Python и не решает задач системной изоляции. Он не предназначен для ограничения CPU, памяти или сетевых ресурсов, а также для балансировки нагрузки между сервисами — эти задачи решаются средствами контейнеризации или оркестрации.

Ещё одно ограничение виртуальных окружений в том, что они изолируют только зависимости Python, но не всю операционную систему. Если разработка ведётся, например, на Windows или macOS, а запуск происходит на Linux-сервере, могут возникнуть проблемы из-за различий в настройках системы, путях, системных библиотеках и серверном ПО.

Сам Flask работает на разных операционных системах. Однако в реальных проектах приложение обычно запускают через дополнительные серверные инструменты (например, uWSGI или Gunicorn) вместе с NGINX. Некоторые из них корректно работают только в Linux-среде. Поэтому одного виртуального окружения Python недостаточно, чтобы полностью повторить рабочую среду.

Нам нужно решение, которое позволит «упаковать» приложение вместе со всей средой — системой, библиотеками и настройками — и запускать его одинаково на любом устройстве. Для этого используются системы

‹ Предыдущий урок Следующий урок ›

4/8 Контейнеризация. Docker и Docker Hub

текущий урок

В 2012–2013 гг. появилось несколько систем контейнеризации, которые позволяли иметь дополнительную операционную систему в изолированном от основной ОС виде и не использовали много ресурсов.

> **Контейнеризация** — это метод виртуализации, при котором ядро операционной системы поддерживает несколько изолированных экземпляров приложений.

Наиболее популярной системой контейнеризации оказался Docker. Сегодня, если кто-то говорит о контейнерах, скорее всего, имеется в виду именно Docker.

SKILLFACTORY

Docker — это открытая платформа для разработки, доставки и запуска приложений. Docker позволяет отделить приложения от инфраструктуры, благодаря чему можно быстрее разрабатывать и разворачивать программное обеспечение. С его помощью инфраструктурой можно управлять так же, как и самим приложением. С практической точки зрения контейнер Docker можно представить как небольшой виртуальный компьютер для одного приложения. В нём находятся все необходимые компоненты: код программы, библиотеки, зависимости и настройки среды. Благодаря этому приложение можно запускать одинаково на разных компьютерах и серверах. Используя подходы Docker для быстрой сборки, тестирования и развёртывания кода, можно значительно сократить время между написанием кода и его запуском в production. Docker можно установить на разных платформах.

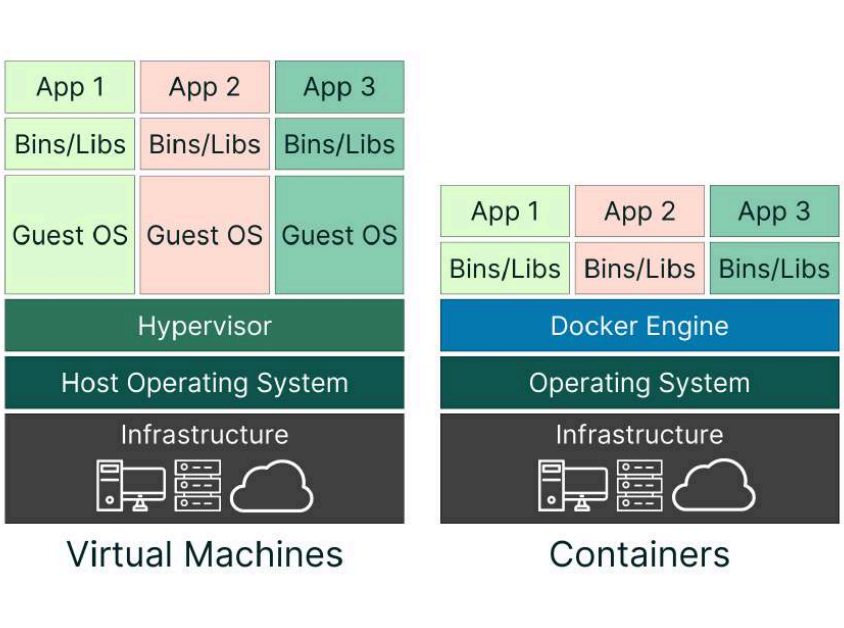
Чтобы лучше понять, как работает Docker и зачем он нужен, полезно сначала разобраться с другой технологией изоляции приложений — виртуальными машинами. Идея запуска программ в изолированной среде появилась задолго до контейнеров и широко используется в IT. Одним из самых известных решений для этого являются виртуальные машины.

Виртуальная машина (Virtual Machine, VM) — это виртуальный компьютер, который работает внутри другого компьютера. Также можно сказать, что это приложение, которое имитирует компьютер со встроенной в него операционной системой.

Виртуальная машина требует выделения ресурсов вашего компьютера (процессор, адаптер, диск) и запускается на изолированном разделе жёсткого диска компьютера-хоста (той системы, в рамках которой запускается ВМ). Проще говоря, виртуальная машина позволяет создать на компьютере ещё один компьютер, который будет использовать его ресурсы, но работать изолированно.

Благодаря технологии виртуальных машин разработчики могут тестировать программное обеспечение в различных окружениях (системах, конфигурациях и т. п.) на одном ПК, не прибегая к дополнительным устройствам.

Подробнее о виртуальных машинах — здесь.



Что изображено на схеме выше?

Докер позволяет собрать приложение со всем его окружением и зависимостями в **контейнер**. В нашем случае приложением может быть модель ML, предсказывающая стоимость авто, его зависимостями — библиотеки sklearn, numpy и pandas, а окружением — ОС, на которой работает приложение.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 1

0 points possible (ungraded)

Как вы думаете, должны ли мы учитывать совместимость зависимостей и ОС?

● Да, должны.

○ Нет, не должны.

✓

Ответ
Верно:
Конечно, должны, ведь внутри ОС есть свои системные зависимости, с которыми могут быть несовместимы зависимости приложения. Например, есть пакеты и инструменты, которые работают только на *Windows*, но не работают на *Linux* или *MacOS*, и наоборот.

Show answer

Отправить

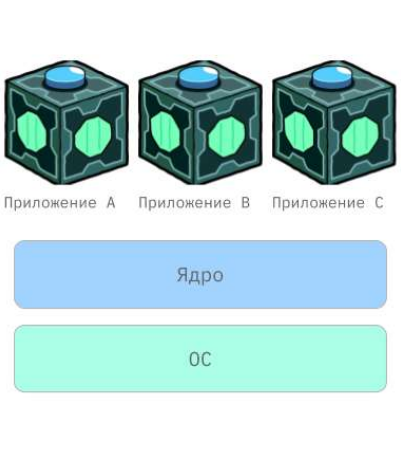
✓ Верных

За счёт того что Docker потребляет не очень много ресурсов машины, на которой находится, можно запускать сразу несколько контейнеров даже на среднестатистическом компьютере. Из-за этого стало принято использовать **небольшие контейнеры** для каждого конкретного сервиса: например, если у нас есть Django-приложение с базой данных, то сам сервер будет находиться в одном контейнере, а база — в другом. Изоляция часто позволяет добиться улучшения производительности и упрощения миграции сервисов.

Источник: developers.redhat.com
У Docker неслучайно такая эмблема и название!

Проще всего это представить, воспользовавшись метафорой кораблей и контейнеров. Если необходимо перевезти несколько типов грузов (продукты, тяжёлые машины и химикаты), то сделать это на одном корабле можно, только используя контейнеры и таким образом изолируя грузы друг от друга. Когда грузы находятся в контейнерах, уже не очень важно, что внутри — оно может быть погружено на корабль.

> Таким образом, контейнеризация — это создание некоторого «ящика» для вашего приложения, в который будут сложены ядро, ОС, библиотеки, ПО и само приложение.



Docker работает с функциями системы Linux и её ядрами, поэтому при работе на других ОС он использует небольшую хитрость: каждый раз в систему устанавливается виртуальная машина с Linux, и Docker работает уже в ней, но мы этого даже не замечаем.

Подведём промежуточный итог

Контейнер Docker можно представить как изолированную файловую систему, в которой находится всё необходимое для запуска приложения: код программы, библиотеки и системные зависимости. При запуске контейнера система использует эту файловую систему и запускает приложение в изолированной среде. При этом контейнер использует ядро операционной системы компьютера, на котором он работает, поэтому он запускается быстрее и требует меньше ресурсов, чем виртуальная машина.

Задание 2

0 points possible (ungraded)

Что включает в себя *Docker*-контейнер с *ML*-моделью?
Отметьте все подходящие варианты ответов.

☐ A Операционную систему

☑ B Приложение — код *ML*-модели

☐ C Данные для обучения *ML*-модели

☑ D Зависимости приложения (необходимые библиотеки)

✓

Ответ
Верно:
B Верно.
D Верно.

Show answer

Отправить

✓ Верных

Задание 3

0 points possible (ungraded)

В каких случаях вам пригодятся *Docker*-контейнеры?
Отметьте все подходящие варианты ответов.

☑ A Ваш коллега, пользователь *Windows*, хочет передать вам код обучения и инференса модели, но вы используете *Ubuntu*.

☑ B Вы с коллегой используете *Ubuntu*, и он хочет передать вам код обучения и инференса модели. При этом коллега вёл разработку на *python 3.8*, а вы уже перешли на *python 3.10*.

☑ C Вы с коллегой используете *Ubuntu*, и он хочет передать вам код обучения и инференса модели. При этом он использует *Ubuntu 20.04*, а вы — *Ubuntu 18.04*.

☐ D Вы с коллегой используете один и тот же удалённый сервер с *Ubuntu 18.04* и *python 3.7*. Он хочет передать вам код обучения и инференса модели, но при этом не хочет делиться подготовленным датасетом для обучения.

☑ E Вы с коллегой используете удалённый сервер с *Ubuntu 18.04* и *python 3.7*. Он хочет передать вам код обучения и инференса модели, при этом он использовал *virtualenv* с sklearn=0.22.2, а вы — *virtualenv* с sklearn=0.23.2.

✓

Ответ
Верно:
A Верно.
B Верно.
C Верно.
E Верно.

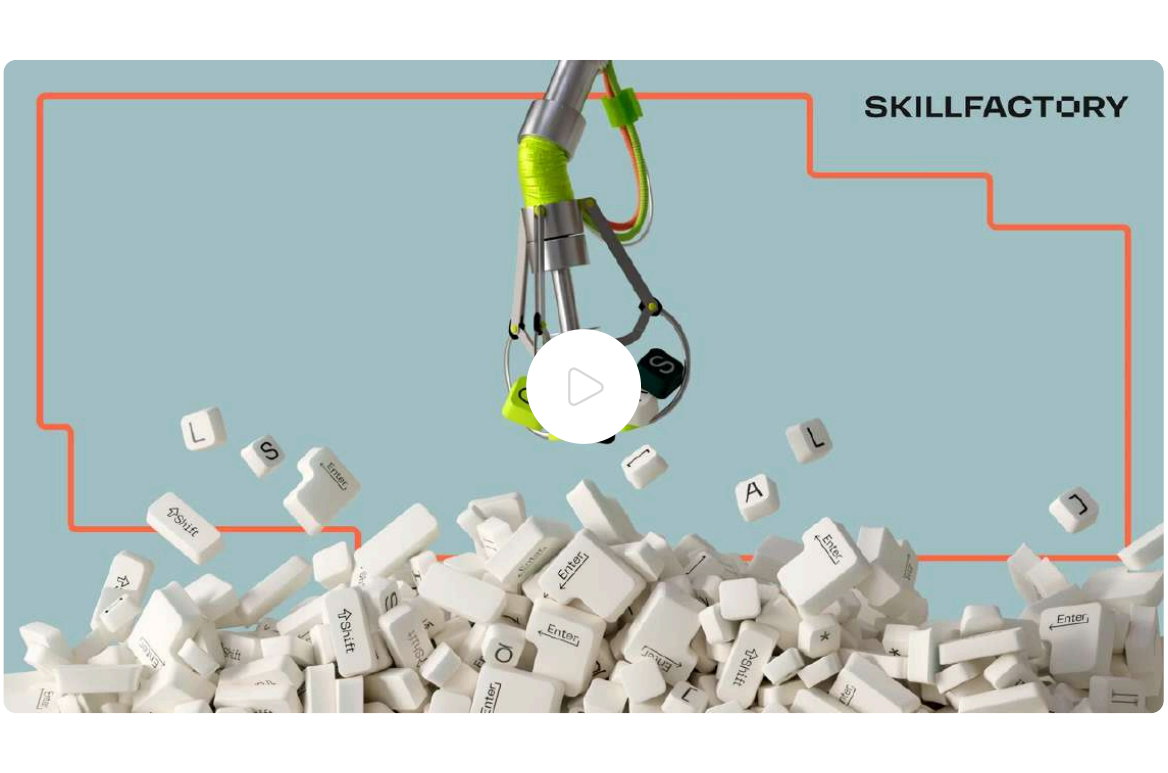
Show answer

Отправить

✓ Верных

Основные компоненты Docker

Поговорим об основных компонентах Docker.

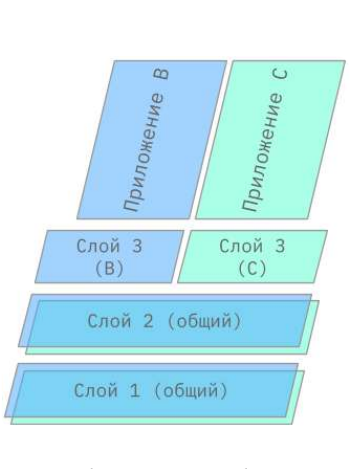


> **Образы (images)** — это основные строительные блоки, на основании которых создаются контейнеры (а в них впоследствии упаковываются приложения).

Образ, по своей сути — это шаблон, своеобразный чертёж или рецепт, в котором содержится образ базовой операционной системы, код приложения и библиотеки.

Запуская на его основе контейнер, мы создаём исполняемый экземпляр, который **инкапсулирует** требуемое программное обеспечение.

Ключевую роль в устройстве образа играет идея о **слоях**.

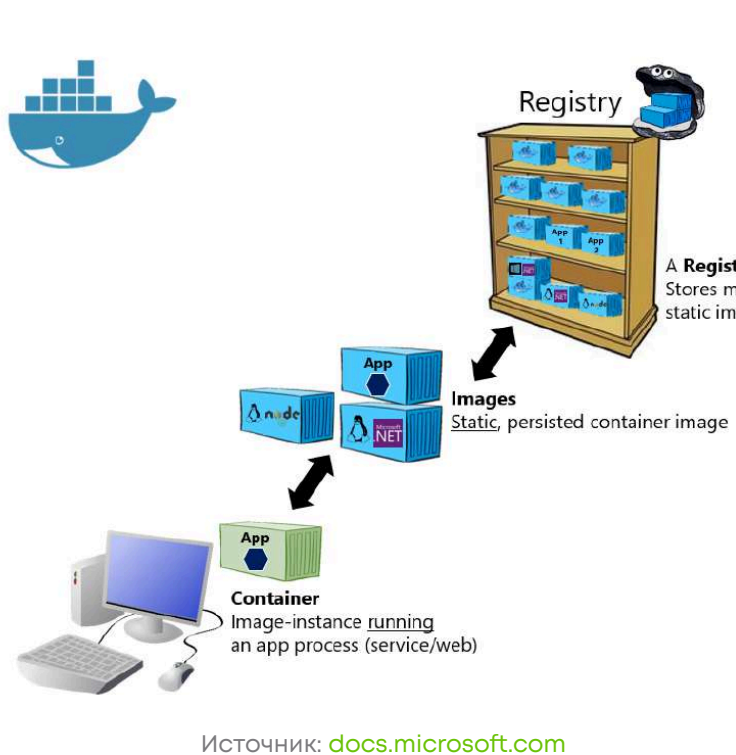


В основе каждого docker лежит базовый образ с операционной системой. С каждым новым слоем в ОС добавляются другие компоненты. Каждый слой представляет из себя подобие diff файловой системы. Например, вы взяли образ системы Ubuntu и поставили туда Python. В таком случае ваш образ будет состоять из двух слоёв — самой ОС и файлов Python поверх неё.

В качестве базового образа для docker можно использовать не только ОС, но и готовые образы с нужными вам компонентами. Кто угодно может добавлять свои образы в **общий регистр**, поэтому в нём очень много готовых образов, доступных для расширения. Таким образом, можно взять готовый образ из публичного репозитория в качестве базового и добавлять в него дополнительные слои. Это делается в соответствии с инструкциями из Dockerfile, которые мы подробно рассмотрим дальше.

Готовые образы хранятся в **Docker Registry**. Они могут быть публичными или приватными: например, официальное публичное хранилище — это Docker Hub. Это аналог GitHub, но если последний используется для хранения кода и работы с Git, то Docker Hub используется для хранения docker-образов и работы с Docker.

В качестве базового можно использовать абсолютно любой образ. Однако стоит помнить о безопасности и удостовериться, что образ не навредит.


Registry
A Registry
Stores many static images
Images
Static, persisted container image
App
Container
Image-instance running an app process (service/web)

Источник: docs.microsoft.com

Основное преимущество Docker заключается в том, что с помощью Docker Registry мы можем делиться образами, а значит, легко переносить созданные нами приложения.

Для работы с Docker Registry достаточно знать две команды (они очень похожи на команды Git) — подробнее о них мы поговорим далее:

1. `docker push` — отправить собранный образ в Docker Registry.
2. `docker pull` — скачать готовый образ из Docker Registry.

Существует множество официальных образов. Например, можно найти образ, где в качестве ОС используется Ubuntu, или, например, Linux с уже установленным Python. Есть даже Docker в Docker! Очень удобно, не правда ли?

Docker-образ управляется Daemon, который отвечает за все действия, связанные с контейнерами, и, конечно, самим клиентом для взаимодействия с ним.

Важно. Зарегистрируйтесь на Docker Hub. Для этого вам понадобится только e-mail. Ваш аккаунт пригодится вам далее при выполнении заданий, а также при работе с самим Docker.

Задание 4

0 points possible (ungraded)

Изучите образ *python* на *Docker Hub*. Выберите версии *Python*, которые на текущий момент не поддерживаются в официальном репозитории *Docker Hub*:

☐ A 2.7

☐ B 3.5

☐ C 3.6

☐ D 3.7

☐ E 3.8

Отправить

Прежде чем перейти к созданию собственного Docker-контейнера, давайте установим сам Docker.

Установка Docker

Docker распространяется в нескольких вариантах. Основой платформы является Docker Engine — открытый движок контейнеризации, который можно использовать бесплатно. Он отвечает за создание, запуск и управление контейнерами, а также за работу с образами, сетями и томами.

Для удобной работы разработчиков на персональных компьютерах используется приложение Docker Desktop. Оно доступно для Windows, macOS и Linux и включает Docker Engine, а также дополнительные инструменты для разработки, настройки и управления контейнерами.

Для организаций Docker предлагает платные тарифы (например, Docker Business), которые предоставляют дополнительные возможности для корпоративной инфраструктуры: управление доступом пользователей, интеграцию с системами аутентификации (SSO), повышенную безопасность образов, централизованное администрирование и другие инструменты.

Для обучения, личных проектов и небольших команд обычно достаточно бесплатных возможностей Docker.

Скачать Docker можно с официальных ресурсов:

- Docker Desktop для Windows, macOS и Linux: docker.com/products/docker-desktop/
- Docker Engine для Linux: docs.docker.com/engine/install/

Хитрость Docker Desktop заключается в том, что для своих контейнеров он запускает Docker на виртуальной машине под операционной системой Linux. То есть, по сути, при сборке контейнеров вы будете использовать ядро ОС Linux, но даже не заметите этого. Благодаря этой особенности ваши контейнеры можно будет запускать на других компьютерах с ОС Linux или ОС, где установлен Docker Desktop.

Мы будем учиться работать с контейнерами, используя инструменты командной строки (терминала) и не прибегая к графическому интерфейсу Docker Desktop. Однако если вы изучите основные концепции работы с консольным Docker, для вас не составит труда разобраться и в графическом интерфейсе этого приложения.

Итак, давайте перейдём к установке. Все её шаги уже описаны в инструкциях с официального сайта — мы лишь приведём их здесь и обратим ваше внимание на некоторые нюансы.

Примечание. Если вы являетесь пользователем Windows, но работаете с дистрибутивами Linux через WSL, то для вас актуальна установка Docker Desktop для Windows с последующей интеграцией Docker в дистрибутив Linux через WSL. Более подробную информацию о работе с Docker через Windows WSL вы можете найти здесь.

Для пользователей Linux

Для дистрибутивов Linux нет необходимости устанавливать Docker Desktop, так как Docker является «родным» для ОС данного типа и все контейнеры запускаются без использования дополнительных виртуальных машин, которые нужны для Windows и MacOS. Кроме того, чаще всего мы работаем с Docker, используя команды в терминале и не прибегая к графическому интерфейсу. В рамках данного курса мы также будем управлять контейнерами только с помощью интерфейса командной строки.

Однако если вы вдруг захотите познакомиться с интерфейсом Docker Desktop и управлять контейнерами с помощью графического интерфейса, то вам повезло: в 2022 году Docker Desktop стал доступен и для Linux (считается, что для Linux ОС этот инструмент избыточен). Ознакомиться с инструкциями по установке и работе с Docker Desktop для Linux вы можете здесь.

Мы же рассмотрим установку «чистого» Docker и его движка для Linux. Сделаем это на примере дистрибутива Ubuntu, следуя этой инструкции. Инструкции по установке на другие дистрибутивы Linux будут отличаться незначительно, вы можете ознакомиться с ними здесь.

Установка будет состоять из ряда этапов (все команды выполняются в терминале):

1. Сначала необходимо выполнить подготовительные шаги.
   - Обновите индекс пакета apt и установите пакеты, чтобы разрешить apt использовать репозиторий через HTTPS:

```
$ sudo apt-get update
$ sudo apt-get install \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

   - Добавьте официальный GPG-ключ Docker:

```
$ sudo mkdir -p /etc/apt/keyrings
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
```

   - Используйте следующую команду для настройки репозитория:

```
$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

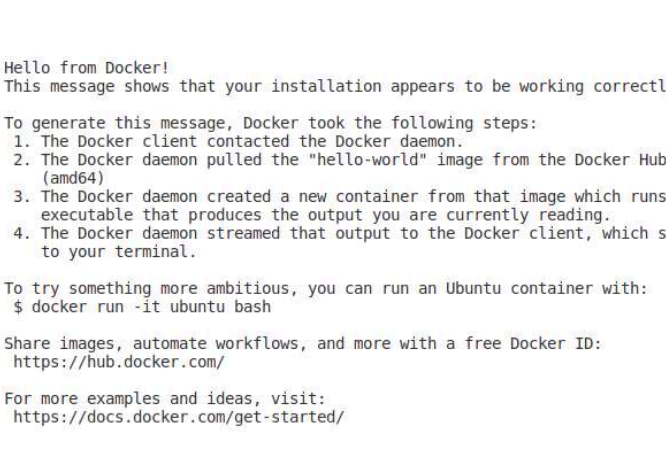
2. Теперь, когда все подготовительные шаги выполнены, мы можем установить движок Docker — Docker Engine. Для этого установите Docker Community Edition (Docker-CE) и ещё несколько компонентов:

```
$ sudo apt-get install docker-ce docker-ce-cli containerd.io docker-
compose-plugin
```

3. Убедимся, что установка Docker прошла успешно, запустив образ hello-world:

```
$ sudo docker run hello-world
```

   Данная команда запускает образ контейнера с именем hello-world. По завершении работы контейнера вы должны увидеть в терминале примерно следующий текст:



```
Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

4. Изначально после установки Docker на Linux ОС команда `docker` может быть запущена только от имени пользователя root или от имени пользователя, указанного в автоматически создаваемой при установке группе пользователей — docker.
   Это означает, что попытка запустить команду `docker` без префикса `sudo` или от имени пользователя, который не находится в группе docker, приведёт к ошибке:

```
Output
docker: Cannot connect to the Docker daemon. Is the docker daemon running
on this host?.
See 'docker run --help'.
```

   Давайте сделаем так, чтобы не нужно было вводить `sudo` каждый раз при запуске Docker. Для этого выполним следующие шаги:

   - Добавьте пользователя, под которым мы сейчас работаем, в список пользователей docker с помощью следующей команды:

```
$ sudo usermod -aG docker ${USER}
```

   - Далее необходимо выйти из системы и войти заново, чтобы изменения вступили в силу. Или вы можете запустить следующую команду, чтобы активировать изменения в группах:

```
$ newgrp docker
```

   - Проверьте, что вы имеете права доступа к docker без указания `sudo`. Для этого запустите образ hello-world:

```
$ docker run hello-world
```

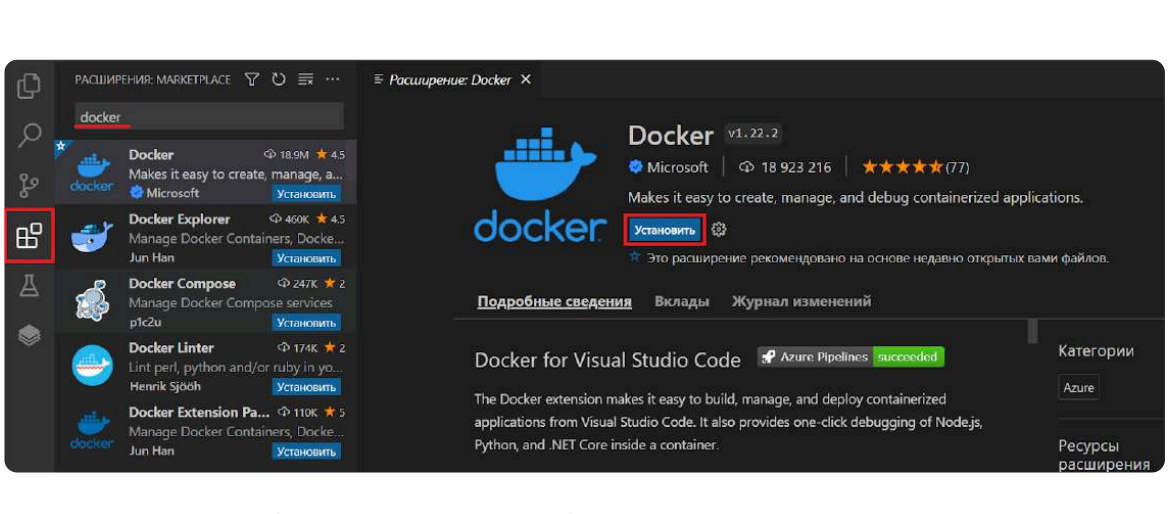
   Вы должны получить тот же результат, что и выше.

Для пользователей Windows 10/11

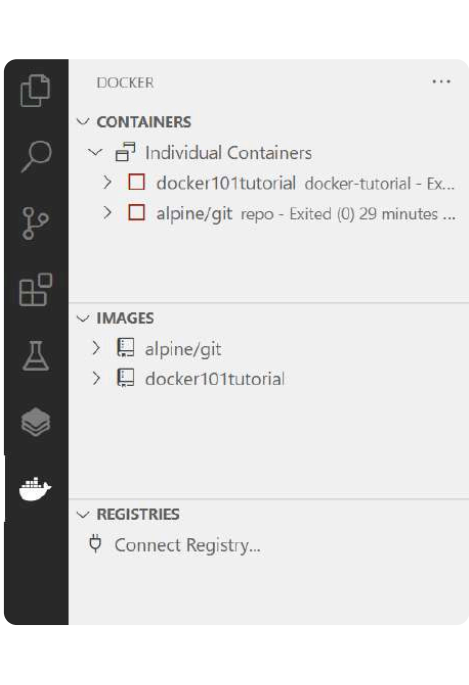
Для пользователей MacOS

Docker в VS Code

В IDE VS Code есть специальные инструменты для удобной работы с контейнерами. Для их подключения к IDE необходимо установить расширение Docker. Однако установка самого Docker также потребуется:



Если процесс Docker запущен, VS Code автоматически подключится к нему и вы сможете управлять вашими уже собранными контейнерами прямо из IDE.



Примечание. Если во вкладке Docker в VS Code возникают ошибки, это означает, что не запущен фоновый процесс Docker.

Чтобы удостовериться, что он работает, откройте скрытые значки на панели инструментов Windows — там должен быть логотип Docker.

При работе с Docker в Windows и MacOS приложение Docker Desktop должно быть всегда запущено.

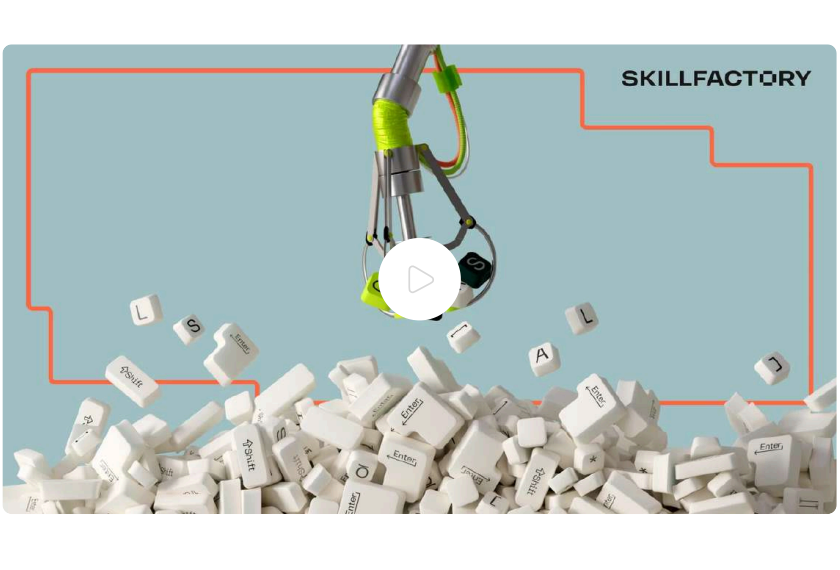
В этом уроке мы рассмотрели основные компоненты Docker и разобрали, как создаётся образ контейнера. Также мы установили Docker на свой компьютер и даже запустили свой первый контейнер. В следующем уроке мы поговорим о том, как создавать собственные образы, познакомимся с

‹ Предыдущий урок   Следующий урок ›

5/8 Создание docker-образов. Dockerfile

текущий урок

Нам уже пора учиться создавать собственные образы, запускать контейнеры и управлять ими. В этом уроке мы рассмотрим основной синтаксис работы с Docker на примере приложения для визуализации данных, а уже в следующем уроке применим технологию контейнеров для нашего веб-сервиса и заставим его работать на любой машине.

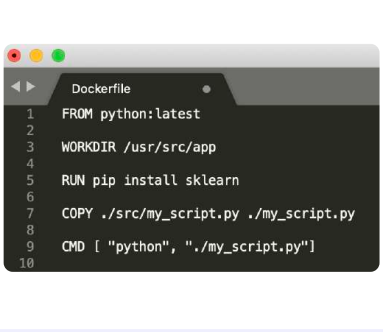


> **Dockerfile** — это специальный файл, в котором содержатся инструкции по сборке контейнера (а точнее, его слоёв): какой тип контейнера и операционную систему использовать, какие дополнительные пакеты установить, какие команды запустить.

Так, например, если нам нужен образ для решения задач машинного обучения, то в Dockerfile мы пропишем инструкцию включить в образ библиотеки sklearn или tensorflow.

Каждая такая команда приводит к созданию отдельного слоя: diff слоя при добавлении файлов будет состоять из файлов, при выполнении команды — из разницы в файловой системе до выполнения и после.

Типичный Dockerfile выглядит примерно так:



```
FROM python:latest

WORKDIR /usr/src/app

RUN pip install sklearn

COPY ./src/my_script.py ./my_script.py

CMD [ "python", "./my_script.py"]
```

Давайте на практике разберёмся, что всё это значит, и соберём свой образ контейнера.

Пишем Dockerfile для приложения визуализации данных

SKILLFACTORY

Внимание! В видео эксперт работает в IDE PyCharm. Вы можете работать в той IDE, к которой привыкли.

Примечание. В видео для построения графиков плотности в файле plot.py эксперт применяет функцию `distplot()` из библиотеки Seaborn. Эта функция будет удалена в версии 0.14.0, и её не рекомендуется использовать. По этой причине в коде, приведённом в тексте модуля, мы заменили функцию `distplot()` на функцию `histplot()` для построения гистограмм с параметром `kde=True` (отображение плотности вероятности).

Также мы обновили версии библиотек до более актуальных (подробнее об этом — ниже, в разделе «Добавляем библиотеки в контейнер»).

Кроме того, в целях сохранения дальнейшей воспроизводимости мы изменили базовый образ на python:3.9 (в видео эксперт использует python:latest).

Мы хотим написать сервис, который создаёт две случайные подвыборки данных из нормальных распределений: первая выборка — с параметрами $\mu = 0$ и $\sigma = 1$ (параметры стандартного нормального распределения), вторая — с параметрами, которые ввёл пользователь.

Для каждой выборки должны строиться и сохраняться в файл plot.png графики плотности распределений. Все файлы с графиками будем помещать в папку output.

Мы хотим запускать это приложение в контейнере и обеспечивать его работу на любом компьютере.

После окончания работы наш проект будет содержать:

- само приложение plot.py;
- папку output, в которой будут храниться результаты работы;
- приложения, то есть графики плотности распределений в формате PNG;
- Dockerfile — описание контейнера, чтобы обеспечить работу приложения на любом компьютере;
- файл requirements.txt — зависимости приложения (библиотеки, которые мы используем).

Примечание. Для простоты изложения в данном проекте не используется виртуальное окружение, но мы уверены, что вы самостоятельно можете его создать и установить в него все необходимые зависимости (numpy, matplotlib, seaborn).

Шаш 1. Пишем приложение

Начнём с написания самого приложения. Пусть сначала исходный код в файле plot.py и папка output будут находиться в корневой директории my_first_container:

```
my_first_container    ├─src
       └─output
       └─plot.py
```

Файл plot.py:

```
import sys
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Создаём бесконечный цикл
while True:
    # Блок обработки исключений на случай неверного ввода
    try:
        # Считываем среднее с клавиатуры
        mean = input('Введите среднее для нормального распределения:')
        # Пытаемся преобразовать введённую строку в число float
        mean = float(mean)
        # Если код преобразования во float отработал, выходим из цикла
        break
    # Если возникло исключение, выводим предупреждение об ошибке
    except:
        print('Это не число!')
# Создаём бесконечный цикл
while True:
    # Блок обработки исключений на случай неверного ввода
    try:
        # Считываем стандартное отклонение с клавиатуры
        deviation = input('Введите стандартное отклонение для нормального 
распределения:')
        # Пытаемся преобразовать введённую строку в число float
        deviation = float(deviation)
        # Если код преобразования во float отработал, выходим из цикла
        break
    # Если возникло исключение, выводим предупреждение об ошибке
    except:
        print('Это не число!')
# Генерируем стандартное нормальное распределение
distribution_n1 = np.random.normal(0,1,1000)
# Генерируем нормальное распределение с параметрами, введёнными пользователем
# Дополнительно умножаем результат на 2
distribution_n2 = np.random.normal(mean,deviation,1000)*2

sns_plot  = sns.histplot(distribution_n1, kde=True, color="orange")
sns_plot  = sns.histplot(distribution_n2, kde=True, color="skyblue")
plt.savefig('/usr/src/app/output/plot.png')

print('Файл успешно сохранен')
```

Чтобы проверить работу приложения, запустите в терминале (находясь в корневой директории my_first_container) команду:

UNIX Windows

```
$ python3 ./plot.py
```

Укажите параметры распределения (помните о том, что стандартное отклонение всегда > 0). В результате выполнения кода в папке src/output/ должен появиться файл plot.png.

Шаг 2. Создаем Docker-образ

Наше мини-приложение работает. Теперь мы можем завернуть его в контейнер, поэтому следующим шагом будет создание образа контейнера. Для этого создайте (если вы этого ещё не сделали) файл Dockerfile (без расширения) в корне папки, в которой лежит ваше приложение.

Традиционно код самого приложения помещают в папку src (от англ. source — источник), а Dockerfile и файл с зависимостями requirements.txt (о последнем мы поговорим далее) располагают рядом с этой папкой в корневой директории проекта. Тогда директория нашего проекта будет выглядеть следующим образом:

```
my_first_container
      ├─src
         └─output
         └─plot.py
      └─Dockerfile
```

Откройте Dockerfile в любом текстовом редакторе или в привычной IDE.

Переходим к описанию нашего контейнера. Первое, что нужно **обязательно** указать в нашем новом Dockerfile, — какой образ мы берём за основу. Базовый образ будет задавать файловую систему контейнера и многие другие его составные конфигурации.

Давайте загрузим в качестве основы образ Linux с уже установленным Python. Для указания базового образа, на основе которого будет собираться контейнер, используется ключевое слово `FROM`. Итак, вот первая строка нашего Dockerfile:

```
FROM python:3.9
```

Примечание. Здесь `:3.9` в названии базового образа указывает на его версию. В данном случае мы используем версию 3.9 — с ней в нашем коде не возникнет предупреждений. Но вы можете использовать другие версии: например, python:latest означает использование последней доступной версии.

Примечание. В описании образа можно ознакомиться с информацией о том, какие бывают сборки и версии. Например, существуют обычная сборка python:<version> и сборка python:<version>-alpine, созданная на базе Alpine Linux. Последняя весит намного меньше, чем большинство базовых образов дистрибутива (~ 5 МБ). Кроме того, есть сборка python:<version>-slim, которая содержит только минимальные пакеты, необходимые для запуска Python, и не включает в себя стандартные пакеты.

Теперь укажем путь к рабочей папке нашего приложения внутри docker-контейнера (вместо `/usr/src/app` вы можете прописать любой путь, по которому хотите поместить файлы внутри контейнера). Для этого используется ключевое слово `WORKDIR`.

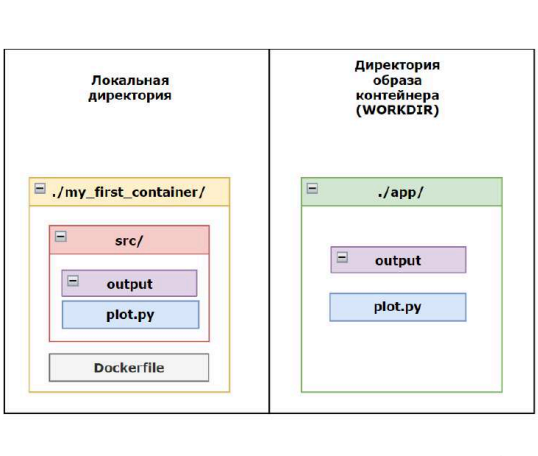
```
WORKDIR /usr/src/app
```

Примечание. Важно понимать, что файловая система контейнера существует отдельно от вашей. Так как образ python, который мы используем в качестве базового, собран на основе Linux, то и файловая система контейнера будет, как в ОС Linux. Ключевое слово `WORKDIR` говорит контейнеру, какой каталог будет использоваться при его запуске. В данном случае мы говорим, что та директория, с которой по умолчанию будет запускаться контейнер, — это `/usr/src/app`.

Далее скопируем в рабочую директорию файлы из корня нашего приложения. Для этого используется ключевое слово `COPY`. В качестве первого аргумента этой директивы указываются папки или файлы, которые мы хотим скопировать с локальной машины, в качестве второго аргумента — куда мы хотим поместить эти копии в контейнере.

```
COPY ./src/ ./
```

Что здесь происходит? Мы указываем, чтобы при создании образа содержимое папки ./src/, в которой находится исходный код нашего приложения, было перемещено в рабочую папку (корневой каталог ./) . В результате при сборке образа контейнера в рабочей директории образа появятся файл plot.py и папка output.



Наконец, напишем команду для запуска скрипта, который будет выполняться вместе с запуском контейнера. Для этого используется директива `CMD` (от англ. command — команда):

```
CMD [ "python", "./plot.py" ]
```

Таким образом, промежуточный Dockerfile будет выглядеть так:

Файл Dockerfile:

```
FROM python:3.9
WORKDIR /usr/src/app
COPY ./src/ .
CMD [ "python", "./plot.py" ]
```

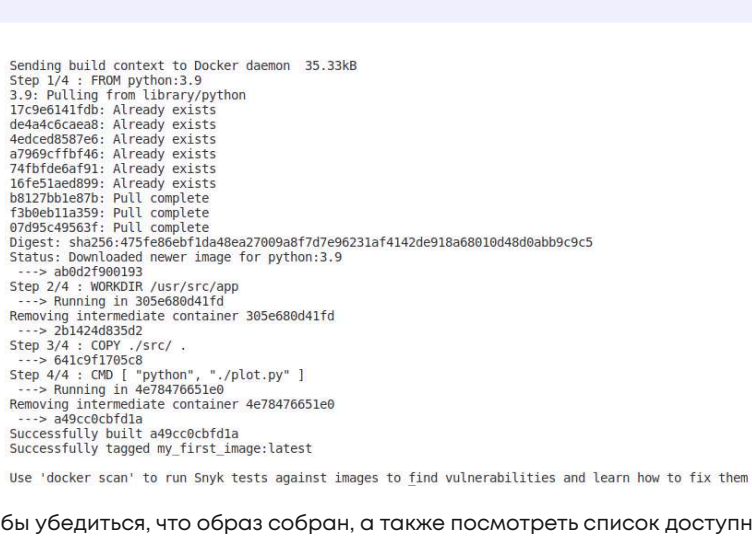
Теперь откроем терминал, перейдём в папку с нашим приложением и запустим команду для сборки контейнера (`docker build`) :

```
$ docker build -t my_first_image .
```

Разберём команду на составляющие:

- `build` сообщает docker, что мы хотим создать образ;
- ключ `-t` указывает на название образа;
- `.` в конце означает, что Dockerfile нужно искать именно в корне. Так как мы запускаем команду из директории my_first_container/, а в ней находится Dockerfile, то Docker автоматически найдёт его. Если ваш Dockerfile находится в директории, отличной от той, в которой вы запускаете команду `docker build`, то вместо " `.` " вам необходимо будет указать путь до него.

Запускаем команду и видим, что образ успешно создан. В справочной информации отражены все наши сборки. Если на каком-то этапе произойдёт ошибка при сборке, docker уведомит вас об этом в терминале.



```
Sending build context to Docker daemon  35.33kB
Step 1/4 : FROM python:3.9
3.9: Pulling from library/python
17c9e6141fdb: Already exists
de4a4c6caea8: Already exists
4edced8587e6: Already exists
a7969cffbf46: Already exists
74fbfde6af91: Already exists
16fe51aed899: Already exists
b8127bb1e87b: Pull complete
f3b0eb11a359: Pull complete
07d95c49563f: Pull complete
Digest: sha256:475fe86ebf1da48ea27009a8f7d7e96231af4142de918a68010d48d0abb9c9c5
Status: Downloaded newer image for python:3.9
 ---> ab0d2f900193
Step 2/4 : WORKDIR /usr/src/app
 ---> Running in 305e680d41fd
Removing intermediate container 305e680d41fd
 ---> 2b1424d835d2
Step 3/4 : COPY ./src/ .
 ---> 641c9f1705c8
Step 4/4 : CMD [ "python", "./plot.py" ]
 ---> Running in 4e78476651e0
Removing intermediate container 4e78476651e0
 ---> a49cc0cbfd1a
Successfully built a49cc0cbfd1a
Successfully tagged my_first_image:latest

Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
```

Чтобы убедиться, что образ собран, а также посмотреть список доступных образов, можно использовать команду:

```
$ docker images
```

Если вы ранее не работали с Docker, скорее всего, сейчас вы получите следующие образы: my_first_image, python (он использовался в качестве базового для my_first_image) и hello-world (мы запускали его на этапе установки Docker).

Команда `docker images` выводит список собранных образов в виде таблицы со следующими столбцами: REPOSITORY (имя образа), TAG (тег образа, в котором обычно указывается его версия), IMAGE ID (идентификатор образа, по которому его можно однозначно найти, — у вас он может отличаться), CREATED (как давно образ был собран), SIZE (размер образа).

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

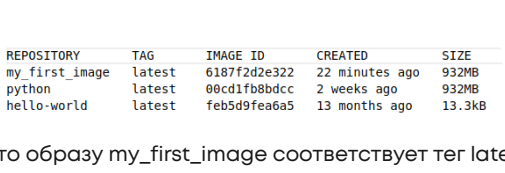
Задание 1

0 points possible (ungraded)

Впишите название тега, который указан напротив образа *my_first_image*:

Отправить

После выполнения команды `docker images` мы увидим на экране примерно следующую таблицу:



```
REPOSITORY       TAG      IMAGE ID       CREATED         SIZE
my_first_image   latest   6187f2d2e322   22 minutes ago  932MB
python           latest   00cd1fb8bdcc   2 weeks ago     932MB
hello-world      latest   feb5d9fea6a5   13 months ago   13.3kB
```

Из неё видно, что образу my_first_image соответствует тег latest.

Примечание. Так как образы занимают место в памяти компьютера, неактуальные образы можно удалять с помощью команды `docker rmi` (от англ. remove image):

```
$ docker rmi <image_id>
```

где `image_id` — идентификатор образа (столбец IMAGE_ID).

Шаг 3. Запускаем docker-контейнер

Попробуем запустить контейнер на основе нашего образа — для этого используется команда `docker run <имя образа>`:

```
$ docker run -it --rm --name=my_first_container my_first_image
```

Расшифруем ключи и аргументы этой команды:

- `-it объединяет команды: -i` оставляет строку для ввода, а `-t` выделяет терминал;
- параметр `--rm` автоматически удаляет контейнер после завершения его работы (в том числе при завершении с ошибкой) — это позволяет не хранить неактивные контейнеры;
- параметр `--name` назначает docker-контейнеру имя (мы задали имя my_first_container).

Но вот незадача — после запуска мы увидим ошибку Python:

```
Traceback (most recent call last):
  File "./plot.py", line 4, in <module>
    import seaborn as sns
ModuleNotFoundError: No module named 'seaborn'
```

Эта ошибка говорит нам, что какие-то библиотеки не установлены. Почему это произошло? Ответ — в самом определении Docker: это инструмент виртуализации. То есть окружение Docker полностью изолировано от нашей операционной системы. Более того, внутри нашего образа и вовсе используется другая операционная система, а это значит, что внутри контейнера нет тех библиотек, которые есть на нашем компьютере.

В таком случае давайте добавим нужные зависимости в Dockerfile.

Шаг 4. Добавляем библиотеки в контейнер

Для этого создадим файл requirements.txt рядом с Dockerfile. Укажем в нём необходимые библиотеки и их версии:

```
my_first_container
      ├─src
         └─output
         └─plot.py
      └─Dockerfile
      └─requirements.txt
```

Файл requirements.txt:

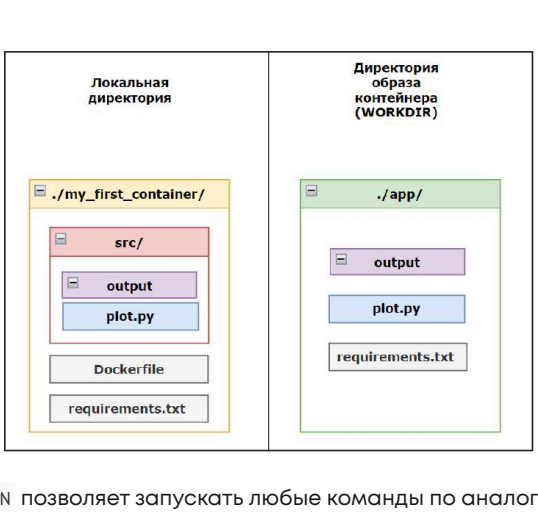
```
numpy == 1.23.4;
matplotlib == 3.6.0;
seaborn == 0.11.2.
```

Теперь необходимо переместить файл с зависимостями в наш контейнер, а после запустить команду для установки зависимостей.

Для этого напишем перед командой `CMD`, запускающей выполнение скрипта в Dockerfile, строки:

```
COPY ./requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
```

Директива `COPY` нам уже знакома: она позволяет копировать файлы из локальной директории в файловую систему контейнера.



Директива `RUN` позволяет запускать любые команды по аналогии с терминалом. Например, поскольку мы работаем на основе базового образа Python, мы можем воспользоваться менеджером пакетов pip для установки зависимостей.

Также для команды `pip install` мы указываем:

- параметр `--no-cache-dir` — позволяет не использовать кэш, а скачать пакеты заново. Вы можете не указывать этот параметр, если уверены, что в дальнейшем версии используемых библиотек не изменятся.
- ключ `-r` — указывает на файлы с зависимостями.

Тогда наш итоговый Dockerfile, на основе которого будет собираться образ контейнера, будет выглядеть следующим образом:

Файл ./Dockerfile:

```
FROM python:3.9
WORKDIR /usr/src/app
COPY ./src/ ./
COPY ./requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
CMD [ "python", "./plot.py" ]
```

Теперь нам необходимо заново собрать образ (сборка займёт некоторое время):

```
$ docker build -t my_first_image .
```

После этого можно запустить контейнер на основе этого образа:

```
$ docker run -it --rm --name=my_first_container my_first_image
```

Примечание. Для того чтобы посмотреть список запущенных контейнеров, можно воспользоваться командой `docker ps`. По умолчанию она выводит список активных контейнеров. Для вывода списка всех контейнеров используйте ключ `-a`, но предварительно перезапустите контейнер без ключа `--rm`, так как данный ключ удаляет контейнеры.

```
$ docker ps -a
```

Запустите команду в соседнем терминале одновременно с контейнером.

Она выводит на экран информацию о контейнерах в виде таблицы со следующими столбцами:

- CONTAINER ID — идентификатор контейнера;
- IMAGE — имя образа, на основе которого запущен контейнер;
- COMMAND — команда, используемая внутри контейнера (то, что мы прописали в директиве `CMD` в Dockerfile);
- CREATED — когда был запущен контейнер;
- STATUS — статус контейнера;
- PORTS — порты, которые использует контейнер (о них поговорим в следующем уроке — сейчас наше приложение не использует веб-интерфейс, и портов у него не будет);
- NAMES — имя контейнера.

Теперь всё работает корректно. После запуска контейнера вас попросят ввести среднее и стандартное отклонение. После исполнения контейнера на экран должна быть выведена фраза `"Файл успешно сохранен"`.

Однако файл plot.png в папке output не обновился с выполнением скрипта в контейнере. Куда же он тогда «успешно сохранён»?

Шаг 5. Синхронизация путей, или куда пропал plot.png

Технически файл plot.png удалился вместе с контейнером после того, как запустился и выполнился скрипт plot.py. Docker не сохраняет файлы внутри контейнера, так как внутри контейнера своя отдельная файловая система.

Что же делать? Ответ очень прост — нам нужно связать контейнер с локальной файловой системой на нашем компьютере.

Для этого укажем параметр `--volume` или ключ `-v` в команде `docker run`.

Ключ `-v` требует указания путей, которые записываются в формате `<путь на локальной машине>:<путь в контейнере>`:

```
$ docker run -it --rm -v $PWD/src/output/:/usr/src/app/output  --
name=my_first_container my_first_image
```

Если всё сделано правильно, после выполнения команды в папке output появится нужный график.

Задание 2

0 points possible (ungraded)

Давайте проверим, что всё работает корректно. Запустите ваш контейнер с помощью приведённой выше команды.

Введите 2.5 как значение среднего и 1 — как значение стандартного отклонения. Посмотрите на полученный график.

На каком значении оси *X* приближённо пересекаются кривые плотностей двух распределений? Ответ округлите до **целого числа**.

Отправить

Задание 3

0 points possible (ungraded)

Сохраняются ли какие-либо файлы внутри контейнера после его удаления?

○ Да, сохраняются по умолчанию.

○ Нет, они безвозвратно теряются.

○ Да, но для этого нужно связать контейнер с локальной файловой системой.

Отправить

Шаг 6. Загружаем образ на Docker Hub

Большой успех Docker во многом обусловлен возможностью делиться образами контейнеров в Docker Hub.

Принцип загрузки образа в Docker Hub очень схож с загрузкой кода на GitHub:

- создаем локальный образ контейнера;
- делаем push образа на Docker Hub.

Пользователь со своей стороны может:

- сделать pull нашего образа;
- запустить образ контейнера на своей локальной машине.

Если мы правильно собрали образ контейнера, то благодаря технологии виртуализации пользователю не нужно знать, как устроено наше приложение, на какой операционной системе оно работает, какие зависимости в нём установлены — ему нужно просто запустить наш образ на своей локальной машине. Это значительно упрощает разработку внутри команды и выведение конечных решений в продакшн.

Рассмотрим механизм загрузки образов в Docker Hub по шагам.

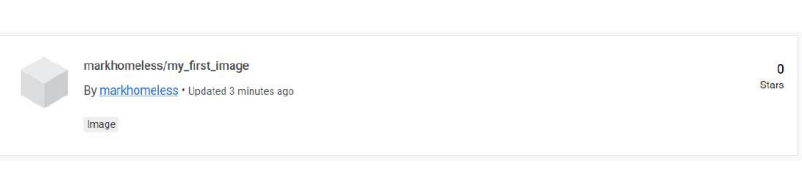
1. Первым делом зарегистрируйтесь на Docker Hub, если не сделали этого ранее.
   Далее необходимо залогиниться под своей учётной записью в самом Docker. Для этого наберите в терминале команду:

   ```
   $ docker login
   ```

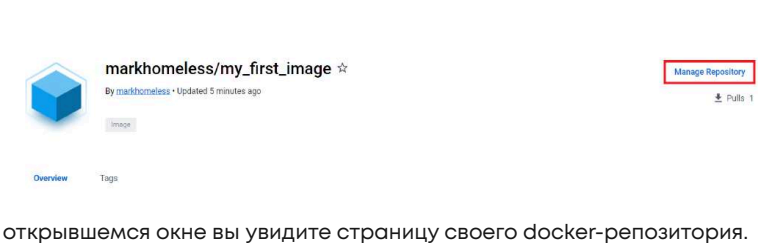
   Если вы ранее залогинились в Docker Desktop, данные учётной записи подгрузятся автоматически. Если вы не устанавливали Docker Desktop и не логинились в нём, Docker попросит вас ввести данные учётной записи на Docker Hub.
2. При отправке образа в Docker Hub необходимо указать имя пользователя как часть имени образа, так как Docker Hub организует репозитории по имени пользователя. Любой репозиторий, созданный под учётной записью, включает имя пользователя в имя образа Docker.
   Поэтому нам необходимо пересобрать наш образ my_first_image, задав ему имя в формате <username>/server_name, где <username> — это ваше имя пользователя в профиле на Docker Hub.

   ```
   $ docker build -t <username>/my_first_image .
   ```
3. Далее делаем push нашего образа на Docker Hub. В аргументах команды необходимо указать имя образа, которым мы хотим поделиться:

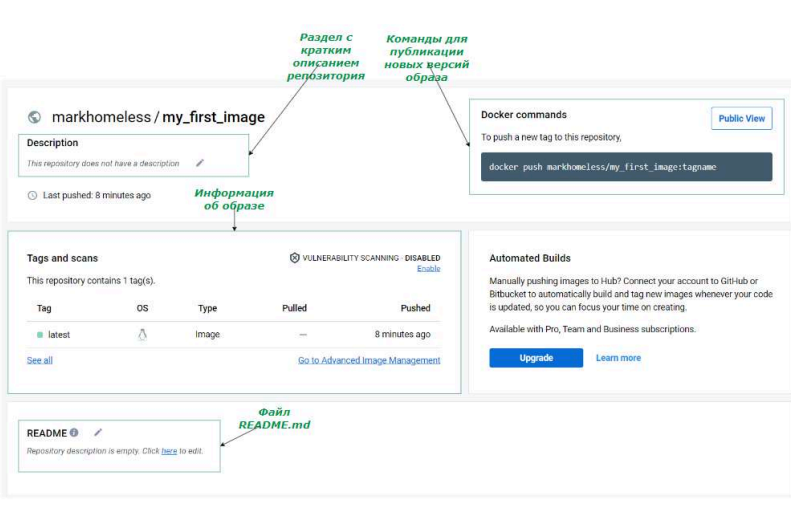
   ```
   $ docker push <username>/my_first_image
   ```
4. Далее откройте ваш профиль на Docker Hub — в нём должен был появиться новый репозиторий с именем <username>/my_first_image:

   
   markhomeless/my_first_image
   By markhomeless • Updated 2 minutes ago
   Image
   0
   Stars

   Вы можете зайти внутрь репозитория и отредактировать его по своему усмотрению. Для этого используется опция Manage Repository:

   В открывшемся окне вы увидите страницу своего docker-репозитория. Здесь вновь всё очень похоже на GitHub: есть раздел с кратким описанием образа (Description), а также файл README.md.

   
   Раздел с кратким описанием репозитория
   Команды для публикации новых версий образа
   markhomeless / my_first_image
   Description
   Информация об образе
   Docker commands
   Public View
   docker push markhomeless/my_first_image:tagname
   Tags and scans
   Automated Builds
   Upgrade
   Learn more
   README
   Файл README.md

   Примечание. При желании в файле README.md вы можете на языке MarkDown описать суть образа, а также команды, которые необходимы для его запуска — всё как с GitHub-репозиториями.

   После того как образ выложен на Docker Hub, любой пользователь может скачать его и воспользоваться им. Для этого используется команда `docker pull`, в аргументах которой указывается имя репозитория на Docker Hub:

   ```
   $ docker pull <username>/my_first_image
   ```

   После этого образ контейнера станет доступным для запуска (это можно отследить с помощью команды `docker images`), и его можно запустить стандартной командой `run`:

   ```
   $ docker run -it --rm -v $PWD/src/output/:/usr/src/app/output  --
   name=my_first_container my_first_image
   ```

Обратите внимание: для запуска образов, которые мы получаем через `pull`, не нужно их собирать, то есть нам не нужен Dockerfile (данный файл по определению предназначен для сборки контейнера). Контейнер уже собран, и нам достаточно запустить его.

На этом все?

Иногда нам может потребоваться задать переменные среды, которые используются для определения констант. Чтобы задать их, в Dockerfile существует инструкция ENV — благодаря ей значение будет доступно в контейнере во время его выполнения.

> **Переменные среды** — так в Linux-подобных системах называются переменные, которые определяются на уровне оболочки и используются различными приложениями во время выполнения. Подробнее о них можно узнать в статье.

Синтаксис ENV можно найти в официальной документации.

В Dockerfile нужно добавить:

```
ENV <key> <value>
```

или

```
ENV <key>=<value> ...
```

Или, к примеру, при запуске контейнера:

```
$ docker run --env <key>=<value>
```

Например, мы хотим, чтобы в нашем контейнере появилась переменная среды `NAME`, в которой будет указываться имя разработчика контейнера. К этой переменной среды можно будет обратиться из любой части контейнера. Тогда в Dockerfile необходимо добавить объявление такой переменной среды:

Файл ./Dockerfile:

```
FROM python:3.9
ENV NAME="Skillfactory"
WORKDIR /usr/src/app
COPY ./src/ ./
COPY ./requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
CMD [ "python", "./plot.py" ]
```

Промежуточный вывод по работе с Docker

В этом уроке мы на примере маленького приложения рассмотрели, как создавать собственные образы, а также основные директивы Dockerfile. Запоминать последние не обязательно — вы всегда можете найти их в документации или в этом гайде. Также мы изучили основные команды, необходимые для работы с Docker.

Перечислим их (и пару дополнительных) ещё раз:

- `docker build` — создать образ контейнера;
- `docker images` — посмотреть список доступных образов;
- `docker rmi` — удалить образ;
- `docker run` — запустить контейнер;
- `docker stop` — остановить контейнер;
- `docker rm` — удалить контейнер;
- `docker ps` — посмотреть список запущенных контейнеров;
- `docker login` - залогиниться в учётной записи Docker Hub;
- `docker push` - отправить образ на Docker Hub;
- `docker pull` - скачать образ с Docker Hub;
- `docker logs` — посмотреть логи;
- `docker kill` — экстренно завершить процесс.

Полезно! У Docker достаточно богатый CLI (Command-Line Interface), который содержит множество команд, помимо тех, что перечислены в модуле. Больше команд и информации о них вы можете увидеть в официальной документации.

Задание 4

0 points possible (ungraded)

Теперь вам предстоит выполнить упражнение с *Docker* самостоятельно.

1. Мы подготовили для вас специальный базовый образ mike0sv/sf_docker_base, в котором зашит секретный код. Чтобы его узнать, вам необходимо создать свой образ на основе базового и добавить в него текстовый файл с фразой I know how to use docker.

2. Создайте переменную среды с названием *(key)* DATA_PATH. Переменная должна содержать добавленный вами путь к файлу *(value)* внутри контейнера (относительный или абсолютный).

3. Запустите контейнер. Если вы всё сделали правильно, он выведет секретный код. Введите его в поле ниже.

Отправить

В следующем уроке мы попрактикуемся в использовании Docker, а также создадим контейнер для нашего веб-сервиса с моделью.

‹ Предыдущий урок   Следующий урок ›

6/8 Создаём образ веб-сервиса

текущий урок

Теперь давайте завернём в контейнер веб-сервис для нашего коллеги Василия, а затем загрузим его на Docker Hub, чтобы этот сервис был доступен для использования и Василий смог развернуть его у себя.

Шаг 1. Заворачиваем Flask-приложение в контейнер

Прежде всего, давайте ещё раз договоримся о расположении файлов в нашей директории. Вынесем файлы с кодом сервера (server.py и папка models) и клиентским приложением (client.py) в папки app и test соответственно. Это нужно для удобства, чтобы не прописывать несколько команд `COPY` в Dockerfile.

```
web
  ├app
     ├models
           └model.pkl
     └server.py
  ├test
     └client.py
  └Dockerfile
  └requirements.txt
```

Примечание. Также в папке web может находиться папка project_venv с конфигурацией вашего виртуального окружения — мы намеренно её опустили, так как она не будет участвовать в создании контейнера. Нам понадобится только файл requirements.txt, который мы создавали ранее.

Момент, который необходимо иметь в виду: когда мы запускаем веб-сервис в контейнере, для тестирования сервера необходимо пользоваться широковещательным адресом (0.0.0.0) — это позволит вам обеспечить доступ к вашему контейнеру. Таким образом, измените в файле server.py следующий код, запускающий сервер:

```
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Теперь, когда мы зафиксировали все вводные, можно переходить к созданию Dockerfile. Это довольно просто, поэтому мы просто приведём описание шагов ниже, а вы с их помощью самостоятельно составите Dockerfile:

1. Оставить те же базовый образ (python:3.9) и рабочую директорию (/usr/src/app).
2. Скопировать локальную папку ./app в рабочую директорию контейнера.
3. Скопировать файл с зависимостями requirements.txt в рабочую директорию проекта.
4. Выполнить команду python ./server.py для запуска сервера.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 1

0 points possible (ungraded)

Заполните недостающие строки *Dockerfile* на основе описанных выше шагов.

```
FROM ___1___
WORKDIR /usr/src/app
COPY ___2___ ___3___
COPY ./requirements.txt ___4___
RUN pip install --no-cache-dir -r requirements.txt
CMD [ "python", "___5___" ]
```

___1___

Выберите опцию ⌄

___2___

Выберите опцию ⌄

___3___

Выберите опцию ⌄

___4___

Выберите опцию ⌄

___5___

Выберите опцию ⌄

Отправить

Задание 2

0 points possible (ungraded)

Создайте образ контейнера на основе получившегося *Dockerfile*. Назовите образ *flask_image*.

В поле для ответа введите команду, которую вы использовали для сборки образа.

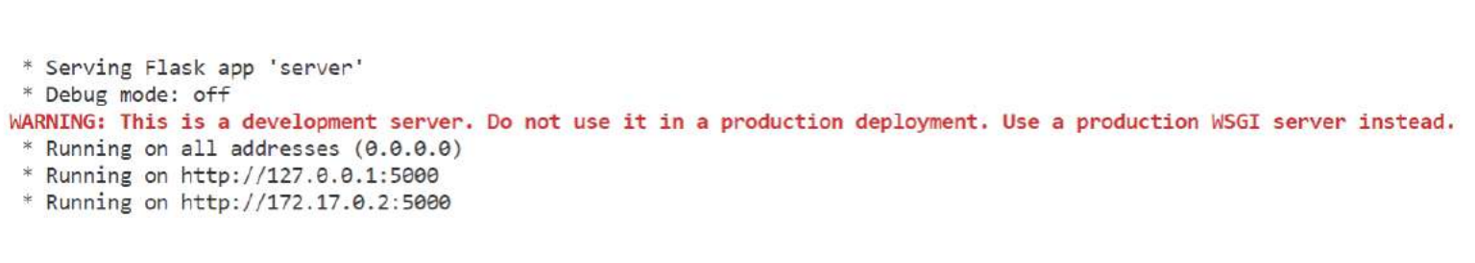
Hint

Отправить

Теперь, когда у нас есть собранный контейнер, нам осталось лишь запустить его. Как мы знаем, это делается с помощью команды `docker run`. Единственное отличие от предыдущего контейнера: так как в нашем контейнере будет работать веб-приложение, нам необходимо открыть внешним приложениям доступ к порту, на котором запущено приложение. Это делается с помощью указания ключа `--publish` или его сокращённой версии `-p`. В аргументе этой команды указывается диапазон из номеров портов контейнера, которые мы хотим сделать публичными. В нашем приложении мы указали, что нам нужен порт 5000, а значит, диапазон будет выглядеть как `5000:5000`. Общий синтаксис команды:

```
$ docker run -it --rm --name=server_container -p=5000:5000 flask_image
```

После запуска контейнера вы должны увидеть примерно следующее сообщение:



```
 * Serving Flask app 'server'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
```

- Первый адрес — localhost (http://127.0.0.1:5000 или http://localhost:5000), к нему мы уже привыкли.
- Второй адрес — адрес вашего контейнера (у вас он может отличаться). По нему контейнеры могут «общаться» друг с другом.

Далее веб-сервер перейдёт в режим прослушивания сети и вы сможете проверить, что он работает, с помощью GET-запроса, обратившись по адресу http://127.0.0.1:5000 через поисковую строку браузера. В браузере должно быть выведено сообщение `"Test message. The server is running"`.

После этого выполните POST-запрос, запустив код клиентского приложения в соседнем терминале:

UNIX Windows

```
$ python3 test/client.py
```

Если в результате выполнения скрипта вернулся код 200 и было выведено сообщение с предсказанием модели, поздравляем — ваше Flask-приложение запущено в контейнере, и вы можете к нему обращаться.

Примечание. Кстати, можно запускать контейнер в фоновом режиме. Для этого добавьте к команде `docker run` ключ `-d` (от англ. detach — отделить):

```
$ docker run -it --rm --name=flask_container -publish=5000:5000
flask_image
```

Итак, наше Flask-приложение запущено, но, как мы знаем из предыдущего модуля, обычного Flask-приложения недостаточно для полноценного веб-сервера. В продакшене используются WSGI-серверы, такие как uWSGI + NGINX (см. предыдущий модуль).

Давайте реализуем связку Flask + uWSGI + NGINX в контейнере.

Примечание. Перед тем как перейти к следующему шагу, остановите контейнер `flask_image` (CTRL + C), так как он будет конфликтовать с контейнером, который мы создадим далее.

Шаг 2. FLASK + UWSGI + NGINX + DOCKER

Если вы проходили урок предыдущего модуля, посвящённый uWSGI и NGINX, вы должны были понять, что настройка этих инструментов — непростая задача, как минимум потому, что запустить uWSGI можно только в UNIX-системах, что ограничивает разработку. К тому же, необходимо прописывать конфигурации серверов uWSGI и NGINX, связывать их работу через сокеты или TCP-порты и многое другое.

Конечно, всё это можно сделать и в контейнере, базовым образом которого является операционная система Linux (например, Ubuntu). Однако в этом случае Dockerfile был бы очень сложным и нам потребовалось бы разбираться в операционной системе внутри контейнера.

К счастью, на Docker Hub выложены образы контейнеров, в которых уже настроена вся необходимая конфигурация uWSGI и NGINX. Всё, что от нас требуется — заменить файл конфигурации uWSGI, который лежит внутри контейнера, на тот, который нужен нам, а также поместить в контейнер исходный код с интерфейсом севера, который будет выполняться сервером uWSGI.

Давайте перепишем наш Dockerfile.

Базовый образ, который мы будем использовать для своего контейнера, — tiangolo/uwsgi-nginx-flask. Это не официальный образ, но, несмотря на это, у него более миллиона скачиваний. Не волнуйтесь — мы проверили образ, он безопасен.

Образ `tiangolo/uwsgi-nginx-flask` создан на базе ОС Linux, и в нём уже настроено взаимодействие серверов uWSGI и NGINX через сокеты. Более подробно о нём вы можете прочитать в документации.

В рабочей директории образа уже лежат файлы некоторого приложения. Давайте проверим, что образ работает — напишем Dockerfile, состоящий только из импорта базового образа (через ":" указывается версия Python, которую мы будем использовать):

Файл ./Dockerfile:

```
FROM tiangolo/uwsgi-nginx-flask:python3.9
```

Соберём образ контейнера и назовём его server_image:

```
$ docker build -t server_image .
```

Примечание. При выполнении этой команды в ОС Linux у вас может возникнуть ошибка примерно следующего содержания:

```
ERROR: UtilConnectToInteropServer:307: connect failed 2
Got permission denied while trying to connect to the Docker daemon
socket at unix:///var/run/docker.sock: Post
"http://%2Fvar%2Frun%2Fdocker.sock/v1.24/build?
buildargs=%7B%7D&cachefrom=%5B%5D&cgroupparent=&cpuperiod=0&cpuquot
a=0&cpusetcpus=&cpusetmems=&cpushares=0&dockerfile=Dockerfile&label
s=%7B%7D&memory=0&memswap=0&networkmode=default&rm=1&shmsize=0&t=se
rver_image&target=&ulimits=null&version=1": dial unix
/var/run/docker.sock: connect: permission denied
```

Она говорит о том, что у контейнера недостаточно прав для доступа к сокетам, через которые общаются uWSGI и NGINX. Для исправления ошибки просто добавьте к команде префикс `sudo`, чтобы она выполнялась от имени главного пользователя root:

```
$ sudo docker build -t server_image .
```

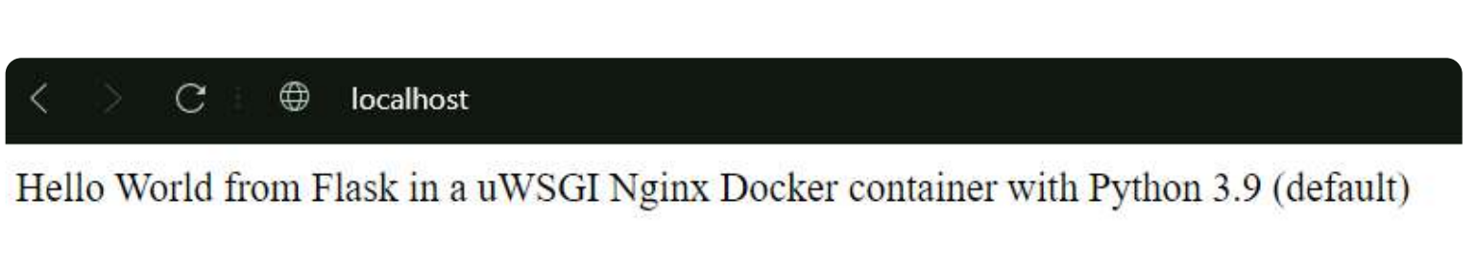
Затем запустим контейнер на порте 80:

```
$ docker run -it --rm --name=server_container -p=80:80 server_image
```

В результате выведется огромное информационное сообщение, в котором описаны все настройки, с которыми был запущен сервер. После этого сервер уйдёт в режим ожидания запросов.

Проверим, что сервер работает. Для этого перейдём по адресу http://localhost или http://127.0.0.1. Обратите внимание, что номер порта указывать необязательно, так как мы запустили сервер на порте 80.

В результате вы должны увидеть в браузере примерно следующее сообщение:



Hello World from Flask in a uWSGI Nginx Docker container with Python 3.9 (default)

Итак, внутри этого образа уже работает какое-то Flask-приложение. Нам лишь нужно заменить его на своё.

Для начала давайте выясним, что представляет собой содержимое этого контейнера. Для этого, не останавливая работу контейнера, в соседнем терминале наберём следующую команду:

```
$ docker exec -it server_container bash
```

После её выполнения мы попадём в файловую систему, окажемся в рабочей директории нашего контейнера и сможем проанализировать его содержимое. Например, выполним команду ls:

```
$ ls
__pycache__  main.py  prestart.sh  uwsgi.ini
```

Примечание. Чтобы посмотреть содержимое файлов, можно воспользоваться командой `cat`. Например, следующая команда позволит вывести содержимое файла main.py:

```
$ cat main.py
```

Что мы видим? Внутри контейнера есть:

- main.py — файл с кодом Flask-приложения;
- uwsgi.ini — файл с настройками uWSGI-сервера;
- prestart.sh — bash-скрипт для запуска;
- __pycache__ — папка с кэшем.

Задание 3

0 points possible (ungraded)

Выведите содержимое файла *uwsgi.ini*, который находится внутри контейнера. Какой модуль указан в конфигурации *uWSGI* (параметр module)? Введите его имя в поле для ответа:

Отправить

Итак, сервер uWSGI в качестве своего интерфейса использует код из файла main.py. Наша задача — заменить файл uwsgi.ini на свой, где в качестве исполняемого приложения будет наш файл server.py.

Давайте создадим в корневом каталоге нашего проекта файл uwsgi.ini. В этом файле мы пропишем, что будем использовать объект app из модуля server.py, количество процессов, обрабатывающих поступающие на сервер запросы, а также добавим master-поток, который будет координировать работу потоков.

Файл ./uwsgi.ini:

```
[uwsgi]

module = server:app
processes = 4
master = true
```

Примечание. Остальные параметры uWSGI, такие как сокет для взаимодействия с NGINX, уже заданы в образе по умолчанию — изменять их не нужно.

Теперь допишем наш Dockerfile.

Файл ./Dockerfile:

```
# Задаём базовый образ
FROM tiangolo/uwsgi-nginx-flask:python3.9
# Копируем содержимое папки ./app в рабочую директорию контейнера
COPY ./app ./
# Копируем файл requirements.txt в рабочую директорию контейнера
COPY ./requirements.txt ./
# Копируем файл uwsgi.ini в рабочую директорию контейнера
COPY ./uwsgi.ini ./
# Запускаем установку необходимых зависимостей
RUN pip install -r requirements.txt
```

Заново соберём наш образ:

```
$ docker build -t server_image .
```

В результате копирования файла uwsgi.ini из локальной директории в файловую систему контейнера мы перезаписали файл uwsgi.ini, который находился в контейнере, на свой вариант, и теперь сервер uWSGI будет работать по нашим правилам.

Давайте убедимся в этом, перезапустив контейнер:

```
$ docker run -it --rm --name=server_container -p=80:80 server_image
```

Теперь в соседнем терминале посмотрим содержимое контейнера:

```
$ docker exec -it server_container bash

$ ls
__pycache__  main.py  models  prestart.sh  requirements.txt  server.py  uwsgi.ini
```

Задание 4

0 points possible (ungraded)

Выведите содержимое файла *uwsgi.ini*, который находится внутри контейнера. Какой модуль указан в конфигурации сервера *uWSGI* (параметр module)? Введите его имя в поле для ответа:

Отправить

Как видите, теперь на сервере для обработки поступающих запросов вместо файла main.py используется файл server.py.

Давайте убедимся, что наш сервер работает. Протестируем GET-запросы: для этого через браузер зайдём по адресу http://localhost или http://127.0.0.1. В результате должно быть выведено сообщение, которое мы прописывали в файле server.py, — `"Test message. The server is running"`. Далее выполним POST-запрос. Помним, что теперь наш сервер работает на порте 80, а не на порте 5000, поэтому в скрипте для отправки запросов изменим URL, по которому происходит отправка запросов на `'http://localhost/predict'`. После внесения изменений запустим код клиентского приложения в соседнем терминале.

UNIX Windows

```
$ python3 test/client.py
```

Если в результате выполнения скрипта вернулся код 200 и было выведено сообщение с предсказанием модели, поздравляем — ваша связка Flask + uWSGI + NGINX работает в контейнере Docker.

Подведём **промежуточный итог**. В результате объединения Flask, uWSGI и NGINX мы получили мощный веб-сервер, способный обрабатывать несколько запросов одновременно, обладающий высокой пропускной способностью, но, что самое важное — этот сервер функционирует в контейнере, а значит, мы легко можем переносить его с одной машины на другую, не беспокоясь о вопросах воспроизводимости и совместимости операционных систем. Для этого нам достаточно разместить результирующий образ на Docker Hub, а затем с любой машины, на которой установлен Docker и есть интернет, можно скачать этот образ, собрать из него контейнер и запустить его.

Давайте разметим наш образ на Docker Hub.

Шаг 3. Делаем Push веб-сервиса на Docker Hub

Залогиньтесь в учётной записи Docker, если вы не сделали этого ранее, с помощью команды `docker login`.

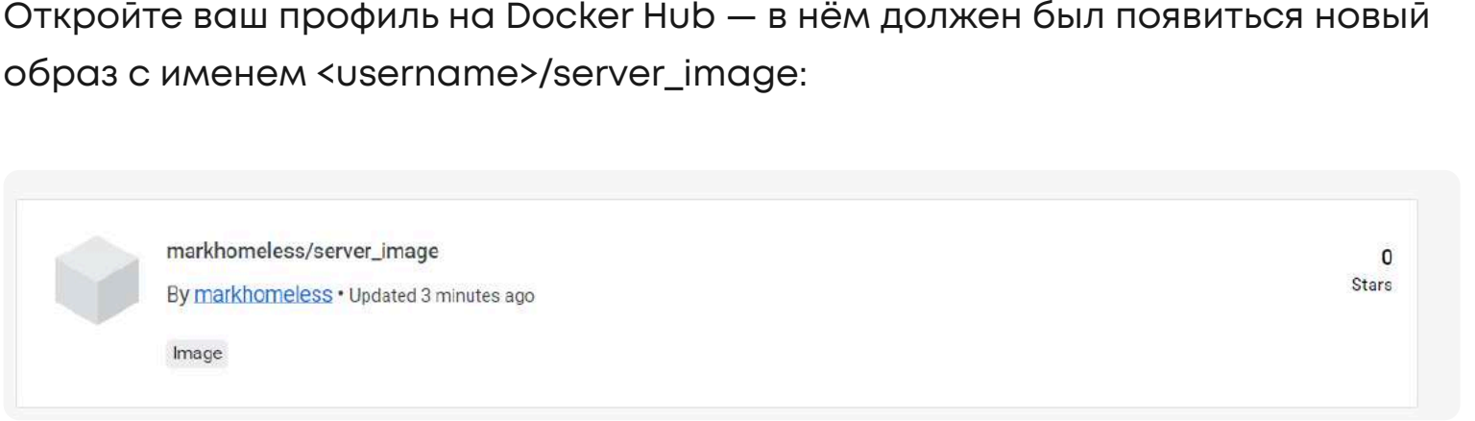
Нам необходимо пересобрать образ, задав ему имя в формате <username>/server_name, где <username> — имя пользователя в профиле на Docker Hub.

```
$ docker build -t <username>/server_image .
```

Образ собран — осталось только загрузить его на Docker Hub:

```
$ docker push <username>/server_image
```

Откройте ваш профиль на Docker Hub — в нём должен был появиться новый образ с именем <username>/server_image:



markhomeless/server_image
By markhomeless • Updated 3 minutes ago
Image
0
Stars

Теперь, чтобы наш коллега Василий смог запустить веб-сервис со своей ML-моделью, ему будет достаточно сделать pull нашего образа контейнера и запустить его. Для справки приведём команды, которые должны быть выполнены на его компьютере:

```
$ docker pull <username>/server_image
$ docker run -it --rm --name=server_container -p=80:80 server_image
```

Согласитесь, очень просто и удобно!

‹ Предыдущий урок   Следующий урок ›

7/8 **Итоги**

текущий урок

Мы с вами проделали большую работу для обеспечения воспроизводимости своего кода.

В этом модуле мы:

- узнали, что такое воспроизводимость и какими инструментами её можно обеспечить;
- познакомились с терминами «виртуализация» и «контейнеризация»;
- научились создавать виртуальные окружения, изолировать среду разработки, устанавливать и фиксировать зависимости в виртуальных окружениях;
- рассмотрели инструмент контейнеризации Docker;
- научились писать Dockerfile, создавать образы контейнеров, запускать их, а также делиться ими с помощью хостинга docker-образов Docker Hub;
- создали docker-образ для нашего веб-сервиса и запустили его в контейнере.

Поместить модель и использующий её веб-сервис в контейнер — это не единственная цель Docker. В реальных системах такой сервис будет лишь частью глобальной системы, состоящей из нескольких или даже нескольких десятков подобных сервисов. Компоненты, из которых будет состоять эта система, называются **микросервисами**, и архитектура такой системы будет по аналогии называться **микросервисной**. Следующий модуль мы посвятим организации микросервисной архитектуры и взаимодействия между её компонентами: мы увидим, как сервисы, внутри которых функционируют модели машинного обучения, становятся частью большой системы, и поговорим о том, как управлять такой системой.

Дополнительные материалы

- Официальная документация Virtualenv.
- Обзор инструментов для создания виртуальных окружений в Python (помимо Virtualenv)*.
- Список команд Docker.
- Материалы по работе с Docker от его амбассадоров.
- Цикл статей «Разворачиваем модель машинного обучения с Docker».

* Если какие-то ссылки не открываются, обратитесь к координатору.

‹ Предыдущий урок Следующий урок ›

1/9 **Введение**

текущий урок

В предыдущих модулях мы научились производить сериализацию и десериализацию моделей, разворачивать собственные простейшие веб-сервисы, создавать для них виртуальные окружения и контейнеризировать их.

> Этот модуль будет посвящён трём глобальным темам, одна из которых — основная, а остальные — бонусные.

Микросервисная архитектура приложения

В основной части модуля мы поговорим о двух подходах к построению архитектуры приложения (**монолитный** и **микросервисный**), рассмотрим их плюсы и минусы и обсудим, когда нужно использовать тот или иной подход.

Затем мы перейдём к обсуждению микросервисного подхода и разберём, какие существуют варианты организации взаимодействия между сервисами и какие инструменты помогают решать эти задачи.В качестве практики мы организуем вокруг модели машинного обучения несколько микросервисов, которые будут взаимодействовать через очередь RabbitMQ.После этого мы разместим наши сервисы в docker-контейнерах и придём к понятию **оркестрации**. С помощью инструмента Docker Compose мы создадим небольшое микросервисное приложение, в котором каждый сервис будет работать внутри отдельного контейнера.

> В промышленной разработке для оркестрации контейнеров чаще используются более мощные системы, такие как Kubernetes. Подробно этот инструмент и подходы к его использованию мы рассмотрим в следующем семестре.

Инструменты управления жизненным циклом модели

В этой части модуля мы поговорим о современных инструментах, которые можно использовать для управления жизненным циклом модели, а именно о DVC и MLflow.

Оценка бизнес-эффективности

В заключительной части модуля мы поговорим о том, как можно оценить эффективность построенных нами моделей машинного обучения с точки зрения бизнеса, какие существуют бизнес-метрики, и обсудим, какие трудности могут нас ожидать.

Цели модуля:

- узнать, какие виды сервисной архитектуры существуют и в чём их особенности;
- научиться настраивать взаимодействие между сервисами с помощью очередей;
- узнать, что такое брокер сообщений и для чего он нужен;
- понять, как можно отслеживать работу сервиса или группы сервисов;
- познакомиться с термином «оркестрация» и узнать, почему она необходима при микросервисной архитектуре;
- научиться работать с Docker Compose;
- рассмотреть несколько инструментов, которые могут помочь на различных этапах жизненного цикла модели;
- узнать, как оценивать эффективность полученных моделей с точки зрения бизнес-эффекта.

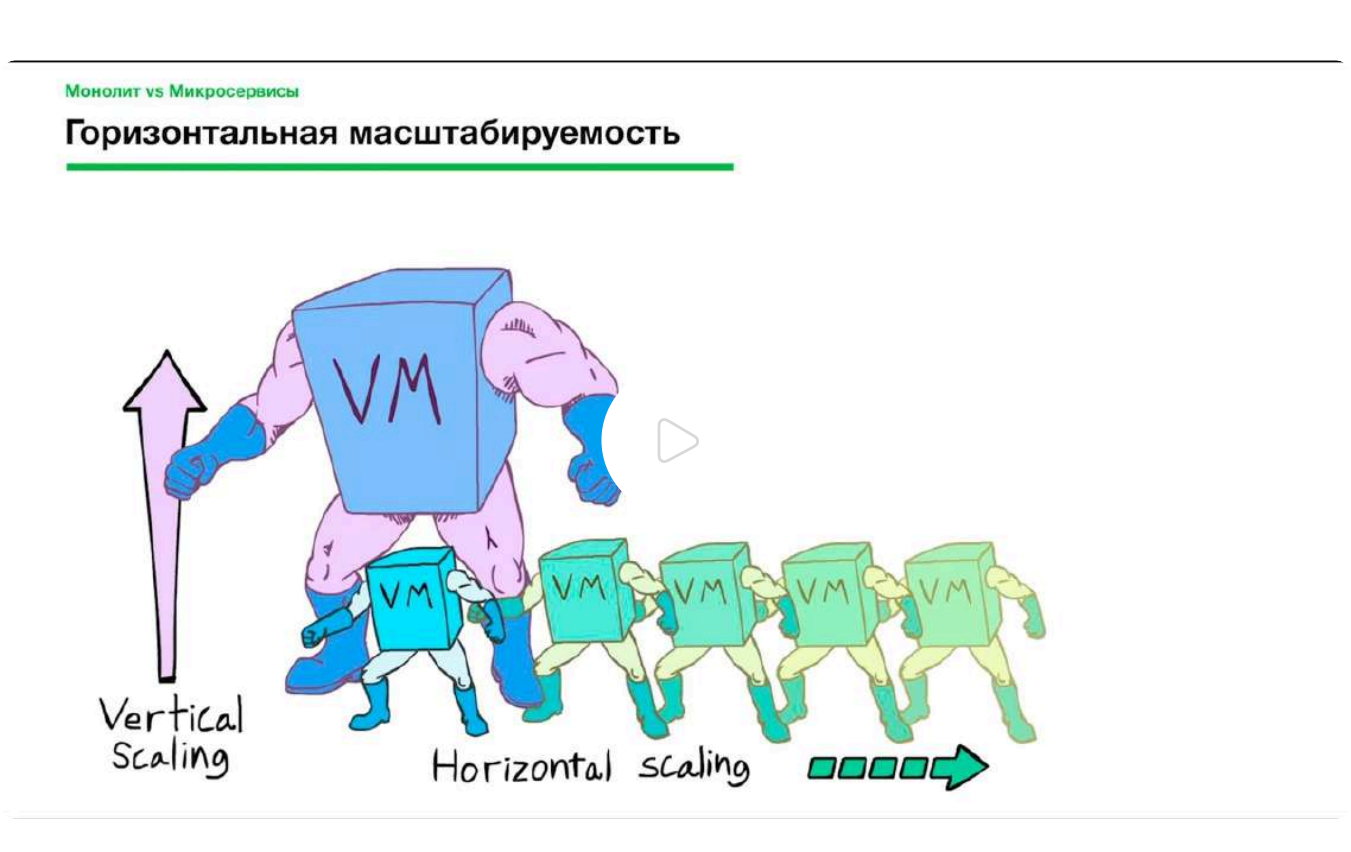
Давайте приступим!

< Предыдущий урок   Следующий урок >

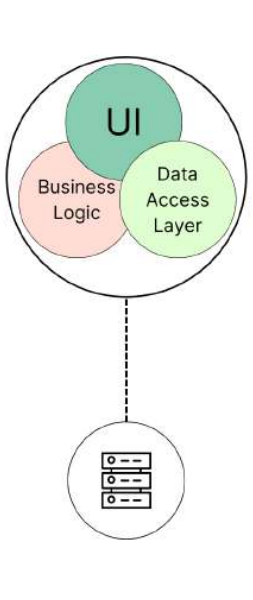
2/9 Монолитная и микросервисная архитектуры. Брокеры и очереди

текущий урок

Требования продакшн-среды влияют и на выбор **архитектуры**. Существует два её основных типа: **монолитная** и **микросервисная**.



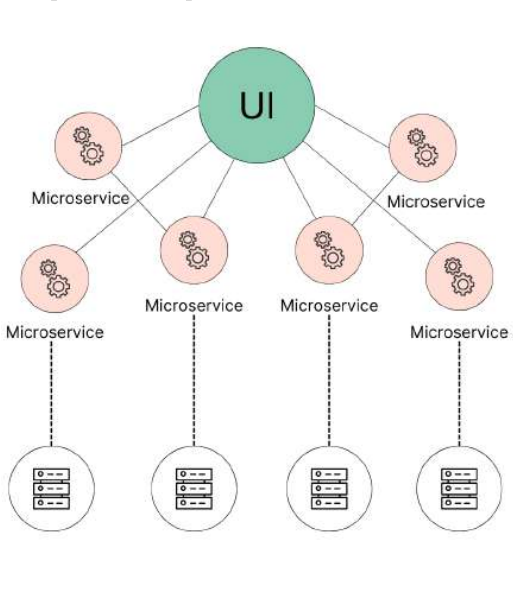
Монолитная



При монолитной архитектуре приложение развёртывается как единая программа. Все компоненты системы находятся в одном приложении и взаимодействуют напрямую через вызовы функций или общие модули.

Таким образом, приложение включает в себя все функции, и они взаимодействуют в одной запущенной программе, написанной на одном конкретном языке программирования.

Микросервисная



Микросервисная архитектура — это подход, позволяющий инкапсулировать определённые контексты приложения: то есть каждая функция приложения вынесена в отдельную небольшую программу или сервис.

Приложение разделено на небольшие, не зависящие друг от друга компоненты — микросервисы. У каждого из них своя задача, например управлять каталогом, хранить и обновлять содержимое корзины или проводить оплату заказа.

Примечание. Впервые о микросервисах заговорили ещё в 2000-х, но концепция архитектуры сформировалась только к началу 2010-х. К 2014 году технологию внедрили такие крупные компании, как Netflix, Amazon и Twitter. Сегодня микросервисы используют гораздо активнее. В 2020 году в отчёте Cloud Microservices Market Research рынок облачных микросервисов оценили в 831,45 млн долларов США. По состоянию на 2026 год этот подход остаётся одним из основных архитектурных стандартов при разработке масштабируемых распределённых приложений.

Преимущества и недостатки микросервисного подхода в разработке

Плюсы

- Повышение автономности различных частей продукта. Мы можем разрабатывать их раздельно и разными командами (при этом отслеживая версионность).
- Гибкость и масштабируемость. За счёт модульности приложения мы можем масштабировать только ту часть, которая требует дополнительных ресурсов. Подробнее об этом мы поговорим ниже.
- Повышение стабильности. Если что-то перестало работать, будет недоступен только один функционал, а не приложение целиком.
- Горизонтальная масштабируемость. Микросервисы позволяют запускать приложение на разных серверах.

Минусы

- Сложность разработки из-за наличия различных технологий.
- Накладные расходы на сериализацию и десериализацию сообщений.
- Время работы напрямую зависит от взаимодействия сервисов.

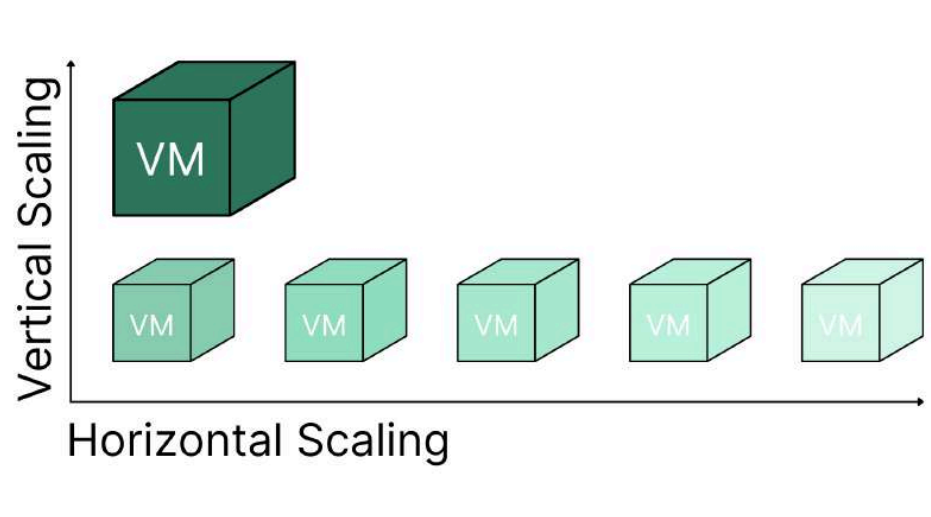
Масштабируемость системы

По мере роста и развития приложения, увеличения числа пользователей и данных у команды неизбежно возникают вопросы:

1. Как обрабатывать N запросов в секунду? Как уменьшить время обработки одного запроса?
2. Как и в чём хранить постоянно увеличивающийся в объёме поток данных от пользователей?
3. Как подстраховаться от падения БД (ведь это трата времени и, следовательно, потеря денег)?
4. Как добиться высокой степени доступности приложения для всех пользователей?
5. Что делать с тяжёлыми запросами, которые могут выполняться часами?

Обобщая эти и подобные вопросы, говорят о **масштабируемости** системы — её способности выдерживать рост нагрузки по мере добавления ресурсов.

Давайте разберёмся в этом подробнее. При создании действительно крупных приложений существует два подхода:

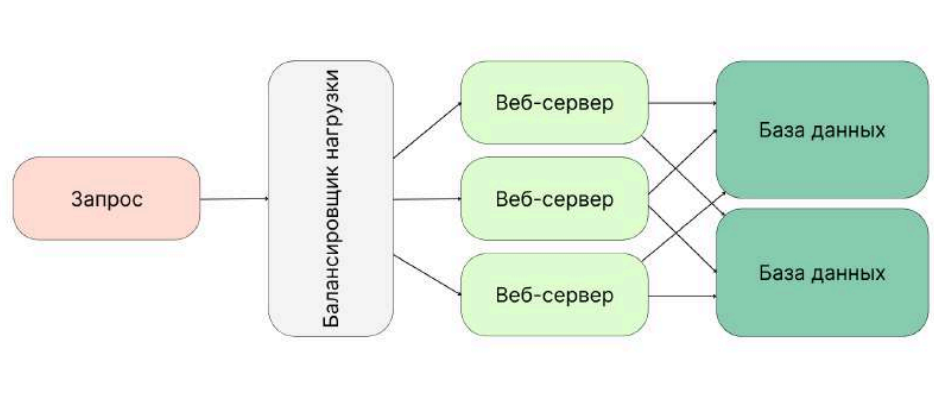


- **Вертикальное масштабирование** (scale up): заменить сервер сервером с большей вместимостью и мощностью. Как вы понимаете, у всего есть предел.
- **Горизонтальное масштабирование** (scale out): добавить ещё один сервер и объединить серверы в кластер.

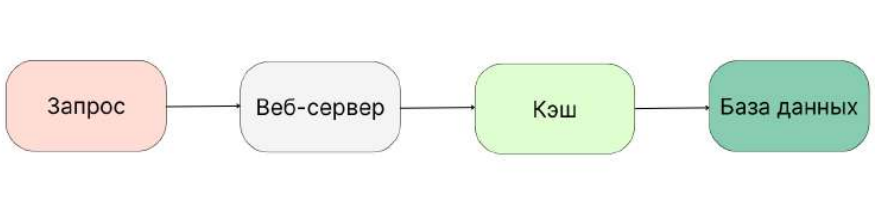
Идеальный вариант — добиться **линейной зависимости**, то есть сделать так, чтобы при добавлении ресурсов приложение выдерживало прямо пропорциональный рост нагрузки. Однако такое встречается редко из-за неоптимальной работы разных частей системы: так, добавление дополнительного сервера, обрабатывающего запросы, может увеличить параметр максимальной нагрузки в два раза, а вот добавление ещё двух серверов вряд ли увеличит этот параметр в четыре раза.

Что мы можем сделать для лучшего масштабирования системы?

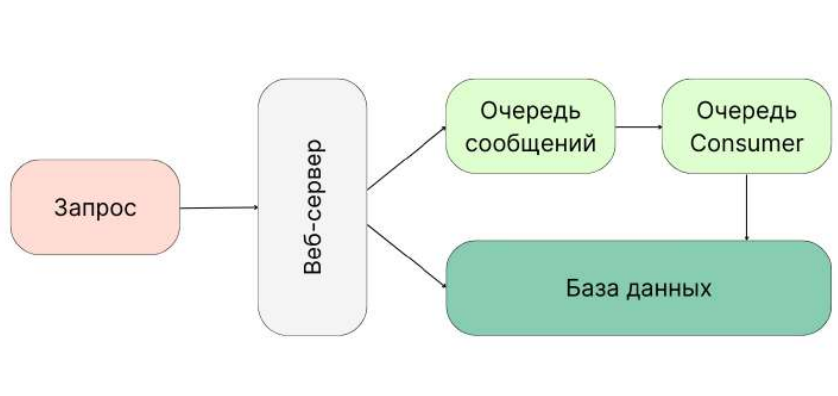
1. **Добавить балансировщики нагрузки** перед дублирующими друг друга серверами. Эти балансировщики будут отправлять запросы на те серверы, на которых нагрузка меньше, чем на остальных.



2. **Добавить кэш** — быстрый «буфер» между базой данных и приложением. В таком случае диспетчер сначала проверит в кэше, не выполнялся ли аналогичный запрос раньше, и только потом направит его на сервер.



3. **Оптимизировать работу БД:**
   - скопировать БД, чтобы застраховаться от её падения;
   - денормализовать данные, чтобы все поля содержались в одной таблице — так мы избежим дополнительной нагрузки на объединение данных;
   - оптимизировать SQL-запросы, повысив скорость их обработки.
4. **Добавить асинхронные очереди сообщений** между частями системы. Подробнее об асинхронном взаимодействии сервисов мы поговорим в следующем блоке.



Когда использовать микросервисную архитектуру?

Выбор архитектуры должен основываться на понимании всех требований к разрабатываемому приложению.

Разработчики Яндекс составили небольшой перечень критериев, по которым вы можете понять, подходит ли вашему проекту микросервисная архитектура. Если ваш проект соответствует хотя бы одному пункту из этого списка, задумайтесь об использовании микросервисов.

- **Большие коллективы.**
  Микросервисы позволяют группам разработчиков не задумываться о синхронизации каждого шага: каждая команда может работать над одним или несколькими микросервисами и использовать свои инструменты.
  Новые фичи можно разрабатывать параллельно и запускать по мере готовности.
- **Объёмные проекты со сложной архитектурой.**
  Обновлять и поддерживать отдельные модули крупных систем намного проще, чем контролировать, как изменения скажутся на системе в целом.
- **Продукты с резко меняющимся трафиком.**
  Если вашим продуктом начинают чаще пользоваться в период праздников или распродаж, микросервисы позволят вам быстро масштабироваться и уменьшить риск отказа системы. Кроме того, вам не придётся платить за дополнительную инфраструктуру, которая нужна только в периоды пиковых нагрузок.
- **Приложения, требующие частых обновлений.**
  Достаточно изменить и отладить только тот модуль, который вы хотите обновить. Это существенно сокращает время разработки и приближает релиз.

Однако главный вопрос, который вам стоит задать себе, прежде чем погружаться в мир микросервисов: можно ли разделить ваш продукт на простые независимые части? Хороший микросервис должен быть легковесным, автономным, обходиться собственной изолированной базой данных и решать одну конкретную бизнес-задачу.

Если ваш проект можно разделить на такие самостоятельные составные части, то вы можете смело выбирать микросервисы в качестве архитектуры своего приложения.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 2.1

0 points possible (ungraded)

Что относится к плюсам микросервисной архитектуры?
Отметьте все подходящие варианты ответов.

☑ **A** Масштабируемость
☑ **B** Автономность различных частей продукта
☐ **C** Почти полное отсутствие сквозных проблем

✓

Ответ
Верно:
A Верно.
B Верно.

Show answer

Отправить

✓ Верных

Задание 2.2

0 points possible (ungraded)

Гарантирует ли использование микросервисной архитектуры лучшую производительность по сравнению с монолитной?

○ Да
● Нет

✓

Ответ
Верно:
При микросервисной архитектуре появляются накладные расходы на взаимодействие между сервисами и сериализацию/десериализацию сообщений, поэтому говорить о гарантированном повышении производительности не приходится.

Show answer

Отправить

✓ Верных

Задание 2.3

0 points possible (ungraded)

Представьте, что вы разрабатываете модель, которая предсказывает скорость оттока клиентов на основе их поведения. При этом весь *backend* приложения работает на *Java*. Как вы будете реализовывать сервис?

● Напишу отдельный микросервис, который будет выполнять предсказания и взаимодействовать с основным приложением.
○ Интегрирую модель в виде функции в общий монолит.
○ Сервис можно реализовать как угодно.

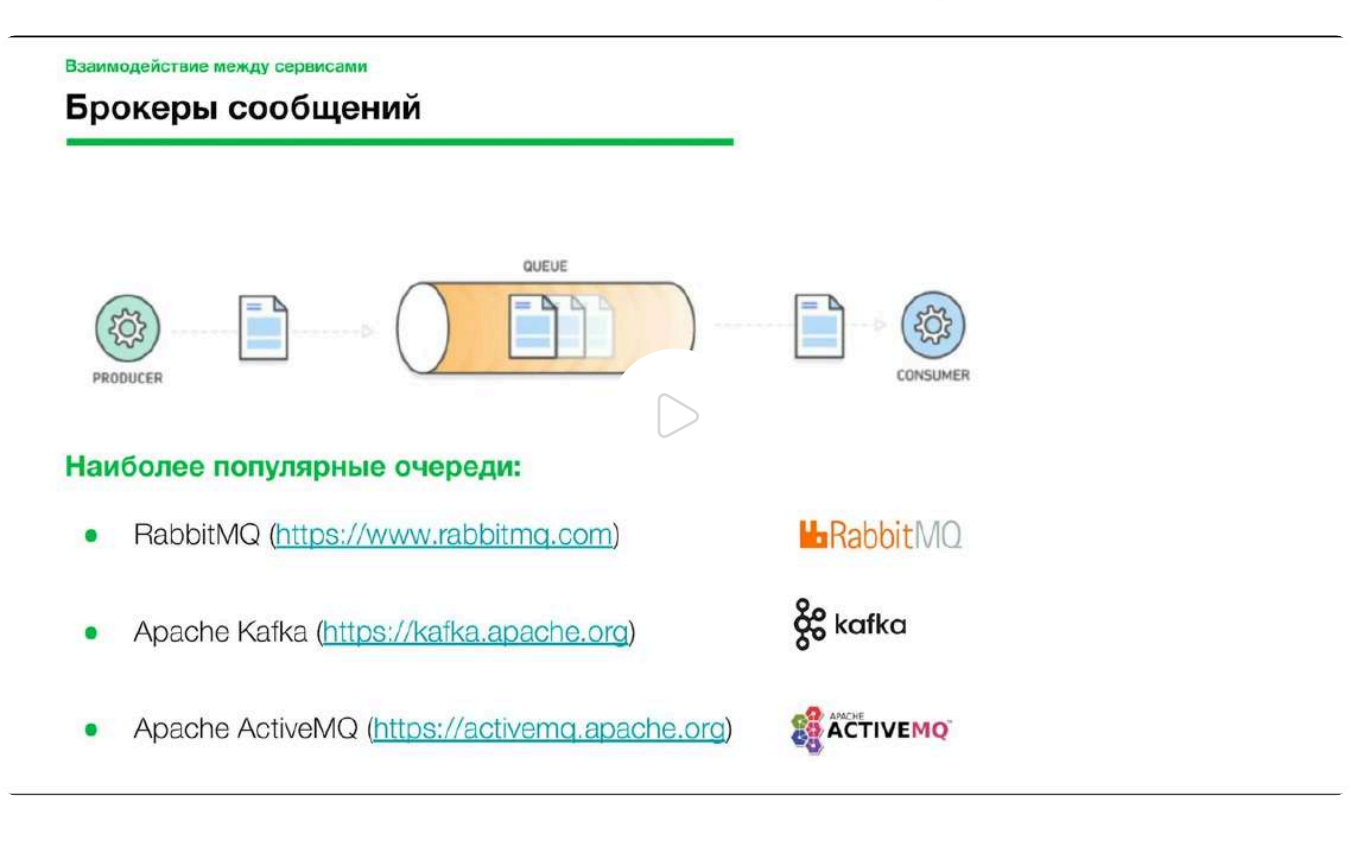
✓

Show answer

Отправить

✓ Верных

Взаимодействие между сервисами


Взаимодействие между сервисами
Брокеры сообщений
PRODUCER
QUEUE
CONSUMER
Наиболее популярные очереди:
RabbitMQ (https://www.rabbitmq.com)
Apache Kafka (https://kafka.apache.org)
Apache ActiveMQ (https://activemq.apache.org)
RabbitMQ
kafka
ACTIVEMQ

Как мы уже поняли, при создании микросервисной архитектуры требуется **организация обмена данными между сервисами**.

Давайте разберёмся, какие существуют типы организации взаимодействия сервисов:

Синхронный

Один сервис обращается к другому и ожидает ответа.

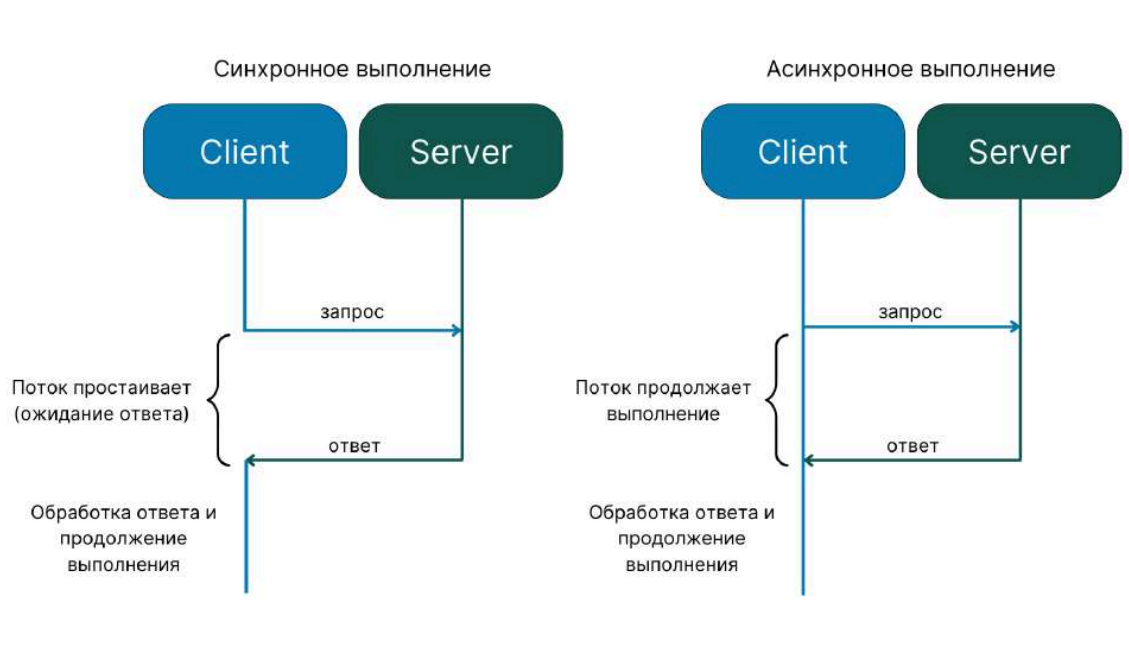
Для организации синхронного взаимодействия используется протокол HTTP или HTTPS. Сервисы обмениваются данными через HTTP-запросы.

Разработка и отладка просты, однако сервис должен быть постоянно доступен — в противном случае обмен сообщениями остановится.

Асинхронный

Сервисы взаимодействуют между собой путём передачи сообщений. Таким образом, сервис не ожидает ответ, а продолжает работу. Нужный сервис принимает сообщение и начинает его обработку.

Для организации асинхронного взаимодействия используются очереди сообщений. О них мы поговорим ниже.


Синхронное выполнение
Client
Server
запрос
Поток простаивает (ожидание ответа)
ответ
Обработка ответа и продолжение выполнения
Асинхронное выполнение
Client
Server
запрос
Поток продолжает выполнение
ответ
Обработка ответа и продолжение выполнения

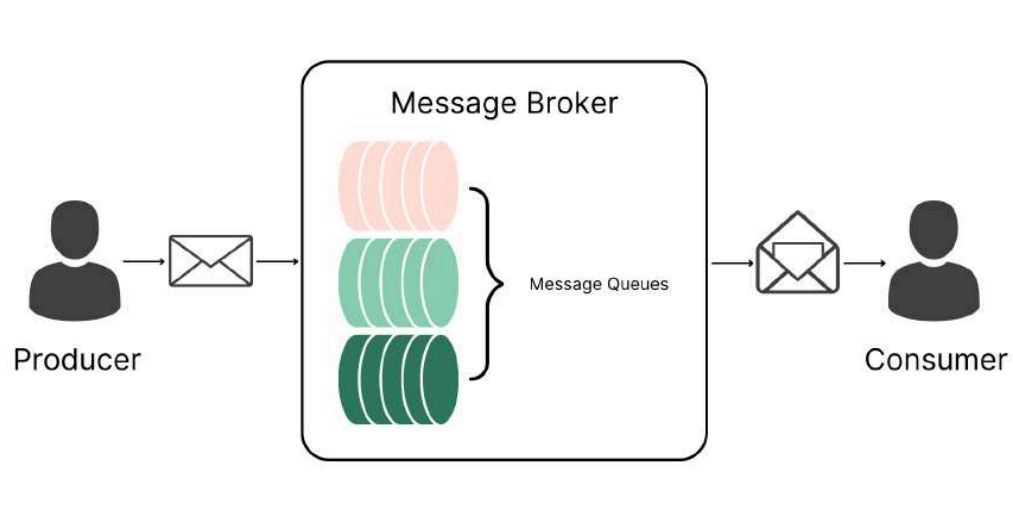
Брокеры сообщений

Брокеры сообщений (также их называют **диспетчеры очереди**) помогают организовать межсистемный обмен сообщениями и являются связующим звеном между различными процессами в приложениях.

Как правило, в таких системах используется паттерн проектирования «**издатель → очередь → подписчик**» (producer → queue → consumer). Под издателем и подписчиком здесь можно понимать всё что угодно (например, микросервисы или целые приложения), то есть это абстрактные понятия, не привязанные к конкретной реализации.

Издатель отправляет сообщение на брокер сообщений, который помещает полученное сообщение в очередь. На одном брокере может быть запущено несколько очередей, например очередь запросов, очередь ответов и служебная очередь, в которую записываются служебные сообщения (логи). После этого сообщение извлекается из очереди и передаётся подписчику.

Издателей и подписчиков может быть несколько, и они могут взаимодействовать друг с другом через очереди в асинхронном режиме, то есть им не нужно останавливать свой процесс работы в ожидании ответного сообщения.

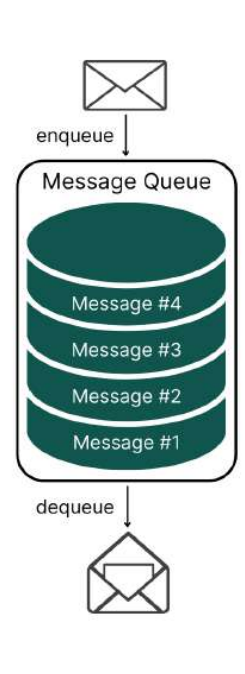


Благодаря брокерам сообщений отправителю не нужно ничего знать о потребителе или потребителях, поэтому связь между сервисами значительно упрощается. Однако поддержка инфраструктуры с участием очереди становится немного сложнее.

Чтобы обеспечить хранение сообщений в очереди до момента их прочтения получателем, доступ к очереди должен быть **асинхронным**.

Структура данных очереди представлена следующими правилами:

- первым «читается» сообщение, пришедшее первым;
- новые элементы добавляются только в конец очереди;
- чтение и удаление происходит только из начала очереди.



Кроме того, для организации очереди необходимо определиться с форматом сообщений (например, XML или JSON). Однако это сильно зависит от типа реализации очереди — бывают случаи, когда определять формат нет необходимости.

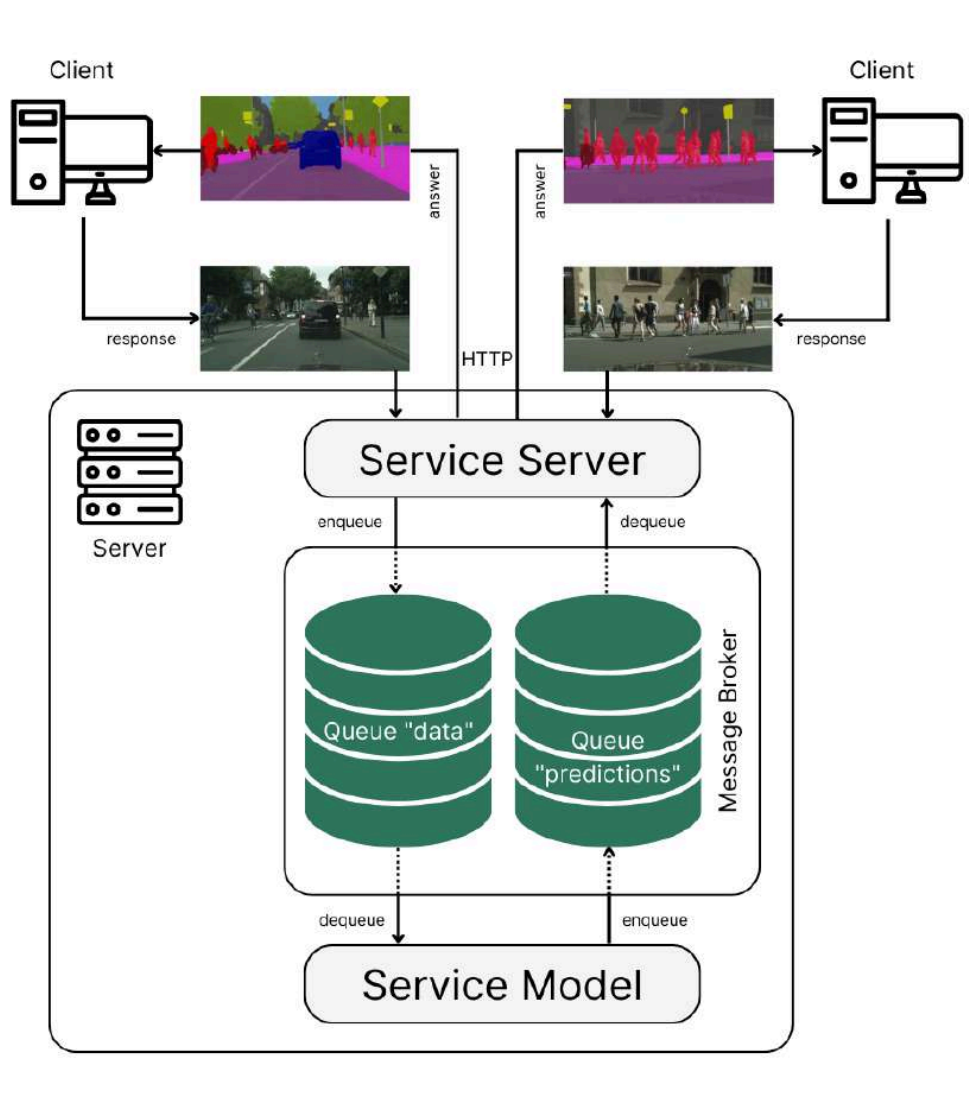
Пример, когда может понадобиться использование очередей

На веб-сервере функционирует огромная нейронная сеть или другая крупная модель. Если клиенты будут отправлять свои данные (например, изображения), а сервер будет принимать эти запросы и обрабатывать данные, подавая их в модель, то время задержки на обработку каждого запроса будет очень большим.

Поэтому идея следующая: пусть сервер — это отдельный микросервис. Назовём его server (это обычное веб-приложение, реализованное, например, через стек Flask + uWSGI + NGINX). Этот микросервис решает одну задачу: принимает сообщения от клиентов, присваивает им идентификаторы, но не обрабатывает их, а помещает в очередь обработки (назовём её data).

Для прогона данных через модель будет реализован отдельный микросервис (назовём его model). Внутри него тоже может быть что угодно, например нейронная сеть, которая проводит сегментацию изображений. Этот микросервис извлекает сообщения с данными от пользователя из очереди, обрабатывает их моделью и отправляет в другую очередь (назовём её predictions).

Сервер в фоновом режиме просматривает очередь predictions и, если в ней появилось новое сообщение (предсказание модели), извлекает его и отправляет нужному клиенту.



Наиболее популярны следующие программные реализации очереди:

- RabbitMQ,
- Apache Kafka,
- ActiveMQ.

Чтобы понять принципы работы с очередями сообщений и сериализацией, мы разберём диспетчер сообщений RabbitMQ. Он прост в использовании, работает на разных системах и пользуется большой популярностью.

Примечание. Также стоит отметить, что с помощью протокола HTTP возможно организовать не только синхронное, но и асинхронное взаимодействие, однако эта тема выходит за рамки нашего курса.

Задание 2.4

0 points possible (ungraded)

Верно ли утверждение «При синхронном взаимодействии сервис продолжает работу, не ожидая ответа после запроса»?

○ Да, верно.
● Нет, не верно.

✓

Show answer

Отправить

✓ Верных

Задание 2.5

0 points possible (ungraded)

Выберите свойства, характерные для очереди:

☐ **A** Первым «читается» сообщение, пришедшее последним.
☑ **B** Новые элементы добавляются только в конец очереди.
☐ **C** Можно удалять сообщения только в конце очереди.
☑ **D** Необходимо обеспечить хранение сообщений в очереди до момента их прочтения получателем.

✓

Ответ
Верно:
B Верно.
D Верно.

Show answer

Отправить

✓ Верных

< Предыдущий урок   Следующий урок >

3/9 Организация взаимодействия через очереди. RabbitMQ

текущий урок

В этом модуле мы рассмотрим основные концепции работы с брокером очередей RabbitMQ: мы создадим несколько микросервисов на Python и организуем их асинхронное взаимодействие через этот брокер.

Сначала мы рассмотрим основную терминологию, которая пригодится нам в работе с RabbitMQ, а затем перейдём к реализации взаимодействия сервисов.

Как вы помните, объект из памяти одного процесса нельзя напрямую перенести в память другого, поскольку объект может иметь сложную структуру и быть разбит на фрагменты в оперативной памяти. Чтобы сохранить целостность объекта, применяется **сериализация**.

При сохранении и загрузке моделей мы проводим эти операции один раз, поэтому это не увеличивает нагрузку на процессы. Для организации взаимодействия сервисов необходимо **сериализовывать** каждое сообщение (например, вектор входных данных) при его отправке, **десериализовывать** его после получения в сервисе для подачи в модель и уже после преобразовывать его в поток битов предсказания. Это требует ресурсов.

Совет. Если добавить сжатие данных, то потребуется больше ресурсов CPU на их чтение и запись, но сам объём данных будет меньше. Таким образом мы уменьшим нагрузку на сеть и диск.

Для обмена сообщениями между сервисами чаще используют формат JSON. В отличие от pickle, он является текстовым и может использоваться между программами, написанными на разных языках.

Для примера работы с брокером очередей мы возьмём интеграцию RabbitMQ и Docker. Для подключения наших процессов к очереди будем использовать библиотеку pika. Давайте разберёмся, как это работает.

ОСНОВЫ РАБОТЫ С RABBITMQ. AMQP

> RabbitMQ — это брокер сообщений, основная цель которого — принимать и отдавать сообщения.

Можете представить себе RabbitMQ как почтовое отделение: когда вы опускаете письмо в ящик, вы можете быть уверены, что рано или поздно почтальон доставит его адресату. В этой аналогии RabbitMQ является одновременно и почтовым ящиком, и почтовым отделением, и почтальоном.

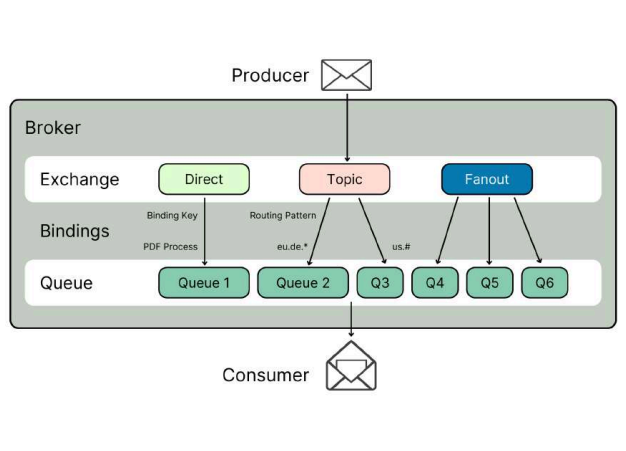
Отличие RabbitMQ от почтового отделения в том, что он не имеет дела с бумажными конвертами, а принимает, хранит и отдаёт сообщения в бинарном (сериализованном) виде.

RabbitMQ использует протокол AMQP (Advanced Message Queuing Protocol).

> **Протокол AMQP** — открытый стандарт передачи сообщений. Он позволяет подсистемам/независимым приложениям обмениваться сообщениями через AMQP-брокер, отвечающий за маршрутизацию, доставку сообщений, распределение нагрузки и так далее.

Основная терминология AMQP:

1. **Message** (сообщение) — передаваемые данные.
2. **Exchange** (точка обмена) — механизм маршрутизации сообщений. Точка обмена получает сообщения и распределяет их по очередям (одно сообщение может уйти в одну или несколько очередей), но при этом сама она не хранит сообщения. В самом простом случае для маршрутизации сообщений используется ключ (routing key), равный названию очереди, в которую нужно отправить сообщение. Иными словами, routing key — это виртуальный адрес очереди.
3. **Bindings** (правила распределения) — правила, по которым точка обмена определяет, куда именно нужно отправить пришедшее сообщение.
4. **Queue** (очередь) хранит сообщения до тех пор, пока какой-нибудь AMQP-клиент не заберёт их.
5. **Producer** (издатель) — клиент, публикующий сообщения в exchange.
6. **Consumer** (подписчик) — клиент, получающий сообщения из очередей.
7. **Connection** (соединение) — служит для физического сетевого соединения между клиентом и брокером и объединения нескольких каналов.
8. **Channel** (канал) — используется для логического соединения между клиентом и брокером.



Примечание. Более подробно о принципах работы протокола AMQP можно узнать здесь.

Чтобы реализовать AMQP на Python, мы воспользуемся библиотекой pika. Мы выбрали эту библиотеку, поскольку она подойдёт для любого брокера, который работает по протоколу AMQP.

Основные функции библиотеки, с которыми мы будем работать:

- `BlockingConnection` — объявление соединения;
- `channel` — объявление канала;
- `queue_declare` — объявление очереди;
- `basic_publish` — отправка сообщения;
- `basic_consume` — получение сообщения;
- `callback` — метод, вызываемый при получении сообщения.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 3.1

0 points possible (ungraded)

Выберите правильный порядок сущностей, по которым проходит передача сообщения, согласно протоколу *AMQP*.

Exchange:

2 ✓

Queue:

3 ✓

Producer:

1 ✓

Consumer:

4 ✓

Show answer

Отправить

✓ Верных

Теперь давайте приступим к практике.

Установка инструментов

Шаг 0. Установка Docker.

Этот шаг вы выполняли ещё в прошлом модуле.

Шаг 1. Установка RabbitMQ.

Теперь, когда Docker уже есть в нашей системе, достаточно просто включить брокер сообщений RabbitMQ — он находится в Docker Registry в виде образа rabbitmq. Мы будем пользоваться версией 3-management. Давайте запустим docker-контейнер на основе этого образа:

```
$ docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

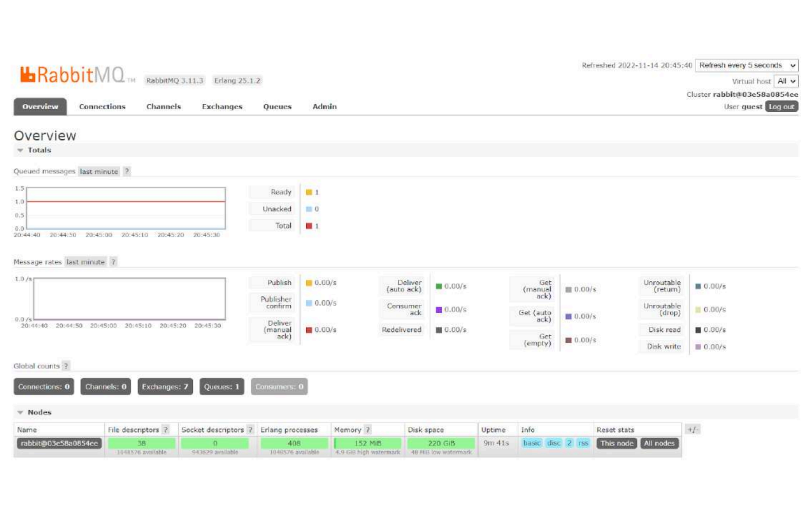
Здесь:

- `-p 5672:5672` — порт для доступа к очереди;
- `-p 15672:15672` — порт для доступа к пользовательскому интерфейсу RabbitMQ.

После выполнения команды на экране появится большое информационное сообщение и брокер уйдёт в режим ожидания сообщений.

Наша очередь работает и готова к приёму сообщений — к ней можно обратиться по адресу localhost:5672.

Примечание. Вы можете войти в режим графического интерфейса управления RabbitMQ. Для этого перейдите по адресу http://localhost:15672. В результате откроется окно графического интерфейса RabbitMQ, с помощью которого вы можете управлять очередями и другими компонентами RabbitMQ.



Сейчас мы не будем использовать графический интерфейс, однако вы можете познакомиться с ним здесь.

Шаг 3. Установка библиотеки pika.

Установим её через pip-команду, сразу зафиксировав нужную нам версию (1.1.0).

```
$ pip install pika (или pika>=1.3)
```

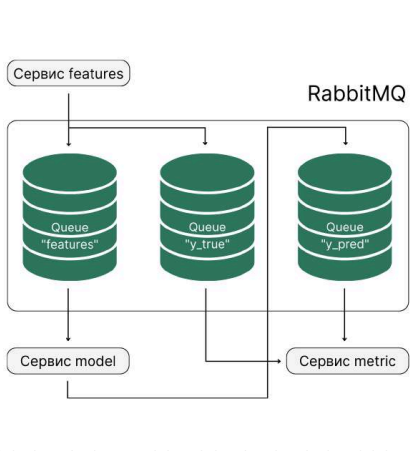
Итак, всё готово к работе. Давайте обсудим план наших действий.

Чем будем заниматься?

Мы реализуем несколько сервисов:

1. Первый будет отправлять признаки в одну очередь и истинный ответ — в другую.
2. Второй сервис будет читать признаки, делать предсказание и отправлять его в очередь с предсказаниями.
3. Третий сервис будет читать очереди с истинными ответами и предсказаниями.

Схематично описанную архитектуру можно представить в следующем виде:



Давайте заранее организуем директорию нашего проекта. Это нужно для дальнейшего удобства, когда мы будем помещать каждый из сервисов в отдельный контейнер.

Рабочую папку проекта назовём microservice_architecture. В этой папке будут три другие папки: features, model и metric. Внутри каждой из этих папок будет по папке src, в которой хранится исходный код для каждого сервиса: features.py, model.py и metric.py. Также в папке model/src/ будет находиться файл с сериализованной моделью — что это за модель, мы посмотрим чуть позже.

```
microservice_architecture
    └─features
        └─src
            └─features.py
    └─model
        └─src
            └─model.py
            └─myfile.pkl
    └─metric
        └─src
            └─metric.py
```

После создания рабочей директории можно переходить к разработке самих микросервисов.

Сервис I. сервис отправки признаков

Приступим к созданию первого сервиса — сервиса для отправки признаков в одну очередь, а истинных ответов — в другую.

Создадим файл features/src/features.py, в котором будем реализовывать сервис. Этот сервис будет брать случайную строку из датасета и отправлять полученные признаки объекта в очередь.

Сначала импортируем необходимые библиотеки, загрузим датасет (используем знакомый нам датасет о диабете) и создадим переменную `random_row`, значение которой будет соответствовать случайному числу от 0 до $N-1$, где N — количество объектов в выборке.

```
import pika
import numpy as np
from sklearn.datasets import load_diabetes
# Загружаем датасет о диабете
X, y = load_diabetes(return_X_y=True)
# Формируем случайный индекс строки
random_row = np.random.randint(0, X.shape[0]-1)
```

Таким образом, в нашем скрипте можно будет получить случайный вектор признаков `X[random_row]` и истинный ответ для него — `y[random_row]`.

Давайте подключимся к брокеру и попробуем отправлять в очередь `y_true` случайную метку `y[random_row]`. Для организации соединения следует создать объект `BlockingConnection`. В его инициализатор необходимо передать объект с параметрами организуемого соединения — `ConnectionParameters`. Наши сервисы будут взаимодействовать через localhost. После того как мы создали объект соединения, необходимо создать канал, по которому сервисы будут обмениваться данными.

Итоговый код для создания подключения к серверу брокера будет выглядеть так:

```
# Подключение к серверу на локальном хосте:
connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

Примечание. Если вы хотите подключиться к удалённому серверу, то есть ваш брокер сообщений работает на удалённом компьютере, вместо `localhost` укажите его IP-адрес.

Далее объявим в канале соединения очередь сообщений с помощью метода `queue_declare()`. Назовём очередь "`y_true`" (параметр `queue`).

```
# Создаём очередь y_true
channel.queue_declare(queue='y_true')
```

Если при выполнении скрипта очередь на брокере ещё не была создана, то она будет создана, иначе мы будем работать с уже имеющейся очередью.

Далее наш микросервис должен отправить сообщение в очередь. Для этого используется метод `basic_publish()`. В нём необходимо указать следующие параметры:

- `exchange` — определяет, в какую очередь отправляется сообщение.

  Как мы знаем, в RabbitMQ сообщения не отправляются сразу в очередь, а проходят через точку обмена (exchange). Сейчас нам достаточно знать, что точку обмена по умолчанию можно определить, указав пустую строку.
- `routing_key` — указывает имя очереди.
- `body` — тело самого сообщения, которое мы хотим поместить в очередь.

```
# Публикуем сообщение
channel.basic_publish(exchange='',
                      routing_key='y_true',
                      body=y[random_row])
print('Сообщение с правильным ответом отправлено в очередь')
```

Добавим ещё одну очередь — `features`. В неё мы будем отправлять признаки, соответствующие случайно выбранному объекту из данных:

```
# Создаём очередь features
channel.queue_declare(queue='features')
# Публикуем сообщение
channel.basic_publish(exchange='',
                      routing_key='features',
                      body=json.dumps(list(X[random_row])))
print('Сообщение с вектором признаков отправлено в очередь')
```

После отправки сообщения мы должны закрыть подключение с помощью метода `close()`.

```
# Закрываем подключение
connection.close()
```

При запуске кода возникает ошибка:

```
TypeError: object of type 'numpy.float64' has no len()
```

Подумайте, почему произошла ошибка, а затем загляните в ответ.

Причина ошибки

Ошибка произошла потому, что объект, который мы отправляем в очередь, не был сериализован. Мы уже говорили, что сообщения в RabbitMQ должны быть представлены в формате JSON, поэтому давайте выполним сериализацию.

Воспользуемся стандартной библиотекой json, чтобы сериализовать истинные метки объектов и вектор признаков в этот формат.

```
import json
```

Напомним, что сериализовать объект можно с помощью метода `json.dumps()`.

Обратите внимание, что в случае отправки вектора признаков в очередь `features` у нас не получится сериализовать объект `array` — его необходимо перевести в список с помощью функции `list()`:

```
channel.basic_publish(exchange='',
                      routing_key='y_true',
                      body=json.dumps(y[random_row]))
channel.basic_publish(exchange='',
                      routing_key='features',
                      body=json.dumps(list(X[random_row])))
```

Таким образом, в качестве параметра `body` мы будем передавать байт-строку вектора признака в формате JSON.

Для тестирования наших микросервисов, в том числе скрипта features.py, давайте зафиксируем датчик случайных чисел:

```
np.random.seed(42)
```

Итоговый код в файле features.py будет выглядеть так:

```
import pika
import numpy as np
import json
from sklearn.datasets import load_diabetes

np.random.seed(42)
# Загружаем датасет о диабете
X, y = load_diabetes(return_X_y=True)
# Формируем случайный индекс строки
random_row = np.random.randint(0, X.shape[0]-1)

# Подключение к серверу на локальном хосте:
connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()

# Создаём очередь y_true
channel.queue_declare(queue='y_true')
# Создаём очередь features
channel.queue_declare(queue='features')

# Публикуем сообщение в очередь y_true
channel.basic_publish(exchange='',
                      routing_key='y_true',
                      body=json.dumps(y[random_row]))
print('Сообщение с правильным ответом отправлено в очередь')

# Публикуем сообщение в очередь features
channel.basic_publish(exchange='',
                      routing_key='features',
                      body=json.dumps(list(X[random_row])))
print('Сообщение с вектором признаков отправлено в очередь')

# Закрываем подключение
connection.close()
```

Выполните скрипт. Если всё прошло успешно, в результате вы должны увидеть на экране следующие сообщения:

```
## Сообщение с правильным ответом отправлено в очередь
## Сообщение с вектором признаков отправлено в очередь
```

Теперь давайте заглянем в RabbitMQ и посмотрим, пришло ли наше сообщение. Для этого в соседнем терминале выполните команду для запуска командной оболочки bash RabbitMQ:

```
$ docker exec -it rabbitmq bash
```

В результате выполнения этой команды вы попадёте внутрь контейнера, где функционирует брокер. Давайте заглянем в очередь `y_true` и посмотрим, появилось ли в ней сообщение. Для этого внутри контейнера выполните команду:

```
$ rabbitmqadmin get queue=y_true count=10
```

Данная команда позволит вывести десять (вы можете указать любое другое число) последних сообщений в очереди с именем `'y_true'` в виде таблицы.

Задание 3.2

0 points possible (ungraded)

В качестве ответа введите истинную метку, которая поступила в очередь 'y_true' (столбец *payload*).

Отправить

Примечание. После завершения работы скрипта удалите строку с фиксацией датчика случайных чисел — больше она нам не понадобится.

Отлично, наш ответ попал в очередь `'y_true'`. Аналогично вы можете просмотреть содержимое очереди `features`. Для этого, находясь внутри командной оболочки контейнера, выполните команду:

```
$ rabbitmqadmin get queue=features count=10
```

Примечание. Иногда при отладке сервисов возникает необходимость очистить очередь, как как в неё были отправлены ненужные сообщения. Это можно сделать тремя способами:

- Из самого RabbitMQ. Для этого, находясь в командной оболочке контейнера, выполните команду:

  ```
  $ rabbitmqctl purge_queue y_true
  ```
- С помощью pika. Для этого используется метод `queue_purge()`:

  ```
  channel.queue_purge(queue='имя очереди')
  ```
- Можно просто остановить работу контейнера rabbitmq и перезапустить его. Так как при запуске контейнера мы указали ключ `--rm`, все данные, создаваемые контейнером, в том числе очереди, будут удалены по завершении его работы.

  ```
  $ docker stop rabbitmq
  ```

Сервис II. Сервис для предсказания

Приступим к написанию второго сервиса — он прочитает признаки из очереди `features`, сделает предсказание и отправит его в очередь `y_pred`.

Создадим второй файл model.py и добавим в него необходимые импорты:

```
import pika
import pickle
import numpy as np
```

Ранее мы уже создавали модель для решения поставленной задачи регрессии. Давайте возьмём наш файл с обученной моделью:

```
with open('myfile.pkl', 'rb') as pkl_file:
    regressor = pickle.load(pkl_file)
```

Подключимся к серверу:

```
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()
```

Укажем, с какой очередью будем работать:

```
channel.queue_declare(queue='Features')
```

Напишем функцию `callback()`, определяющую, как работать с полученным сообщением. Эта функция в дальнейшем будет передана для обработки очереди, поэтому у неё должен быть шаблонный синтаксис: стандартные аргументы `ch`, `method`, `properties` и `body`. В данный момент нас интересует только аргумент `body` — тело сообщения из очереди. Наша функция-обработчик будет просто выводить на экран сообщение из очереди `features` (вектор признаков).

```
def callback(ch, method, properties, body):
    print(f'Получен вектор признаков {body}')
```

На следующем шаге микросервис должен будет извлечь сообщение из очереди `features` с помощью метода `basic_consume()`. В аргументах данного метода укажем:

- `queue` — имя очереди;
- `on_message_callback` — функцию-обработчик очереди, которая устанавливает, какие действия должны быть произведены с полученным из очереди сообщением;
- `auto_ack` — параметр, определяющий, использовать ли режим автоматического подтверждения (подробнее об этом — здесь).

```
# Извлекаем сообщение из очереди features
# on_message_callback показывает, какую функцию вызвать при получении сообщения
channel.basic_consume(
    queue='features',
    on_message_callback=callback,
    auto_ack=True
)
print('...Ожидание сообщений, для выхода нажмите CTRL+C')
```

Мы указали все параметры, необходимые для работы микросервиса. Осталось запустить его в режиме ожидания прихода сообщений. Для этого используется метод `start_consuming()`. Скрипт будет работать до принудительной остановки (CTRL+C) — так мы не пропустим ни одного сообщения.

```
# Запускаем режим ожидания прихода сообщений
channel.start_consuming()
```

Важно! Обратите внимание, что после того как сообщение будет получено из очереди и обработано функцией-обработчиком, оно удаляется по правилам очереди.

Второй сервис готов. На данный момент код в файле features.py выглядит так:

```
import pika
import pickle
import numpy as np

# Читаем файл с сериализованной моделью
with open('myfile.pkl', 'rb') as pkl_file:
    regressor = pickle.load(pkl_file)

# Создаём подключение к серверу на локальном хосте:
connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

# Объявляем очередь features
channel.queue_declare(queue='features')

# Создаём функцию callback для обработки данных из очереди y_pred
def callback(ch, method, properties, body):
    print(f'Получен вектор признаков {body}')

# Извлекаем сообщение из очереди features
# on_message_callback показывает, какую функцию вызвать при получении сообщения
channel.basic_consume(
    queue='features',
    on_message_callback=callback,
    auto_ack=True
)
print('...Ожидание сообщений, для выхода нажмите CTRL+C')

# Запускаем режим ожидания прихода сообщений
channel.start_consuming()
```

Запустите оба скрипта одновременно. Для этого откройте два терминала.

- В одном из них перейдите в директорию features/src и запустите скрипт:

  UNIX Windows

  ```
  $ cd features/src
  $ python3 features.py
  ```
- В другом перейдите в директорию model/src и запустите скрипт:

  UNIX Windows

  ```
  $ cd model/src
  $ python3 model.py
  ```

Вы увидите, что после отработки features.py в терминале работающего model.py принимаются сериализованные сообщения. На это указывает `b'` в начале строки.

Для завершения работы второго микросервиса не хватает одной маленькой детали, а именно **предсказания**.

После того как мы извлекли вектор признаков из очереди `features`, нам необходимо сделать для него предсказание моделью `regressor` и отправить результат в очередь `y_pred`. Помним о том, что сообщения приходят в бинарном виде, а значит, следует выполнить десериализацию.

Задание 3.3

0 points possible (ungraded)

В функции callback с помощью функции loads библиотеки *json* десериализуйте пришедшее сообщение (переменная body) и сделайте предсказание расконсервированной моделью regressor. Полученный ответ модели отправьте в очередь y_pred, предварительно объявив её (объявление очереди происходит вне функции callback).

Обратите внимание на три нюанса:

- Десериализованный список необходимо будет привести обратно к *numpy*-массиву размерности $(1, m)$, где m — количество признаков (у нас их десять). Пример:

  ```
  shaped_features = np.array(features).reshape(1, -1)
  ```
- Предсказание модели также находится в *numpy*-массиве — для получения числа обратитесь по индексу 0: pred[0].
- Перед отправкой сообщения в очередь его необходимо сериализовать с помощью функции dumps.

После отправки предсказания в очередь y_pred скрипт должен выводить на экран фразу 'Предсказание {prediction} отправлено в очередь y_pred'.

Для проверки работы микросервиса очистите очереди features и y_true от имеющихся сообщений (если они там есть).

После этого одновременно запустите скрипты *features.py* и *model.py*. В результате выполнения скриптов в очереди y_pred должно появиться предсказание модели.

В качестве ответа на задание введите это **предсказание**. В файле *features.py* должен быть зафиксирован датчик случайных чисел (seed=42), чтобы получать воспроизводимые результаты. Полученное число округлите до **целого**.

Отправить

Решение задания 3.3

Ниже представлена часть скрипта, которая меняется:

```
/* … */
# Объявляем очередь features
channel.queue_declare(queue='features')
# Объявляем очередь y_pred
channel.queue_declare(queue='y_pred')

# Создаём функцию callback для обработки данных из очереди y_pred
def callback(ch, method, properties, body):
    print(f'Получен вектор признаков {body}')
    features = json.loads(body)
    pred = regressor.predict(np.array(features).reshape(1, -1))
    channel.basic_publish(exchange='',
                          routing_key='y_pred',
                          body=json.dumps(pred[0]))
    print(f'Предсказание {pred[0]} отправлено в очередь y_pred')
/* … */
```

Обратите внимание, что сервис features.py при запуске отправляет одно сообщение в очередь и прекращает свою работу. Сервис model.py, напротив, постоянно принимает сообщения после запуска.

Сервис III. Самостоятельная работа

Последний микросервис, который нам осталось реализовать, должен извлекать сообщения из очередей `y_true` и `y_pred` и выводить их на экран.

Задание 3.4

0 points possible (ungraded)

Напишите сервис *metric.py*, который будет читать очереди y_true и y_pred с истинными ответами и предсказаниями. Сообщения из очередей должны выводиться на экран в следующем формате:

```
"Из очереди {имя очереди} получено значение {сообщение}"
```

Примечание. Узнать, из какой очереди получено сообщение, можно с помощью атрибута method.routing_key, где method — второй аргумент функции callback().

Дополните код ниже:

```
importpikaimport___1___# Создаём подключение к серверу на локальном хосте
connection = ___2___(pika.ConnectionParameters(host='localhost'))
channel = connection.___3___

# Объявляем очередь y_true
channel.queue_declare(queue='y_true')
# Объявляем очередь y_pred
channel.queue_declare(queue='___4___')

# Создаём функцию callback для обработки данных из очереди
def callback(ch, method, properties, body):
    print(f'Из очереди {method.routing_key} получено значение {json.___5___(body)}')

# Извлекаем сообщение из очереди y_true
channel.basic_consume(
    queue='y_true',
    on_message_callback=___6___,
    auto_ack=True
)
# Извлекаем сообщение из очереди features
channel.basic_consume(
    queue='y_pred',
    on_message_callback=___7___,
    auto_ack=True
)

# Запускаем режим ожидания прихода сообщений
print('...Ожидание сообщений, для выхода нажмите CTRL+C')
channel.___8___()
```

1 Выберите опцию
5 Выберите опцию
2 Выберите опцию
6 Выберите опцию
3 Выберите опцию
7 Выберите опцию
4 Выберите опцию
8 Выберите опцию

Отправить

После запуска сервис metric.py уйдёт в режим ожидания поступления сообщений в прослушиваемые очереди. Если в очереди уже были данные, он извлечёт их оттуда и выведет на экран. После этого очереди будут очищены до ожидания новых данных.

Протестируйте обмен данных в вашем микросервисном приложении, запустив три сервиса одновременно. Для этого откройте три терминала.

- В первом перейдите в директорию features/src и запустите скрипт features.py:

  UNIX Windows

  ```
  $ cd features/src
  $ python3 features.py
  ```
- Во втором перейдите в директорию model/src и запустите скрипт model.py:

  UNIX Windows

  ```
  $ cd model/src
  $ python3 model.py
  ```
- В третьем перейдите в директорию model/src и запустите скрипт metric.py:

  UNIX Windows

  ```
  $ cd metric/src
  $ python3 metric.py
  ```

Обратите внимание, что сервисы model.py и metric.py работают в фоновом режиме, ожидая поступающих в очереди сообщений, а сервис features.py прекращает работу после запуска. Если вы хотите несколько раз протестировать работу микросервисного приложения, несколько раз запустите скрипт features.py.

В результате запуска сервиса features в терминале, соответствующем сервису model, должно появляться сообщение, что получен вектор признаков из очереди `features` и отправлено предсказание модели в очередь `y_pred`, а в терминале, соответствующем сервису metric, появятся сообщения из очередей `y_pred` и `y_true`.

Резюмируем

В итоге мы получили три микросервиса, которые обмениваются между собой данными через очередь в брокере сообщений RabbitMQ.

- Сервис features имитирует клиентскую часть ML-приложения, которая отправляет данные об объекте на сервис, на котором функционирует модель.
- Сервис model выполняет функцию обработки данных моделью и отправки результата.
- Сервис metric имитирует подсистему логирования работы нашей модели. С её помощью мы можем фиксировать результаты работы модели и сравнивать их с истинными показателями: например, мы можем определять абсолютную ошибку между истиной и предсказанием, следить за изменением этой ошибки во времени, усреднять её и так далее.

Конечно, полученные микросервисы «игрушечные» — в реальных условиях всё устроено несколько сложнее: например, микросервис с клиентской частью приложения должен содержать хотя бы минимальный графический интерфейс, на микросервисе с моделью должна быть предусмотрена предобработка данных перед подачей в модель (она может быть зашита в виде pipeline в pickle-файле с моделью) и ещё много нюансов.

Самое главное — в настоящих микросервисных приложениях всегда должна быть предусмотрена защита от возникающих при работе сервисов ошибок, ведь если, например, из-за неверного формата переданных данных перестанет работать скрипт сервиса с моделью, приложение потеряет свою основную функциональность.

Однако вручную отслеживать работу всех микросервисов — неблагодарное дело. Для этого должна быть предусмотрена система автоматического логирования и мониторинга работы сервисов. В следующем уроке мы рассмотрим инструменты, которые можно использовать для таких целей.

Если у вас остались вопросы по работе с RabbitMQ:

- Официальное руководство от создателей RabbitMQ (англ.).
- То же руководство на русском языке (рус.).
- Официальная документация библиотеки pika (англ.)

‹ Предыдущий урок Следующий урок ›

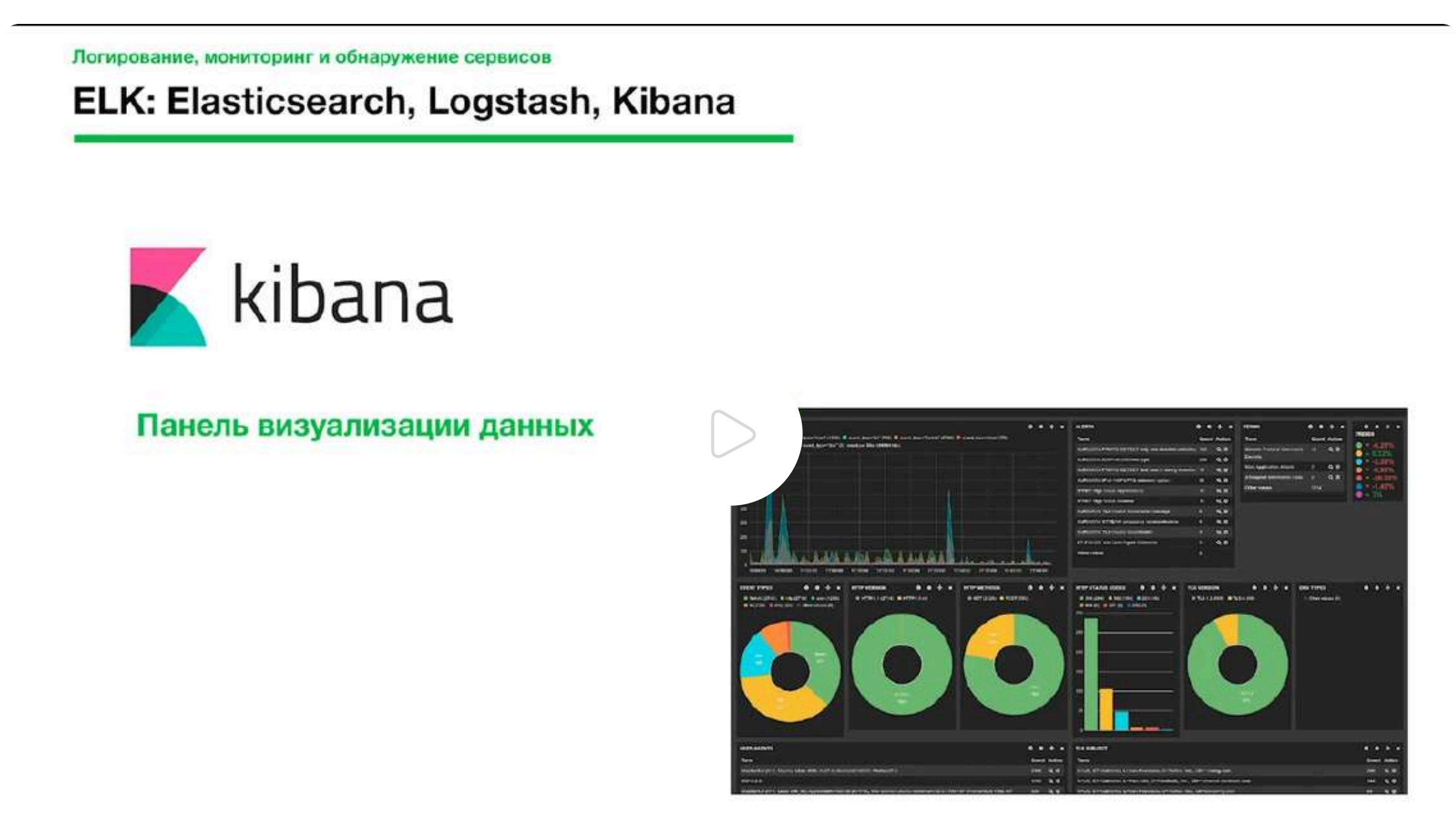
4/9 Логирование, мониторинг, Service Discovery

текущий урок

В разных компаниях логирование, мониторинг и обнаружение сервисов (service discovery) устроены по-разному и зависят от решаемых бизнес-задач. Некоторые компании и вовсе используют самописные решения. В этом уроке мы обсудим несколько популярных инструментов для логирования и мониторинга микросервисных систем. В этой сфере часто появляются новинки — старайтесь их отслеживать.

Наша задача — расширить своё понимание микросервисной архитектуры и выяснить, как микросервисы работают в реальных крупных компаниях.

Конечно, в обязанности дата-сайентиста не входит настройка логирования и мониторинга работы всей системы — это задачи других специалистов, однако общее понимание того, как это работает, должно быть у каждого, кто участвует в разработке приложения или системы.


Логирование, мониторинг и обнаружение сервисов
ELK: Elasticsearch, Logstash, Kibana
kibana
Панель визуализации данных

Мониторинг микросервисов — задача непростая: в режиме реального времени нужно отслеживать состояние как отдельных компонентов, так и системы в целом. Задача усложняется, если нужно следить не только за техническими показателями, но и за метриками, значимыми для бизнеса.

Если посмотреть на микросервисную архитектуру со стороны, можно увидеть, что каждый микросервис, по сути, живёт своей жизнью, и зачастую они поддерживаются разными командами. Поэтому при организации их взаимодействия что-то обязательно идёт не так.

Конечно, в хорошо продуманной микросервисной архитектуре каждый сервис в случае **ошибки** отправит вызывающему сервису подробности произошедшего, а тот, в свою очередь, завернёт это в ответ с указанием причины. Однако, чтобы всегда быть в курсе событий, нам необходимо постоянно отслеживать все цепочки взаимодействия.

В идеале каждый сервис хранит логи сообщений независимо от других, а затем логи всех сервисов собираются специальным агрегатором.

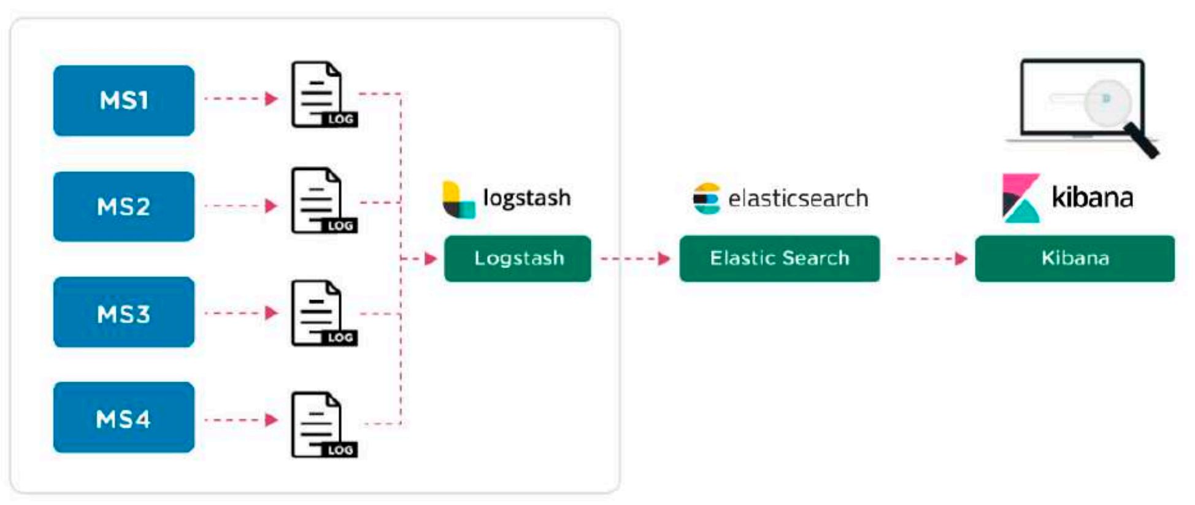
Стек ELK

Одним из наиболее известных наборов инструментов для централизованного логирования является стек ELK: Elasticsearch, Logstash и Kibana.

- Elasticsearch — это не только база данных NoSQL, но и полноценный движок полнотекстового поиска и аналитики. Написан на языке Java.
- Logstash — это инструмент для сбора данных с открытым исходным кодом и возможностью конвейерной обработки данных в реальном времени. Logstash позволяет динамически идентифицировать данные из различных источников и нормализовать их с помощью выбранных фильтров.
- Kibana — это панель визуализации данных. Данные представляются в виде различных диаграмм.

Все указанные инструменты работают в связке:

- Logstash отвечает за агрегацию логов для каждого микросервиса и поставляет входящий поток данных в Elasticsearch для хранения, классификации и поиска.
- Kibana получает доступ к данным Elasticsearch для их визуализации в различных форматах, например в виде дашборда.



Примечание. Помимо стека ELK, существует связка Prometheus* и Grafana.

Prometheus используется для сбора и хранения метрик, а также для настройки системы мониторинга и оповещений (alerting). А Grafana помогает построить дашборд для визуализации данных.

* Если какие-то ссылки не открываются, обратитесь к координатору.

Service discovery

Представим, что у нас есть десятки тысяч сервисов, распределённых по тысячам серверов. Как сервисы будут находить друг друга? Как один сервис будет определять состояние другого? Конечно, хотелось бы, чтобы это происходило автоматически, например с помощью журнала, в который сервисы вносили бы своё состояние и адреса.

Для решения этой задачи используются протоколы обнаружения сервисов (Service Discovery). Их дополняет ряд инструментов, например:

- Consul,
- etcd,
- ZooKeeper.

Consul от компании HashiCorp — один из известных инструментов для service discovery. Он позволяет обнаруживать сервисы, проверять их доступность и управлять конфигурацией сервисной инфраструктуры.

Логирование — полезный инструмент не только в разработке микросервисов.

Существует готовая библиотека на Python для логирования работы сервисов — logging. Использование этой библиотеки уже давно стало **правилом хорошего тона** при любой разработке.

Мы рекомендуем вам ознакомиться с пошаговыми руководствами по работе с библиотекой logging:

- Инструкция на Хабре (рус.).
- Официальная документация (англ.).

Это задание для самостоятельной работы, за него не предусмотрены баллы. Однако оно поможет вам закрепить новый материал.

Задание 4.1

0 points possible (ungraded)

Соотнесите инструмент из стека *ELK* с его назначением.

Инструмент для сбора данных:

Logstash ✓

Инструмент для визуализации:

Kibana ✓

База данных полнотекстового поиска:

Elasticsearch ✓

Show answer

Отправить

✓ Верных

Теперь вы знаете, какими инструментами можно пользоваться для мониторинга работы микросервисов, чтобы обеспечить их надёжность.

Нам осталось рассмотреть последнюю тему по работе с сервисами — оркестрация контейнеров при микросервисной разработке. Давайте перейдём к ней.

< Предыдущий урок Следующий урок >

5/9 Оркестрация. Docker Compose

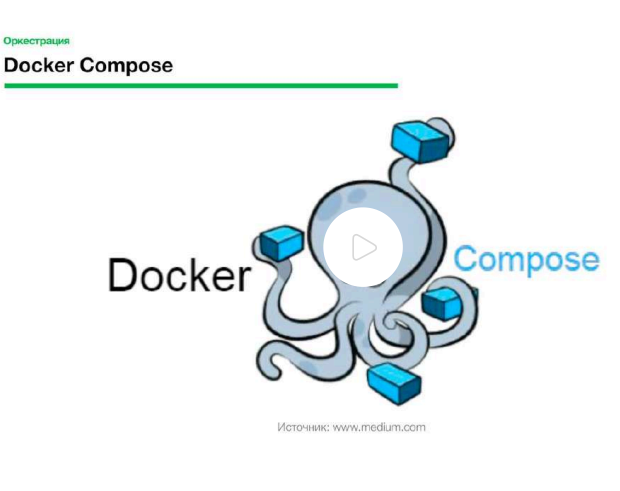
Теперь, когда у нас есть микросервисное приложение, наша следующая цель — поместить каждый сервис в собственный docker-контейнер. Для этого нам нужно прописать последовательность сборки и запуска этих контейнеров, а также указать, как они взаимодействуют друг с другом.

> Процесс управления несколькими контейнерами (микросервисами) называется **оркестрацией**, а инструменты, с помощью которых можно организовать этот процесс, называются **оркестраторами** или **системами оркестрации**.

Термин не зря происходит из музыки — такие системы можно сравнить с дирижёрами оркестра: они организуют расположение и координируют взаимосвязь инструментов в одном проекте, распределяют задачи между ними и контролируют их выполнение.

Давайте научимся пользоваться такими системами.

Docker compose

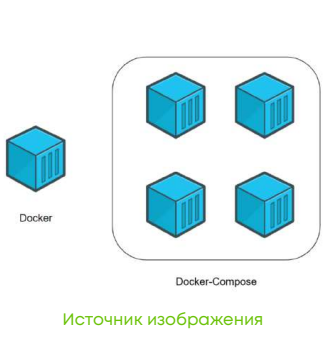


При разработке приложений на основе **микросервисной архитектуры** появляется множество сервисов, каждый из которых живёт в отдельном контейнере. Нам необходимо автоматизировать или максимально облегчить процессы сборки, запуска и остановки таких контейнеров.

К счастью, для решения этой задачи уже придумана масса инструментов.

> Один из таких инструментов — Docker Compose. Он позволяет управлять запуском и взаимодействием нескольких контейнеров на одной машине. Его основная задача — помогать разворачивать проекты, состоящие из нескольких контейнеров, и эффективно управлять ими.

Проще говоря, Docker Compose нужен, когда есть несколько контейнеров, которые обмениваются между собой данными. Это как раз пример нашей микросервисной архитектуры.



Источник изображения

Работа с Compose предполагает несколько этапов:

1. Предварительная подготовка: создание Dockerfile для каждого сервиса, чтобы их можно было воспроизвести где угодно.
2. Создание описания всех сервисов, которые требуется запускать вместе, в специальном файле docker-compose.yml.
3. Сборка и запуск проекта через docker-compose, чтобы приложение развернулось автоматически.

Давайте посмотрим, как реализуется каждый из этих этапов на практике.

В этой части мы будем практиковаться управлять сервисами, которые писали в практике по RabbitMQ (скрипты features.py, metric.py, model.py).

Предварительная подготовка

Первый шаг — подготовить наши микросервисы для контейнеризации, создать файлы с зависимостями, а затем создать Dockerfile для каждого сервиса.

Начнём с небольшой модернизации файлов.

1. Во-первых, теперь взаимодействие сервисов на базе RabbitMQ будет происходить не через localhost, а средствами docker-compose, поэтому необходимо задать другую точку взаимодействия. Подробнее о том, почему так происходит, мы поговорим ниже, когда будем реализовывать сам compose-файл.

   Пока что договоримся, что адрес, через который сервисы будут подключаться к очереди, будет называться rabbitmq. Это означает, что во всех трёх сервисах при инициализации параметров соединения в `ConnectionParameters` нам необходимо будет поменять `'localhost'` на `'rabbitmq'`.
2. Во-вторых, мы хотим, чтобы все наши сервисы работали в фоновом режиме.

   Например, сервисы model и metric постоянно мониторят поступление сообщений в очереди, а вот сервис features завершает свою работу сразу после отправки сообщения в очередь. Поэтому давайте модернизируем features так, чтобы он в бесконечном цикле выбирал случайные объекты из выборки и отправлял вектор признаков в соответствующую очередь.
3. В-третьих, давайте обернём код каждого сервиса в конструкцию try-except, чтобы в случае, если при исполнении скрипта сервиса произошла ошибка, он не остановил свою работу, а вывел эту ошибку на экран.

Итоговый код будет иметь примерно следующий вид:

Файл ./features/src/features.py

```
import pika
import numpy as np
import json
from sklearn.datasets import load_diabetes

# Создаём бесконечный цикл для отправки сообщений в очередь
while True:
    try:
        # Загружаем датасет о диабете
        X, y = load_diabetes(return_X_y=True)
        # Формируем случайный индекс строки
        random_row = np.random.randint(0, X.shape[0]-1)

        # Создаём подключение по адресу rabbitmq:
        connection =
pika.BlockingConnection(pika.ConnectionParameters('rabbitmq'))
        channel = connection.channel()

        # Создаём очередь y_true
        channel.queue_declare(queue='y_true')
        # Создаём очередь features
        channel.queue_declare(queue='features')

        # Публикуем сообщение в очередь y_true
        channel.basic_publish(exchange='',
                              routing_key='y_true',
                              body=json.dumps(y[random_row]))
        print('Сообщение с правильным ответом отправлено в очередь')

        # Публикуем сообщение в очередь features
        channel.basic_publish(exchange='',
                              routing_key='features',
                              body=json.dumps(list(X[random_row])))
        print('Сообщение с вектором признаков отправлено в очередь')

        # Закрываем подключение
        connection.close()
    except:
        print('Не удалось подключиться к очереди')
```

Файл ./model/src/model.py

```
import pika
import pickle
import numpy as np
import json

# Читаем файл с сериализованной моделью
with open('myfile.pkl', 'rb') as pkl_file:
    regressor = pickle.load(pkl_file)

try:
    # Создаём подключение по адресу rabbitmq:
    connection =
pika.BlockingConnection(pika.ConnectionParameters(host='rabbitmq'))
    channel = connection.channel()

    # Объявляем очередь features
    channel.queue_declare(queue='features')
    # Объявляем очередь y_pred
    channel.queue_declare(queue='y_pred')

    # Создаём функцию callback для обработки данных из очереди
    def callback(ch, method, properties, body):
        print(f'Получен вектор признаков {body}')
        features = json.loads(body)
        pred = regressor.predict(np.array(features).reshape(1, -1))
        channel.basic_publish(exchange='',
                      routing_key='y_pred',
                      body=json.dumps(pred[0]))
        print(f'Предсказание {pred[0]} отправлено в очередь y_pred')

    # Извлекаем сообщение из очереди features
    # on_message_callback показывает, какую функцию вызвать при получении
сообщения
    channel.basic_consume(
        queue='features',
        on_message_callback=callback,
        auto_ack=True
    )
    print('...Ожидание сообщений, для выхода нажмите CTRL+C')

    # Запускаем режим ожидания прихода сообщений
    channel.start_consuming()
except:
    print('Не удалось подключиться к очереди')
```

Файл ./metric/src/metric.py

```
import pika
import json

try:
    # Создаём подключение к серверу на локальном хосте
    connection =
pika.BlockingConnection(pika.ConnectionParameters(host='rabbitmq'))
    channel = connection.channel()

    # Объявляем очередь y_true
    channel.queue_declare(queue='y_true')
    # Объявляем очередь y_pred
    channel.queue_declare(queue='y_pred')

    # Создаём функцию callback для обработки данных из очереди
    def callback(ch, method, properties, body):
        print(f'Из очереди {method.routing_key} получено значение
{json.loads(body)}')

    # Извлекаем сообщение из очереди y_true
    channel.basic_consume(
        queue='y_true',
        on_message_callback=callback,
        auto_ack=True
    )
    # Извлекаем сообщение из очереди y_pred
    channel.basic_consume(
        queue='y_pred',
        on_message_callback=callback,
        auto_ack=True
    )

    # Запускаем режим ожидания прихода сообщений
    print('...Ожидание сообщений, для выхода нажмите CTRL+C')
    channel.start_consuming()
except:
    print('Не удалось подключиться к очереди')
```

Далее нам необходимо подготовить модернизированные сервисы к контейнеризации. Для этого внутри каждой директории микросервиса создадим Dockerfile и файлы с зависимостями — requirements.txt.

```
microservice_architecture
    └features
        └src
            └features.py
        └Dockerfile
        └requirements.txt
    └model
        └src
            └model.py
            └myfile.pkl
        └Dockerfile
        └requirements.txt
    └metric
        └src
            └metric.py
        └Dockerfile
        └requirements.txt
```

Начнём с файлов с зависимостями.

> Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 5.1

0 points possible (ungraded)

От каких библиотек зависит сервис *features*?

Отметьте только те библиотеки, которые **нуждаются в дополнительной установке**, то есть не входят в стандартную библиотеку *Python*.

- ☑ A *numpy*
- ☐ B *pandas*
- ☐ C *json*
- ☑ D *scikit-learn*
- ☑ E *pika*
- ☐ F *pickle*
- ☐ G *joblib*

✓

Ответ
Верно:
A Верно.
D Верно.
E Верно.

Show answer

Отправить

✓ Верных

Задание 5.2

0 points possible (ungraded)

От каких библиотек зависит сервис *model*?

Отметьте только те библиотеки, которые **нуждаются в дополнительной установке**, то есть не входят в стандартную библиотеку *Python*.

Обратите внимание, что в скрипте *model.py* используется десериализованная модель из библиотеки *scikit-learn*. Подумайте, можно ли использовать эту модель, если в окружении сервиса не установлена данная библиотека.

- ☑ A *numpy*
- ☐ B *pandas*
- ☐ C *json*
- ☑ D *scikit-learn*
- ☑ E *pika*
- ☐ F *pickle*
- ☐ G *joblib*

✓

Ответ
Верно:
A Верно.
D Верно.
E Верно.

Show answer

Отправить

✓ Верных

Задание 5.3

0 points possible (ungraded)

От каких библиотек зависит сервис *metric*?

Отметьте только те библиотеки, которые **нуждаются в дополнительной установке**, то есть не входят в стандартную библиотеку *Python*.

- ☐ A *numpy*
- ☐ B *pandas*
- ☐ C *json*
- ☐ D *scikit-learn*
- ☑ E *pika*
- ☐ F *pickle*
- ☐ G *joblib*

✓

Ответ
Верно:
E Верно.

Show answer

Отправить

✓ Верных

Итак, мы разобрались с тем, какие зависимости нужны в каждом из контейнеров. Ниже представлены примеры файлов requirements.txt с зависимостями для каждого сервиса.

Файл ./features/requirements.txt

```
numpy>=1.23
scikit-learn>=1.1
pika>=1.3
```

Файл ./model/requirements.txt

```
numpy>=1.23
scikit-learn>=1.1
pika>=1.3
```

Файл ./metric/requirements.txt

```
pika>=1.3
```

> **Примечание.** Строго говоря, устанавливать библиотеку numpy для сервисов features и model было необязательно, так как она поставляется вместе с библиотекой scikit-learn.

> **Следующий шаг** — создать Dockerfile. Мы неслучайно создали типизированные папки для каждого микросервиса — благодаря этому будет удобно составлять шаблонный Dockerfile.

Шаги сборки образа контейнера:

1. Подключить базовый образ. Мы будем использовать образ Python версии 3.9.
2. Задать рабочую директорию контейнера. Назовём её /app.
3. Скопировать содержимое папки src в рабочую директорию контейнера.
4. Скопировать файл с зависимостями в рабочую директорию контейнера.
5. Установить все необходимые зависимости.
6. Запустить скрипт для работы сервиса.

Задание 5.4

0 points possible (ungraded)

На основе описанных выше шагов заполните пропущенные директивы в *Dockerfile* для сервиса *features*:

```
FROM ___1___
WORKDIR /usr/src/app
COPY ___2___ ___3___
COPY ./requirements.txt ___4___
RUN pip install --no-cache-dir -r ___5___
CMD [ "python", "___6___" ]
```

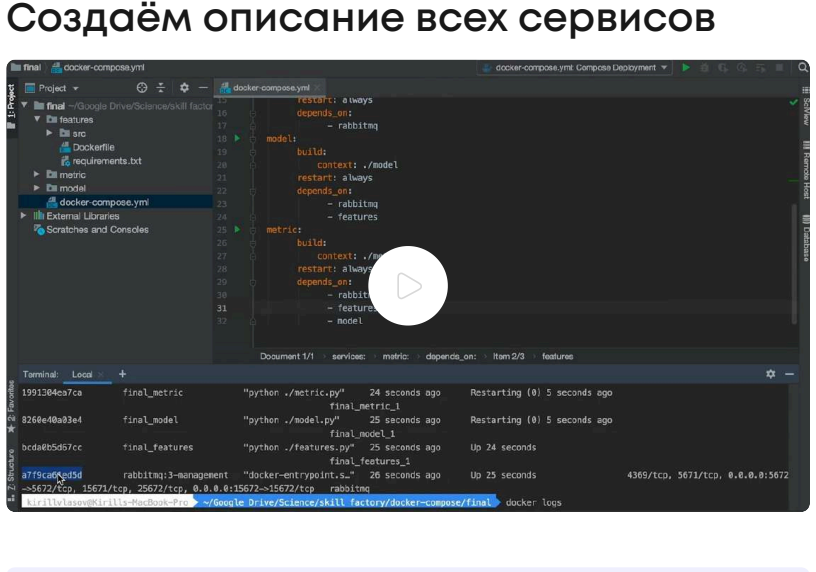
1 Выберите опцию
2 Выберите опцию
3 Выберите опцию
4 Выберите опцию
5 Выберите опцию
6 Выберите опцию

Отправить

> **Примечание.** Файлы Dockerfile для сервисов model и metric будут выглядеть аналогично, за тем исключением, что в директиве CMD указываются соответствующие скрипты (model.py и metric.py). Составьте инструкции по сборке образа для этих двух сервисов самостоятельно.
>
> Вы сами можете протестировать свои Dockerfile, собрав их образы с помощью команды `docker build`. Однако это не обязательно — мы укажем процедуру сборки в docker-compose.
>
> Обратите внимание, что при вызове команды для создания образа (`docker_build`) из корневой директории проекта (у нас она называется `microservice_architecture`) необходимо указать путь до папки, где находится Dockerfile. Например, команда для сборки образа сервиса features будет выглядеть следующим образом:
>
> ```
> $ docker build -t feature_image ./features
> ```
>
> **Важно!** Запускать сами контейнеры пока не нужно, так как мы ещё не прописали логику взаимодействия между сервисами — за это как раз и будет отвечать docker-compose.

Предварительная подготовка завершена — инструкции по сборке образов контейнеров готовы. Переходим к описанию регламентов взаимодействия наших сервисов — иначе говоря, давайте знакомиться с оркестрацией.

Создаём описание всех сервисов



> **Примечание.** Перед тем как приступить к изучению Docker Compose, убедитесь, что он установлен на вашем компьютере, введя в терминале или командной строке:
>
> ```
> $ docker-compose version
> ```
>
> Если Docker Compose не установлен, сделайте это с помощью официального руководства.

Ранее мы запускали очередь RabbitMQ и ещё три сервиса на Python. Как вы помните, мы рекомендовали использовать docker для запуска очереди с помощью команды:

```
$ docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-
management
```

Однако теперь, помимо образа `rabbitmq:3-management`, который можно воспринимать как отдельный микросервис для управления очередями, у нас есть ещё три почти контейнеризированных сервиса. Запускать каждый сервис отдельной командой и открывать десятки терминалов, в которых выводятся сообщения от контейнера, крайне неудобно. Поэтому хотелось бы написать некоторую инструкцию по сборке и запуску всех контейнеров с возможностью задавать последовательность. Такие инструкции прописываются в файлах docker-compose.yml. Давайте рассмотрим синтаксис такого файла.

Создайте в корневой директории проекта (у нас она называется `microservice_architecture`) файл docker-compose.yml с помощью любого текстового редактора и напишите в нём:

Файл ./docker-compose.yml

```
version: '3.7'
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    hostname: rabbitmq
    restart: always
    ports:
       - 5672:5672
       - 15672:15672
```

Сравните содержимое файла docker-compose.yml с docker-командой для запуска RabbitMQ. Много сходств, не так ли?

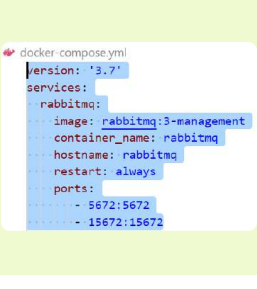
Поясним, что мы написали:

- `version: '3.7'` — версия docker-compose;
- `services` — начало блока, в котором будут описаны все сервисы;
- `rabbitmq` — имя сервиса (контейнера) rabbitmq;
- `image` — образ, на основе которого будет запускаться контейнер;
- `restart: always` — в случае падения контейнер должен перезапускаться автоматически;
- `hostname` — имя хоста;

  Почему нельзя указать в качестве хоста localhost?
- `ports` — TCP-порты, которые использует контейнер.

> **Примечание.** Можно задать и другие условия перезапуска:
>
> `restart: "no"` — контейнер не будет перезапускаться ни при каких обстоятельствах (опция по умолчанию);
>
> `restart: on-failure` — перезапуск контейнера, если предыдущий запуск завершился ошибкой.

> **Обратите внимание.** В docker-compose.yml важны отступы. Для того чтобы обозначить вложенность настроек друг в друга, используются два или более пробелов. Например, в видео выше эксперт использует четыре и даже шесть пробелов для наглядности.



```
version: '3.7'
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    hostname: rabbitmq
    restart: always
    ports:
      - 5672:5672
      - 15672:15672
```

> **Следующий шаг** — прописать сборку и запуск наших «самописных» сервисов.

Разберём структуру описания compose-инструкций на примере сервиса features. Снова откроем файл docker-compose и добавим в него описание features (ещё раз обратите внимание на отступы).

Файл ./docker-compose.yml

```
version: '3.7'
version: '3.7'
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    hostname: rabbitmq
    restart: always
    ports:
       - 5672:5672
       - 15672:15672
  features:
    build:
      context: ./features
    restart: always
    depends_on:
      - rabbitmq
```

Мы дописали следующее:

- Директива `build` указывает, что образ требуется собрать перед запуском контейнера. При этом `context` указывает путь на размещение Dockerfile относительно файла docker-compose.yml.
- Директива `depends_on` указывает на зависимость от других сервисов и означает, что compose не будет запускать сервис features без запущенного rabbitmq.

Настройка оставшихся сервисов производится аналогично.

Задание 5.5 (самопроверка)

Самостоятельно добавьте в файл docker-compose.yml описание работы двух оставшихся двух сервисов — model и metric.

Перед тем как перейти к составлению инструкций, поразмышляйте над следующими вопросами:

- Должны ли docker-образы для данных сервисов собираться заранее? Если да, то где лежат их Dockerfile?
- Когда должен перезапускаться каждый из сервисов?
- Наконец, самое главное: какие сервисы должны быть запущены предварительно? От каких сервисов зависит рассматриваемый сервис?

После того как вы самостоятельно составите файл docker-compose.yml, загляните в ответ.

Ответ

Сервис model:

- Необходима предварительная сборка образа на основе Dockerfile, который лежит в директории ./models.
- Сервис должен автоматически перезапускаться при возникновении в нём ошибок.
- Сервис не должен запускаться раньше, чем запускается сервисы rabbitmq и features.

Сервис metric:

- Необходима предварительная сборка образа на основе Dockerfile, который лежит в директории ./metric.
- Сервис должен автоматически перезапускаться при возникновении в нём ошибок.
- Сервис не должен запускаться раньше, чем запускается сервисы rabbitmq, features и model.

Файл docker-compose.yml будет иметь следующий вид:

```
version: '3.7'
services:
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    hostname: rabbitmq
    restart: always
    ports:
       - 5672:5672
       - 15672:15672
  features:
    build:
      context: ./features
    restart: always
    depends_on:
      - rabbitmq
  model:
    build:
      context: ./model
    restart: always
    depends_on:
      - rabbitmq
      - features
  metric:
    build:
      context: ./metric
    restart: always
    depends_on:
      - rabbitmq
      - features
      - model
```

Сборка и запуск приложения

После того как в docker-compose.yml внесены все необходимые инструкции, проект необходимо собрать. Этот шаг напоминает использование команды `docker build`, но соответствующая команда имеет отношение к нескольким сервисам. Перейдите в терминале в корневую директорию вашего проекта и запустите в нём команду:

```
$ docker-compose build
```

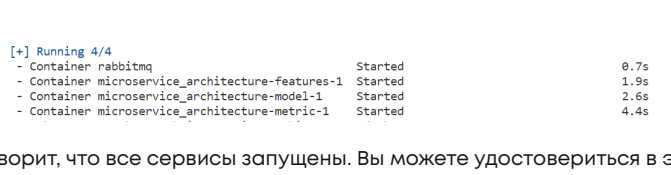
После этого запустится сборка всех необходимых образов — дождитесь её завершения.

Когда всё будет готово, останется лишь запустить проект. Этот шаг аналогичен шагу, на котором при работе с отдельными контейнерами выполняется команда `docker run`.

Для запуска проекта используется команда `up`. Однако если просто запустить её, терминал будет бесконечно обновляться, так как в него будут постоянно выводиться логи сервисов. Чтобы отвязать логи от терминала и запустить сервисы в фоновом режиме, воспользуемся ключом `-d`:

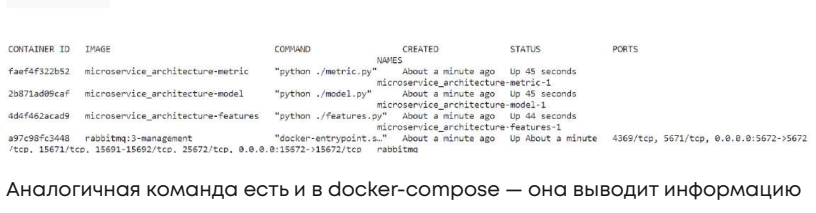
```
$ docker-compose up -d
```

Если всё сделано верно, после запуска команды вы увидите примерно следующее сообщение:



```
[+] Running 4/4
 - Container rabbitmq                                 Started                    0.7s
 - Container microservice_architecture-features-1     Started                    1.9s
 - Container microservice_architecture-model-1        Started                    2.6s
 - Container microservice_architecture-metric-1       Started                    4.4s
```

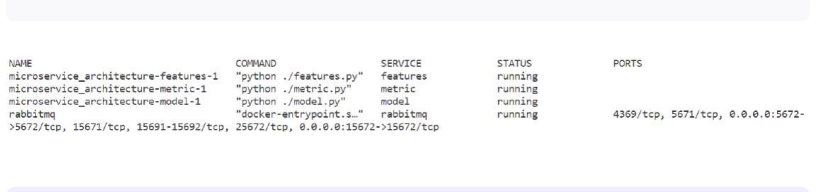
Оно говорит, что все сервисы запущены. Вы можете удостовериться в этом и вывести список активных контейнеров, воспользовавшись командой `docker ps`:



```
CONTAINER ID   IMAGE                                 COMMAND                  CREATED              STATUS              PORTS
                                                                                                                      NAMES
faef4f322b52   microservice_architecture-metric      "python ./metric.py"     About a minute ago   Up 46 seconds
                                                                              microservice_architecture-metric-1
2b871ad80caf   microservice_architecture-model       "python ./model.py"      About a minute ago   Up 46 seconds
                                                                              microservice_architecture-model-1
6dbf462acad0   microservice_architecture-features    "python ./features.py"   About a minute ago   Up About a minute
                                                                              microservice_architecture-features-1
a97c90fc2448   rabbitmq:3-management                 "docker-entrypoint.s…"   About a minute ago   Up About a minute   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672
/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp   rabbitmq
```

Аналогичная команда есть и в docker-compose — она выводит информацию о запущенных сервисах:

```
$ docker-compose ps
```



```
NAME                                   COMMAND                  SERVICE             STATUS              PORTS
microservice_architecture-features-1   "python ./features.py"   features            running
microservice_architecture-metric-1     "python ./metric.py"     metric              running
microservice_architecture-model-1      "python ./model.py"      model               running
rabbitmq                               "docker-entrypoint.s…"   rabbitmq            running             4369/tcp, 5671/tcp, 0.0.0.0:5672-
>5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp
```

> **Примечание.** Содержимое таблицы, получаемой в результате выполнения команды `docker ps`, мы обсуждали в предыдущем модуле, когда изучали docker-команды.

Ещё несколько полезных команд

С помощью команды `docker logs <ID контейнера>` можно посмотреть логи контейнера. Вместо ID необходимо подставить идентификатор запущенного контейнера. Например, следующая команда запускает контейнер, соответствующий сервису metric (ваш ID контейнера может отличаться):

```
$ docker logs faef4f322b52
```

Аналогичная команда есть в docker-compose, только вместо ID контейнера нужно указать имя сервиса. Например, следующая команда позволяет отследить логи сервиса metric:

```
$ docker-compose logs -f metric
```

Чтобы остановить запущенные сервисы, следует воспользоваться командой `down`:

```
$ docker-compose down
```

Иногда требуется пересобирать образы, например при изменении кода. Для этого удобно пользоваться ключом `--build`. В таком случае команда запуска будет выглядеть следующим образом:

```
$ docker-compose up -d --build
```

Задание 5.6 (самопроверка)

В функцию callback в файле metric.py добавьте код, который будет записывать логи, содержащие информацию о том, из какой очереди было получено сообщение, в файл labels_log.txt. Пример формата логов:

```
'Из очереди <имя очереди> получено значение <тело сообщения>'
```

Файл с логами разместите в директории logs рядом с вашим compose-файлом. В итоге у вас должна получиться примерно следующая структура директории:

```
microservice_architecture
    └features
        └src
            └features.py
        └Dockerfile
        └requirements.txt
    └model
        └src
            └model.py
            └myfile.pkl
        └Dockerfile
        └requirements.txt
    └metric
        └src
            └metric.py
        └Dockerfile
        └requirements.txt
    └logs
        └labels_log.txt
    └docker_compose.yml
```

Чтобы файл был доступен из локальной файловой системы, в compose-файле в разделе сервиса metric необходимо прописать нужную папку с помощью директивы volumes, которая является аналогом `-v` в команде `docker run`. Чтобы связать локальную директорию и директорию контейнера, используется следующий формат:

```
volumes:
      - ./logs/:/usr/src/app/logs/
```

Если вы захотите поближе познакомиться с директивой volumes, посмотрите пример её использования в официальной документации по синтаксису compose-файла.

О том, как организовать запись в файл, вы можете почитать здесь.

Ответ

Файл ./metric/src/metric.py

```
…

def callback(ch, method, properties, body):
    answer_string = f'Из очереди {method.routing_key} получено
значение {json.loads(body)}'
    with open('./logs/labels_log.txt', 'a') as log:
        log.write(answer_string +'\n')

…
```

Файл docker-compose.yml

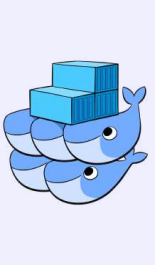
```
/*Описание сервисов*/
…
  metric:
    build:
      context: ./metric
    restart: always
    depends_on:
      - rabbitmq
      - features
      - model
    volumes:
      - ./logs/:/usr/src/app/logs/
```

Для завершения темы оркестрации нам осталось лишь упомянуть инструменты оркестрации, которыми пользуются профессиональные DevOps-специалисты, занимающиеся автоматизацией развёртывания приложений.

Другие инструменты оркестрации

Мы подробно разобрали Docker Compose — достаточно мощный и эффективный инструмент для запуска нескольких сервисов на одной машине. Однако он имеет пусть один, но весьма существенный недостаток: по умолчанию в нём **нет средств для горизонтального масштабирования**.

> В таких случаях больше подойдёт Docker Swarm, созданный специально для развёртывания контейнеров на кластере. По сути, он преобразует набор хостов Docker в один **последовательный виртуальный хост** под названием Swarm. Docker Swarm обеспечивает высокую производительность работы, равномерно распределяя нагрузку внутри кластера.



> Однако наиболее популярным средством для оркестрации контейнеров в кластере является Kubernetes. В литературе можно встретить акроним K8s.
>
> Kubernetes — инструмент с открытым исходным кодом. Его начали разрабатывать в Google, а сейчас сервис поддерживается многими компаниями, среди которых Microsoft, RedHat, IBM и, конечно, сам Docker.

Kubernetes упрощает управление развёртыванием, сетевой маршрутизацией, расходом ресурсов, балансировкой нагрузки и отказоустойчивостью запускаемых приложений.

На этом мы завершаем знакомство с оркестрацией контейнеров. Итак, в этом уроке мы:

- познакомились с инструментом оркестрации контейнеров Docker Compose;
- посмотрели, как выглядит простейший файл docker-compose с настройками сборки, запуска и взаимодействия между сервисами;
- научились запускать оркестратор Docker Compose и просматривать логи каждого сервиса;
- узнали, какие ещё существуют сервисы для оркестрации.

> Также рекомендуем вам заглянуть в **дополнительные источники**:
>
> - Ещё один пример оркестрации микросервисного приложения с двумя сервисами (клиентская и серверная части приложения).
> - Официальная документация по синтаксису compose-файла.
> - Официальная документация по командам docker-compose

Предыдущий урок
Следующий урок

6/9 Обзор инструментов: DVC и MLflow

текущий урок

В этом уроке мы рассмотрим инструменты, с которыми вы можете столкнуться в своей профессиональной деятельности на различных этапах жизненного цикла моделей машинного обучения.

Процесс классической разработки программного обеспечения довольно **прямолинеен**: мы просто уточняем требования, проводим разработку и сдаём результат.

В Data Science этот процесс скорее **цикличен**: мы постоянно вносим изменения в модели и пытаемся улучшить результат, разбираясь в природе данных, бизнес-процессе, признаках и их влиянии на результат.

Есть ещё несколько отличий разработки в Data Science от классической разработки ПО:

- Никто не может гарантировать, что полученное с помощью методов DS решение будет лучше, чем существующее.
- Никто не может быть уверен, что решение возможно будет внедрить или что задача вообще может быть решена.

Чтобы разработка DS-продукта была качественной и проходила без потрясений, принято использовать инструменты управления жизненным циклом модели. Давайте познакомимся с ними поближе.

DVC

Вы уже знакомы с системой управления версиями Git. Слабая сторона этой системы в том, что она не предназначена для работы со слишком большими файлами, например с моделями. Наверняка вы уже успели столкнуться с тем, что модель может занимать достаточно много места на диске, особенно когда мы говорим о глубоких нейросетях.

Представьте, что вы работаете над Data Science-проектом, в котором постоянно меняются входные данные для моделирования, потому что коллеги в вашей команде параллельно настраивают сбор данных и их предварительную обработку. Вам просто не обойтись без системы, которая поможет **версионировать** их.

DVC* позволяет управлять версиями файлов и каталогов данных, промежуточными результатами и моделями ML с использованием системы Git. При этом в репозитории сохраняется не содержимое файлов, а только их **метаданные**.

* Если какие-то ссылки не открываются, обратитесь к координатору.

Для фактического хранения данных DVC поддерживает различные типы **удалённого хранилища**: например, можно использовать обычные (при помощи SSH-доступа) или облачные серверы, такие как Amazon S3, Microsoft Azure, Google Диск и другие. Это позволяет легко сохранять данные и обмениваться ими.

MLFLOW

> MLflow — это платформа с открытым исходным кодом. Она позволяет мониторить эксперименты, обеспечивает воспроизводимость и помогает развёртывать модели.

Компоненты MLflow:

- MLflow Tracking позволяет логировать эксперименты и сохранять для каждого из них код, данные, настройки алгоритмов и результаты обучения.
- MLflow Projects требуется для упаковки кода обученных моделей в специальный формат.
- MLflow Models отвечает за деплой обученных моделей машинного обучения.
- Model Registry — это, по сути, хранилище для моделей. В нём можно сохранить информацию и метаданные для каждой модели и управлять ими.

Основная особенность MLflow заключается в **трекинге экспериментов**: мы фиксируем модель и её параметры и сохраняем метрики на выходе. Однако деплой с помощью MLflow нельзя назвать **production-ready**: например, в нём нет мониторинга запущенных образов.

В этом уроке мы вкратце познакомились с двумя инструментами, которые упрощают работу с ML-моделями. Вы сможете подробно изучить все их возможности и воспользоваться ими на практике уже в своей профессиональной деятельности. Нам осталось обсудить, как оценивать эффективность работы модели с точки зрения бизнеса — мы займёмся этим в следующем уроке.

‹ Предыдущий урок Следующий урок ›

7/9 Оценка эффективности модели. Бизнес-метрики

текущий урок

Итак, мы обучили ML-модель и развернули её в продакшене — она работает и отдаёт бизнесу предсказания. Как оценивать эффективность модели? Как выбирать метрики в зависимости от решаемой задачи? Как модель влияет на итоговые показатели?

Задача 1

Для примера возьмём задачу предотвращения оттока клиентов в телекоммуникационной компании. Наша цель — выявлять клиентов в группе риска, чтобы компания вовремя предлагала им скидку.

Для начала вспомним, что такое **матрица ошибок**.

- Как вы помните, False Positive является **ошибкой первого рода**, когда алгоритм предсказывает, что объект относится к положительному классу, но на самом деле объект таковым не является.
- False Negative, в свою очередь, относится к **ошибкам второго рода**, когда алгоритм присваивает объекту положительного класса отрицательную метку.

Метрика точности

$$precision = \frac{TP}{TP+FP}$$

Доля объектов, которые алгоритм предсказал как положительные и которые действительно являются таковыми.

Метрика полноты

$$recall = \frac{TP}{TP+FN}$$

Доля положительных объектов, которую выделил классификатор.

Нетрудно догадаться, что precision оптимизирует ошибку первого рода, а recall — второго. Какую ошибку важнее оптимизировать с точки зрения бизнеса?

Конечно, можно взять F-меру:

$$F_\beta = (1+\beta^2)\,\frac{precision \times recall}{\beta^2 precision + recall}$$

$$F = \frac{2 \times precision \times recall}{precision + recall}$$

Вес метрик precision и recall определяет β:

- Если $0 < \beta < 1$, то precision важнее.
- Если $\beta > 1$, то recall важнее.
- Если $\beta = 1$, это среднее гармоническое.

Примечание. F-мера достигает максимума при precision и recall, равных единице, и близка к нулю, если один из аргументов близок к нулю.

Вопрос в том, как выбрать β. Давайте определимся с ценой ошибки.

Это задание для самостоятельной работы, за него не предусмотрены баллы. Однако оно поможет вам закрепить новый материал.

Задание 8.1

0 points possible (ungraded)

Если мы предскажем, что клиент собирается уходить, выдадим ему скидку, а уходить он на самом деле не собирался, то какого рода ошибку мы совершим?

● Первого рода

○ Второго рода

✓

Ответ
Верно: В таком случае мы понесём расходы, равные размеру скидки.

Show answer

Отправить

✓ Верных

Ошибиться с предсказанием собирающегося уйти от нас клиента означает потерять выручку, соответствующую его LTV (Lifetime Value). Метрика LTV определяется бизнесом. Очевидно, что ценность LTV гораздо выше предоставленной скидки, а значит, оптимизация recall важнее.

True Positive также стоит денег: мы дадим клиенту скидку и тем самым уменьшим его LTV. В случае с True Negative мы ничего не теряем и сохраняем LTV клиента, то есть метрика для оптимизации на самом деле выглядит так:

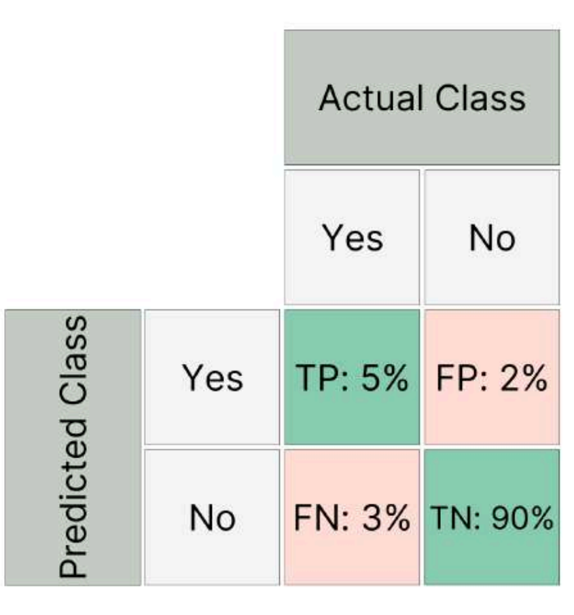
$$TP\,(LTV - D_{cost}) + TN\,(LTV) + FP\,(LTV - D_{cost}) + FN\,(-LTV) \to max$$

Однако и здесь стоит дополнительно оценивать вероятность того, что после предоставления скидки клиент действительно решит остаться.

Задача 2

Наша задача — предсказать внеплановую поломку парогазовой турбины в ближайшее время. Если шанс поломки есть, это повлечёт превентивный ремонт.

Представим, что мы решили задачу и на валидации получили такую матрицу ошибок:

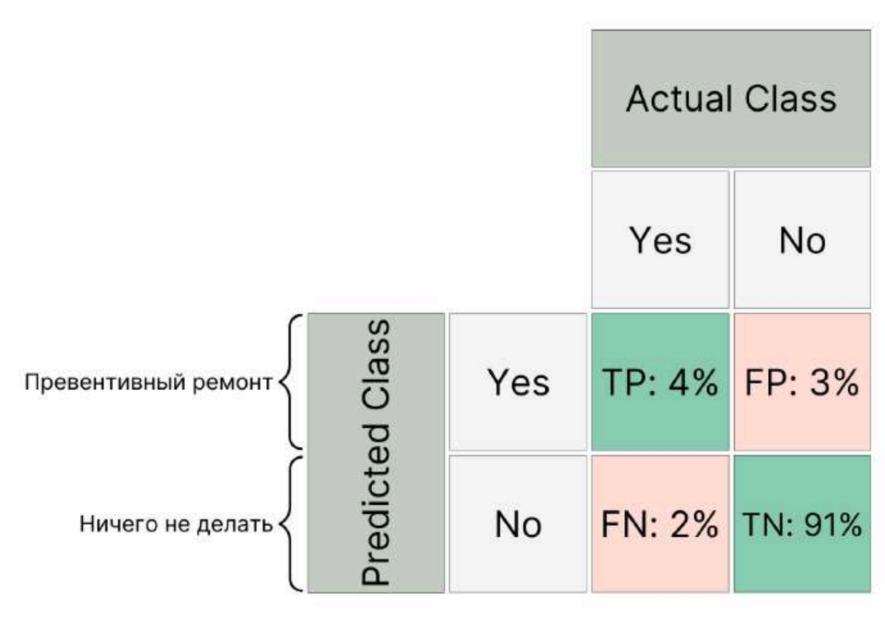


		Actual Class	
		Yes	No
Predicted Class	Yes	TP: 5%	FP: 2%
	No	FN: 3%	TN: 90%

$$precision = \frac{TP}{TP+FP} = \frac{0.05}{0.05+0.02} = 0.71$$

$$recall = \frac{TP}{TP+FN} = \frac{0.05}{0.05+0.03} = 0.63$$

Мы завернули нашу модель в веб-сервис, упаковали всё в контейнер и развернули на продакшене. Спустя некоторое время мы получили такую матрицу ошибок:


Превентивный ремонт
Ничего не делать

		Actual Class	
		Yes	No
Predicted Class	Yes	TP: 4%	FP: 3%
	No	FN: 2%	TN: 91%

Кажется, что теперь всё выглядит хуже, а если взглянуть на метрики, то значительно.

По сути, каждый раз, получив положительное предсказание, мы делали превентивный ремонт. Однако мы не можем утверждать, что поломка после него обязательно должна или не должна случиться. Более того, мы даже не можем это проверить. Если ремонт сделан, мы хотим, чтобы поломки не произошло, а значит, на фактических данных False Positive будет расти за счёт уменьшения True Positive.

Бизнес-эффект

Давайте посмотрим на эффект, который мы получили за счёт внедрения нашего решения.

Посмотрим на столбец Actual Class в валидационной выборке: сумма составляет 8 %. Именно столько внеплановых ремонтов случалось раньше. Если сложить этот же столбец в матрице после внедрения, то мы увидим 6 %. Значит, внеплановых ремонтов стало меньше на 2 %. Но какое реальное влияние оказало наше решение на бизнес?

Ситуация ДО

$A : 8\%\ Cost_{\text{авария}}$

Ранее затраты на внеплановый ремонт составляли 8 % случаев.

Ситуация ПОСЛЕ

$B : 6\%\ Cost_{\text{авария}} + 7\%\ Cost_{\text{превент}}$

Мы сохранили затраты на внеплановый ремонт в 6 % случаев. При этом каждый раз, когда мы предсказывали положительный класс, мы несли затраты на превентивный ремонт (в 7 % случаев). Это сумма первой строки матрицы ошибок.

Реальный эффект составил: $A - B : 2\%\ Cost_{\text{авария}} - 7\%\ Cost_{\text{превент}}$.

ROI

Теперь обратимся к экономике внедрения и оценим показатель ROI.

> **Return on Investment (ROI)** — это показатель эффективности, используемый для оценки инвестиций или сравнения различных проектов. ROI показывает сумму прибыли на конкретный проект относительно суммы инвестиций.

Для расчёта этого показателя прибыль делится на стоимость инвестиций. Результат выражается в процентах или соотношении.

$$ROI = \frac{Current\ Value\ of\ Investment - Cost\ of\ Investment}{Cost\ of\ Investment}$$

Чтобы посчитать ROI по нашему проекту, необходимо сосчитать все трудозатраты, которые потребовались для разработки и внедрения решения. Представим их в виде числа I_0. Также понадобится стоимость поддержки — обозначим её как $Cost_{\text{поддержки}}$.

Тогда ROI составит:

$$ROI = \frac{2\%\ Cost_{\text{авария}} - 7\%\ Cost_{\text{превент}} - Cost_{\text{поддержки}} - I_0}{I_0}$$

Итак, мы прошли все этапы от валидации модели до оценки экономического эффекта, полученного от внедрения решения.

В этом уроке мы рассмотрели лишь некоторые подходы к тому, как оценивать эффективность работы ML-моделей. Процесс определения качества моделирования достаточно специфичен для каждой области и решаемой задачи. Основная идея заключается в проведении классического A/B-теста, чтобы сравнить результаты до и после.

Чтобы грамотно провести эксперимент после внедрения модели, следует правильно его запланировать ещё на этапе постановки задачи, оценив бизнес-задачу и корректно выбрав метрики.

‹ Предыдущий урок Следующий урок ›

8/9 Итоги

текущий урок

По итогу пройденных материалов вы:

1. Научились:
    - правильно сохранять модели, используя библиотеки pika и joblib;
    - реализовывать простейшие веб-сервисы с помощью фреймворка Flask;
    - заворачивать сервисы и приложения в контейнеры, чтобы быстро их разворачивать и соблюдать требования воспроизводимости.
2. Узнали, как создавать сервисы, какими они бывают и как их подбирать в зависимости от своих задач.
3. Познакомились с различными инструментами, которые позволяют отслеживать логи и оркестрировать запуск контейнеров, а также помогают создать свой DS-пайплайн, начиная от сбора данных и заканчивая деплоем моделей

Конечно, в каждой компании все эти инструменты используются в различных комбинациях, а за каждый этап могут отвечать разные команды. Однако мы надеемся, что дали вам представление о жизненном цикле модели и теперь вы сможете увереннее управлять им в любой команде.

‹ Предыдущий урок
Следующий урок ›

1/9 Введение

текущий урок

Ранее вы уже изучали A/B-тестирование как отдельную дисциплину — в основном с математической и статистической точек зрения. В этом модуле мы не будем начинать тему с нуля: наша задача — повторить ключевые концепции и рассмотреть их применение в контексте задач машинного обучения.

Это особенно важно, поскольку именно умение применять A/B-тестирование в ML-задачах потребуется вам при выполнении итогового проекта.

> Мы также настоятельно рекомендуем посещать вебинары по данной теме или обязательно посмотреть их в записи — это поможет лучше структурировать знания и существенно упростит выполнение практической части проекта.

В предыдущих модулях мы уже познакомились со статистической проверкой гипотез. Однако рассматриваемые ранее примеры были достаточно абстрактны и слабо связаны с реальными бизнес-задачами. Теперь мы перейдём к применению этих методов в условиях, максимально приближенных к практике.

Умение проводить и анализировать A/B-тесты — один из ключевых навыков специалиста в области Data Science. Они позволяют отвечать как на бизнес-вопросы (например, «увеличится ли прибыль при введении скидок для определённых групп пользователей?»), так и на задачи, напрямую связанные с машинным обучением (например, «действительно ли новая модель рекомендаций работает лучше текущей?»).

A/B-тестирование даёт возможность объективно оценивать результаты внедрения решений в реальных условиях и лучше понимать поведение и потребности пользователей.

Для анализа результатов A/B-тестирования нам также пригодятся навыки построения доверительных интервалов — с этим инструментом мы уже знакомы и теперь применим его в более практическом контексте.

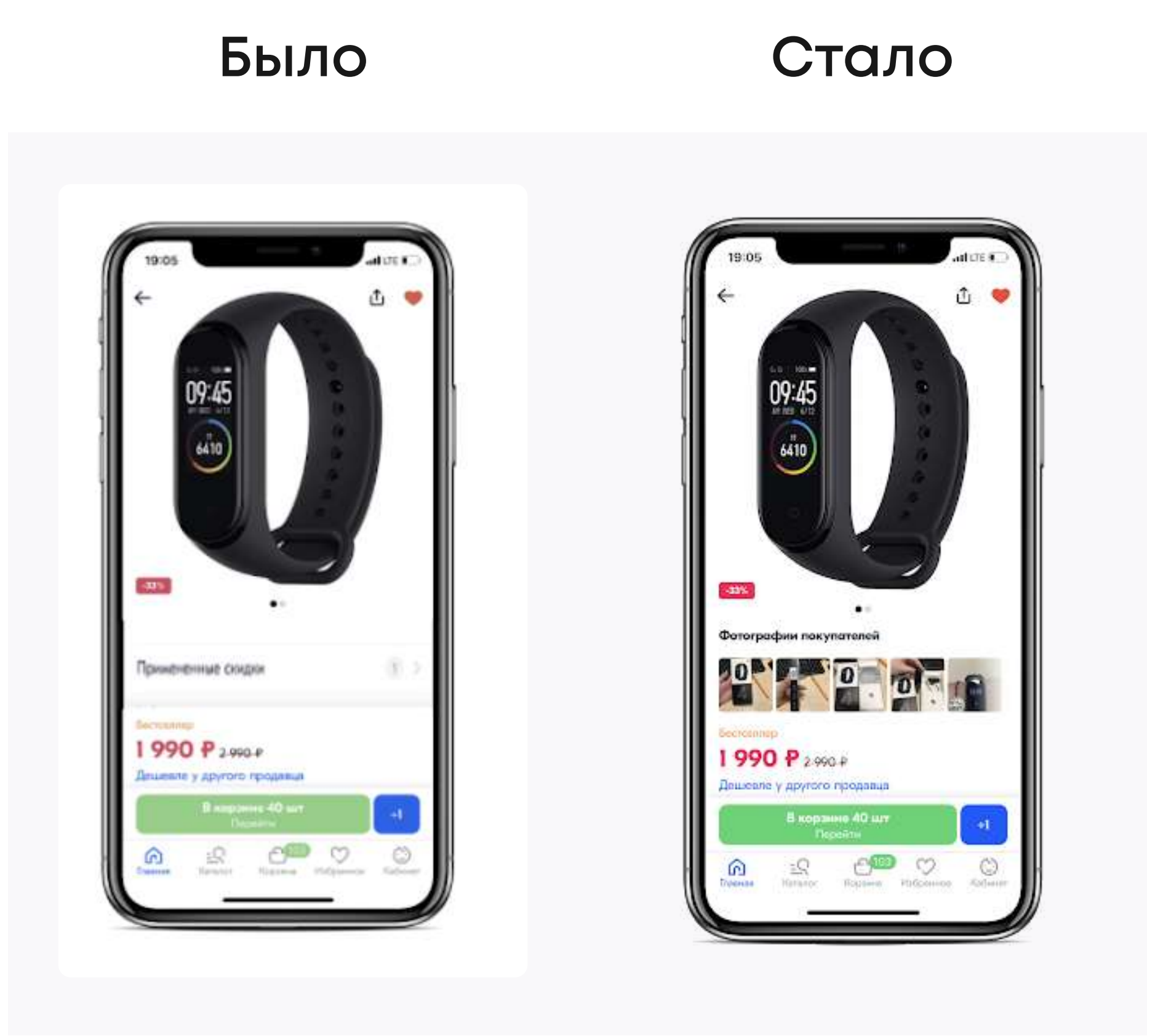
‹ Предыдущий урок   Следующий урок ›

2/9 Суть и сферы применения A/B-тестирования

текущий урок

Прежде чем перейти к определению A/B-тестирования, рассмотрим реальный кейс компании Ozon, связанный с изменениями карточки товара.

Цель кейса состояла в том, чтобы увеличить конверсию добавления товара в корзину пользователями приложения Ozon. Команда предположила, что обычных фотографий товара может не хватать для принятия решений. Поэтому было решено переместить фотографии покупателей вместе с отзывами в верхнюю часть карточки товара.



Гипотеза и метрика

Исследование всегда начинается с выдвижения гипотезы. Напомним, **гипотеза** — это утверждение, предполагающее доказательство.

При формировании гипотезы мы сразу высказываем предположение о том, как реализация проекта повлияет на метрики.

Метрика — численное отображение целевого показателя. Это может быть значение конверсии, среднего чека, посещаемости заведения и так далее. Метрика позволяет нам от субъективных оценок «хорошо» и «плохо» перейти к конкретным числам.

Пример составления гипотезы: Конкретное изменение X повлияет на метрику Y на Z единиц.

В случае с кейсом Ozon гипотеза звучала следующим образом: «Заданное на второй картинке расположение фотографий (X) увеличит конверсию добавления товара в корзину с карточки товара (Y) на 2 % (Z)».

Итак, возникает вопрос: каким образом можно проверить данную гипотезу? Как с определённой долей уверенности убедиться в том, что она является истинной? На эти вопросы нам помогает ответить матушка-статистика и A/B-тестирование!

> **A/B-тестирование** — это метод, пришедший к нам из маркетинга. Он заключается в сравнении текущей версии продукта (версии А) с его изменённой версией (версией B) на основании данных, полученных до введения обновления в продукт и после него.

Метод основан на проверке статистической значимости результатов эксперимента и позволяет заранее задать границу уверенности в результатах исследования **(уровень надёжности**).

Для кейса Ozon результаты A/B-тестирования оказались вполне успешными: удалось подтвердить гипотезу команды, а при дальнейшем анализе обнаружились категории товаров, в которых эту функциональность не стоило запускать сразу. Проведение A/B-тестирования и проверка гипотезы позволило компании увеличить прибыль, лишь поменяв дизайн вкладки товара.

Давайте же разберёмся, в чём же заключается A/B-тест?

Механизм A/B-тестирования

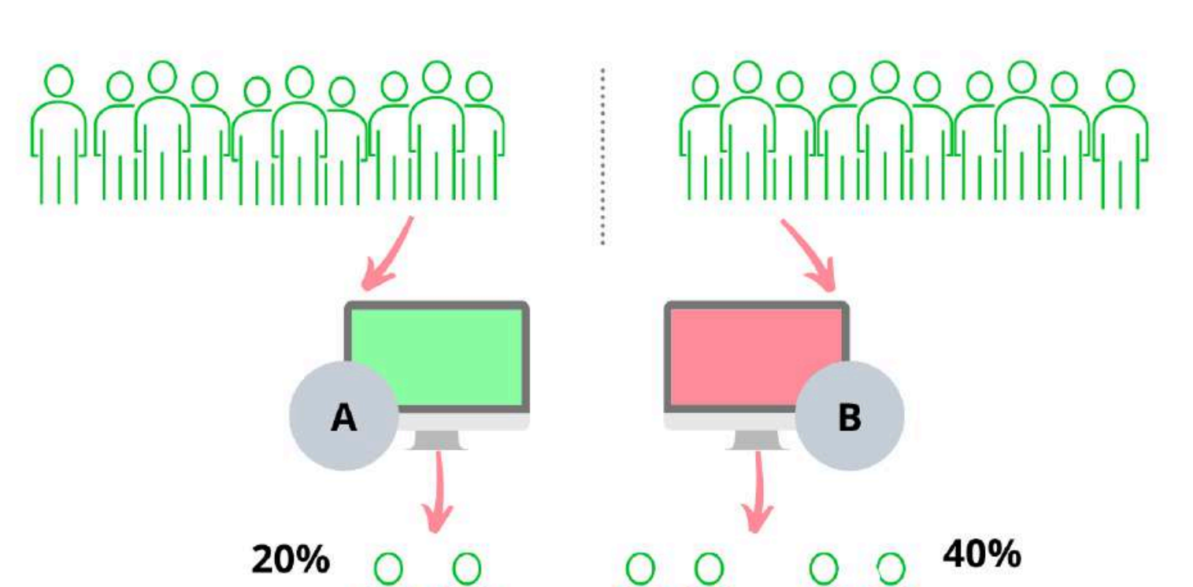
A/B-тестирование и все производные от него (A/A, A/A/B, A/B/C) также принято называть **сплит-тестами** (от англ. split, разделять), что объясняется основополагающим принципом их проведения.

Опишем механику A/B-тестов в общих чертах.

Чтобы протестировать какую-либо гипотезу при помощи A/B-теста, аудиторию разделяют на две части:

- группа А продолжает использовать (видеть) старую версию продукта;
- группа B начинает использовать новую.

В реальном времени собирается информация об обеих группах теста, проводятся замеры важных показателей. После этого проводится сравнение этих показателей и принимается решение об эффективности влияния гипотезы на показатели продукта.



Пояснение к картинке

На картинке изображено схематическое представление механизма A/B-тестирования. Есть два варианта дизайна: **зелёный** и **красный**. Каждый вариант дизайна предоставляется соответствующей группе пользователей, и считается метрика (например, конверсия). На основании рассчитанной метрики определяется более эффективный вариант.

На первый взгляд может показаться, что ничего сложного в этом процессе нет. Но на деле тестирование — достаточно дорогой и непростой инструмент. A/B-тестирование очень чувствительно к получаемым данным и требует большого внимания на всех его этапах.

О тонкостях процесса проведения тестирования и оценке его результатов мы поговорим в дальнейших уроках.

Сферы применения A/B-тестирования

A/B-тесты используют везде и всюду, где необходимо проверять какие-то гипотезы. Давайте рассмотрим несколько примеров отраслей, где применяют A/B-тесты.

Медицина

Например, с помощью A/B-теста можно выявить положительные эффекты в препаратах.

Все вы слышали про **эффект плацебо**. Применение плацебо — не что иное, как A/B-тестирование, когда одной группе тестируемых дают реальное лекарство, а другим — пустышку, и смотрят на итоговые результаты.

Розничная торговля

Большие ритейл-компании, такие как X5, тестируют различные варианты расстановки товаров в магазине и влияние позиции товаров на продажи конкретного магазина. Например, можно тестировать положение брендов на полке: в одном магазине на полке с консервами перед глазами лежит Bonduelle, а в другом — Heinz. В итоге можно посмотреть, изменилась ли доля продаж бренда в зависимости от магазина.

У такого A/B-теста есть сложности: на входе в магазин нельзя поставить турникет, который бы мог одних пользователей отправлять налево, а других — направо.

В таком случае аналитики сначала выбирают максимально похожие друг на друга магазины по площади, ассортиментной матрице, текущему обороту, местоположению и так далее. Как вы понимаете, это очень сложный и трудоёмкий процесс.

Интернет-маркетинг

В этой сфере можно тестировать разные виды баннеров, рекламных кампаний, призывов к действию.

Довольно много проводят A/B-тестов в рекламных кампаниях Google Рекламы, Яндекс.Директа, Facebook. Рекламные сервисы уже сейчас позволяют случайным образом расщеплять трафик на части и показывать пользователям разные тексты объявлений.

Сегодня вообще весь интернет и всё больше сервисов проводят десятки A/B-тестов ежедневно:

- Google и Яндекс тестируют ранжирование поисковой выдачи, количество рекламных блоков на странице.
- Ozon тестирует карточку товара или процесс оформления заказа.
- Другой большой e-commerce магазин тестирует, как маркетинговые акции и системы лояльности сказываются на покупательской способности в дальнейшем.

A/B-тестирование ML-моделей

Помимо того, что A/B-тестирование позволяет отвечать на вопросы бизнеса о пользователях, оно может стать очень важным инструментом выявления эффективности построенной дата-сайентистом модели и позволить понять, как она показывает себя в реальных условиях.

Итак, давайте погрузимся в банковский кейс.

Представьте, что вы — дата-сайентист в банке и занимаетесь построением моделей для оптимизации мобильного приложения этого банка.

В приложении уже присутствует некоторая модель искусственного интеллекта, которая позволяет пользователям планировать свой бюджет на следующий месяц на основе расходов и доходов на предыдущие месяцы и поправок, введённых пользователем (планы на отпуск, повышение на работе и прочие внешние факторы). Для конкретики представим, что в среднем эта модель ошибается на 7 000 рублей.

Как мы знаем, мир искусственного интеллекта не стоит на месте. И вот в научно-популярном журнале выходит статья от некоторого университета, где подробно разбирается новая, якобы более совершенная методика решения задачи прогнозирования. Вы с коллегами изучили эту статью и построили на её основе новую модель для планирования бюджета.

И вот вы добрались до этапа оценки модели. Вы «прогоняете» вашу модель на припасённых заранее тестовых данных. Тесты показывают, что в среднем модель ошибается в своих прогнозах на 3 000 рублей, что является лучшим результатом, чем есть сейчас, то есть модель оправдывает ваши ожидания! У вас уже чешутся руки, чтобы отправить прошлую модель в небытие и поставить новую.

Спешим вас расстроить… Ваши показатели на тестовой выборке пока что мало что значат. Разница в показателях может быть обусловлена чистой случайностью или внешними факторами. Модель должна начать работать на реальных данных и доказать свою эффективность «в бою». Чтобы проверить эту эффективность можно провести сравнительное тестирование в условиях, приближенных к реальным. Тут-то вам и поможет A/B-тестирование!

В качестве контрольного варианта (варианта А) возьмём текущую версию приложения банка, в которой стоит старая модель. В качестве тестового варианта (варианта B) возьмём новую версию модели и внедрим её в раздел Планирование бюджета в приложении.

Разделим клиентов на две группы: группу А, которая будет пользоваться текущей версией приложения, и группу B, которой предоставим новую функциональность.

Для простоты рассмотрим предсказания моделей только на один месяц вперёд. В качестве метрики мы можем взять какую-то бизнес-метрику и/или метрику качества модели (о них мы поговорим в блоке по ML). Значение показателя будем считать для двух моделей в абсолютно равных условиях.

Тогда мы будем рассматривать следующую **гипотезу**: Новая версия модели увеличит <выбранную метрику> на <n пунктов>.

На основании результатов A/B-тестирования мы можем сделать вывод о том, стоит ли производить замену текущей модели на новую или же результат новой модели статистически незначим.

Таким образом, A/B-тестирование позволяет нам оценить работу модели в реальных условиях и понять, имеет ли смысл вводить её в продакшен.

Задание для самостоятельной работы, за него не предусмотрены баллы. Однако оно поможет вам закрепить новый материал.

Задание 2.1

0 points possible (ungraded)

Выберите кейсы, в которых стоит воспользоваться *A/B*-тестированием.

☑ **A.** Команда дата-сайентистов *Netflix* построила новую рекомендательную систему сериалов и фильмов и хочет сравнить её эффективность с текущим вариантом системы рекомендаций.

☐ **B.** Дата-сайентист Василий хочет построить модель для автоматического позиционирования беспилотных дронов.

☑ **C.** Дизайнеры *Instagram** предложили новый вариант отображения фотографий в профиле. Команде дата-сайентистов поручают задание сравнить новый вариант дизайна со старым и определить его эффективность.

☐ **D.** Команда разработчиков *YouTube* хочет проанализировать распределение посещаемости ресурса пользователями в зависимости от времени суток.

✓

Show answer

Отправить

✓ Верных

*В материалах курса упоминается социальная сеть Фейсбук/Инстаграм, принадлежащая Meta Platforms Inc., деятельность которой запрещена на территории РФ в части реализации данных социальных сетей на основании осуществления ею экстремистской

‹ Предыдущий урок
Следующий урок ›

3/9 Алгоритм, принципы и параметры A/B-тестирования

текущий урок

A/B-тестирование — это настоящий эксперимент, в котором вы выдвигаете уже знакомые вам статистические гипотезы (например: «Варианты A и B равнозначны» (помним, что нулевая гипотеза в статистике чаще формулируется как отсутствие различий (или эффекта)) против альтернативной гипотезы «Вариант B лучше, чем вариант A по показателю Y на Z %»), ставите «опыты» над пользователями и смотрите на изменение определённой заранее метрики.

Как и в любом эксперименте, при проведении A/B-тестирования очень важно соблюдать «правила безопасности»: тестирование необходимо проводить по определённым канонам, которые были разработаны на практике другими специалистами.

Если не соблюдать эти правила, существенно повышается вероятность совершить следующие типичные ошибки:

- **Отвергнуть верную нулевую гипотезу — совершить ошибку I рода**. Например, на основе полученных данных вы можете принять решение, что вариант B является более предпочтительным, однако такая разница была обусловлена лишь случайностью или внешними факторами. В таком случае, помимо времени и денег, потраченных на A/B-тестирование, вы бессмысленно тратите деньги и время ещё и на глобальную интеграцию варианта B, а его эффективность оказывается на уровне или даже хуже варианта A.
- **Принять неверную нулевую гипотезу — совершить ошибку II рода**. Например, на основе полученных данных вы можете принять решение, что для пользователей нет разницы между вариантами A и B, хотя на самом деле вы произвели некорректное сравнение и вариант B был лучшим. В таком случае вы теряете только время и деньги, потраченные на проведение A/B-тестирования, и упустите возможный дополнительный доход от улучшения.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 3.1

0 points possible (ungraded)

Платформа онлайн-образования, посмотрев на отзывы своих студентов, принимает решение запустить новый вариант некоторого модуля.

Нулевая гипотеза состояла в том, что модули одинаково эффективны по значению метрики. Альтернативная гипотеза – новый вариант модуля увеличит текущее значение метрики (зависящей от обратной связи студентов) в 1,5 раза.

A/B-тестирование показало, что новый вариант модуля в два раза превосходит по метрике старый вариант, поэтому «на радостях» старый вариант безвозвратно удалили с платформы, заменив его новым.

⛔ По прошествии некоторого времени оказывается, что новый вариант модуля стал показывать ещё более низкие оценки, чем старый.

Выяснилось, что тест проводился некорректно: показатели метрики по новому варианту модуля оценивались на выборке из 10 человек, а тестирование проводилось всего неделю на людях, которые уже знакомы с изучаемой в модуле темой.

Какую ошибку допустила команда курса?

○ Ошибку третьего рода
● Ошибку первого рода
○ Ошибку второго рода
○ Ошибку в формулировке гипотезы

✓

Show answer

Отправить

✓ Верных

Итак, давайте разбираться, как правильно проводить A/B-тестирование! Начнём с разбора этапов, которые мы должны пройти при проведении тестирования.

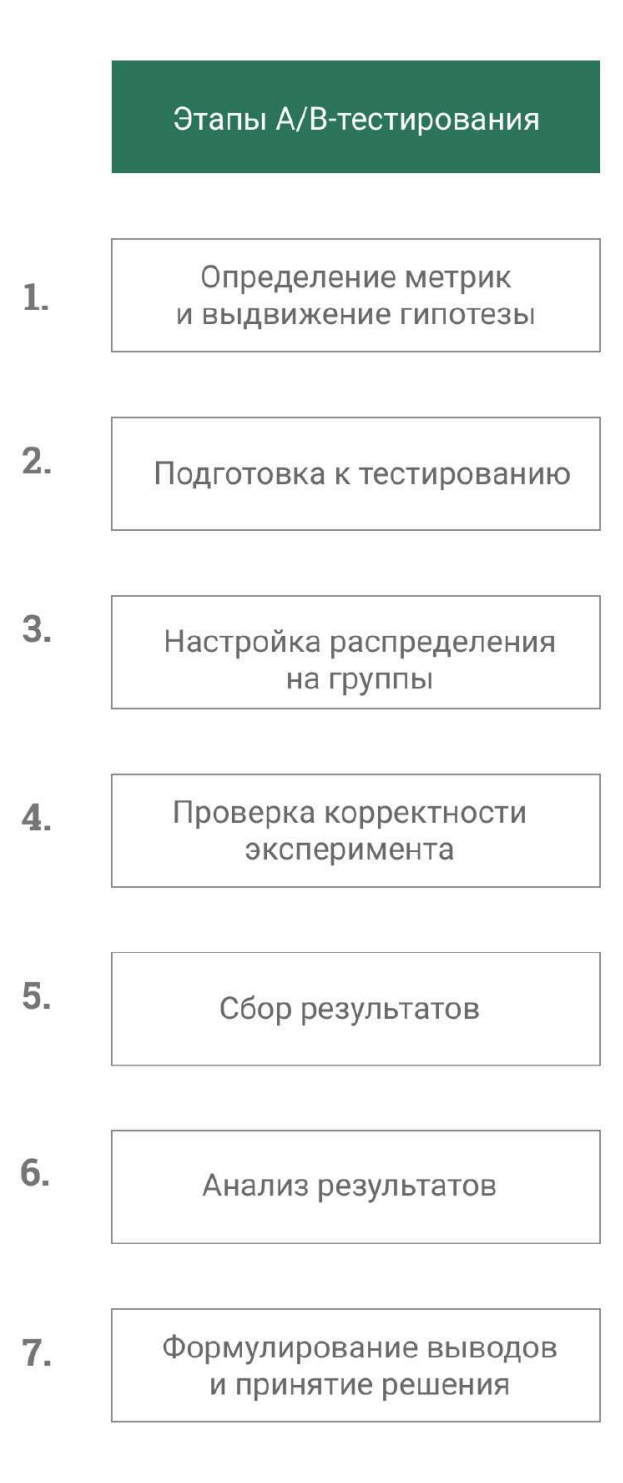
Алгоритм A/B-тестирования

Рассмотрим алгоритм проведения A/B-тестирования на следующей задаче.

Представьте, что ваша команда занимается разработкой рекомендательной системы, которая подбирает пользователям персональные предложения по продажам. Сейчас ваш заказчик производит рассылку предложений по почте и делает это, не опираясь на информацию о пользователе: все возможные предложения сыплются одним потоком.

Цель нашего исследования — проверить эффективность работы построенной рекомендательной системы и понять, какой из вариантов лучше: персонализировать предложения с помощью искусственного интеллекта или рассылать все возможные предложения всем клиентам.

Схематично A/B-тестирование можно разделить на следующие этапы.


Этапы A/B-тестирования
1. Определение метрик и выдвижение гипотезы
2. Подготовка к тестированию
3. Настройка распределения на группы
4. Проверка корректности эксперимента
5. Сбор результатов
6. Анализ результатов
7. Формулирование выводов и принятие решения

1. Определение метрик и выдвижение гипотез

На этом этапе мы определяем, какую метрику мы будем пытаться оптимизировать при внедрении варианта продукта B (дизайна, функциональности, модели, препарата и так далее).

Это могут быть специализированные метрики задач машинного обучения, если мы сравниваем две модели между собой, или маркетинговые и SMM-метрики: средний чек, конверсия покупки товара, количество лайков или репостов, количество просмотров, количество негативных комментариев, отток клиентов и так далее.

При определении метрики важно соблюдать несколько правил:

- Метрика должна быть измерима, то есть должна иметь численное выражение, и у вас должна быть возможность как можно более точно измерить этот показатель.
- У вас есть возможность влиять на метрику.

Плохие примеры метрик

- Уровень счастья пользователя в момент совершения покупки (метрика плохо измерима).
- Курс доллара (вы не можете повлиять на курс доллара, только если ваша компания не является международным финансовым гигантом).

Примечание. Очень часто в компаниях есть своя метрика, которая высчитывается специальным образом и включает в себя сразу несколько показателей.

Итак, предположим, что в качестве метрики для нашей задачи оценки эффективности системы рекомендаций мы придумали свою комплексную метрику, которую назвали «**вовлечённость пользователей**». Она измеряется от 0 до 100 и учитывает сразу несколько показателей: долю перешедших по рекламной ссылке пользователей, средний чек, конверсию покупки…

Как только мы определились с метрикой, мы выдвигаем гипотезу о том, что вариант продукта B увеличит (уменьшит) нашу метрику на Z единиц. Если вы выбрали несколько метрик, то количество выдвигаемых гипотез должно быть не меньше количества метрик.

Как и к метрике, к гипотезе тоже есть свои требования:

- Гипотеза должна быть проверяемой, если предположение нельзя проверить, то это не гипотеза, а фантазия.
- Гипотеза должна не иметь логических противоречий.
- Должно быть понятно, на что влияет гипотеза, то есть она должна быть связана с конкретным показателем.

Плохие примеры гипотез

- «Добавление искусственного интеллекта в приложение уменьшит уровень стресса пользователей в 3 раза» (вы не можете проверить данную гипотезу, если вы не заставите пользователей принудительно проходить психологический тест на уровень стресса).
- «Смена цвета кнопки продажи с красного на зелёный увеличит средний чек в 150 раз» (гипотеза является нереальной, дизайн не может так сильно влиять на показатель среднего чека).
- «Цветные изображения товаров в приложении в 1,5 раза эффективнее, чем чёрно-белые» (непонятно, на что влияет гипотеза, к какой метрике она привязана, по какому критерию определяется «эффективность»).

Хорошие примеры гипотез

- «Замена текущей модели SSD для детектирования предметов на модель семейства YOLO увеличит метрику IOU на 25 %».
- «Новый дизайн сайта поможет увеличить конверсию для новых пользователей на 5 % и не ухудшит конверсию старым».
- «Новый алгоритм ранжирования в рекомендациях в корзине увеличит итоговый средний чек на 3 %».

Аналитики провели некоторое исследование и составили следующую гипотезу: «При внедрении персональных предложений вовлечённость пользователей увеличится на 10 %».

2. Подготовка к тестированию

Этот этап подразумевает проведение подготовительных мероприятий и включает в себя следующие компоненты.

Определение размера выборки и длительности теста

Размер выборки имеет ключевое значение при проведении A/B-тестирования. Не собрав достаточную для тестирования выборку, мы получим результаты, которым попросту нельзя будет доверять, и эксперимент будет произведён впустую.

Рассмотрим пример.

Представим, что вы пытаетесь сравнить эффективность рекомендательной системы по выборке размером в 10 человек в каждой группе. При этом ваш интернет-магазин ежедневно посещают тысячи пользователей. Как можно судить о многотысячной аудитории по такой маленькой выборке?

Ответ: никак.

Размер выборки определяется с помощью специальных статистических методов, для которых реализовано множество онлайн-калькуляторов.

Аналогично и с **длительностью теста**: чтобы быть уверенным в результатах тестирования, необходимо проводить тест на достаточном для этого отрезке времени. Длина данного временного отрезка также вычисляется с помощью статистических методов, и для этого есть специальные калькуляторы.

Примечание. О том, как вычислять размер выборки и длительность теста для конверсии, мы поговорим ниже.

Предположим, что мы вычислили необходимый размер выборки для нашей задачи, и он составил 29 000 посетителей, а также вычислили длительность проведения тестирования, и она составила 16 дней.

Принятие решения о целесообразности проведения тестирования

На этом шаге мы производим оценку наших ресурсов и отвечаем на примерно следующие вопросы:

- Хватит ли у нас ресурсов для проведения A/B-тестирования заданной длительности и с необходимым размером выборки?
- Окупится ли проведение A/B-тестирования при внедрении варианта B?
- Значителен ли предполагаемый эффект внедрения варианта продукта B для бизнеса?

Разработка варианта B

На этом этапе мы занимаемся проектированием, разработкой и внедрением нового варианта нашего продукта по заявленному техническому заданию. Для этого этапа могут быть привлечены внешние специалисты.

Наш вариант B — это модернизированная с помощью подсистемы рекомендаций система рассылки предложений. Таким образом, на этом шаге мы разрабатываем DS-продукт — модель машинного обучения — и выводим её в продакшен, то есть внедряем её в систему рассылок предложений нашего заказчика.

3. Настройка распределения на группы

Когда новый вариант продукта уже разработан и готов к внедрению, необходимо распределить наших пользователей на группы A и B.

Этот этап намного сложнее, чем кажется, так как существует вероятность того, что полученные группы могут оказаться **неравнозначны**.

Вы должны постараться сделать так, чтобы группы были максимально схожи по всем возможным признакам. Идеального результата добиться никогда не получится, однако нужно попытаться учесть как можно больше факторов.

Пример плохого разделения на группы

Представим, что вы разделили посетителей вашего сайта на две группы и пытаетесь сравнить конверсию покупки в этих группах. Допустим, вы учли геолокацию посетителей и разделили их так, чтобы в каждой группе было одинаковое количество пользователей из Москвы, Санкт-Петербурга, Саратова и так далее.

Но вот незадача: вы не учли тот факт, что пользователи приходят через различные маркетинговые каналы (через рекламу по ссылке, по советам друзей или находят вас целенаправленно).

Представим, что в группе B оказалось более половины клиентов, которые целенаправленно пришли к вам. Вполне логично, что такие клиенты более лояльны к вашей продукции и вероятность покупки ими вашего товара выше. Можно заранее с немалой уверенностью предположить, что в группе B конверсия покупки будет выше.

Несбалансированное разделение выборки на группы искажает результаты A/B-тестирования. Если пропустить доскональное исследование данных и не продумать разделение на группы, вы придёте к необходимости повторного проведения эксперимента.

Примечание. Мы не будем углубляться в тонкости данного этапа, так как он очень индивидуален, и для каждой задачи придётся продумывать свои манипуляции с разделением аудитории. В этом модуле мы больше сосредоточимся на этапе анализа результатов A/B-тестирования.

Если это необходимо, на этом этапе также проектируется место для хранения данных, возникающих в процессе проведения эксперимента.

Представим, что мы спроектировали небольшую базу данных для нашего примера, куда записываем данные о каждом переходе клиентов по ссылке из рассылки, информацию о том, совершил ли пользователь целевое действие, и сумму его покупки, а также время, проведённое на сайте после перехода по ссылке.

4. Проверка корректности эксперимента

На этом этапе мы проверяем, что все условия для проведения эксперимента работают корректно:

- группа A использует (видит) вариант продукта A, а группа B использует (видит) вариант продукта B; отсутствуют пересечения между этими группами;
- отсутствует внезапное влияние внешних факторов (не появляются неожиданные масштабные события, которые могут повлиять на результаты эксперимента);
- результаты централизованно поступают в подсистему хранения данных о процессе проведения эксперимента (если такая предусмотрена);
- другие специфичные условия.

В случае нашей задачи при проведении эксперимента мы должны проверить, что клиенты получают рекламу, а персональные предложения работают корректно. Также мы должны следить за тем, чтобы сообщения поступали в одинаковые промежутки времени, пристально наблюдать за резкими скачками «вовлечённости» наших клиентов, если такие возникают.

5. Сбор результатов

Когда процесс тестирования доходит до своей финальной точки, нам необходимо выгрузить результаты: данные об изменении целевого показателя (метрики) на заданном промежутке времени.

Итак, представим, что мы выгрузили данные о показателях, на основе которых будет считать нашу метрику («вовлечённость») из спроектированной базы.

6. Анализ результатов

На этом этапе мы производим сравнение метрик в двух группах на основе собранных данных. Для этого могут использоваться методы визуализации и визуальное сравнение показателей («на глаз»).

Однако, когда значимость разницы показателей неочевидна, то есть нельзя с большой долей уверенности сказать, какой вариант продукта (A или B) оказался «лучше», мы должны воспользоваться **статистическими тестами**. Они позволяют с заданным уровнем значимости ответить на вопрос: имеет ли статистическую значимость полученная разница в показателях или же она является случайной?

Помимо статистических тестов, для анализа результатов часто используются **доверительные интервалы**, которые позволяют построить возможные границы для истинного значения метрики в группах и для разницы между показателями в группах. Доверительные интервалы помогут нам ответить на вопрос: «Насколько изменилось значение целевой метрики в варианте B, если изменения произошли?»

Мы рассмотрим реализацию этапа анализа результатов A/B-тестирования на Python в следующем уроке и подробно разберём все тонкости определения статистической значимости результатов.

Предположим, что в результате анализа данных проведённого A/B-тестирования для системы персональных рекомендаций выяснилось, что «вовлечённость» пользователей группы B на 3 % выше, чем та же метрика для группы A. Статистический тест и доверительный интервал для разницы «вовлечённостей» показали, что данная разница является статистически значимой. А значит, пора переходить к выводам!

7. Формирование выводов и принятие решения

На основании анализа результатов A/B-тестирования мы принимаем или отклоняем первоначальную гипотезу и формируем один из трёх возможных выводов:

1. Вариант продукта A является более эффективным (по заданной метрике), чем вариант B.
2. Вариант продукта B является более эффективным (по заданной метрике), чем вариант A.
3. Варианты A и B являются идентичными, то есть между ними нет разницы с точки зрения заданной метрики.

Когда сделан вывод, сформирован отчёт о проведении A/B-теста и его результатах, принимается решение о внедрении варианта B или необходимости дальнейшего исследования.

Хотелось бы отметить, что не существует хорошего или плохого результата. Цель A/B-тестирования — это сделать верные выводы в целях рационального использования средств бизнеса. Если в результате получилось так, что новый вариант не оправдал себя, нужно с чистой совестью принять такой исход событий.

Итак, по итогам нашего вымышленного A/B-тестирования мы приходим к тому, что новая система рассылки предложений с внедрённой в неё моделью для подбора персональных рекомендаций является эффективной, и это доказано результатами эксперимента.

Однако следует помнить о том, что статистические методы проверки гипотез не являются абсолютно точными и могут гарантировать результат только с заданным уровнем значимости. Поэтому существует вероятность, что даже при соблюдении всех условий эксперимента вы допустите ошибку I или II рода.

Задание 3.2

0 points possible (ungraded)

Какая из гипотез *A/B*-тестирования сформулирована правильно?

● Если в корзине при оформлении заказа на сайте сократить число полей для заполнения в два раза, то это увеличит конверсию в два раза.
○ При переходе на сайт красная кнопка срабатывает лучше чёрной в 3,5 раза.
○ Рекламное объявление о распродаже, в котором используется изображение Джеймса Бонда, в 1,5 раза эффективнее, чем рекламное объявление с «Дюной».

✓

Ответ
Верно:
Гипотеза сформулирована верно. Она отражает новый вариант продукта, метрику и её изменение.

Show answer

Отправить

✓ Верных

Задание 3.3

0 points possible (ungraded)

Для чего в *A/B*-тестировании нужны статистические тесты?

○ определить зависимость между признаками
● определить, есть ли статистическая значимость разницы между полученными показателями метрик в группах *A* и *B*
○ определить статистическую значимость метрики для бизнеса
○ определить границы доверительного интервала для метрики в группах *A* и *B*

✓

Show answer

Отправить

✓ Верных

При организации A/B-тестирования очень важно помнить о нескольких принципах проведения эксперимента. Следуя принципам A/B-тестирования, вы снижаете шансы допустить ошибку при организации эксперимента, если на вас ляжет такая ответственность. Давайте их рассмотрим.

Принципы A/B-тестирования

Принцип 1: Исключить влияние извне

Как уже отмечалось, для качественного результата тестирования необходимо разделить группы так, чтобы пользователь, который попал в одну из групп, в течение всего теста находился только в ней. Это даёт нам уверенность в том, что на принятие решения о совершении целевого действия его подтолкнул только тот вариант продукта, который был ему показан.

В идеале на пользователя вообще не должны оказывать влияние никакие внешние факторы, будь то конкуренция, экономические и социальные изменения (вспомним COVID-19), сезонность и так далее.

На практике сделать это крайне тяжело: мы не можем влиять на акции, которые проводят наши конкуренты, или на пользователя, который может заходить на наш сайт с разных устройств и видеть разные версии сайта, предназначенные для разных групп теста.

Если всё-таки оказалось так, что некоторые пользователи попали в обе группы теста, то нужно смотреть на их количество. Как правило, если их число невелико (менее 5-10 %), их исключают из анализа.

Поэтому крайне важно правильно рассчитать длительность теста, чтобы накопить достаточное количество данных для принятия правильного решения!

Принцип 2: Данных должно быть много

Согласно закону больших чисел, чем больше раз повторяется эксперимент, тем ближе будет частота события к его вероятности.

Иначе говоря, если наблюдений мало, их разброс будет больше, а если наблюдений много, случайные выбросы будут друг друга компенсировать.

Давайте рассмотрим пример.

Когда мы подбрасываем монетку, то с одинаковой вероятностью может выпасть как орёл, так и решка. Вероятность каждого исхода — 50 %.

Даже если в начале эксперимента пять раз подряд выпал орёл, то с увеличением количества повторений (например, если мы решим подбросить монетку 10 000 000 раз) подобные выбросы компенсируют друг друга и количество выпавших орлов и решек будет одинаково.

Поэтому данных и должно быть много!

Принцип 3: Правильный инструмент анализа

Как мы уже говорили, проверка гипотез — статистический инструмент. Она позволяет судить о больших совокупностях по меньшим выборкам.

То есть, измеряя результаты групп A и B, мы на самом деле хотим понять различие не самих этих групп (выборок), а различие генеральной совокупности (всех пользователей).

И статистика даёт нам такую возможность! По выборке мы можем судить о реальном среднем значении всей генеральной совокупности.

Важно определиться с показателями, по которым производится сравнение в группах, и правильно выбрать статистические инструменты (тип теста и доверительного интервала) для проверки гипотезы. Неправильные инструменты — неверный результат.

Обычно, проводя тестирование, группы разделяют равномерно — 50/50. То есть ровно половина пользователей видит вариант A, а вторая половина — вариант B. Однако также допускается разделять группы в других пропорциях, например 70/30.

Также можно проводить **множественные сравнения**: когда аудитория делится не на две группы, а на большее количество, и каждая видит разные варианты. Такие исследования называют **множественным тестированием**, или A/B/C-тестированием.

Задание 3.4

0 points possible (ungraded)

Представим ситуацию, что при анализе результатов *A/B*-тестирования вы нашли 3 % пользователей, которые попали в обе группы: контрольную и тестовую. Что вы будете делать?

● исключу пересекающихся пользователей из обеих групп
○ оставлю всё как есть
○ проведу *A/B*-тестирование заново

✓

Show answer

Отправить

✓ Верных

Задание 3.5

0 points possible (ungraded)

Зачем при *A/B*-тестировании большое количество данных?

● уменьшить разброс и компенсировать выбросы
○ увеличить объём базы данных
○ увеличить суммарный показатель метрики

✓

Show answer

Отправить

✓ Верных

Расчёт параметров A/B-теста

Для расчёта размера выборки и длительности проведения теста обычно используют специальные калькуляторы. В интернете можно найти много подобных калькуляторов. Мы рассмотрим некоторые их них.

- Калькулятор Эвана Миллера

Поможет определить размер выборки для задачи повышения конверсии в группах.

Например, вы пытаетесь повысить долю пользователей, которые переходят на ваш сайт по рекламе в соцсетях или повышаете конверсию действия «добавить товар в корзину».

На вход подаются такие параметры, как:

- Baseline Conversion Rate: базовая конверсия (конверсия существующего варианта);
- Minimum Detectable Effect: минимально желаемый результат изменения метрики (относительно базовой конверсии или абсолютный);
- Statistical Power 1−β: мощность теста (вероятность не допустить ошибку II рода);
- Significance Level α: уровень значимости (вероятность ошибки I рода).

Подставив необходимые параметры, получим размер выборки для каждой из групп теста.

Попробуйте воспользоваться этим калькулятором для решения следующей задачи.

Задание 3.6

0 points possible (ungraded)

Вы тестируете форму ввода личных данных клиента. В исходной версии клиент должен обязательно указать номер телефона, имя и адрес, а в новой версии обязательным для заполнения является только поле с номером телефона (имя и адрес можно не оставлять).

Вам необходимо рассчитать размер выборки для *A/B*-теста сценария оформления заказа на сайте.

У вас есть данные за год (365 дней). Вы знаете, что из 2 000 000 пользователей, которые открыли эту форму за год, 600 000 пользователей заполняли её.

Вы ожидаете, что новая форма покажет минимальное увеличение конверсии относительно базовой в 5 % (опция *Relative*). Рассчитайте необходимый размер выборки при мощности теста в 80 % и уровне значимости в 5 %.

Вычислите необходимый размер выборки посетителей страницы.

Отправить

- Калькулятор VWO

Позволит рассчитать длительность теста для получения статистически значимого результата. Калькулятор используется для задач повышения конверсии.

На вход подаются следующие данные:

- Estimated Existing Conversion Rate (%): базовый уровень конверсии;
- Minimum Improvement in Conversion Rate you want to detect (%): минимально желаемый результат изменения метрики;
- Number of Variations/Combinations (including control): общее количество групп в тесте;
- Average Number of Daily Visitors: среднее ожидаемое количество посетителей в день в обеих группах;
- Percent Visitors Included in Test: процент посетителей, включенных в тест.

Подставив необходимые параметры, получим необходимую длительность теста.

Попробуйте воспользоваться этим калькулятором для решения следующей задачи.

Задание 3.7

0 points possible (ungraded)

Вы тестируете новую версию системы рекомендаций товаров. Версии сравниваются по конверсии действия «добавить товар в корзину».

Вы знаете, что в среднем на сайт интернет-магазина ежедневно заходит 6 000 пользователей. Из них в среднем 720 пользователей добавляют товар из рекомендуемых в корзину.

Вы ожидаете, что новая рекомендательная система покажет минимальное увеличение конверсии относительно базовой в 5 %. В тестирование вовлечены все пользователи.

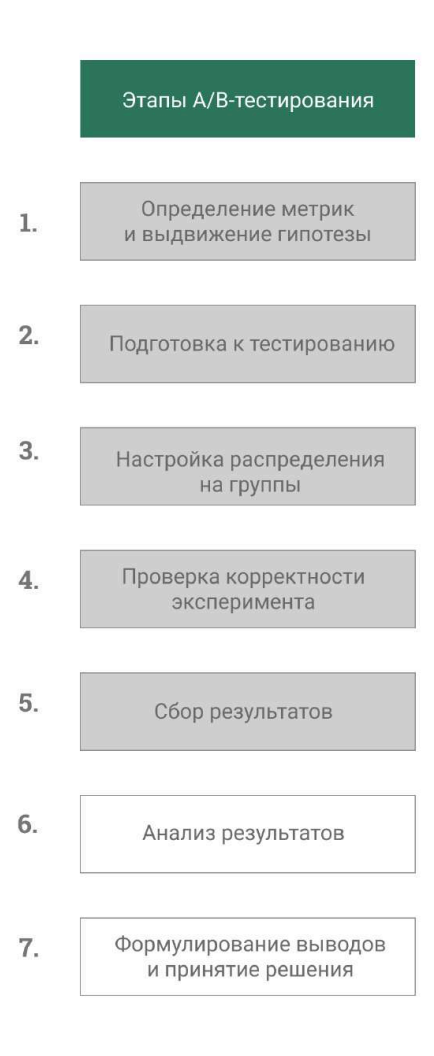
Рассчитайте необходимую длительность *A/B*-теста и запишите количество дней.

Отправить

4/9 Анализ результатов A/B-тестирования: кумулятивные метрики

текущий урок

Итак, давайте рассмотрим, как проводится анализ результатов A/B-тестирования (в нашем алгоритме он был обозначен под номером 6) с помощью Python.


Этапы A/B-тестирования
1. Определение метрик и выдвижение гипотезы
2. Подготовка к тестированию
3. Настройка распределения на группы
4. Проверка корректности эксперимента
5. Сбор результатов
6. Анализ результатов
7. Формулирование выводов и принятие решения

В качестве примера давайте попробуем решить задачу об анализе конверсии на некотором интернет-магазине, который занимается продажей различных товаров. Другие возможные задачи анализа результатов A/B-тестирования решаются похожим образом с учётом специфичности рассматриваемой метрики, длительности тестирования и характера распределения целевого показателя.

Описание задачи

Проводится A/B-тестирование для двух вариантов рекомендаций сопутствующих товаров в профиле товара некоторого интернет-магазина.

- **Вариант 1.** Дополнительные товары рекомендуются на основе статистических данных уже совершённых покупок — товары, которые другие пользователи часто покупают с основным.
- **Вариант 2.** Дополнительные товары рекомендуются на основе совершённых пользователем ранее покупок с помощью рекомендательной системы, построенной на основе гибридного подхода.

Варианты сравниваются по доле пользователей, которые добавляют хотя бы один из предложенных товаров в корзину (метрикой является конверсия действия «добавить в корзину сопутствующий товар»).

Для этого пользователи были разделены на две группы.

- **Контрольной (группе А)** доступен первый вариант рекомендаций.
- **Тестовой (группе B)** доступен второй вариант.

Предполагается, что все условия и принципы проведения A/B-теста были соблюдены, то есть:

- выборка имеет достаточный для принятия решения объём;
- выбран достаточный интервал времени для тестирования;
- группы максимально сбалансированы;
- внешнее вмешательство минимально.

Итак, в результате проведения A/B-тестирования были получены следующие данные:

Скачать данные

На основании этих данных нам предстоит сделать вывод о том, какой из вариантов рекомендаций (A или B) является более эффективным по показателю конверсии.

Знакомимся с данными

Прежде, чем переходить к этапу анализа, необходимо совершить некоторые преобразования над данными, для этого импортируем библиотеки, которые нам пригодятся:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Импортировав библиотеки, откроем в ноутбуке сохранённый в папке data датасет:

```
ab_data = pd.read_csv('data/ab_data.csv')
```

И выведем пять первых строк таблицы:

```
ab_data.head()
```

	user_id	timestamp	group	converted
0	851104	2017-01-21	A	0
1	804228	2017-01-12	A	0
2	661590	2017-01-11	B	0
3	853541	2017-01-08	B	0
4	864975	2017-01-21	A	1

Итак, мы получили таблицу со следующими полями:

Название поля	Описание
user_id	идентификатор пользователя, зашедшего на сайт
timestamp	дата события
group	группа теста (контрольная или тестовая)
converted	совершил пользователь целевое действие (нажатие кнопки «добавить в корзину» на сопутствующем товаре) или нет. 1 — да, 0 — нет.

Посмотрим тип данных и содержание каждого столбца, для этого применим к датафрейму метод info():

```
ab_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 290482 entries, 0 to 290481
Data columns (total 4 columns):
 #   Column     Non-Null Count   Dtype 
---  ------     --------------   ----- 
 0   user_id    290482 non-null  int64 
 1   timestamp  290482 non-null  object
 2   group      290482 non-null  object
 3   converted  290482 non-null  int64 
dtypes: int64(2), object(2)
memory usage: 8.9+ MB
```

Итак, в наших данных нет пропусков, однако нам нужно перевести столбец с информацией о дате события (timestamp) в формат datetime. Напомним: для этого мы можем воспользоваться функцией `pandas to_datetime()`:

```
ab_data['timestamp'] = pd.to_datetime(ab_data['timestamp'], format='%Y-%m-%d')
```

В итоге мы получили таблицу, в которой отображены входы клиента на сайт (сессия) и его действия. Для того, чтобы получить показатель конверсии в группах на каждый день, нам необходимо предварительно сгруппировать наши данные по дате и группе и вычислить количество пользователей, заходивших на сайт, и суммарное количество совершённых целевых действий.

Дополнительно для удобства сбросим индекс на сгруппированной таблице с помощью метода reset_index() и переименуем столбец `user_id` в `users_count` с помощью метода rename().

```
daily_data = ab_data.groupby(['timestamp','group']).agg({
    'user_id':'count',
    'converted':'sum'
}).reset_index().rename(columns={'user_id': 'users_count'})
daily_data.head()
```

	timestamp	group	users_count	converted
0	2017-01-02	A	2851	357
1	2017-01-02	B	2848	339
2	2017-01-03	A	6584	749
3	2017-01-03	B	6620	754
4	2017-01-04	A	6573	797

Примечание. Количество пользователей в день в контрольной и тестовой группе определяется как количество значений в столбце user_id в сгруппированных данных, а количество совершённых целевых действий — это просто сумма по столбцу converted (1 в строке означает положительный исход, то есть целевое действие было совершено, а 0 — отрицательный исход; просуммировав все 1 и 0 в группе, получим количество положительных исходов).

Для завершения этапа подготовки данных к анализу нам осталось рассчитать конверсию на каждый день проведения A/B-тестирования. Напомним: ежедневная конверсия рассчитывается как отношение числа посетителей сайта, выполнивших на нём какие-либо целевые действия в день, к ежедневному общему числу посетителей сайта, умноженная на 100 %:

$$conv_t = \frac{N_t^+}{N_t} \times 100$$

- N_t^+ — количество посетителей сайта, выполнивших целевое действие в день t;
- N_t — общее количество посетителей сайта в день t;
- $conv_t$ — конверсия в день t.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 4.1

0 points possible (ungraded)

Рассчитайте конверсию на каждый день по приведённой формуле. Результат занесите в столбец *conversion*. Чему была равна конверсия (в процентах) в группе *A* 5 января 2017 года? Ответ округлите до второго знака после запятой.

Отправить

Вычислив ежедневную конверсию, мы получим следующую таблицу:

```
display(daily_data.head())
```

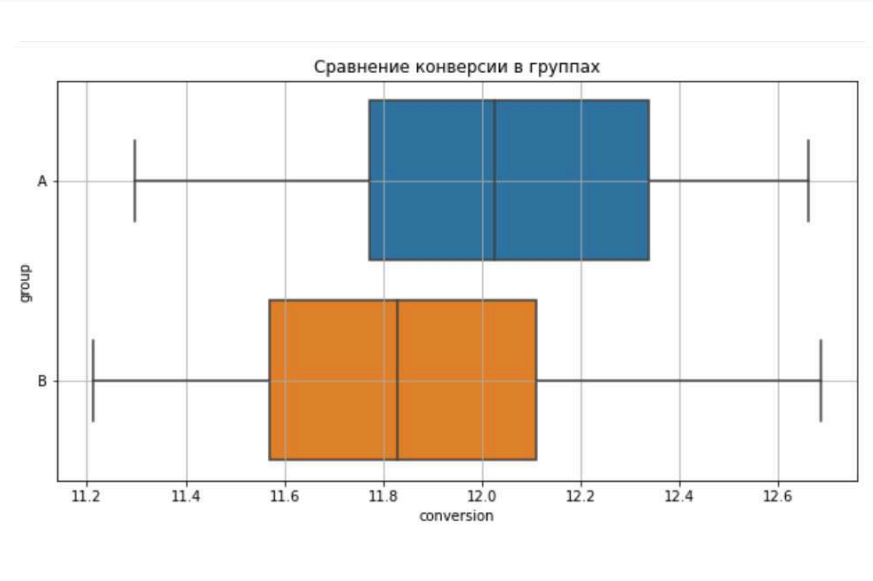
	timestamp	group	users_count	converted	conversion
0	2017-01-02	A	2851	357	12.521922
1	2017-01-02	B	2848	339	11.903090
2	2017-01-03	A	6584	749	11.376063
3	2017-01-03	B	6620	754	11.389728
4	2017-01-04	A	6573	797	12.125361

ВИЗУАЛЬНЫЙ АНАЛИЗ КОНВЕРСИИ В ГРУППАХ

Нам необходимо ответить на вопрос: в какой группе (A или B) показатель конверсии был выше, и на этой основе принять решение о том, какой из вариантов более эффективный.

Давайте построим диаграмму boxplot для сравнения параметров распределения ежедневной конверсии по группам.

```
# создаём фигуру размером 8x4
fig = plt.figure(figsize=(8, 4))
# добавляем систему координат
ax = fig.add_axes([1, 1, 1, 1])
# строим boxplot для conversion по признаку group
sns.boxplot(data=daily_data, x='conversion', y='group', ax=ax)
# задаём подпись к графику
ax.set_title('Сравнение конверсии в группах')
# задаём отображение сетки
ax.grid(True);
```



Что можно сказать исходя из этого графика? Во-первых, медианное значение ежедневной конверсии в группе А, выше, чем в группе B. Во-вторых, разброс конверсии в группе B несколько больше, чем в группе A.

Тут неопытный специалист воскликнет: версия рекомендаций А лучше, чем версия B (медиана конверсии варианта А больше, да и разброс поменьше, то есть конверсия более стабильна) — и завершит эксперимент.

Но выводы делать ещё рано! И сейчас попробуем понять, почему.

Примечание. Прежде чем делать выводы по визуальному представлению данных, всегда обращайте внимание на численные значения показателей. Они могут различаться в масштабе графика, но на практике разница между ними не будет иметь значения. Также всегда стоит убедиться, что в данных нет выбросов (базовый вариант — сравнить медиану и среднее между собой). Далее мы ещё поговорим о стабилизации показателя во времени.

Давайте вычислим средний и медианный показатель конверсии в контрольной и тестовой группе и умножим результат на 100 (получим результат в процентах):

```
conversion_piv = daily_data.groupby('group')['conversion'].agg(
    ['mean', 'median']
)
display(conversion_piv)
```

	mean	median
group		
A	12.032876	12.024261
B	11.872697	11.829582

Из полученной таблицы видно, что медиана и среднее в каждой группе практически совпадают. При этом разница между показателями в группах менее 0.3 %!

Возникает вопрос: а значима ли вообще такая разница в показателях или она обусловлена случайностью, и варианты A и B одинаково эффективны по метрике конверсии?

Прежде чем отвечать на этот вопрос, нам нужен ответ на другой вопрос: а стабилен ли показатель конверсии в группах во времени или он колеблется как «уж на сковородке»?

Примечание. Вопрос о стабильности метрик является обязательным. Прогонять результаты через статистические тесты можно только после того, как вы убедитесь, что разница в ваших показателях стабилизировалась во времени. Это связано с проблемой подглядывания, речь о которой пойдёт ниже.

Тут нам помогут кумулятивные метрики.

КУМУЛЯТИВНЫЕ МЕТРИКИ

В течение всего теста рекомендуется отслеживать поведение целевых метрик. Для этого чаще всего используются кумулятивные метрики.

> **Кумулятивная метрика** — это отображение целевой метрики, когда вы отслеживаете её поведение за каждый день — накопленным итогом по дням.

Почему это важно?

Ранее мы говорили, что для принятия решения в A/B-тестировании требуется большой объём данных, что соотносится с законом больших чисел: чем больше данных — тем ближе получаемый результат к математическому ожиданию (истинному среднему).

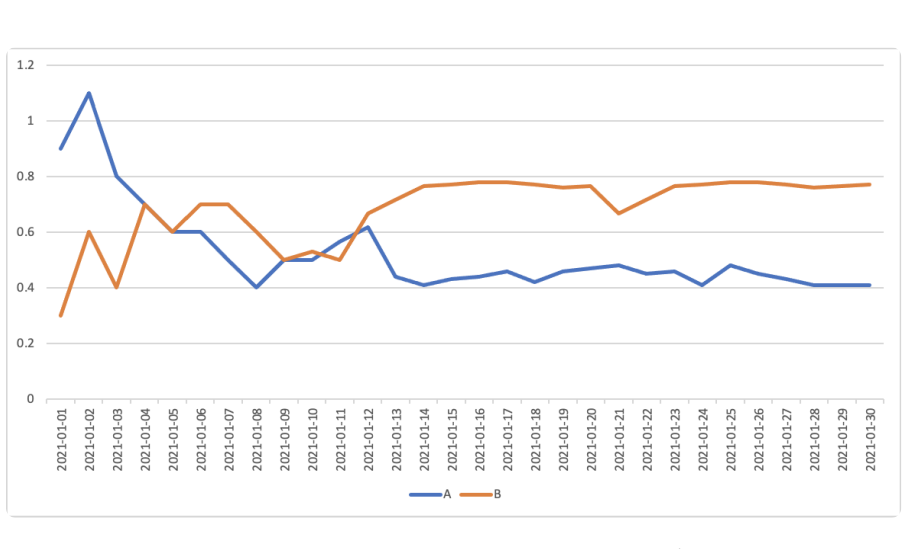
Вспомните пример про подбрасывание монетки. Если подбросить монетку 10 раз, то велика вероятность, что один из двух результатов будет значительно превышать второй. Но если подбросить её 1 000 раз, то результаты компенсируют друг друга, и мы увидим, что итог крайне близок к разделению 50/50 (количество выпавших орлов и решек будет практически одинаковым).

Так же и в A/B-тестировании: с самого начала любой из вариантов может показывать лучшие результаты, и статистическая значимость может быть достигнута уже в первые дни теста. Это явление называется **проблемой подглядывания**.

Важно! Не стоит торопиться: как минимум дождитесь накопления запланированных объёмов выборки и длительности теста. Но и тут не стоит заканчивать тест — проверьте, стабилизировались ли метрики!

То есть в начале теста любой полученный за день результат будет оказывать сильное влияние на результат теста. Но со временем данные будут накапливаться, и доля получаемых за день данных будет всё меньше и меньше воздействовать на уже накопленный результат.

Кумулятивный график конверсии может выглядеть, например, так:



Как видите, в начале данные подвержены сильному колебанию, но со временем они выравниваются.

Примечание. Кумулятивная метрика считается стабилизированной, когда на графике прекращаются резкие пики и спады показателя, и линия постепенно выравнивается в горизонтальную прямую.

Важно! Принимать какие-либо решения стоит только после того, как метрика стабилизируется!

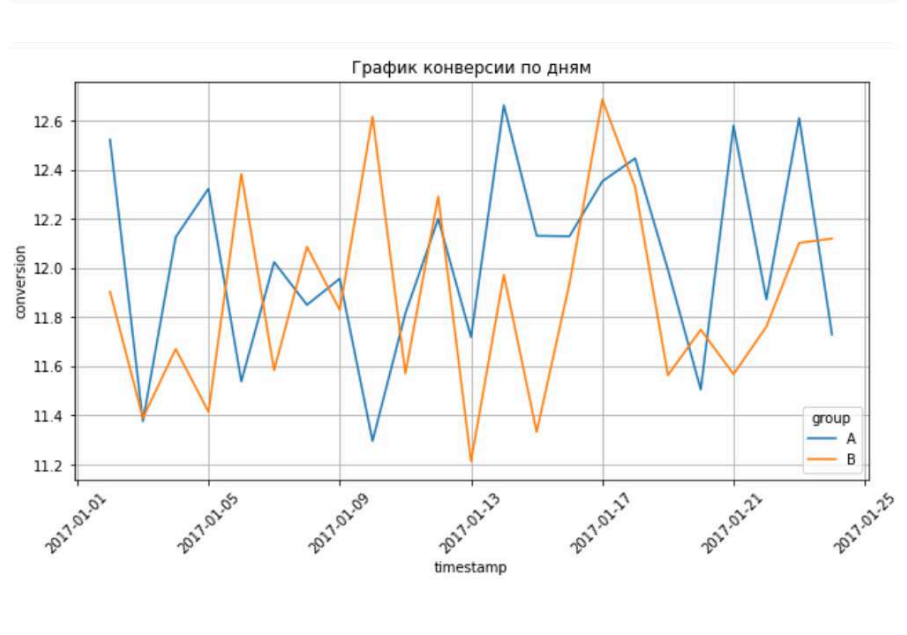
Согласно центральной предельной теореме и закону больших чисел, с увеличением объёма данных метрика стабилизируется и будет стремиться к реальному среднему.

Примечание. В качестве кумулятивной метрики можно считать не только конверсию, но и выручку, средний чек, количество заказов, процент негативных комментариев, точность модели и так далее.

Итак, давайте применим новые знания к нашим данным и вычислим кумулятивную конверсию, а затем построим её график.

Но для начала ради эксперимента посмотрим, как вообще изменялись показатели ежедневной конверсии в каждой группе во времени. Для этого построим линейные графики для каждой группы: по оси абсцисс отложим дату, а по оси ординат — показатель конверсии:

```
# создаём фигуру размером 8x4
fig = plt.figure(figsize=(8, 4))
# добавляем систему координат
ax = fig.add_axes([1, 1, 1, 1])
# строим lineplot для конверсии во времени в каждой группе
sns.lineplot(
    data=daily_data,
    x='timestamp',
    y='conversion',
    hue='group',
    ax=ax
)
# задаём подпись к графику
ax.set_title('График конверсии по дням')
# задаём поворот меток на оси абсцисс
ax.xaxis.set_tick_params(rotation=45)
# задаём отображение сетки
ax.grid(True);
```



Что можно сказать по такому графику? Да практически ничего! Конверсия у обеих групп «скачет» день ото дня. Понять, какая из групп лучше, — практически невыполнимая задача.

Именно поэтому и принято смотреть график кумулятивной конверсии, где за каждый день отображаются данные накопленным итогом.

Для расчёта кумулятивного показателя суммы в Pandas есть специальный метод cumsum(). Давайте посмотрим, как он работает — сначала на данных о ежедневном количестве посетителей для группы А.

Выберем данные для группы А и создадим в полученной таблице новый столбец cum_users_count, в который запишем результат работы метода cumsum() по столбцу users_count:

```
# выделяем данные группы А
daily_data_a = daily_data[daily_data['group'] == 'A']
# считаем кумулятивное количество посетителей
daily_data_a.loc[:, 'cum_users_count'] = daily_data_a['users_count'].cumsum()
# выводим время, количество посетителей и кумулятивное количество посетителей
display(daily_data_a[['timestamp', 'users_count', 'cum_users_count']].head())
```

	timestamp	users_count	cum_users_count
0	2017-01-02	2851	2851
2	2017-01-03	6584	9435
4	2017-01-04	6573	16008
6	2017-01-05	6419	22427
8	2017-01-06	6604	29031

Обратите внимание, что в столбце с кумулятивной суммой за 2017-01-03 указано 9 435 пользователей. Сложите данные за 2017-01-02 и 2017-02-03 из столбца users_count — получите те же 9 435. Аналогично и для других дней.

То есть кумулятивная сумма — это накопительная сумма; каждое её значение — это результат сложения показателя за предыдущие дни с показателем за текущий день.

Кумулятивную сумму можно записать в виде рекурсивной формулы:

$$S_t = S_{t-1} + x_t$$

- x_t — значение показателя в день t;
- S_t — значение суммы в день t.

Как сделать такой расчёт для двух групп (A и B) сразу? Воспользуемся методом `groupby`: сгруппируем данные по признаку группы, и в каждой группе будем вычислять кумулятивные суммы признаков количества посетителей (users_count) и количества совершённых целевых действий (converted).

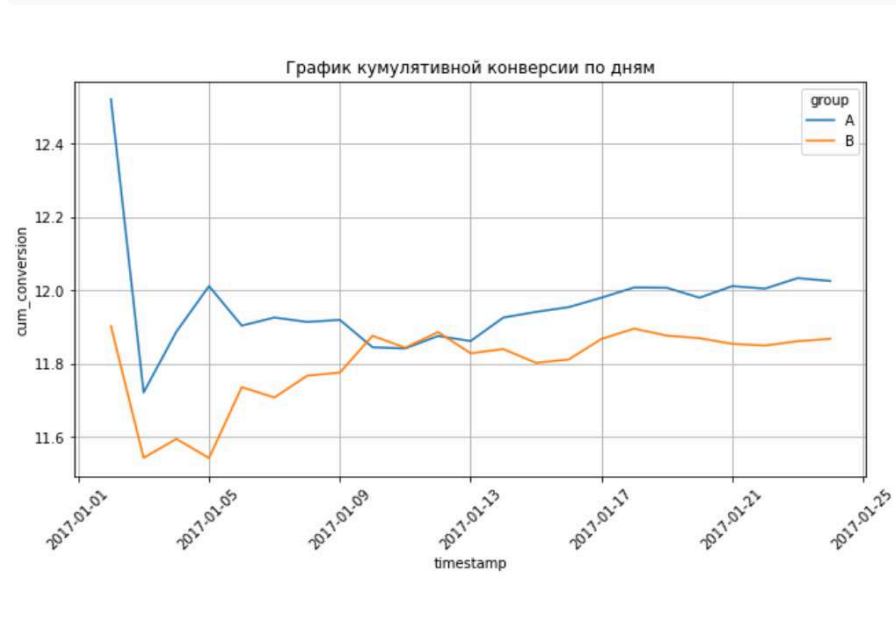
После этого рассчитаем **кумулятивную конверсию** — это отношение кумулятивного числа совершённых целевых действий к кумулятивному числу посетителей:

```
# вычисляем кумулятивную сумму количества посетителей
daily_data['cum_users_count'] = daily_data.groupby(['group'])
['users_count'].cumsum()
# вычисляем кумулятивную сумму количества совершённых целевых действий
daily_data['cum_converted'] = daily_data.groupby(['group'])['converted'].cumsum()
# вычисляем кумулятивную конверсию
daily_data['cum_conversion'] =
daily_data['cum_converted']/daily_data['cum_users_count'] * 100
display(daily_data.head())
```

	timestamp	group	users_count	converted	conversion	cum_users_count	cum_converted	cum_conversion
0	2017-01-02	A	2851	357	12.521922	2851	357	12.521922
1	2017-01-02	B	2848	339	11.903090	2848	339	11.903090
2	2017-01-03	A	6584	749	11.376063	9435	1106	11.722311
3	2017-01-03	B	6620	754	11.389728	9468	1093	11.544149
4	2017-01-04	A	6573	797	12.125361	16008	1903	11.887806

А теперь построим графики кумулятивной конверсии по дням в каждой группе:

```
# создаём фигуру размером 8x4
fig = plt.figure(figsize=(8, 4))
# добавляем систему координат
ax = fig.add_axes([1, 1, 1, 1])
# строим lineplot для кумулятивной конверсии во времени в каждой группе
sns.lineplot(x='timestamp', y='cum_conversion', data=daily_data, hue='group',
ax=ax)
# задаём подпись к графику
ax.set_title('График кумулятивной конверсии по дням')
# задаём поворот меток на оси абсцисс
ax.xaxis.set_tick_params(rotation = 45)
# задаём отображение сетки
ax.grid(True);
```



Что можно сказать теперь? До 2017-01-09 конверсия была нестабильна, но после 2017-01-11 резкие скачки прекратились (кривая графика переходит в прямую). Судя по графику, конверсия группы B стабильно хуже конверсии группы А на всём промежутке времени.

Обратите внимание на различие в конверсии в первые дни теста: тут наглядно отображена проблема подглядывания, когда статистическая значимость ошибочно могла быть достигнута уже в самом начале теста.

На основании стабилизированного кумулятивного графика мы можем сделать вывод, что длительность эксперимента была достаточной для того, чтобы судить о статистической значимости разности между показателем конверсии в группах.

Значит, мы можем переходить к проведению статистических тестов. Но прежде предлагаем решить несколько заданий.

Задание 4.2

0 points possible (ungraded)

Почему при проведении анализа результатов *A/B*-тестирования нельзя делать выводы, основываясь только на визуальном анализе и вычисленных показателях в группах?

○ визуальный анализ всегда «врёт»

● разница между метриками в группах может оказаться статистически незначимой

○ визуальный анализ не даёт представления о разнице между показателями в группах

✓

Show answer

Отправить

✓ Верных

Задание 4.3

0 points possible (ungraded)

Что такое проблема подглядывания?

● ошибочное принятие решения о статистической значимости результатов до ожидания окончания статистического теста

○ предсказание конечных результатов теста с помощью модели машинного обучения

○ несбалансированность групп для тестирования

○ запуск теста без предварительного определения гипотезы и метрик

✓

Show answer

Отправить

✓ Верных

Задание 4.4

0 points possible (ungraded)

С помощью какого метода *Pandas* можно вычислить кумулятивную сумму по столбцу?

○ `cumcount()`

○ `cummean()`

○ `sum()`

● `cumsum()`

✓

Show answer

Отправить

✓ Верных

Вводные данные для задач 4.6–4.9

Проводится A/B-тестирование для двух вариантов дизайна сайта. Результаты тестирования собраны в двух файлах (данные для группы А и для группы B).

→ Данные по группе А

→ Данные по группе B

```
sample_a = pd.read_csv('data/ab_test-redesign_sample_a.csv')
sample_b = pd.read_csv('data/ab_test-redesign_sample_b.csv')
```

Каждая таблица имеет следующие поля:

- date — дата наблюдения;
- cid — идентификатор посетителя;
- transactions — количество совершённых целевых действий (транзакций);
- revenue — сумма покупки;
- group — группа (тестовая — А, контрольная — B).

Используйте эти данные для ответа на следующие вопросы.

Задание 4.5

0 points possible (ungraded)

Постройте таблицу со следующими ежедневными показателями:

- количество посетителей сайта;
- суммарное число транзакций;
- общая сумма покупок (выручка).

Для этого сгруппируйте данные по признаку *date*, предварительно преобразовав его к типу *datetime*.

Чему равны значения ежедневных показателей в группе *A* на 8 ноября 2020 года?

Количество посетителей сайта:

Суммарное число транзакций:

Общая сумма покупок:

Hint

Отправить

Задание 4.6

0 points possible (ungraded)

Добавьте в каждую из таблиц кумулятивные метрики:

- кумулятивное количество посетителей сайта;
- кумулятивное суммарное число транзакций;
- кумулятивная общая сумма покупок (кумулятивная выручка).

Рассчитайте кумулятивную конверсию действия покупки (в процентах) и кумулятивный средний чек (кумулятивная выручка/кумулятивное суммарное число транзакций).

Чему равна средняя кумулятивная конверсия в группах за весь период проведения *A/B*-теста? Ответ округлите до сотых, знаки процентов не вводите.

Hint

Отправить

Задание 4.7

0 points possible (ungraded)

Постройте графики кумулятивной конверсии в каждой группе. Какие выводы по ним вы можете сделать?

○ Конверсия в группе *B* стабильно выше конверсии в группе *A* на всём промежутке времени.

○ С уверенностью можно сказать, что, если продолжить тест, конверсия в группе *A* будет выше конверсии в группе *B*.

○ Начиная с 9 ноября 2020 года конверсии в группах имеют приблизительно идентичное поведение (одинаковые точки роста и спада). При этом конверсия в группе *B* стабильно выше конверсии в группе *A* на данном периоде.

○ Никаких выводов сделать нельзя.

Hint

Отправить

Задание 4.8

0 points possible (ungraded)

Постройте графики кумулятивного среднего чека в каждой группе. Какие выводы по ним вы можете сделать?

○ Средний чек в группе *B* стабильно выше среднего чека в группе *A* на всём промежутке времени.

○ В начале периода наблюдения средний чек в группе *B* был меньше, чем средний чек в группе *A*. Однако при стабилизации метрик наблюдается обратный результат.

○ В начале периода наблюдения средний чек в группе *A* был меньше, чем средний чек в группе *B*. Однако при стабилизации метрик наблюдается обратный результат.

○ Никаких выводов сделать нельзя.

Hint

Отправить

< Предыдущий урок Следующий урок >

5/9 Анализ результатов A/B-тестирования: статистические тесты

текущий урок

Наша задача состоит в том, чтобы проверить гипотезу о равенстве (различии) между конверсиями в группах, исходя из выборочных данных.

Примечание. Говоря о равенстве здесь и далее, мы не имеем в виду фактическое равенство показателей ($2 = 2$) — мы хотим узнать, есть ли между полученными значениями статистическое различие. Однако для простоты обычно говорят просто о равенстве или неравенстве.

Например, конверсия для одной группы может быть равна 4 %, а для другой — 4.5 %. Фактически показатели не равны, но это не говорит о том, что конверсия для другой группы лучше: возможно, подобное отклонение — в пределах допустимого для рассматриваемых данных, и тогда нельзя говорить о статистическом различии.

То есть мы, по сути, хотим ответить на вопрос: «Существует ли статистическая значимость в эффективности (разнице конверсий) контрольного и тестового вариантов?». Причём мы хотим проверить статистическую значимость результатов для всей генеральной совокупности (всех пользователей приложения), имея только выборки в виде данных тестовой и контрольной групп.

Статистические тесты — как раз один из инструментов, который позволяет исследовать значения параметров генеральной совокупности на основе выборочных данных.

Мы можем вычислить p-value для проверки нулевой и альтернативной гипотез и сравнить его с уровнем значимости для рассматриваемых нулевой и альтернативной гипотез и сравнить это значение с заранее заданным пороговым значением — допустимой вероятностью совершить ошибку I рода (уровень значимости α). На основе этого сравнения можно определить справедливость выдвинутых гипотез и принять важное бизнес-решение.

Какой уровень значимости выбрать?

Стараются придерживаться уровней значимости $\alpha = 0.05$ или $\alpha = 0.1$. Помните, что больший уровень значимости подразумевает больший риск совершить ошибку I рода (отклонить верную нулевую гипотезу). Вы сами определяете уровень значимости в зависимости от гипотезы и того, насколько критично в вашем случае допустить ошибку I рода.

Как правило, если возникают спорные ситуации, то склоняются в пользу уже существующего варианта А, так как он не требует временных и денежных затрат на внедрение. В нашем примере варианты А и B оказались равными по признаку конверсии при уровне значимости $\alpha = 0.05$, однако при уровне значимости $\alpha = 0.1$ вариант А оказывается более эффективным. Так или иначе, немедленное внедрение варианта B — определённо не лучшая идея: необходимо либо доработать его и заново провести тестирование, либо отказаться от него вовсе.

Мы знаем, что важно уметь правильно выбирать статистический тест для конкретного вида задач. Давайте рассмотрим наиболее популярные статистические тесты, применяемые при A/B-тестировании: с большинством из них вы уже знакомы, но мы также рассмотрим и те тесты, которые не освещали ранее.

СТАТИСТИЧЕСКИЕ ТЕСТЫ ДЛЯ КОНВЕРСИИ

Оказывается, интересующий нас признак (совершил ли пользователь конверсию) — это бинарная случайная величина, имеющая распределение Бернулли. Такая случайная величина имеет два возможных исхода: успех (1) — пользователь совершил целевое действие и неуспех (0) — пользователь не совершил целевое действие.

> **Конверсия** — это доля (пропорция) пользователей, которые совершили целевое действие (например, добавили сопутствующий товар в корзину). Истинная конверсия (в смысле генеральной совокупности) — это вероятность «успеха» (традиционно её обозначают как p), то есть **вероятность того, что пользователь совершит целевое действие**.

При такой постановке задачи наша цель — сравнить истинные конверсии в группах А и B на основе выборочных данных.Для проверки гипотезы равенства (при альтернативной гипотезе о различиях) пропорций мы можем воспользоваться следующим рядом статистических тестов:

- Z-тест для пропорций (**Z-критерий**). Самый популярный критерий для определения статистической значимости изменения конверсии.
- χ^2-тест (хи-квадрат, **χ^2-критерий**) для пропорций — это знакомый нам статистический тест для проверки независимости двух категориальных признаков. Однако его можно использовать и при проверке различия пропорций для сравнения наблюдаемых и ожидаемых частот в каждой группе, которые можно рассчитать на основе гипотетической модели независимости.
- Критерий Мак-Немара — ещё один знакомый нам статистический тест для проверки равенства пропорций в двух зависимых (связанных) между собой выборках. Как правило, при A/B-тестировании стараются использовать именно независимые выборки, то есть показывать разным пользователям разные версии продукта. Однако если по каким-то причинам произвести независимое A/B-тестирование невозможно и группы являются зависимыми, можно воспользоваться этим критерием.

Вернёмся к нашей задаче об оценке разницы конверсий действия «Добавить товар в корзину» для двух вариантов рекомендательных систем.

Сначала подготовим данные к проведению статистических тестов. Для этого нам нужно рассчитать количество пользователей, совершивших конверсию, и общее количество пользователей в каждой контрольной и тестовой группах.

Сгруппируем таблицу ab_data по признаку группы и рассчитаем количество «успехов» (сумма по столбцу converted) и общее количество наблюдений (количество непустых значений в столбце converted) для каждой группы:

```
converted_piv= ab_data.groupby('group')['converted'].agg(
    ['sum', 'count']
)
display(converted_piv)
```

group	sum	count
A	17467	145241
B	17238	145241

Теперь у нас есть всё для того, чтобы провести тестирование.

Мы знаем, что группы независимые: пользователи в группах А и B не пересекаются. Значит, нам нужно воспользоваться либо Z- либо χ^2 -критерием. Давайте рассмотрим тонкости применения обоих критериев.

Z-ТЕСТ ДЛЯ ПРОПОРЦИЙ

При использовании Z-теста выдвигают следующие нулевую (H_0) и альтернативную (H_1) гипотезы:

Нулевая гипотеза: разница пропорций в группах А и B равна некоторому заранее заданному числу value.

$H_0 : p_a - p_b = value$

Альтернативная гипотеза: разница пропорций в группах А и B не равна некоторому заранее заданному числу value.

$H_1 : p_a - p_b \neq value$

По умолчанию $value = 0$, соответственно, для случая двусторонних гипотез тестируется нулевая гипотеза о равенстве между истинными пропорциями p_a и p_b ($H_0 : p_a = p_b$) против альтернативной об их неравенстве ($H_1 : p_a \neq p_b$).

Альтернативная гипотеза также может быть и односторонней. Тогда гипотезы задаются в следующем виде:

- Правосторонняя альтернативная:

  $H_0 : p_a - p_b \leq value$

  $H_1 : p_a - p_b > value$
- Левосторонняя альтернативная:

  $H_0 : p_a - p_b \geq value$

  $H_1 : p_a - p_b < value$

При $value = 0$ мы получаем $H_0 : p_a \leq p_b$ против $H_1 : p_a > p_b$ для правосторонней альтернативной гипотезы и $H_0 : p_a \geq p_b$ против $H_1 : p_a < p_b$ — для левосторонней альтернативной гипотезы.

Z-тест для пропорций реализован в функции proportions_ztest() модуля proportion библиотеки statsmodels. Импортируем его:

```
from statsmodels.stats.proportion import proportions_ztest
```

Для получения p-value необходимо передать в функцию следующие аргументы:

- `count` — количество «успехов» (в каждой из групп). В данном случае это количество пользователей, которые совершили целевое действие в каждой группе.
- `nobs` — количество наблюдений (в каждой из групп), то есть размер выборки.
- `alternative` — вид альтернативной гипотезы (`'two-sided'` — двусторонняя, `'smaller'` — левосторонняя, `'larger'` — правосторонняя). По умолчанию параметр равен `'two-sided'`.
- `value` — число, с которым сравнивается разница между двумя пропорциями. По умолчанию равно 0.

Примечание. Этот параметр может пригодиться, если необходимо проверить разницу между пропорциями на соответствие конкретному числу: например, гипотезы вида «Конверсия в группе А меньше, чем конверсия в группе B на 3 %». Тогда в качестве значения `value` можно установить число 0.03.

Давайте применим Z-тест для пропорций на примере двух типов рекомендаций нашего интернет-магазина. Возьмём в качестве альтернативной двустороннюю гипотезу о неравенстве пропорций с параметром `value=0`. То есть у нас будут две следующие конкурирующие гипотезы:

- Нулевая (будет задавать отсутствие эффекта): конверсии в группах А и B равны.

  $H_0 : p_a = p_b$
- Альтернативная (будет задавать наличие эффекта): конверсии в группах А и B не равны.

  $H_1 : p_a \neq p_b$

По умолчанию функция `proportions_ztest()` возвращает z-статистику и p-value. Z-статистика нас пока что не интересует, а вот p-value мы занесём в отдельную переменную для дальнейшего сравнения с уровнем значимости.

Зададим уровень значимости $\alpha = 0.05$ и будем сравнивать полученное p-value с ним.

Примечание. Давайте вспомним, что:

- Если полученное значение p-value **ниже уровня значимости** ($p \leq \alpha$), это означает, что результаты статистически значимы и согласуются с альтернативной гипотезой.
- Если полученное значение p-value **выше уровня значимости** ($p < \alpha$), результаты считаются статистически незначимыми и нельзя отвергнуть нулевую гипотезу.

```
alpha = 0.05 # уровень значимости
# вычисляем значение p-value для z-теста для пропорций
_, p_value = proportions_ztest(
    count=converted_piv['sum'], #число «успехов»
    nobs=converted_piv['count'], #общее число наблюдений
    alternative='two-sided',
)
# выводим результат на экран
print('p-value: ', round(p_value, 3))
# сравниваем полученное p-value с уровнем значимости
if (p_value <= alpha):
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value:  0.19
# У нас нет оснований отвергнуть нулевую гипотезу
```

Итак, мы получили p-value = 0.19. Это больше, чем заданный уровень значимости 0.05, а значит, мы должны принять нулевую гипотезу о равенстве конверсий.

Что это значит для нас и для бизнеса?

Так как Z-тест для пропорций показал, что истинные значения конверсий в группах равны (с уровнем значимости 0.05), то для нас это значит, что альтернативная гипотеза не подтверждается статистически.

Переводим на язык бизнеса: разницы между вариантами рекомендаций сопутствующих товаров нет — они одинаково эффективны **по метрике конверсии**.

На основе этого вывода мы принимаем решение о том, что делать дальше.

Возможные варианты:

- отказаться от внедрения ML-модели для рекомендаций сопутствующих товаров и оставить текущую версию рекомендаций;
- попытаться усовершенствовать модель и провести тестирование снова;
- использовать оба варианта системы рекомендаций и посмотреть на результат в более долгосрочной перспективе.

А что, если мы хотим уточнить исследование и проверить гипотезу о том, что варианты А и B не просто идентичны между собой, а вариант А даже лучше варианта B по метрике конверсии? Тогда нам нужно переформулировать гипотезы таким образом, чтобы альтернативная гипотеза отражала наш бизнес-вопрос:

Нулевая (будет задавать отсутствие эффекта): конверсия в группе А меньше либо равна конверсии в группе B.

$H_0 : p_a \leq p_b$

Альтернативная (будет задавать наличие эффекта): конверсия в группе А больше, чем конверсия в группе B.

$H_1 : p_a > p_b$

Тогда код будет выглядеть следующим образом:

```
alpha = 0.05 # уровень значимости
# вычисляем значение p-value для Z-теста для пропорций
_, p_value = proportions_ztest(
    count=converted_piv['sum'], # число «успехов»
    nobs=converted_piv['count'], # общее число наблюдений
    alternative='larger' # правосторонняя альтернативная гипотеза
)
# выводим результат на экран
print('p-value: ', round(p_value, 3))
# сравниваем полученное p-value с уровнем значимости
if (p_value <= alpha):
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value:  0.095
# У нас нет оснований отвергнуть нулевую гипотезу
```

Итак, p-value = 0.095 вновь больше, чем уровень значимости 0.05. А значит, что статистика говорит нам о том, что вариант А все таки не лучше, чем вариант B, что, в принципе, было ожидаемым из предыдущего тестирования о равенстве конверсий.

Подробнее с Z-тестом для пропорций и его математической основой мы предлагаем вам познакомиться здесь.

χ^2-ТЕСТ ДЛЯ ПРОПОРЦИЙ

Теперь давайте воспользуемся χ^2 -тестом для оценки разницы в пропорциях.

Этот тест основан на уже знакомом нам χ^2-критерии, который мы применяли для тестирования гипотезы о независимости двух категориальных признаков. Оказывается, этот же критерий с небольшими модификациями можно использовать и для установления статистической значимости разницы между пропорциями.

Этот тест применяется только для двусторонних гипотез:

Нулевая гипотеза: разница пропорций в группах А и B равна некоторому заранее заданному числу $value$:

$H_0 : p_a - p_b = value$

Альтернативная гипотеза: разница пропорций в группах А и B отлична от числа $value$:

$H_1 : p_a - p_b \neq value$

По умолчанию $value = 0$, соответственно, проверяется нулевая гипотеза равенства между истинными пропорциями p_a и p_b ($H_0 : p_a = p_b$) против альтернативной гипотезы об их неравенстве ($H_1 : p_a \neq p_b$).

Тестом не предусмотрена возможность проверки односторонних гипотез.

χ^2-тест для пропорций реализован в функции proportions_chisquare() модуля proportion библиотеки statsmodels. Импортируем его:

```
from statsmodels.stats.proportion import proportions_chisquare
```

Для получения χ^2-статистики и p-value необходимо передать в функцию следующие аргументы:

- `count` — количество «успехов» (в каждой из групп). В данном случае это количество пользователей, которые совершили целевое действие в каждой группе.
- `nobs` — количество наблюдений (в каждой из групп), то есть размер выборки.
- `value` — число, с которым сравнивается разница между двумя пропорциями. По умолчанию равно 0.

Давайте применим χ^2-тест для пропорций на примере двух типов рекомендаций интернет-магазина с параметром `value=0`. То есть у нас будут две следующие конкурирующие гипотезы:

Нулевая (будет задавать отсутствие эффекта): конверсии в группах А и B равны.

$H_0 : p_a = p_b$

Альтернативная (будет задавать наличие эффекта): конверсии в группах А и B не равны.

$H_1 : p_a \neq p_b$

По умолчанию функция `proportions_chisquare()` возвращает χ^2-статистику, p-value и таблицу сопряжённости (используется во внутренних расчётах p-value). Из всего перечня возвращаемых значений нас интересует только p-value. Занесём его в отдельную переменную для дальнейшего сравнения с уровнем значимости.

Зададим уровень значимости $\alpha = 0.05$ и будем сравнивать с ним полученное p-value.

```
alpha = 0.05 # уровень значимости
# вычисляем значение p-value для Z-теста для пропорций
_, p_value, c = proportions_chisquare(
    count=converted_piv['sum'], #число «успехов»
    nobs=converted_piv['count'] #общее число наблюдений
)
# выводим результат на экран
print('p-value: ', round(p_value, 3))
# сравниваем полученное p-value с уровнем значимости
if (p_value <= alpha):
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value:  0.19
# У нас нет оснований отвергнуть нулевую гипотезу
```

Мы получили то же самое значение p-value, что и при тестировании гипотез с помощью Z-теста. Это неудивительно, ведь Z-критерий основан на нормальном распределении, а χ^2-критерий основан на χ^2-распределении, которое получается как сумма квадратов случайных величин, распределённых нормально.

То есть способы вычисления статистики для Z- и χ^2-критериев отличаются математически, однако при большом количестве наблюдений p-value для этих критериев будут совпадать.

Подробнее о распределении χ^2 и о семействе χ^2-критериев вы можете узнать в блоге Skillfactory и в этой статье на Хабре.

О том, при каких условиях наступает математическая эквивалентность Z- и χ^2-критерия, можно прочитать здесь и здесь.

Итак, согласно критерию χ^2, у нас нет оснований отвергнуть нулевую гипотезу, а значит, мы в очередной раз подтвердили, что разницы между рекомендациями по метрике конверсии нет.

Отсюда возникает закономерный вопрос: какой из критериев использовать?

- Если есть две независимых выборки и необходимо проверить равенство пропорций в них, то можно использовать Z-критерий для пропорций или χ^2-критерий для пропорций.

  Если объём выборок достаточно большой (обычно больше 30 наблюдений в каждой группе), то оба критерия дают сравнимые результаты. В противном случае рекомендуется применять χ^2 для пропорций.

  Также в литературе часто указывают, что при маленьких частотах в группах (менее пяти наблюдений в каждой ячейке таблицы сопряжённости) вместо χ^2-критерия рекомендуется применять точный критерий Фишера. Однако в реальных задачах A/B-тестирования стараются не применять тест на маленьких выборках, чтобы избежать эффекта подглядывания, поэтому тест Фишера вам вряд ли пригодится.
- Если по каким-то причинам нельзя обеспечить независимость групп, можно использовать критерий Мак-Немара. Однако он подходит только для парных данных, то есть каждому наблюдению в одной выборке должно соответствовать наблюдение в другой выборке.

Таким образом, если соблюдаются все условия проведения A/B-теста, в подавляющем большинстве случаев для проверки статистической значимости разницы конверсий в группах будет достаточно использовать Z-тест для пропорций.

Примечание. Важно отметить, что тесты для пропорций можно применять не только к конверсии. Они подойдут для любых бизнес-метрик, которые отражают некоторые долевые соотношения: например, так можно оценивать долю курящих людей, долю просроченных кредитов и т. д.

СТАТИСТИЧЕСКИЕ ТЕСТЫ ДЛЯ ДРУГИХ МЕТРИК

На самом деле наша задача об оценке эффективности двух вариантов была сильно упрощена: мы рассматривали только одну метрику — конверсию добавления в корзину. В реальности смотрят на несколько показателей в комплексе.

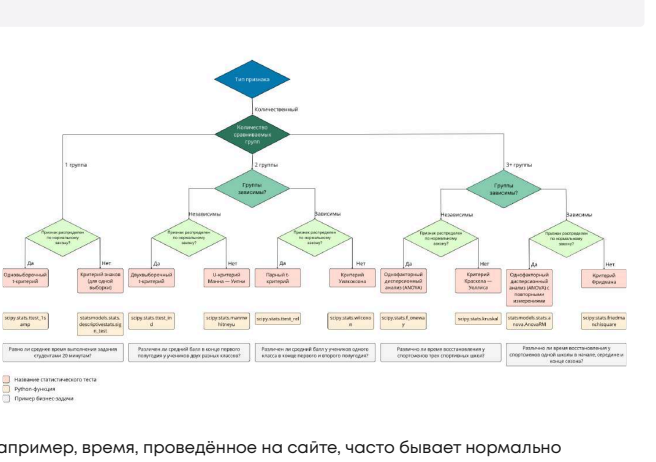
Помимо конверсии (категориальной метрики), нужно рассматривать и другие, числовые метрики, например средний чек. Ведь возможно такое, что конверсия в группах одинаковая, а средние чеки существенно различаются. В таком случае проводят статистические тесты для обеих метрик и принимают решение в пользу варианта, который эффективнее по более приоритетной метрике (например, если мы хотим повысить прибыль, это будет средний чек).

Что делать с другими метриками?

То же самое: провести визуальный анализ показателей, посмотреть на кумулятивные графики, убедиться в достаточном накопительном эффекте, после чего провести статистический тест (построить доверительный интервал, речь о котором пойдёт далее) и определить статистическую значимость разницы в показателях.

Однако стоит помнить о том, что любой статистический метод имеет свою область применения, которая зависит от задачи и распределения данных. Перед тем как проводить статистический тест, важно узнать распределение метрики, по которой вы будете сравнивать.

Диаграмма-алгоритм по выбору статистического теста:



Например, время, проведённое на сайте, часто бывает нормально распределённым. Тогда мы можем использовать двухвыборочный T-критерий для средних. А вот денежные метрики, такие как средний чек, часто (но не всегда) напоминают логнормальное распределение: большинство наблюдений сосредоточены около нуля, и частота постепенно падает.

Напомним, что для проверки распределения на нормальность мы всегда можем воспользоваться самым популярным и знакомым нам тестом Шапиро — Уилка или любым другим, например, тестом Д'Агостино K^2.

ПАРАМЕТРИЧЕСКИЕ ТЕСТЫ

Давайте рассмотрим пример.

Мы проводим A/B-тест, чтобы оценить, влияет ли тема оформления на привлекательность сайта для пользователя. Гипотеза заключается в том, что тёмная тема более привлекательна для посетителей, поэтому их сильнее заинтересует наш сайт, следовательно, они проведут на нём больше времени. Уровень значимости равен 0.05.

Результаты тестирования представлены в этом файле.

Прочитаем его:

```
time_data = pd.read_excel('data/time_data.xlsx')
display(time_data.head())
```

	time(A)	time(B)
0	435.0	549.2
1	454.5	630.2
2	447.0	437.2
3	480.0	637.2
4	561.0	712.4

Для начала сравним среднее время в группах на глаз. Рассчитаем среднее и медианное время в каждой группе:

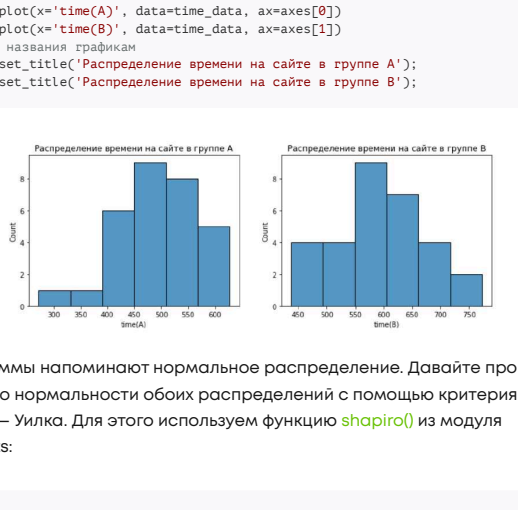
```
time_data[['time(A)', 'time(B)']].agg(['mean', 'median'])
```

	time(A)	time(B)
mean	492.90	595.406667
median	482.25	600.400000

По предварительным подсчётам, среднее время, проводимое на сайте, в группе B выше, чем в группе А. Однако делать выводы ещё рано — необходимо определить статистическую значимость полученных результатов.

Затем построим гистограмму времени, которое провели на сайте пользователи в каждой из групп:

```
# создаём два графика размером 12x4
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
# строим гистограмму для времени на сайте в каждой группе
sns.histplot(x='time(A)', data=time_data, ax=axes[0])
sns.histplot(x='time(B)', data=time_data, ax=axes[1])
# задаём названия графикам
axes[0].set_title('Распределение времени на сайте в группе A');
axes[1].set_title('Распределение времени на сайте в группе B');
```



Гистограммы напоминают нормальное распределение. Давайте проверим гипотезу о нормальности обоих распределений с помощью критерия Шапиро — Уилка. Для этого используем функцию shapiro() из модуля scipy.stats:

```
from scipy.stats import shapiro
```

Зададим уровень значимости $\alpha = 0.05$ и проверим гипотезу о нормальности каждого из распределений (нулевая гипотеза — распределение нормальное, альтернативная — распределение отлично от нормального):

```
alpha = 0.05 # уровень значимости
# вычисляем результат теста Шапиро — Уилка для выборок
shapiro_result_a = shapiro(time_data['time(A)'])
shapiro_result_b = shapiro(time_data['time(B)'])
print('p-value группы А', round(shapiro_result_a.pvalue, 2))
print('p-value группы B', round(shapiro_result_b.pvalue, 2))

# сравниваем полученное p-value для группы А с уровнем значимости
if shapiro_result_a.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной. Распределение в группе А отлично от нормального")
else:
    print("Принимаем нулевую гипотезу. Распределение в группе А является нормальным")

# сравниваем полученное p-value для группы B с уровнем значимости
if shapiro_result_b.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной. Распределение в группе B отлично от нормального")
else:
    print("Принимаем нулевую гипотезу. Распределение в группе B является нормальным")

# p-value группы А 0.43
# p-value группы B 0.12
# Принимаем нулевую гипотезу. Распределение в группе А является нормальным
# Принимаем нулевую гипотезу. Распределение в группе B является нормальным
```

Итак, распределение в обеих группах нормальное, а значит, мы можем воспользоваться T-тестом, чтобы сравнить среднее время посещения сайта в группах.

Напомним формулировки статистических гипотез для T-теста:

Нулевая гипотеза (об отсутствии эффекта): среднее время, которое проводят на сайте пользователи из группы А, равно среднему времени, которое проводят на сайте пользователи из группы B:

$H_0 : \mu_a = \mu_b$

Альтернативная гипотеза (о наличии эффекта): среднее время, которое проводят на сайте пользователи из группы А, отличается от среднего времени, которое проводят на сайте пользователи из группы B:

$H_1 : \mu_a \neq \mu_b$

Примечание. Ещё раз обратите внимание на то, что именно альтернативная гипотеза отвечает на бизнес-вопрос «Увеличилось ли время, которое пользователь проводит на сайте, после внедрения варианта B?».

Применим двухвыборочный T-тест для сравнения средних — функцию ttest_ind() из модуля scipy.stats.

```
from scipy.stats import ttest_ind
```

Функция принимает на вход следующие параметры:

- `a` и `b` — выборки, соответствующие каждой из групп.
- `alternative` — вид альтернативной гипотезы (`'two-sided'` — двусторонняя, `'less'` — левосторонняя, `'greater'` — правосторонняя). По умолчанию используется двусторонняя гипотеза.

```
alpha = 0.05 #уровень значимости
# вычисляем результат T-теста для выборок
results = ttest_ind(
    a=time_data['time(A)'],
    b=time_data['time(B)'],
    alternative='two-sided'
)
print('p-value:', round(results.pvalue, 2))

# сравниваем полученное p-value с уровнем значимости
if results.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value: 0.0
# Отвергаем нулевую гипотезу в пользу альтернативной
```

Итак, в результате проведения T-теста мы получили p-value = 0, что меньше, чем уровень значимости $\alpha = 0.05$. Это значит, что мы отвергаем нулевую гипотезу в пользу альтернативной.

Что это значит для нас и для бизнеса?

Так как истинные значения конверсий в группах не равны (с уровнем значимости 0.05), то для нас это значит, что альтернативная гипотеза подтверждается статистически.

Переводим на язык бизнеса: существует разница между вариантами дизайна сайта **по метрике времени, которое пользователь проводит на сайте**. На основе данных мы видим, что тёмная тема более привлекательна для посетителей, чем светлая.

Обратите внимание: так как при двусторонней гипотезе p-value = 0, не нужно проверять одностороннюю гипотезу о том, что среднее время, которое проводят на сайте пользователи из группы А, меньше, чем среднее время для пользователей из группы B. Мы можем сделать итоговый вывод об этом только на основе данных, ведь мы доказали статистическую значимость разницы в метриках. Однако при желании вы можете проверить одностороннюю гипотезу самостоятельно — для этого в функции установите параметр `alternative='less'`.

НЕПАРАМЕТРИЧЕСКИЕ ТЕСТЫ

Давайте рассмотрим ещё один пример.

Мы проводим A/B-тест, чтобы оценить, влияет ли наличие рекомендательной системы сопутствующих товаров на средний чек покупки. Гипотеза заключается в том, что рекомендательная система мотивирует пользователей, помимо основных товаров, добавлять в покупку сопутствующие, тем самым повышая средний чек. Уровень значимости равен 0.05.

Результаты тестирования представлены в этом файле.

```
check_data = pd.read_excel('data/check_data.xlsx')
check_data.head()
```

	date	mean_check_a	mean_check_b
0	2020-11-06	68701.239649	88665.123322
1	2020-11-07	26136.309375	46230.394893
2	2020-11-08	119470.770656	139356.451490
3	2020-11-09	71110.880188	90867.073191
4	2020-11-10	85896.024022	105977.473589

В данных содержится информация о значениях ежедневного среднего чека в группах А и B. Период проведения A/B-тестирования — 18 дней.

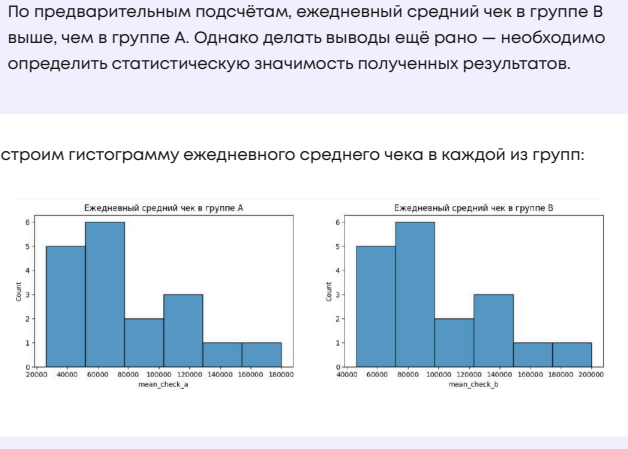
Сначала сравним среднее время в группах на глаз. Рассчитаем среднее и медианное время в каждой группе:

```
check_data[['mean_check_a', 'mean_check_b']].agg(['mean', 'median'])
```

	mean_check_a	mean_check_b
mean	76684.763071	96658.897304
median	62883.707793	82854.201467

По предварительным подсчётам, ежедневный средний чек в группе B выше, чем в группе А. Однако делать выводы ещё рано — необходимо определить статистическую значимость полученных результатов.

Построим гистограмму ежедневного среднего чека в каждой из групп:



Распределения не являются нормальными. Можно проверить это с помощью теста Шапиро — Уилка, однако мы опустим этот шаг (при желании вы можете сделать это самостоятельно). Формы распределений больше напоминают логнормальные: большинство наблюдений сосредоточено около нуля, и частота постепенно падает.

Итак, поскольку данные не распределены нормально, для сравнения истинных средних чеков T-тест уже не подойдёт. Здесь нам придётся обратиться к непараметрическим тестам, например, к критерию Манна — Уитни (U-тест), критерию Колмогорова — Смирнова и другим. Напомним, что эта категория тестов применяется в случаях, когда нет информации о распределении.

Примечание о критерии Манна — Уитни

Постоянно ведутся споры о том, целесообразно ли при A/B-тестировании применять критерий Манна — Уитни, чтобы проверить, является ли статистически значимой разница в метриках между группами А и B.

Часто в учебных материалах для начинающих дата-сайентистов и аналитиков говорится, что критерий Манна — Уитни — это непараметрический аналог T-теста, и вместо сравнения средних он определяет разницу между медианами. Однако это не совсем так, почему — отлично показано в статье Дмитрия Лунина, аналитика из «Авито». Мы настоятельно рекомендуем вам ознакомиться с ней, если вы хотите лучше понять тонкости применения статистических тестов при обработке результатов A/B-тестирования.

Основные идеи этой статьи:

- Критерий Манна — Уитни **не проверяет равенство средних или медиан** в группах. Он позволяет убедиться, что выборки из контрольной и тестовой групп соответствуют одному и тому же распределению, то есть **проверяет наличие (отсутствие) сдвига между распределениями**. Например, если тестовая выборка имеет распределение $F(u)$, а тестовая — $G(u)$, то тест будет проверять нулевую гипотезу $H_0 : F(u) = G(u)$ против альтернативной $H_1 : F(u) \neq G(u)$.
- Пытаясь использовать критерий Манна — Уитни для установления равенства средних/медиан при A/B-тестировании, вы повышаете вероятность возникновения ошибки I рода более чем в два раза.
- Критерий Манна — Уитни чаще, чем T-тест, завышает вероятность наличия эффекта в данных, что может привести к искажению результатов A/B-тестирования.
- Применение логарифмирования или других трансформаций к целевой метрике не поможет уменьшить ошибку U-теста.
- Для реальных задач A/B-тестирования критерий Манна — Уитни стоит использовать с осторожностью, подкрепляя его результаты другими статистическими тестами.

В учебных целях мы будем использовать U-критерий Манна — Уитни для случаев, когда метрики не распределены нормально. Однако при этом будем помнить, что результатам этого критерия при p-value, близким к уровню значимости, доверять не стоит.

Давайте воспользуемся тестом Манна — Уитни, чтобы проверить, статистически значима ли разница ежедневных средних чеков для двух вариантов рекомендательной системы.

Для критерия Манна — Уитни гипотезы будут выглядеть следующим образом:

Нулевая гипотеза (об отсутствии эффекта): распределение, лежащее в основе среднего чека в группе А, идентично распределению, лежащему в основе среднего чека в группе B:

$H_0 : F_a(u) = F_b(u)$

Альтернативная гипотеза (о наличии эффекта): распределение, лежащее в основе среднего чека в группе А, отлично от распределения, лежащего в основе среднего чека в группе B:

$H_1 : F_a(u) \neq F_b(u)$

Напомним, что U-тест Манна — Уитни реализован в функции mannwhitneyu() из модуля scipy.stats.

```
from scipy.stats import mannwhitneyu
```

Функция принимает на вход следующие параметры:

- `x` и `y` — выборки, соответствующие каждой из групп.
- `alternative` — вид альтернативной гипотезы (`'two-sided'` — двусторонняя, `'less'` — левосторонняя, `'greater'` — правосторонняя). По умолчанию используется двусторонняя гипотеза.

```
alpha = 0.05 # уровень значимости

# вычисляем результат теста Манна — Уитни для выборок
results = mannwhitneyu(
    x=check_data['mean_check_a'],
    y=check_data['mean_check_b'],
    alternative='two-sided'
)
print('p-value:', round(results.pvalue, 2))

# сравниваем полученное p-value с уровнем значимости
if results.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value: 0.06
# У нас нет оснований отвергнуть нулевую гипотезу
```

Итак, согласно результатам тестирования, p-value = 0.06, что больше, чем уровень значимости $\alpha = 0.05$. Значит, у нас нет оснований отвергнуть нулевую гипотезу.

Что это значит для нас и для бизнеса?

Средние чеки в группах А и B статистически равны между собой. Отсюда можно сделать бизнес-вывод, что рекомендательная система не влияет на мотивацию пользователей добавлять сопутствующие товары в корзину.

Однако нас должно насторожить, что p-value очень близко к уровню значимости. Поэтому давайте дополнительно проведём односторонний U-тест со следующими гипотезами:

Нулевая гипотеза (об отсутствии эффекта): распределение, лежащее в основе среднего чека в группе А, статистически больше либо равно распределению, лежащему в основе среднего чека в группе B:

$H_0 : F_a(u) \geq F_b(u)$

Левосторонняя альтернативная гипотеза (о наличии эффекта): распределение, лежащее в основе среднего чека в группе А, статистически меньше распределения, лежащего в основе среднего чека в группе B:

$H_1 : F_a(u) < F_b(u)$

Код будет выглядеть следующим образом:

```
alpha = 0.05 # уровень значимости

# вычисляем результат теста Манна — Уитни для выборок
results = mannwhitneyu(
    x=check_data['mean_check_a'],
    y=check_data['mean_check_b'],
    alternative='less' # изменили альтернативную гипотезу
)
print('p-value:', round(results.pvalue, 2))

# сравниваем полученное p-value с уровнем значимости
if results.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value: 0.03
# Отвергаем нулевую гипотезу в пользу альтернативной
```

Получаем значение p-value = 0.03, что меньше, чем уровень значимости $\alpha = 0.05$. Значит, мы должны отвергнуть нулевую гипотезу в пользу альтернативной. Таким образом, односторонний тест говорит нам, что разница в средних чеках всё-таки статистически значима. То есть рекомендательная система всё-таки мотивирует пользователей добавлять сопутствующие товары в корзину и повышает средний чек, а значит, её можно внедрять для всех пользователей сайта.

Итак, мы получили два противоречивых вывода: двусторонний тест говорит, что эффекта нет, а односторонний — что есть. Какой вывод правильный?

Ответ

Итак, мы разобрались, как применять статистические тесты для анализа результатов A/B-тестирования, рассмотрели примеры использования разных статистических тестов и обсудили их тонкости. В следующем уроке мы узнаем, как для этих же целей использовать ещё один статистический инструмент, а именно **доверительные интервалы**.

Стоит отметить, что для всех приведённых ранее примеров мы принимали решение, исходя только из одной бизнес-метрики, так как другими мы и не обладали. В финальном задании модуля мы проведём комплексный анализ результатов A/B-тестирования по нескольким метрикам. А пока предлагаем вам решить несколько задач.

Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 5.1

0 points possible (ungraded)

Проводится *A/B*-тестирование нового варианта расстановки товаров в магазине. Предполагается, что число покупок определённого товара вырастет, если расположить его рядом с кассой. Какой статистический тест вы будете использовать для сравнения вариантов расстановки по конверсии покупки этого товара?

● *Z*-тест для пропорций

○ *T*-тест

○ Тест Шапиро-Уилка

○ *ANOVA*-тест

✓

Ответ

Верно:

Конверсия является долевым показателем. Для проверки гипотезы о равенстве долей (пропорций) используется *Z*-тест для пропорций.

Show answer

Отправить

✓ Верных

Для решения следующих заданий используйте наборы данных sample_a и sample_b из предыдущего урока (перед заданием 4.6), где приведены результаты A/B-тестирования двух вариантов дизайна одного и того же сайта.

```
sample_a = pd.read_csv('data/ab_test-redesign_sample_a.csv')
sample_b = pd.read_csv('data/ab_test-redesign_sample_b.csv')

# преобразуем тип столбца date к типу datetime
sample_a['date'] = pd.to_datetime(sample_a['date'])
sample_b['date'] = pd.to_datetime(sample_b['date'])
```

Задание 5.2

0 points possible (ungraded)

С помощью *Z*-теста для пропорций проанализируйте результаты проведённого *A/B*-тестирования и определите, какой из вариантов дизайна (*A* или *B*) более предпочтителен по метрике конверсии. В качестве уровня значимости возьмите $\alpha = 0.05$. Используйте функцию proportions_ztest() из модуля *statsmodels.stats.proportion*.

Какой вывод можно сделать на основании результатов тестирования?

○ Вариант *A* более предпочтителен, чем вариант *B*, по метрике конверсии.

○ Вариант *A* менее предпочтителен, чем вариант *B*, по метрике конверсии.

○ Варианты *A* и *B* равнозначны по метрике конверсии.

○ Нельзя сделать однозначный вывод.

Hint

Отправить

Давайте вычислим ежедневный средний чек — это ежедневная сумма заказов, делённая на ежедневную сумму транзакций:

```
daily_sample_a = sample_a.groupby('date')[['transactions', 'revenue']].sum()
daily_sample_a['mean_check'] =
daily_sample_a['revenue']/daily_sample_a['transactions']
daily_sample_b = sample_b.groupby('date')[['transactions', 'revenue']].sum()
daily_sample_b['mean_check'] =
daily_sample_b['revenue']/daily_sample_b['transactions']
```

Задание 5.3

0 points possible (ungraded)

Проверьте гипотезу о нормальности распределений ежедневного среднего чека в контрольной и тестовой группах с помощью критерия Шапиро — Уилка. В качестве уровня значимости возьмите $\alpha = 0.05$.

Каковы результаты тестирования?

○ Распределения в обеих группах являются нормальными.

○ Распределения в одной или обеих группах отличны от нормального.

Отправить

Задание 5.4

0 points possible (ungraded)

С помощью критерия Манна — Уитни проанализируйте результаты проведённого *A/B*-тестирования и определите, какой из вариантов дизайна (*A* или *B*) более предпочтителен по метрике ежедневного среднего чека. В качестве уровня значимости возьмите $\alpha = 0.05$. Используйте функцию mannwhitneyu() из модуля *scipy.stats*.

Каков результат тестирования?

○ Вариант *A* более предпочтителен, чем вариант *B*, по метрике ежедневного среднего чека.

○ Варианты *A* и *B* равнозначны по метрике ежедневного среднего чека.

○ Вариант *A* менее предпочтителен, чем вариант *B*, по метрике ежедневного среднего чека.

○ Нельзя сделать однозначный вывод.

‹ Предыдущий урок Следующий урок ›

6/9 Анализ результатов A/B-тестирования: доверительные интервалы

текущий урок

Ещё один важный статистический метод, который нам необходимо изучить — это **доверительные интервалы**. Давайте ненадолго отвлечёмся от нашей задачи про конверсию на сайте и рассмотрим небольшой, но очень важный для понимания кусочек статистической теории.

Итак, мы с вами знаем, что на основе выборки (части наблюдений) мы всегда можем измерить следующие показатели:

- **выборочное среднее** (центральная мера распределения);
- **выборочное стандартное отклонение** (мера разброса);
- **выборочную пропорцию** (доля объектов из выборки, выбранных по условию, например доля пользователей, совершивших покупки).

Однако эти оценки не являются нашим основным предметом интереса. На самом деле нас интересуют параметры генеральной совокупности:

- истинное среднее — **математическое ожидание** (μ);
- **истинное стандартное отклонение** (σ);
- **истинная пропорция** (p).

Если вы взяли выборку клиентов и выяснили их предпочтения, вам важно не то, что вы узнали предпочтения именно этих клиентов — вам важно, что теперь вы знаете предпочтения всех ваших клиентов (**генеральная совокупность**).

Важно помнить о разнице между выборочными статистиками и параметрами генеральной совокупности.

> **Выборочная статистика** является случайной величиной, ведь она рассчитана на случайной выборке. Если мы возьмём одну выборку и посчитаем значение среднего, мы получим одно значение; возьмём другую — получим другое значение.

> А вот параметр **генеральной совокупности** — это константа, конкретное число, которое, как правило, нам неизвестно и никогда известным не станет. Однако его можно приблизительно оценить!

Существует два способа оценки параметров генеральной совокупности:

- Точечные оценки
- Интервальные оценки

С **точечными оценками** на самом деле мы уже знакомы и не раз их вычисляли — это просто выборочные статистики. Например, мы собрали данные о росте прохожих людей и посчитали среднее, стандартное отклонение и так далее. Почему они точечные? Потому что они дают ответ точечно, то есть в виде одного числа.

С одной стороны, это удобно: взяли выборку, посчитали выборочное среднее и получили ответ, к примеру «средний рост в этой выборке равен 178 см».

Однако есть у такого подхода и существенный недостаток: полученное значение практически никогда не равно истинному.

> **Интервальные оценки** — это ещё один способ оценки параметров генеральной совокупности, при использовании которого ответ даётся не в виде одного числа, а в виде интервала.

Тогда мы скажем так: «Истинный средний рост лежит в интервале от 170 до 179 см с такой-то долей уверенности». Скорее всего, истинный параметр будет находиться внутри этого интервала.

Поиск интервальной оценки заключается в построении доверительного интервала.

> **Доверительный интервал** — интервал, который с заданной надёжностью покрывает значение неизвестного параметра.

Например, если вам говорят, что построен 90 %-ый доверительный интервал для среднего роста и имеет вид: (170, 179), то это означает, что с **надёжностью** в 90 % истинный средний рост (его математическое ожидание) лежит в пределах от 170 см до 179 см. То есть с вероятностью в 90 % ожидается (в среднем), что любой человек из рассматриваемой генеральной совокупности имеет рост от 170 до 179 см.

Доверительные интервалы бывают **двусторонними**, **левосторонними** и **правосторонними** (как гипотезы). Мы с вами будем рассматривать только построение двусторонних доверительных интервалов, так как они являются наиболее популярными.

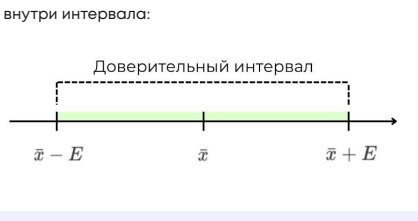
Любой двусторонний доверительный интервал обладает следующей структурой:

Параметр = Выборочная оценка ± Предел погрешности

Выборочной оценкой может быть среднее, стандартное отклонение, пропорция, медиана и так далее. А о том, как считается предел погрешности, и от чего он зависит, мы поговорим позже.

В соответствии с этой формулой можно нарисовать доверительный интервал. Обозначим выборочную оценку как $\bar{x}$, а предел погрешности как E.

Тогда истинное значение параметра (с заданной степенью надёжности) лежит где-то внутри интервала:



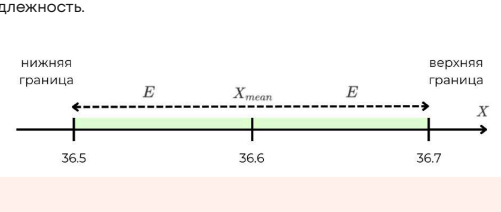
На оси возможных значений параметра откладывается точка $\bar{x}$. Влево и вправо от $\bar{x}$ откладывается погрешность E. Обратите внимание, что выборочная оценка всегда находится в центре двустороннего доверительного интервала. В результате построения интервала мы получаем зону, в которой предположительно лежит истинное значение параметра.

Пример. Мы замеряем температуру тела человека несколько раз. Сделав замеры, мы рассчитали среднюю температуру X_{mean}, она оказалась 36.6 C⁰, а также рассчитали погрешность E=0.1 C⁰.

Тогда границы доверительного интервала для истинного среднего температуры будут следующими:

- левая (нижняя) граница: $X_{mean} - E = 36.6C^0 - 0.1C^0 = 36.5C^0$;
- правая (верхняя) граница: $X_{mean} + E = 36.6C^0 + 0.1C^0 = 36.7C^0$.

То есть истинное среднее температуры лежит в диапазоне от 36.5 C⁰ до 36.7 C⁰. Математически это пишется так: $\in (36.5, 36.7)$. Символ $\in$ обозначает принадлежность.



Эти задания для самостоятельной работы, за них не предусмотрены баллы. Однако они помогут вам закрепить новый материал.

Задание 6.1

0 points possible (ungraded)

Чем отличается точечная оценка от интервальной?

○ Точечная оценка надёжнее.

● Точечная оценка позволяет оценить неизвестный параметр по выборке в виде числа, а интервальная оценка задаёт интервал, в который с заданной надёжностью попадает неизвестный параметр.

○ Точечная оценка позволяет оценить неизвестный параметр с заданной надёжностью по выборке в виде числа, а интервальная оценка задаёт произвольный интервал, в который может попасть неизвестный параметр.

○ Точечные и интервальные оценки — это синонимы.

✓

Show answer

Отправить

✓ Верных

Задание 6.2

0 points possible (ungraded)

Что всегда является центром доверительного интервала?

○ предел погрешности

○ ноль

● точечная (выборочная) оценка

○ произвольное, заданное исследователем число

✓

Show answer

Отправить

✓ Верных

Задание 6.3

0 points possible (ungraded)

В результате проведения SMM-эксперимента было выяснено, что выборочное стандартное отклонение количества лайков в профиле Ольги Бузовой составило $X_{std} = 50325$ шт. При этом вычисленная погрешность составила $E = 10524$ шт. Что можно сказать по результатам такого исследования?

● Истинное стандартное отклонение количества лайков в профиле лежит в интервале от 39 801 до 60 849.

○ Истинное среднее количества лайков в профиле лежит в интервале от 39 801 до 60 849.

○ Истинное стандартное отклонение количества лайков в профиле лежит в от 50 325 до 60 849.

○ По приведённым результатам нельзя ничего сказать.

✓

Show answer

Отправить

✓ Верных

ДОВЕРИТЕЛЬНЫЙ ИНТЕРВАЛ ДЛЯ СРЕДНЕГО ПРИ ИЗВЕСТНОМ ИСТИННОМ СТАНДАРТНОМ ОТКЛОНЕНИИ

Самым частым параметром, который хотелось бы оценить, является истинное среднее (**математическое ожидание**). И начнём мы сразу с задачи:

Была опрошена случайная выборка из 36 жителей региона об их затратах на продукты питания за последний месяц. Выборочное среднее оказалось равным 16 100 рублей.Допустим, откуда-то известно, что истинное стандартное отклонение расходов равно 12 000 рублей.Нам нужно построить доверительный интервал для истинных средних расходов жителя данного региона на продукты питания в месяц. Интервал должен определять истинное значение с надёжностью в 95 %.

Страшно и непонятно? Давайте разбираться.

По условию задачи у нас есть выборка размером $n = 36$ человек. Далее, просуммировав затраты жителей региона и поделив на их количество, было получено выборочное среднее $X_{mean} = 16100$ рублей. Ещё нам дано значение истинного (НЕ выборочного) стандартного отклонения $\sigma = 12000$ рублей. То есть в среднем разброс затрат составляет 12 000 рублей. Ещё нам сказали, что рассчитанный нами интервал должен быть надёжным с уровнем надёжности $\gamma = 0.95$.

Теперь, когда мы ввели все обозначения, пора нам познакомиться с формулой **доверительного интервала для истинного среднего при известном стандартном отклонении**:

- $\mu = X_{mean} \pm z_{крит} \times \frac{\sigma}{\sqrt{n}}$

Другая запись этой формулы:

- $\mu \in (X_{mean} - z_{крит} \times \frac{\sigma}{\sqrt{n}}; X_{mean} + z_{крит} \times \frac{\sigma}{\sqrt{n}})$

Оба варианта формулы читаются следующим образом: истинное среднее μ лежит в диапазоне от $X_{mean} - z_{крит} \times \frac{\sigma}{\sqrt{n}}$ до $X_{mean} + z_{крит} \times \frac{\sigma}{\sqrt{n}}$.

Примечание. Не будем сильно углубляться в статистику, скажем лишь то, что данная формула выводится из центральной предельной теоремы — одной из основных теорем теории вероятностей.

Если сопоставить полученную формулу со структурой доверительного интервала, о которой мы говорили выше, можно заметить, что за выборочную компоненту отвечает выборочное среднее $\bar{x} = X_{mean}$, а за погрешность отвечает выражение $E = z_{крит} \times \frac{\sigma}{\sqrt{n}}$.

Подставляя значения в эту формулу и подобрав правильное значение z-критического, мы можем сказать, что истинное среднее лежит в интервале с определённой надёжностью.

И всё в этой формуле было бы прекрасно, если бы мы знали, что такое $z_{крит}$ и как его вычислять. Давайте узнаем!

Откуда берётся z-критическое?

Вообще z в формуле доверительного интервала для среднего — это случайная величина, имеющая нормальное распределение с математическим ожиданием (центром) $\mu = 0$ и стандартным отклонением (разбросом) $\sigma = 1$.

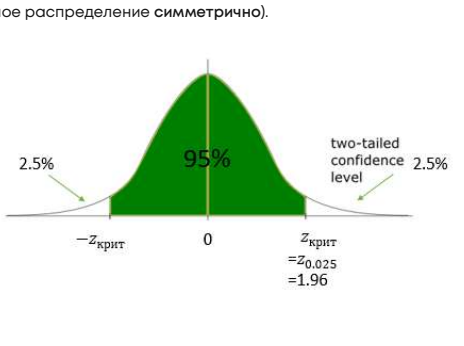
Компонент $z_{крит}$ в доверительном интервале определяется уровнем надёжности. А что вообще такое **уровень надёжности** (его ещё называют **уровнем доверия**)?

А вот что: $\gamma = 1 - \alpha$, где α — уровень значимости или вероятность ошибки I рода. Тогда уровень надёжности — вероятность НЕ допустить ошибку I рода.

Например, мы хотим, чтобы интервал был надёжен на 95 % ($\gamma = 0.95$). Это означает, что мы хотим покрыть 95 % процентов исходов (всех возможных выборочных средних). Вероятность допустить ошибку I рода — не захватить истинное значение в построенный доверительный интервал — будет равно $\alpha = 1 - \gamma = 1 - 0.95 = 0.05$ (5 %).

Вывод. Под уровнем надёжности понимается вероятность того, что истинное значение параметра окажется в построенном интервале. А под уровнем значимости — вероятность того, что построенный доверительный интервал «промахнётся» и не захватит истинное значение параметра.

То есть мы хотим взять такое значение, при котором в интервале от $-z$-критического до z критического лежало бы 95 % всех исходов. В двустороннем интервале эти 95 % мы будем брать как раз **посередине**, чтобы справа и слева осталось 5 %, по 2.5 % с каждой стороны (помним, что нормальное распределение **симметрично**).



> Таким образом, $z_{крит}$ — это значение, которое отсекает критическую область нормального распределения.

Это константа, которая берётся из таблицы, и её значение мы задаём, когда указываем надёжность, с которой хотим построить интервал:

$z_{крит} = z_{\frac{1-\gamma}{2}} = z_{\frac{\alpha}{2}}$,

- где γ — заданный уровень надёжности (доверия);
- $\alpha = 1 - \gamma$ — уровень значимости.

Значение z-критического мы можем найти при помощи специальных таблиц, библиотек Python или калькулятора. Он поможет нам выяснить, что $z_{0.025}$ приблизительно равно 1.96.

Тогда:

$\mu = X_{mean} \pm z_{\frac{\alpha}{2}} \times \frac{\sigma}{\sqrt{n}}$

Теперь ничто не мешает решить нашу задачу! Вычисления будем производить с помощью **Python**.

Задаём необходимые для вычисления переменные:

```
n = 36 # размер выборки
x_mean = 16100 # выборочное среднее
sigma = 12000 # истинное стандартное отклонение
gamma = 0.95 # уровень надёжности
alpha = 1 - gamma # уровень значимости
```

Найти значение z-критического для любого уровня значимости можно при помощи функции `norm.ppf()` из модуля scipy.stats. По специфическим причинам реализации функция возвращает не совсем z-критическое, а его отрицательный модуль, поэтому мы берём его со знаком минус.

```
from scipy.stats import norm
z_crit = -norm.ppf(alpha/2) # z-критическое
print(z_crit)
# 1.959963984540054
```

Итак, теперь у нас есть всё, чтобы построить доверительный интервал. Напишем функцию `z_mean_conf_interval()`, которая будет принимать на вход:

- `mean` — выборочное среднее;
- `sigma` — истинное стандартное отклонение;
- `n` — размер выборки;
- `gamma` — уровень надёжности.

Функция будет вычислять нижнюю и верхнюю границы по формуле и возвращать их в виде кортежа.

Округлим результат до целого и выведем на экран:

```
def z_mean_conf_interval(x_mean, sigma, n, gamma=0.95):
    alpha = 1 - gamma  # уровень значимости
    z_crit = -norm.ppf(alpha/2)  # z-критическое
    eps = z_crit * sigma/(n ** 0.5) # погрешность
    lower_bound = x_mean - eps  # левая (нижняя) граница
    upper_bound = x_mean + eps  # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound

# строим доверительный интервал для среднего при известном СКО
lower_bound, upper_bound = z_mean_conf_interval(x_mean, sigma, n)

# выводим результат
print('Доверительный интервал: {}'.format((round(lower_bound),
round(upper_bound))))

# Доверительный интервал: (12180, 20020)
```

Примечание. Заметьте, что это универсальный код. Если перед вами возникнет другая задача расчёта доверительного интервала для истинного среднего, то вам достаточно будет правильно поменять значения переменных.

Полученный результат интерпретируется следующим образом: мы на 95 % уверены, что истинные расходы жителей на продукты питания лежат в интервале от 12 180 рублей до 20 020 рублей.

Примечание

Чтобы построить доверительный интервал для среднего при известном стандартном отклонении, можно использовать функцию norm.interval() из модуля norm библиотеки scipy. В неё нужно нужно передать следующие параметры:

- `confidence` — уровень надёжности γ;
- `loc` — центр интервала (выборочное среднее);
- `scale` — погрешность (стандартное отклонение, нормированное на $\sqrt{n}$).

Пример вызова функции:

```
# строим доверительный интервал
lower, upper = norm.interval(gamma, loc=x_mean, scale=sigma / n**0.5)
print('Доверительный интервал: {}'.format((round(lower), round(upper))))
# Доверительный интервал: (12180, 20020)
```

Важно! Доверительный интервал не даёт нам 100 % результата. Всегда есть шанс, что мы промахнулись и не захватили истинного значения. Этот шанс как раз равен уровню значимости α.То есть в нашем примере существует 5 %-ый шанс, что истинное среднее расходов вовсе не лежит в интервале от 12 180 до 20 020. Существует вероятность, что математическое ожидание на самом деле равно 5 000 рублей или 25 000 рублей. Но такая вероятность составляет менее 5 %, и это нас устраивает.

К сожалению, нельзя построить доверительный интервал, охватывающий 100 % возможных значений, так как границы такого доверительного интервала — $-\infty$ и $+\infty$. Как мы знаем, это абстрактные математические понятия, неприменимые на практике.

Задание 6.4

0 points possible (ungraded)

Что такое z-критическое?

● значение, которое определяется уровнем надёжности (уровнем значимости) на основе нормального распределения

○ запрещённое значение надёжности интервала, выше которого надёжность нельзя повысить

○ переменная величина, зависящая от параметров выборки, показывает разброс параметра

○ вероятность «промахнуться» доверительным интервалом

✓

Show answer

Отправить

✓ Верных

Задание 6.5

0 points possible (ungraded)

Аккумуляторный завод производит батарейки, срок службы которых имеет истинное стандартное отклонение в 2.4 месяца. Средний срок службы батареек в случайной выборке из 64 штук составил 12.35 месяца. Необходимо построить 90 %-ый доверительный интервал для истинного среднего срока службы производимой батарейки и ответить на вопросы заданий ниже.

Чему равно значение z-критического для такого интервала? Введите число с двумя знаками после разделительного знака.

Теперь постройте 90 %-ный доверительный интервал. Используйте критическое значение, округлённое до двух знаков. Ответ введите в виде нижней и верхней границы интервала, округляя результат до двух знаков после разделительной точки.

Нижняя граница:

Верхняя граница:

? **Подсказка (1 из 2):** z-критическое можно определить с помощью функции `norm.ppf(alpha/2)` , умножив результат на -1. *alpha* — уровень значимости `(1-gamma = 1-0.9 = 0.1)` . Next Hint

Hint

Отправить

ДОВЕРИТЕЛЬНЫЙ ИНТЕРВАЛ ДЛЯ СРЕДНЕГО ПРИ НЕИЗВЕСТНОМ СТАНДАРТНОМ ОТКЛОНЕНИИ

Внимательный студент заметил, что в формуле доверительного интервала для истинного среднего мы использовали значение истинного стандартного отклонения σ.

Для правомерного использования формулы истинное стандартное отклонение выборки должно быть известно. Но, как правило, нам неизвестно это значение (если только оно не свалилось на нас озарением), ведь для этого нам нужно знать все значения генеральной совокупности. А если они у нас есть, то нам и доверительные интервалы не нужны — зная всю генеральную совокупность, всегда можно найти истинные параметры.

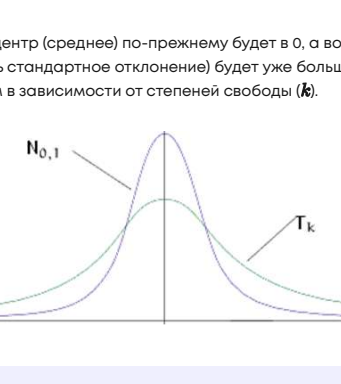
В реальной жизни нам приходится заменять значение σ значением **выборочного стандартного отклонения**, которое мы можем посчитать на основании выборки.

Когда мы заменяем константу на случайную величину, точность всей конструкции явно становится меньше. Вероятно, итоговая величина будет похожа на нормальное распределение, но будет обладать бо́льшим разбросом.

Такая случайная величина хорошо описывается распределением Стьюдента, или **T-распределением.**

> **Распределение Стьюдента** — это целое семейство родственников стандартного нормального распределения.

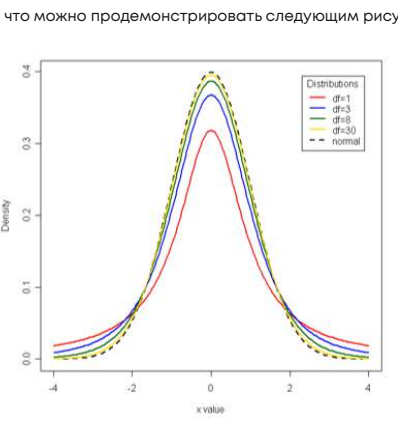
Действительно, центр (среднее) по-прежнему будет в 0, а вот разброс значений (то есть стандартное отклонение) будет уже больше. Этот разброс будет различным в зависимости от степеней свободы (k).



На рисунке **фиолетовым** цветом изображено нормальное распределение с параметрами 0 и 1, а **зелёным** цветом — распределение Стьюдента. Отчётливо видно, что распределение Стьюдента несколько «шире» — обладает большим разбросом.

Степени свободы обозначаются **d.f.** (от англ. degrees of freedom). Они и говорят нам о том, насколько велик разброс в значениях распределения Стьюдента. Этот параметр вычисляется как $k = n - 1$, где n — размер выборки.

Чем больше k, тем меньше разброс распределения и тем ближе оно к нормальному, что можно продемонстрировать следующим рисунком:



Тогда в формуле для доверительного интервала случайная нормальная величина Z заменится на случайную величину T, а истинное стандартное отклонение σ заменится на выборочное — X_{std}:

$\mu = X_{mean} \pm t_{крит} \times \frac{X_{std}}{\sqrt{n}}$

Или другая запись этой формулы:

$\mu \in (X_{mean} - t_{крит} \times \frac{X_{std}}{\sqrt{n}}; X_{mean} + t_{крит} \times \frac{X_{std}}{\sqrt{n}})$

Значения $t_{крит}$ вычисляются аналогично $z_{крит}$: задаётся уровень надёжности (или уровень значимости α), и на его основе по таблицам или калькуляторам для распределения Стьюдента вычисляется число. Отличие состоит в том, что для вычисления $t_{крит}$ нужно задать ещё и число степеней свободы $k = n - 1$:

$t_{крит}(k) = t_{\frac{(1-\gamma)}{2}}(k) = t_{\frac{\alpha}{2}}(k)$

А теперь давайте решим задачу!

Управляющий супермаркета хочет оценить, сколько денег покупатели тратят за один поход в супермаркет в среднем. На основании ранее собранных данных известно, что расходы одного покупателя распределены приблизительно нормально. Управляющий взял случайную выборку из 15 покупателей и выяснил, что их средние расходы равны 2 000 рублей, а выборочное стандартное отклонение равно 400 рублей.

Для оценки управляющий хочет построить доверительный интервал с надёжностью в 95 %.

Давайте поможем управляющему, решив несколько задачек с помощью Python.

Итак, зададим необходимые для решения задачи переменные:

```
n = 15 # размер выборки
k = n - 1 # число степеней свободы
x_mean = 2000 # выборочное среднее
x_std = 400 # выборочное стандартное отклонение
gamma = 0.95 # уровень надёжности
alpha = 1 - gamma # уровень значимости
```

Найти значение t-критического для любого уровня значимости можно, используя функцию `t.ppf()` из модуля scipy.stats. Для этого в функцию передаётся интересующий уровень значимости ($\frac{\alpha}{2}$) и число степеней свободы ($k = n - 1$). Аналогично z-критическому функция возвращает отрицательный модуль t-критического, поэтому мы берём его со знаком минус:

```
from scipy.stats import t
t_crit = -t.ppf(alpha/2, k) # t-критическое
print(t_crit)
# 2.1447866879169273
```

Примечание. Интересен тот факт, что значение t-критического при том же уровне надёжности чуть выше, чем значение z-критического, например $z_{0.025} = 1.96$, а вот $t_{0.025} = 2.1402$. Благодаря этому доверительный интервал при неизвестном стандартном отклонении гарантированно будет шире, чем при известном, если $\sigma = X_{std}$.

Теперь составим доверительный интервал для истинного среднего. Давайте напишем функцию для вычисления доверительного интервала для среднего — `t_mean_conf_interval()`, модифицировав функцию для z-интервала согласно приведённым выше рассуждениям. Функция всё так же будет возвращать левую и нижнюю границу по формуле.

Округлим результат и выведем на экран:

```
def t_mean_conf_interval(x_mean, x_std, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    t_crit = -t.ppf(alpha/2, k) # t-критическое
    eps = t_crit * x_std/(n ** 0.5) # погрешность
    lower_bound = x_mean - eps # левая (нижняя) граница
    upper_bound = x_mean + eps # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound

# строим доверительный интервал для среднего при неизвестном СКО
lower_bound, upper_bound = t_mean_conf_interval(x_mean, x_std, n)

# выводим результат
print('Доверительный интервал: {}'.format((round(lower_bound),
round(upper_bound))))

# Доверительный интервал: (1778, 2222)
```

Итак, мы на 95 % уверены, что истинные траты покупателей в нашем магазине лежат в интервале от 1 778 до 2 222 рублей.

Примечание

Чтобы построить доверительный интервал для среднего при известном стандартном отклонении, можно использовать функцию t.interval() из модуля t библиотеки scipy. В неё нужно передать следующие параметры:

- `confidence` — уровень надёжности γ;
- `df` — количество степеней свободы для распределения Стьюдента;
- `loc` — центр интервала (выборочное среднее);
- `scale` — погрешность (стандартное отклонение, нормированное на $\sqrt{n}$.

Пример вызова функции:

```
# строим доверительный интервал для среднего при неизвестном СКО
lower, upper = t.interval(gamma, df=k, loc=x_mean, scale=x_std/(n ** 0.5))
print('Доверительный интервал: {}'.format((round(lower), round(upper))))
# Доверительный интервал: (1778, 2222)
```

Важно отметить, что при достаточной выборке (примерно более 30 наблюдений) распределение Стьюдента очень похоже на стандартное нормальное распределение. В этом случае можно использовать первую формулу (z-критическое) даже при неизвестном стандартном отклонении. Однако используя вторую формулу при оценке среднего, вы не ошибётесь.

Ещё один важный факт состоит в том, что обе формулы работают только при соблюдении двух условий:

- Выборка должна быть случайной.
- Выборка должна быть достаточно велика. Но есть исключение: если выборка маленькая, но сама величина распределена нормально, то выборочное среднее тоже будет распределено нормально. В таком случае мы можем пользоваться формулой доверительного интервала.

Задание 6.6

0 points possible (ungraded)

Чиновника интересует сумма подоходного налога, выплачиваемого домохозяйствами за год. На случайной выборке из 25 домохозяйств был посчитан средний размер выплаченного налога, и он составил $ 3 540. Также известно выборочное отклонение величины подоходного налога, равное $ 1 150.

Необходимо найти 90 %-ый доверительный интервал для истинного среднего значения налоговых выплат за год. Используйте формулу с t-критическим.

Укажите границы построенного доверительного интервала для истинного среднего. Ответ округлите до целого.

Нижняя граница:

Верхняя граница:

Отправить

ДОВЕРИТЕЛЬНЫЙ ИНТЕРВАЛ ДЛЯ ПРОПОРЦИИ

Настало время вернуться к нашей задаче об анализе конверсии в двух группах A/B-тестирования.

Давайте ещё раз вспомним, что такое конверсия.

> **Конверсия** — это доля пользователей, совершивших целевое действие, по отношению к общей массе пользователей.

Ранее мы уже выяснили, что признак того, осуществлена конверсия пользователем или нет — это случайная величина, имеющая распределение Бернулли. Такая случайная величина имеет два возможных исхода:

- успех (1) — конверсия осуществлена (пользователь совершил целевое действие);
- неуспех (0) — пользователь не совершил целевое действие.

То есть конверсия — это вероятность того, что пользователь совершит целевое действие (вероятность успеха традиционно обозначается как p):

$p = conv = \frac{N^+}{N}$

- N^+ — количество успехов (количество пользователей, осуществивших конверсию);
- N — общее количество исходов (количество пользователей);
- $conv$ — конверсия.

Примечание. При расчётах конверсии обычно долю ещё умножают на 100 %.

В контексте задачи A/B-тестирования нам бы хотелось знать истинную конверсию (а не ту, которая подсчитана по выборке) в контрольной и тестовой группе, а также оценить истинную разницу между конверсиями.

Если подумать, то на самом деле мы решаем задачу оценки истинной пропорции генеральной совокупности по выборке. А значит, нужно научиться строить **доверительный интервал для пропорции**.

Давайте вспомним, что истинное среднее (**математическое ожидание**) случайной величины, распределённой по закону Бернулли, равно $\mu = p$, а **истинное стандартное отклонение** $\sigma = \sqrt{p(1-p)}$.

Истинную вероятность успеха p, мы, конечно же, не знаем, так как нам неизвестна вся генеральная совокупность (всех возможных пользователей). Вместо неё мы используем **выборочную пропорцию** (обозначим её как X_p) — долю совершивших конверсию пользователей, то есть **выборочную конверсию**.

Заметим, что истинная пропорция p полностью совпадает с истинным средним μ. Если немного поколдовать с формулой доверительного интервала для истинного среднего, то можно вывести **формулу доверительного интервала для истинной пропорции** (истинной конверсии):

$p = \mu = X_p \pm z_{крит} \times \sqrt{\frac{X_p(1-X_p)}{n}}$

Другая запись этой формулы:

$\mu = p \in (X_p - z_{крит} \times \sqrt{\frac{X_p(1-X_p)}{n}}; X_p + z_{крит} \times \sqrt{\frac{X_p(1-X_p)}{n}})$

Данная формула позволяет построить доверительный интервал для истинной конверсии в контрольной (A) и тестовой (B) группах.

Если сравнить со структурой доверительного интервала, введённой ранее, то можно заметить, что за выборочную оценку отвечает **выборочная пропорция** $\bar{x} = X_p$, а за погрешность интервала отвечает выражение $E = z_{крит} \times \sqrt{\frac{X_p(1-X_p)}{n}}$.

Примечание. Если размер выборки < 30, то вместо $z_{крит}$ используется $t_{крит}$. Но в задачах A/B-тестирования, как правило, такой размер выборки не используется. Чаще всего мы имеем дело с сотнями или тысячами наблюдений.

Пора приступать к реализации на Python. Будем работать с данными о посетителях веб-сайта из прошлого урока:

```
ab_data = pd.read_csv('data/ab_data.csv')
ab_data.head()
```

	user_id	timestamp	group	converted
0	851104	2017-01-21	A	0
1	804228	2017-01-12	A	0
2	661590	2017-01-11	B	0
3	853541	2017-01-08	B	0
4	864975	2017-01-21	A	1

Для удобства создадим отдельные таблицы для каждой из групп пользователей:

```
# фильтруем данные группы А
a_data = ab_data[ab_data['group'] == 'A']
# фильтруем данные группы B
b_data = ab_data[ab_data['group'] == 'B']
```

Давайте на этот раз оформим расчёт в виде функции: объявим функцию `proportion_conf_interval()`. У данной функции будет три аргумента: `x_p` — выборочная пропорция, `n` — размер выборки и `gamma` — уровень надёжности (по умолчанию он равен 0.95). Функция будет возвращать кортеж из вычисленных границ доверительного интервала, умноженных на 100 % и округлённых до второго знака после запятой.

```
def proportion_conf_interval(x_p, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    z_crit = -norm.ppf(alpha/2) # z-критическое
    eps = z_crit * (x_p * (1 - x_p) / n) ** 0.5 # погрешность
    lower_bound = x_p - eps # левая (нижняя) граница
    upper_bound = x_p + eps # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound
```

Теперь применим нашу функцию к данным группы А и группы B, результат умножим на 100 % и округлим до сотых:

```
# строим доверительный интервал для конверсии в группе А
lower_bound_a, upper_bound_a  = proportion_conf_interval(
    x_p=a_data['converted'].mean(), # выборочная пропорция
    n=a_data['user_id'].count() # размер выборки
)

# строим доверительный интервал для конверсии в группе B
lower_bound_b, upper_bound_b  = proportion_conf_interval(
    x_p=b_data['converted'].mean(), # выборочная пропорция
    n=b_data['user_id'].count() # размер выборки
)

# выводим результат
print('Доверительный интервал для конверсии в группе А:
{}'.format((round(lower_bound_a * 100, 2), round(upper_bound_a * 100, 2))))
print('Доверительный интервал для конверсии в группе B:
{}'.format((round(lower_bound_b * 100, 2), round(upper_bound_b * 100, 2))))

# Доверительный интервал для конверсии группы А: (11.86, 12.19)
# Доверительный интервал для конверсии группы B: (11.7, 12.03)
```

Итак, можно сделать вывод, что с надёжностью в 95 % конверсия группы А лежит в интервале от 11.86 % до 12.19 %, а конверсия группы B лежит в интервале от 11.7 % до 12.03 %.

Примечание

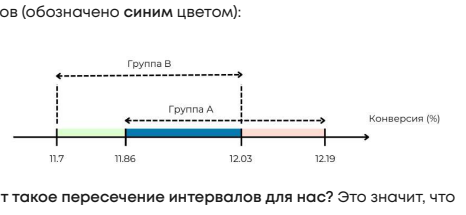
Вычисление доверительного интервала можно было реализовать с помощью библиотечных инструментов, например, функцией proportion.proportion_confint() из модуля stats библиотеки statsmodels. Функция принимает на вход следующие аргументы:

- `count` — количество успехов (в нашем случае это суммарное количество пользователей, выполнивших целевое действие);
- `nobs` — количество наблюдений;
- `alpha` — уровень значимости.

Пример вызова функции:

```
from statsmodels.stats import proportion

# вычисляем доверительный интервал для конверсии в группе А с уровнем доверия
0.95
lower_bound_a, upper_bound_a = proportion.proportion_confint(
    count=a_data['converted'].sum(),
    nobs=a_data['user_id'].count(),
    alpha=0.05
)

# вычисляем доверительный интервал для конверсии в группе B с уровнем доверия
0.95
lower_bound_b, upper_bound_b = proportion.proportion_confint(
    count=b_data['converted'].sum(),
    nobs=b_data['user_id'].count(),
    alpha=0.05
)
# выводим результат
print('Доверительный интервал для конверсии в группе А:
{}'.format((round(lower_bound_a * 100, 2), round(upper_bound_a * 100, 2))))
print('Доверительный интервал для конверсии в группе B:
{}'.format((round(lower_bound_b * 100, 2), round(upper_bound_b * 100, 2))))
# Доверительный интервал для конверсии в группе А: (11.86, 12.19)
# Доверительный интервал для конверсии в группе B: (11.7, 12.03)
```

Если схематично изобразить построенные доверительные интервалы для конверсии в группах А и B, можно отчётливо заметить пересечение интервалов (обозначено **синим** цветом):



Что значит такое пересечение интервалов для нас? Это значит, что существует вероятность, что истинное значение конверсии в обеих группах может совпасть (попасть в **синюю** зону). То есть мы не можем однозначно сказать, есть ли разница в конверсиях. Можно даже рассчитать эту вероятность, но лучше пойти другим путём.

Лучше построить **доверительный интервал разницы пропорций** (конверсий).

Не будем уходить в статистические дебри, просто приведём формулу для расчёта доверительного интервала разницы конверсий в группах. Запоминать её, как и считать руками, не нужно, мы реализуем расчёты с помощью Python. Однако важно понимать её составляющие:

$$\Delta p = \Delta X_p \pm z_{крит} \times \sqrt{\frac{X_{p_a}(1-X_{p_a})}{n_a} + \frac{X_{p_b}(1-X_{p_b})}{n_b}}$$

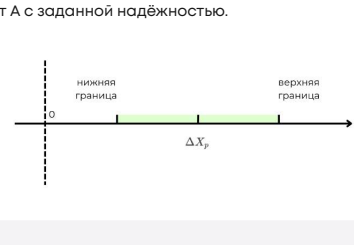
- индексы a и b обозначают принадлежность параметра группе А и B соответственно;
- $\Delta p = p_b - p_a$ — истинная разница конверсий групп B и А;
- $\Delta X_p = X_{p_b} - X_{p_a}$ — выборочная разница конверсий групп B и А.

Таким образом, в центре доверительного интервала находится ΔX_p — это выборочная оценка, — а границы интервала задаются погрешностью $E = z_{крит} \times \sqrt{\frac{X_{p_a}(1-X_{p_a})}{n_a} + \frac{X_{p_b}(1-X_{p_b})}{n_b}}$.

Допустим, мы построим такой доверительный интервал. Как тогда на его основе сделать выводы и принять решение? Возможны три случая:

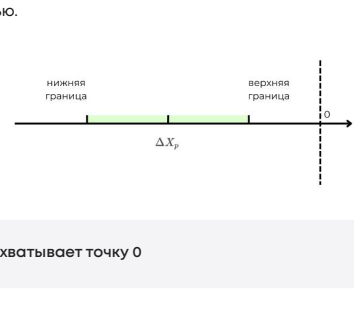
Обе границы доверительного интервала являются положительными (больше 0)

Истинная разница в конверсии $\Delta p = p_b - p_a$ положительная. В таком случае говорят, что вариант B более эффективен по метрике конверсии, чем вариант А с заданной надёжностью.



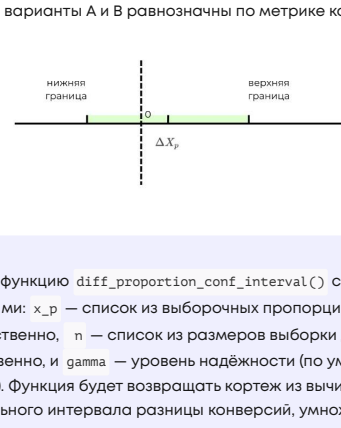
Обе границы доверительного интервала являются отрицательными (меньше 0)

Истинная разница в конверсии $\Delta p = p_b - p_a$ отрицательная. Тогда вариант B хуже по метрике конверсии, чем вариант А с заданной надёжностью.



Интервал охватывает точку 0

Левая граница доверительного интервала отрицательная, а правая — положительная. То есть истинная разница в конверсии $\Delta p = p_b - p_a$ может быть как положительной, так и отрицательной. Тогда это будет значить, что варианты А и B равнозначны по метрике конверсии.



Создадим функцию `diff_proportion_conf_interval()` со следующими аргументами: `x_p` — список из выборочных пропорций для групп А и B соответственно, `n` — список из размеров выборки для групп А и B соответственно, и `gamma` — уровень надёжности (по умолчанию он равен 0.95). Функция будет возвращать кортеж из вычисленных границ доверительного интервала разницы конверсий, умноженных на 100 % и округлённых до второго знака после запятой.

```
def diff_proportion_conf_interval(x_p, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    diff = x_p[1] - x_p[0] # выборочная разница конверсий групп B и A
    z_crit = -norm.ppf(alpha/2) # z-критическое
    eps = z_crit * (x_p[0] * (1 - x_p[0])/n[0] + x_p[1] * (1 - x_p[1])/n[1]) **
0.5 # погрешность
    lower_bound = diff - eps # левая (нижняя) граница
    upper_bound = diff + eps # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound
```

Вызовем функцию и построим интересующий нас доверительный интервал:

```
# размеры выборок групп А и B
n = [a_data['user_id'].count(), b_data['user_id'].count()]
# выборочная пропорция групп A и B
x_p = [a_data['converted'].mean(), b_data['converted'].mean()]


# строим доверительный интервал для разности пропорций
lower_bound, upper_bound = diff_proportion_conf_interval(x_p=x_p, n=n)


# выводим результат
print('Доверительный интервал для разности конверсий:
{}'.format((round(lower_bound*100, 2), round(upper_bound*100, 2))))
```

Итак, что можно сказать теперь? Построенный доверительный интервал с 95 %-ой надёжностью говорит нам, что разница в конверсии между вариантом B и вариантом А лежит в интервале от -0.39 % до 0.08 %.

То есть разница в конверсии с 95 %-ой надёжностью может быть как положительной, так и отрицательной. Мы имеем дело с третьим случаем — интервал охватывает точку 0.

Отсюда можно сделать вывод, что статистической разницы в конверсии между вариантами А и B не существует, то есть они являются равнозначными по данной метрике.

В заключение хотелось бы отметить, что доверительный интервал для пропорции применяется не только для оценки конверсии и разницы конверсий в группах!

Под истинной пропорцией может пониматься доля успешно угаданных моделью объектов (accuracy), доля выздоровевших пациентов или доля ушедших клиентов (**отток**). То есть такой доверительный интервал может быть применён в любой задаче A/B-тестирования, где признак, по которому считается метрика, **бинарный** (имеет два возможных исхода — «успех» и «неуспех»). А это очень большой спектр задач!

Таким образом, доверительные интервалы — это ещё один вспомогательный инструмент проведения A/B-тестирования, который позволяет с определённой заранее надёжностью статистически сравнить варианты продукта А и B по заданной метрике.

Обычно доверительные интервалы используются в комплексе со статистическими тестами. Статистический тест показывает, значимы ли наши результаты, а доверительный интервал показывает пределы изменения метрик и их разницы в группах.

Задание 6.7

0 points possible (ungraded)

Выберите кейсы, для которых может быть использован доверительный интервал для пропорций.

☑ **A** Оценить средний рост студентов в университете

☑ **B** Оценить вероятность того, что случайно выбранный студент в университете имеет рост 189 сантиметров

☐ **C** Оценить истинную долю регистраций на сайте

☐ **D** Оценить среднюю дальность полёта новой ракеты

Отправить

Задание 6.8

0 points possible (ungraded)

Было проведено *A/B*-тестирование главной страницы сайта. В контрольной версии сайта на главной странице использовалась фотография, а в тестовой версии — видео. Варианты сравниваются по доле зарегистрировавшихся пользователей. Расчёты на основе результатов тестирования показали, что доверительный интервал разницы долей зарегистрировавшихся на сайте пользователей (в процентах) между вариантами *B* и *A* с 95 % надёжностью имеет вид: $(5.32, 8.51)$. Какие выводы можно сделать из построенного интервала?

○ Разница в конверсии регистрации находится в интервале от 5.32 % до 8.51 %. Обе границы положительны, что говорит о том, что для повышения регистраций на сайте на главной странице следует использовать фотографию.

○ Разница в конверсии регистрации находится в интервале от 5.32 % до 8.51 %. Обе границы положительны, что говорит о том, что версии сайта идентичны.

○ Разница в конверсии регистрации находится в интервале от 5.32 % до 8.51 %. Обе границы положительны, что говорит о том, что для повышения регистраций на сайте на главной странице следует использовать видео.

○ Никаких выводов сделать нельзя.

Отправить

Задание 6.9

0 points possible (ungraded)

Проводится *A/B*-тестирование вариантов отображения страницы корзины на сайте интернет-магазина. Есть два варианта отображения страницы: с товарными рекомендациями «Хиты продаж» и персональными, подобранными под интерес конкретно этого пользователя. Метрикой является конверсия действия «добавить в корзину товар из рекомендаций». Были собраны следующие данные:

Группа	Число добавленных в корзину товаров	Размер выборки
A (Персональные скидки)	45	1000
B (Хиты продаж)	50	1000

Постройте 95 %-ый доверительный интервал разности полученных конверсий. Ответ приведите в процентах и округлите границы интервала до второго знака после запятой. При расчёте используйте *z*-критическое.

Нижняя граница:

Верхняя граница:

Какой вывод можно сделать на основе полученного доверительного интервала?

○ вариант *A* является более предпочтительным варианту *B* по показателю конверсии

○ вариант *B* является более предпочтительным варианту *A* по показателю конверсии

○ варианты *A* и *B* равнозначны по показателю конверсии

Hint

Отправить

‹ Предыдущий урок Следующий урок ›

7/9 Итоги модуля

текущий урок

Модуль, посвящённый A/B-тестированию, подошёл к концу! Это большая и сложная тема — охватить её полностью практически невозможно. Но вы сделали важный шаг: систематизировали знания и закрепили ключевые подходы к работе с A/B-тестами в контексте задач анализа данных и машинного обучения.

Методов для проверки гипотез существует гораздо больше, и многие из них могут заменять друг друга. Однако того объёма, который вы повторили и закрепили в этом модуле, достаточно для уверенного старта и решения большинства практических задач.

Заложенный фундамент позволит вам решать значительную часть задач, связанных с анализом A/B-тестов. Остальное вы сможете углубить уже в процессе практики и профессиональной деятельности.

Итак, в этом модуле мы:

- повторили основные концепции A/B-тестирования и рассмотрели примеры применения;
- разобрали этапы проведения A/B-теста и ключевые принципы его корректной организации;
- закрепили подходы к определению параметров теста: размера выборки и длительности;
- отработали инструменты анализа результатов A/B-тестирования: кумулятивные графики, статистические тесты и доверительные интервалы.

И напоследок напомним: никогда не забывайте про предварительный анализ и предобработку данных!

< Предыдущий урок Следующий урок >

8/9 Подсказки и эталонные ответы к заданиям модуля

текущий урок

4. Анализ результатов А/В-тестирования: кумулятивные метрики

Задание 4.6

```
# преобразуем тип столбца date к типу datetime
sample_a['date'] = pd.to_datetime(sample_a['date'])
sample_b['date'] = pd.to_datetime(sample_b['date'])
# группируем данные по дате и рассчитываем агрегированные  показатели в
группе А
daily_sample_a = sample_a.groupby('date').agg({
    'cid': 'count',
    'transactions': 'sum',
    'revenue': 'sum',
}).reset_index()
# группируем данные по дате и рассчитываем агрегированные  показатели в
группе В
daily_sample_b = sample_b.groupby('date').agg({
    'cid': 'count',
    'transactions': 'sum',
    'revenue': 'sum',
}).reset_index()
print(daily_sample_a[daily_sample_a['date'] == pd.to_datetime('2020-11-
08')])
```

Задание 4.7

```
# объявляем функцию для расчёта кумулятивных показателей
def calculate_cum(data):
    data['cum_users_count'] = data['cid'].cumsum()
    data['cum_transactions'] = data['transactions'].cumsum()
    data['cum_revenue'] = data['revenue'].cumsum()
    data['cum_conversion'] =
data['cum_transactions']/data['cum_users_count'] * 100
    data['cum_mean_check'] =
data['cum_revenue']/data['cum_transactions']
    return data
# применяем функцию для каждого набора данных:
daily_sample_a = calculate_cum(daily_sample_a)
daily_sample_b = calculate_cum(daily_sample_b)
# вычислим среднюю кумулятивную конверсию в каждой группе
print(round(daily_sample_a['cum_conversion'].mean(), 2))
print(round(daily_sample_b['cum_conversion'].mean(), 2))
```

Задание 4.8

```
# создаём фигуру размером 8x4
fig = plt.figure(figsize=(8, 4))
# добавляем систему координат
ax = fig.add_axes([1, 1, 1, 1])
# строим lineplot для кумулятивной конверсии во времени в каждой группе
sns.lineplot(x='date', y='cum_conversion', data=daily_sample_a, ax=ax,
label='A')
sns.lineplot(x='date', y='cum_conversion', data=daily_sample_b, ax=ax,
label='B')
# задаём подпись к графику
ax.set_title('График кумулятивной конверсии по дням')
# задаём поворот меток на оси абсцисс
ax.xaxis.set_tick_params(rotation = 45)
#задаём отображение сетки
ax.grid(True);
```

Задание 4.9

```
# создаём фигуру размером 8x4
fig = plt.figure(figsize=(8, 4))
# добавляем систему координат
ax = fig.add_axes([1, 1, 1, 1])
# строим lineplot для кумулятивного среднего чека во времени в каждой
группе
sns.lineplot(x='date', y='cum_mean_check', data=daily_sample_a, ax=ax,
label='A')
sns.lineplot(x='date', y='cum_mean_check', data=daily_sample_b, ax=ax,
label='B')
# задаём подпись к графику
ax.set_title('График кумулятивного среднего чека по дням')
# задаём поворот меток на оси абсцисс
ax.xaxis.set_tick_params(rotation = 45)
# задаём отображение сетки
ax.grid(True);
```

5. Анализ результатов А/В-тестирования: статистические тесты

Задание 5.2

Выдвигаем гипотезы:

Нулевая (об отсутствии эффекта): конверсии в группах А и B равны.

$$H_0 : p_a = p_b$$

Альтернативная (о наличии эффекта): конверсии в группах А и B не равны.

$$H_1 : p_a \neq p_b$$

Проверяем гипотезы:

```
from statsmodels.stats.proportion import proportions_ztest

alpha = 0.05 # уровень значимости

# вычисляем значение p-value для z-теста для пропорций
_, p_value = proportions_ztest(
    count=[sample_a['transactions'].sum(),
sample_b['transactions'].sum()], # число «успехов»
    nobs=[sample_a['cid'].count(), sample_b['cid'].count()], # общее
число наблюдений
    alternative='two-sided',
)
# выводим результат на экран
print('p-value: ', round(p_value, 2))
# сравниваем полученное p-value с уровнем значимости
if (p_value <= alpha):
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")

# p-value:  0.44
# У нас нет оснований отвергнуть нулевую гипотезу
```

Итак, p-value = 0.44, что больше уровня значимости $\alpha = 0.05$. Значит, мы не можем отвергнуть нулевую гипотезу о равенстве конверсий в группах. С точки зрения бизнеса это означает, что варианты А и B равнозначны по метрике конверсии.

ЗАДАНИЕ 5.3

```
from scipy.stats import shapiro

alpha = 0.05 # уровень значимости
# вычисляем результат теста Шапиро — Уилка для выборок
shapiro_result_a = shapiro(daily_sample_a['mean_check'])
shapiro_result_b = shapiro(daily_sample_b['mean_check'])
print('p-value группы А', round(shapiro_result_a.pvalue, 2))
print('p-value группы B', round(shapiro_result_b.pvalue, 2))

# сравниваем полученное p-value для группы А с уровнем значимости
if shapiro_result_a.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной.
Распределение в группе А отлично от нормального")
else:
    print("Принимаем нулевую гипотезу. Распределение в группе А является
нормальным")

# сравниваем полученное p-value для группы B с уровнем значимости
if shapiro_result_b.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной.
Распределение в группе B отлично от нормального")
else:
    print("Принимаем нулевую гипотезу. Распределение в группе B является
нормальным")
# p-value группы А 0.01
# p-value группы B 0.03
# Отвергаем нулевую гипотезу в пользу альтернативной. Распределение в
группе А отлично от нормального
# Отвергаем нулевую гипотезу в пользу альтернативной. Распределение в
группе B отлично от нормального
```

Задание 5.4

Выдвигаем гипотезы:

Нулевая гипотеза (об отсутствии эффекта): распределение, лежащее в основе среднего чека в группе А, идентично распределению, лежащему в основе среднего чека в группе B:

$$H_0 : F_a\left(u\right) = F_b\left(u\right)$$

Альтернативная гипотеза (о наличии эффекта): распределение, лежащее в основе среднего чека в группе А, отлично от распределения, лежащего в основе среднего чека в группе B:

$$H_1 : F_a\left(u\right) \neq F_b\left(u\right)$$

```
from scipy.stats import mannwhitneyu


alpha = 0.05 # уровень значимости


# вычисляем результат теста Манна — Уитни для выборок
results = mannwhitneyu(
    x=daily_sample_a['mean_check'],
    y=daily_sample_b['mean_check'],
    alternative='two-sided'
)
print('p-value:', round(results.pvalue, 2))


# сравниваем полученное p-value с уровнем значимости
if results.pvalue <= alpha:
    print("Отвергаем нулевую гипотезу в пользу альтернативной")
else:
    print("У нас нет оснований отвергнуть нулевую гипотезу")
# p-value: 0.12
# У нас нет оснований отвергнуть нулевую гипотезу
```

Итак, p-value = 0.12, что больше уровня значимости $\alpha = 0.05$. Значит, мы не можем отвергнуть нулевую гипотезу о равенстве средних чеков в группах. С точки зрения бизнеса это означает, что варианты А и B равнозначны по метрике ежедневного среднего чека.

6. Доверительные интервалы

Задание 6.5

```
def z_mean_conf_interval(x_mean, sigma, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    z_crit = -norm.ppf(alpha/2) # z-критическое
    eps = z_crit * sigma/(n ** 0.5) #погрешность
    lower_bound = x_mean - eps # левая (нижняя) граница
    upper_bound = x_mean + eps # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound

n = 64 # размер выборки
x_mean = 12.35 # выборочное среднее
sigma = 2.4 # истинное стандартное отклонение
gamma = 0.90 # уровень надёжности

# строим доверительный интервал для среднего при известном СКО
lower_bound, upper_bound = z_mean_conf_interval(x_mean, sigma, n, gamma)

# выводим результат
print('Доверительный интервал: {}'.format((round(lower_bound, 2),
round(upper_bound, 2))))
```

Задание 6.6

```
def t_mean_conf_interval(x_mean, x_std, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    t_crit = -t.ppf(alpha/2, k) # t-критическое
    eps = t_crit * x_std/(n ** 0.5) # погрешность
    lower_bound = x_mean - eps # левая (нижняя) граница
    upper_bound = x_mean + eps # правая (верхняя) граница
    # возвращаем кортеж из границ интервала
    return lower_bound, upper_bound

n = 25 # размер выборки
k = n - 1 # число степеней свободы
x_mean = 3540 # выборочное среднее
x_std = 1150 # выборочное стандартное отклонение
gamma = 0.90 # уровень надёжности

# строим доверительный интервал для среднего при неизвестном СКО
lower_bound, upper_bound = t_mean_conf_interval(x_mean, x_std, n, gamma)

# выводим результат
print('Доверительный интервал: {}'.format((round(lower_bound),
round(upper_bound))))
```

Задание 6.9

```
def diff_proportion_conf_interval(x_p, n, gamma=0.95):
    alpha = 1 - gamma # уровень значимости
    diff = x_p[1] - x_p[0] # выборочная разница конверсий групп B и A
    z_crit = -norm.ppf(alpha/2) #z-критическое
    eps = z_crit * (x_p[0] * (1 - x_p[0])/n[0] + x_p[1] * (1 -
x_p[1])/n[1]) ** 0.5 # погрешность
    lower_bound = diff - eps # левая (нижняя) граница
    upper_bound = diff + eps # правая (верхняя) граница
    # возвращаем кортеж из  границ интервала
    return lower_bound, upper_bound


n = 1000 # размер выборки обеих групп
n_positive_a = 45 # число пользователей, совершивших конверсию, в группе
A
n_positive_b = 50 # число пользователей, совершивших конверсию, в группе
B
xp_a = n_positive_a/n # выборочная пропорция группы A
xp_b = n_positive_b/n # выборочная пропорция группы B


# строим доверительный интервал для разности пропорций
lower_bound, upper_bound = diff_proportion_conf_interval(
    x_p=[xp_a, xp_b],
    n=[n, n]
)


# выводим результат
print('Доверительный интервал для разности конверсий:
{}'.format((round(lower_bound*100, 2), round(upper_bound*100, 2))))
```

‹ Предыдущий урок Следующий урок ›